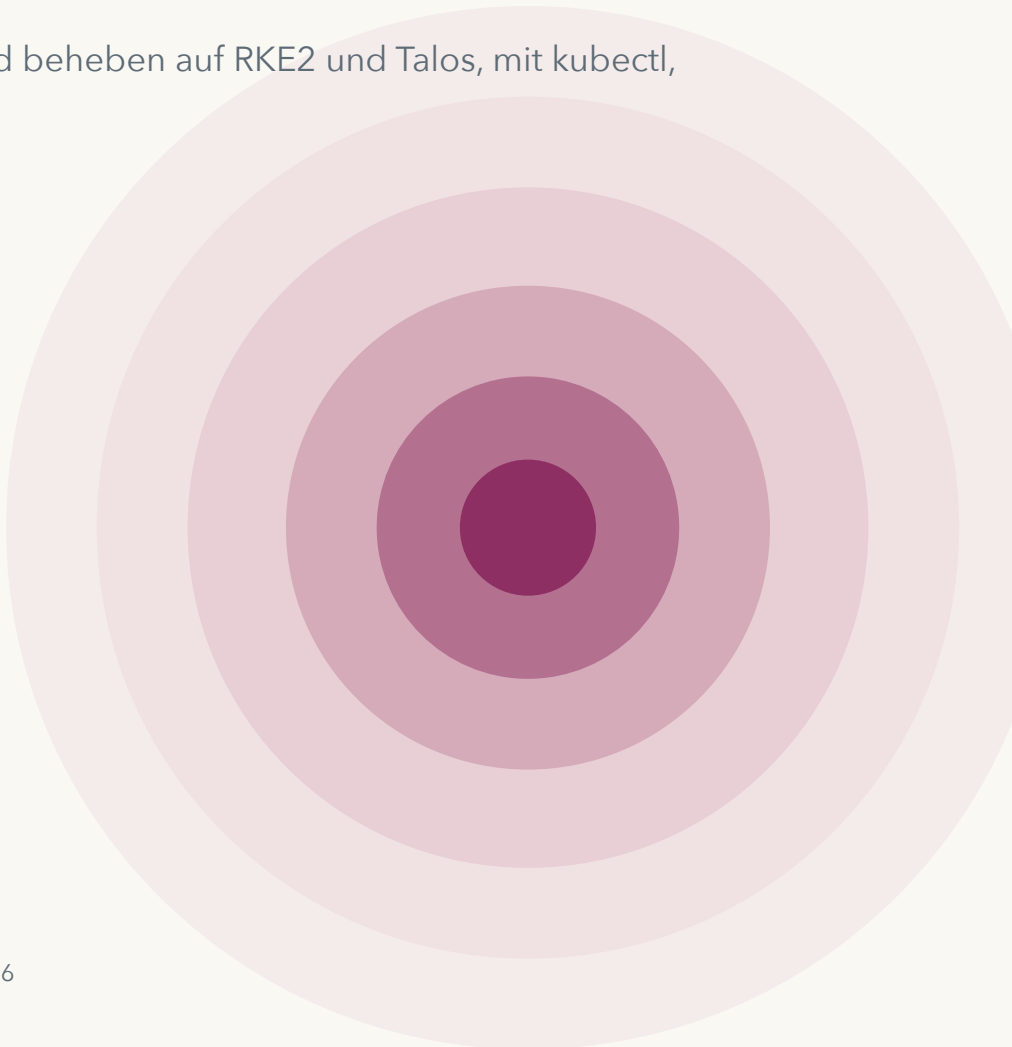
HANDS-ON-HANDBUCH

# Kubernetes debuggen

Fehler finden und beheben auf RKE2 und Talos, mit kubectl, k9s und talosctl



**Thomas Zachmann**

Stand: 9. November 2026

# Inhalt

**So benutzt du dieses Buch ..... 6**
Warum zwei Distributionen ..... 6
Zuerst das Repo klonen ..... 7
Wo du einen Befehl ausführst ..... 7
Die Kästen ..... 7
Wie ein Szenario abläuft ..... 8
Was geprüft wurde und was nicht ..... 9
Was du brauchst ..... 9
**1 Die Methode ..... 12**
1.1 Raten oder eingrenzen ..... 12
1.2 Die Landkarte ..... 13
1.3 Phase, State und Reason ..... 14
1.4 Exit-Codes ..... 15
1.5 Die Grundwerkzeuge ..... 16
1.6 Ein Beispiel von Anfang bis Ende ..... 18
**2 Das Labor ..... 22**
2.1 Was du aufbaust ..... 22
2.2 Werkzeuge auf dem Mac ..... 23
2.3 RKE2: die Konfiguration ..... 24
2.4 RKE2: der Server ..... 25
2.5 RKE2: der Agent ..... 26
2.6 Die Kubeconfig von RKE2 ..... 27
2.7 Talos: booten ..... 27
2.8 Talos: die Patches ..... 28
2.9 Talos: Konfiguration erzeugen ..... 29
2.10 Talos: anwenden und bootstrappen ..... 30
2.11 Storage: local-path-provisioner ..... 31
2.12 k9s einrichten ..... 33
2.13 Abschluss: beide Cluster im Überblick ..... 33
**3 k9s als Cockpit ..... 36**
3.1 Warum ein zweites Werkzeug ..... 36
3.2 Starten und Orientieren ..... 37

3.3 Ressourcen, Namespaces und Filter ... 38
3.4 Ein Pod im Detail ... 39
3.5 Logs lesen ... 40
3.6 Events, XRay und Pulses ... 41
3.7 Port-Forward ... 41
3.8 Kontexte und Read-only ... 42
3.9 Die Plugins dieses Buchs ... 42
3.10 Wo kubectl besser ist ... 45
**4 Pods bleiben Pending ... 48**
4.1 Was in diesem Kapitel kaputtgeht ... 48
4.2 Der Scheduler erklärt sich selbst ... 48
4.3 Zu große Requests ... 49
4.4 nodeSelector und Taint ... 51
4.5 PVC ohne StorageClass ... 53
4.6 ResourceQuota: Der Pod, den es nicht gibt ... 56
**5 Images und Container-Start ... 60**
5.1 Was in diesem Kapitel kaputtgeht ... 60
5.2 Drei Pull-Fehler, drei Meldungen ... 60
5.3 Auf dem Knoten nachsehen ... 63
5.4 Rate-Limit und Registry-Mirror ... 64
5.5 CreateContainerConfigError ... 66
5.6 exec format error ... 68
**6 Abstürze und Neustarts ... 72**
6.1 Was in diesem Kapitel kaputtgeht ... 72
6.2 CrashLoopBackOff: Befehl und Anwendung ... 72
6.3 OOMKilled ... 75
6.4 Probes: zu streng und falsch gezielt ... 77
6.5 readOnlyRootFilesystem ... 79
6.6 Distroless: Debuggen ohne Shell ... 81
**7 Services, DNS und NetworkPolicies ... 86**
7.1 Was in diesem Kapitel kaputtgeht ... 86
7.2 Der Service findet keine Pods ... 87
7.3 Endpoints da, Verbindung trotzdem abgelehnt ... 89
7.4 Namen, Namespaces und ndots ... 90

7.5 Default-Deny blockiert DNS ... 92
7.6 Die Policy, die nichts tut ... 95
**8 Ingress, Gateway und TLS ... 98**
8.1 Was in diesem Kapitel kaputtgeht ... 98
8.2 Traefik auf beiden Clustern ... 98
8.3 404: Der Controller fühlt sich nicht zuständig ... 101
8.4 404 trotz richtiger Klasse ... 102
8.5 503: Die Route steht, aber niemand ist bereit ... 104
8.6 TLS: das falsche Zertifikat ... 105
8.7 Das Dashboard und die Gateway API ... 108
**9 Storage ... 110**
9.1 Was in diesem Kapitel kaputtgeht ... 110
9.2 Der PVC findet seine StorageClass nicht ... 111
9.3 Talos: Der Helper-Pod darf nicht starten ... 112
9.4 Zwei Replikas, ein Volume, zwei Knoten ... 115
9.5 Permission denied auf dem Volume ... 117
9.6 Wenn der Platz ausgeht ... 119
**10 RBAC und Pod Security ... 122**
10.1 Was in diesem Kapitel kaputtgeht ... 122
10.2 Eine Forbidden-Meldung lesen ... 122
10.3 RoleBinding im falschen Namespace ... 123
10.4 Role statt ClusterRole ... 126
10.5 Pod Security: Was die Distributionen vorgeben ... 128
10.6 hostPath: läuft auf RKE2, nicht auf Talos ... 128
10.7 restricted: Deployment ja, Pods nein ... 131
**11 Fehler durch Kyverno ... 136**
11.1 Was in diesem Kapitel kaputtgeht ... 136
11.2 Kyverno installieren ... 136
11.3 Policy lehnt das Deployment ab ... 137
11.4 Policy lehnt nur die Pods ab ... 139
11.5 Mutation ändert den Pod unerwartet ... 141
11.6 Webhook nicht erreichbar ... 144
11.7 PolicyException ohne Wirkung ... 146
11.8 Policies testen, bevor sie stören ... 148

**12 Falco: Alarme und Störungen ... 152**
12.1 Was in diesem Kapitel kaputtgeht ... 152
12.2 Falco installieren ... 152
12.3 Falco startet nicht auf Talos ... 153
12.4 Alarm nach `kubectl exec` ... 155
12.5 Fehlalarm durch einen legitimen Job ... 157
12.6 Keine Kubernetes-Metadaten ... 160
**13 Knoten ... 164**
13.1 Was in diesem Kapitel kaputtgeht ... 164
13.2 Der Werkzeugkasten für Knoten ... 164
13.3 Knoten wird NotReady ... 165
13.4 Pods hängen nach einem Knotenausfall ... 169
13.5 Pods werden evicted ... 171
13.6 Drain hängt an einem PodDisruptionBudget ... 174
13.7 Aufräumen ... 177
**14 Control Plane, etcd und Webhooks ... 178**
14.1 Was in diesem Kapitel kaputtgeht ... 178
14.2 Ist die API gesund? ... 178
14.3 Vor jedem Eingriff: etcd prüfen und sichern ... 179
14.4 Der API-Server startet nicht ... 181
14.5 Abgelaufenes Client-Zertifikat ... 183
14.6 Ein Webhook blockiert den Namespace ... 186
14.7 Der Namespace bleibt in Terminating ... 188
**15 Rollouts, Autoscaling und Helm ... 192**
15.1 Was in diesem Kapitel kaputtgeht ... 192
15.2 Der Rollout hängt ... 192
15.3 Der HPA misst nicht ... 195
15.4 Das Helm-Release hängt ... 198
15.5 Der Job gibt auf ... 200
15.6 Die ConfigMap-Änderung kommt nicht an ... 201
15.7 Aufräumen ... 202
**16 Anhang ... 204**
16.1 Spickzettel: Von der Meldung zum ersten Befehl ... 204
16.2 RKE2 und Talos: dieselbe Frage, zwei Befehle ... 206

16.3 k9s: die wichtigsten Tasten ..... 207
16.4 Versionen ..... 208
16.5 Das Repo im Überblick ..... 208
16.6 Offizielle Dokumentation ..... 209

# So benutzt du dieses Buch

Ein Kubernetes-Cluster gibt selten zu, was ihm fehlt. Er zeigt dir `Pending`, `CrashLoopBackOff` oder ein knappes `Forbidden` und überlässt dir den Rest. Wer schon einmal eine Stunde auf einen Pod gestarrt hat, der einfach nicht startet, kennt das Gefühl. Meist lag die Antwort die ganze Zeit in einem Event, das niemand gelesen hat.

Dieses Buch macht aus der Fehlersuche eine Routine. Du baust dir zwei kleine Cluster und machst sie absichtlich kaputt, immer wieder und auf viele Arten. Zu jedem Fehler siehst du, wie er sich zeigt, wie du ihn mit `kubectl` und k9s untersuchst, was die eigentliche Ursache ist und wie du ihn behebst. Nach einigen Kapiteln wirst du merken, dass sich die Fragen wiederholen. Genau das ist das Ziel.

## Warum zwei Distributionen

Kubernetes ist überall gleich, die Plattform darunter nicht. Die Fehlersuche endet oft nicht beim Pod, sondern auf dem Knoten: beim Kubelet, bei containerd, beim etcd oder bei einer Einstellung, die die Distribution mitbringt. Deshalb arbeitest du mit zwei sehr unterschiedlichen Vertretern:

| | RKE2 | Talos Linux |
|---|---|---|
| Betriebssystem | Ubuntu 24.04, RKE2 als systemd-Dienst | Talos, unveränderliches Linux nur für Kubernetes |
| Zugang zum Knoten | SSH, `journalctl`, `crictl` | keine Shell, nur die API mit `talosctl` |
| Konfiguration | `/etc/rancher/rke2/config.yaml` | Machine-Config, per `talosctl` angewendet |
| Netzwerk | Canal, setzt NetworkPolicies durch | Flannel, ignoriert NetworkPolicies |
| Mitgeliefert | Traefik, metrics-server, CoreDNS | CoreDNS, sonst bewusst wenig |

RKE2 fühlt sich an wie ein klassischer Linux-Server. Bei Talos funktionieren viele gewohnte Handgriffe nicht, weil es keine Shell gibt. Wo sich die Fehlersuche unterscheidet, zeigt ein Kasten beide Wege nebeneinander:

**RKE2**

Hier steht, wie du auf RKE2 vorgehst, meist per SSH auf dem Knoten.

**TALOS**

Hier steht der Weg über `talosctl`, ohne Shell auf dem Knoten.

## Zuerst das Repo klonen

Alle Dateien, auf die sich das Buch bezieht, liegen in einem öffentlichen Repository: github.com/thomaszachmann/kubernetes-debuggen. Jedes Fehlerszenario hat dort einen eigenen Ordner unter `szenarien/` mit einer Datei, die den Fehler erzeugt, und einer, die ihn behebt.

MAC Schritt 1

```
git clone https://github.com/thomaszachmann/kubernetes-debuggen.git
cd kubernetes-debuggen
```

Lädt das Repo und wechselt ins Wurzelverzeichnis. Von hier aus laufen alle Befehle mit dem Etikett **Mac**. Pfade wie `szenarien/04-pending/` sind immer relativ zu diesem Ordner.

## Wo du einen Befehl ausführst

Jeder Befehl trägt ein farbiges Etikett:

| Etikett | Bedeutung |
|---|---|
| MAC | Dein Terminal auf dem Laptop, im Wurzelverzeichnis des Repos. Von hier aus steuerst du beide Cluster mit `kubectl --context rke2` oder `--context talos` und die Talos-Knoten mit `talosctl`. |
| K9S | Tastatureingaben in k9s. `⏎` steht für die Eingabetaste. |
| RKE2-CP | Eine SSH-Sitzung auf dem RKE2-Server `rke2-cp` (`172.16.0.160`). |
| RKE2-W1 | Eine SSH-Sitzung auf dem RKE2-Agent `rke2-w1` (`172.16.0.161`). |
| POD | Eine Shell in einem Container, geöffnet mit `kubectl exec` oder `kubectl debug`. |

Für Talos gibt es kein eigenes Etikett. Auf einen Talos-Knoten loggst du dich nie ein, `talosctl` läuft immer auf dem Mac.

**WARUM DAS WICHTIG IST**

Mit zwei Clustern ist der häufigste Fehler der falsche Kontext. Deshalb nennt jeder Befehl ausdrücklich `--context rke2` oder `--context talos`. Den aktiven Standardkontext von `kubectl` nutzt dieses Buch nie. Gewöhn dir das auch im Alltag an: Ein Fix im falschen Cluster ist schlimmer als gar kein Fix.

## Die Kästen

**SYMPTOM**

Was du siehst, wenn der Fehler auftritt. So beginnt jede Fehlersuche.

**URSACHE**

Was wirklich dahintersteckt, und warum Kubernetes sich so verhält.

**DAS SOLLTEST DU SEHEN**

Das erwartete Ergebnis. Weicht deine Ausgabe ab, geh nicht weiter, sondern kläre zuerst den Unterschied.

**WARUM DAS WICHTIG IST**

Der Hintergrund, der dir beim nächsten, ähnlichen Problem hilft.

**STOLPERFALLE**

Was typischerweise schiefgeht und woran du es erkennst.

**PROBIER ES SELBST**

Hier bist du dran. Überlege zuerst selbst, bevor du nachschlägst.

## Wie ein Szenario abläuft

Ab Kapitel 4 folgt jedes Szenario demselben Muster. Du wendest eine Datei an, die einen Fehler enthält. Dann siehst du das Symptom, untersuchst es Schritt für Schritt und findest die Ursache. Erst danach behebst du den Fehler und prüfst das Ergebnis. Lies die Ursache nicht vorab. Der Lerneffekt entsteht beim Suchen, nicht beim Lesen der Lösung.

Jedes Szenario läuft in einem eigenen Namespace, etwa `dbg-04` für Kapitel 4. Was du dort kaputtmachst, bleibt dort. Am Ende eines Kapitels räumst du mit einem einzigen Befehl auf:

MAC — Schritt 2

```
kubectl --context rke2 delete namespace dbg-04
kubectl --context talos delete namespace dbg-04
```

Löscht den Namespace samt allen Objekten darin, auf beiden Clustern.

Die Kapitel bauen lose aufeinander auf. Kapitel 1 bis 3 solltest du der Reihe nach lesen. Danach kannst du springen, etwa direkt zu Kyverno in Kapitel 11, wenn dich das gerade beschäftigt.

## Was geprüft wurde und was nicht

Ein Buch voller Befehle ist nur so gut wie seine Prüfung. Deshalb steht hier ehrlich, wie weit sie ging.

**Geprüft am 9. November 2026:**

- Alle Versionen stammen aus den offiziellen Release-Seiten und Helm-Repositories: RKE2, Talos, k9s, Kyverno, Falco, Traefik, metrics-server und local-path-provisioner.
- Alle Kubernetes-Manifeste im Repo wurden mit `kubeconform` gegen Kubernetes 1.36 geprüft.
- Alle Kyverno-Policies bestehen `kyverno test` mit der Kyverno-CLI 1.19.1 gegen die Testressourcen im Repo.
- Alle Helm-Werte wurden mit `helm template` gegen die genannten Chart-Versionen gerendert, die Shell-Skripte bestehen `shellcheck`.
- Die Talos-Patches wurden in eine vollständige Konfiguration eingebaut und mit `talosctl validate` geprüft, allerdings mit `talosctl` 1.12. Befehle, die erst Talos 1.14 kennt, wurden in Dokumentation und Quellcode von 1.14.5 nachgeschlagen.
- Fehlermeldungen von Kubernetes, Kyverno, Traefik und Helm wurden im Quellcode der genannten Versionen nachgelesen. Befehle, Flags und Pfade von RKE2, k9s und `kubectl` stammen aus der offiziellen Dokumentation und den CLIs selbst.

**Nicht geprüft:**

- Das Buch wurde **nicht** auf echten Clustern durchgespielt. Alle Ausgaben in den Kästen und Codeblöcken sind Beispiele, die dem Format der echten Werkzeuge folgen. Namen von Pods, Zeitangaben und IDs werden bei dir anders aussehen.
- Fehlermeldungen können sich zwischen Versionen leicht im Wortlaut ändern. Der Kern, also das Schlüsselwort wie `Insufficient cpu` oder `violates PodSecurity`, bleibt in aller Regel gleich.

Rechne also damit, dass an einer Stelle etwas abweicht. Für dieses Buch ist das ein Geschenk: Es ist ein echter Fehler, und du hast gerade das Werkzeug in der Hand, ihn zu finden.

## Was du brauchst

- Vier VMs mit festen IP-Adressen im selben Netz:

| Rolle | Hostname | IP | Größe |
|---|---|---|---|
| RKE2-Server | `rke2-cp` | `172.16.0.160` | 4 CPUs, 8 GB RAM, 60 GB Disk, Ubuntu 24.04 |
| RKE2-Agent | `rke2-w1` | `172.16.0.161` | 2 CPUs, 4 GB RAM, 40 GB Disk, Ubuntu 24.04 |
| Talos-Control-Plane | `talos-cp` | `172.16.0.170` | 4 CPUs, 8 GB RAM, 60 GB Disk, Talos-ISO |

| Rolle | Hostname | IP | Größe |
|---|---|---|---|
| Talos-Worker | `talos-w1` | `172.16.0.171` | 2 CPUs, 4 GB RAM, 40 GB Disk, Talos-ISO |

- Auf dem Laptop `kubectl`, `talosctl`, `helm`, `k9s`, `jq`, `curl`, `openssl` und `git`. Für Kapitel 11 zusätzlich die Kyverno-CLI.
- SSH-Zugang zu den beiden Ubuntu-VMs mit einem Benutzer, der `sudo` darf.
- Kubernetes-Grundwissen: Du weißt, was Pods, Deployments, Services und Namespaces sind. Alles Weitere erklärt das Buch.

**ANDERE IP-ADRESSEN?**

Die vier Adressen stehen in den Konfigurationen unter `labor/` und in vielen Befehlen. Hast du andere, ersetze sie überall, bevor du mit Kapitel 2 beginnst. Ein `grep -rn "172.16.0.1[67]" .` zeigt dir jede Stelle.

KAPITEL 1

# Die Methode

*Du ordnest jedes Symptom einer Schicht des Clusters zu, kennst die Grundwerkzeuge von `kubectl` für jede Schicht und liest Pod-Status, Container-State und Exit-Code richtig. Ein kleines Beispiel zeigt den Ablauf von Anfang bis Ende.*

**Dauer:** 45 Minuten **Voraussetzung:** Kubernetes-Grundwissen; für das Beispiel das Labor aus Kapitel 2

## 1.1 Raten oder eingrenzen

Wer zum ersten Mal vor einem Pod in `CrashLoopBackOff` steht, probiert gern aus: Image neu bauen, Pod löschen, Deployment neu anwenden. Manchmal hilft das sogar. Verstanden hast du danach nichts, und beim nächsten Mal fängst du wieder von vorn an.

Dieses Buch arbeitet anders. Jeder Fehler durchläuft dieselben fünf Schritte:

| Schritt | Was du tust |
|---|---|
| Reproduzieren | Den Fehler gezielt auslösen. Was du nicht auslösen kannst, kannst du auch nicht sicher als behoben erklären. |
| Untersuchen | Befehle in fester Reihenfolge, von außen nach innen. Du sammelst Fakten, keine Vermutungen. |
| Verstehen | Die Ursache in zwei, drei Sätzen benennen, mit dem Mechanismus dahinter. |
| Beheben | Die kleinste Änderung, die die Ursache beseitigt. Nicht das Symptom. |
| Prüfen | Zeigen, dass es wieder läuft, und zwar am selben Signal, an dem du den Fehler gesehen hast. |

Der wichtigste Schritt ist der zweite. Gute Fehlersucher sind nicht schneller im Raten, sie stellen bessere Fragen in besserer Reihenfolge. Jede Frage halbiert den Raum der möglichen Ursachen. Nach drei, vier Fragen bleibt meist nur noch eine übrig.

**WARUM DAS WICHTIG IST**

Schreib dir während der Suche kurz auf, was du geprüft hast und was dabei herauskam. Das klingt umständlich, verhindert aber, dass du denselben Befehl dreimal ausführst. Und wenn du den Fall an Kollegen übergibst, liegt die halbe Analyse schon vor.

## 1.2 Die Landkarte

Ein Kubernetes-Cluster besteht aus Schichten, die aufeinander aufbauen. Fast jedes Symptom lässt sich einer davon zuordnen. Wer die Schicht kennt, kennt auch die ersten Befehle.

| Schicht | Typisches Symptom | Erste Frage | Kapitel |
|---|---|---|---|
| Scheduling | Pod bleibt `Pending` | Warum findet der Scheduler keinen Knoten? | 4 |
| Image und Runtime | `ImagePullBackOff`, `CreateContainerConfigError` | Kann der Knoten das Image holen und den Container anlegen? | 5 |
| Workload | `CrashLoopBackOff`, `OOMKilled`, nicht ready | Was sagt der Prozess selbst, und womit endet er? | 6 |
| Netzwerk | Timeout, `connection refused`, DNS-Fehler | Kommt das Paket an, und antwortet jemand? | 7, 8 |
| Storage | PVC `Pending`, `Permission denied` | Gibt es ein Volume, und darf der Prozess hineinschreiben? | 9 |
| Policy und Admission | `Forbidden`, `denied the request` | Wer hat abgelehnt: RBAC, Pod Security oder ein Webhook? | 10, 11 |
| Laufzeit | Falco-Alarm, Prozess verhält sich unerwartet | Ist das ein Angriff oder ein legitimer Ablauf? | 12 |
| Knoten | `NotReady`, Evictions, `Unknown` | Lebt das kubelet, und hat der Knoten genug Ressourcen? | 13 |
| Control Plane | `kubectl` hängt, Änderungen kommen nicht an | Sind API-Server, etcd und Webhooks gesund? | 14 |
| Rollout | Deployment hängt, HPA zeigt `<unknown>` | Wartet der Controller auf etwas, das nie eintritt? | 15 |

Die Schichten haben eine Richtung. Ein Fehler weiter unten zeigt sich oft weiter oben. Ein Knoten mit voller Platte führt zu Evictions, die du zuerst als verschwundene Pods siehst. Ein hängender Webhook in der Control Plane lässt jedes `kubectl apply` scheitern, obwohl deine Manifeste fehlerfrei sind. Du beginnst deshalb oben beim Symptom, gehst aber bereitwillig eine Schicht tiefer, sobald die Fakten dorthin zeigen.

### Von außen nach innen

Für die Untersuchung gilt eine feste Reihenfolge. Sie ist so gewählt, dass jeder Schritt billig ist und den nächsten vorbereitet:

1. `kubectl get`: Wie ist der Zustand, und seit wann?
2. `kubectl describe`: Was weiß Kubernetes über dieses Objekt, welche Events hängen daran?
3. `kubectl events`: Was ist drumherum passiert, auch an Objekten, die du noch nicht im Blick hattest?
4. `kubectl logs`: Was sagt der Prozess, auch der vorherige Lauf?
5. `kubectl debug`: Wie sieht es von innen aus, im Netzwerk und im Dateisystem des Pods?
6. Knoten: kubelet, containerd, Kernel. Auf RKE2 per SSH, auf Talos mit `talosctl`.
7. Control Plane: API-Server, etcd, Webhooks.

Die meisten Fehler sind nach Schritt 4 erklärt. Die Schritte 6 und 7 brauchst du selten, aber wenn, dann dringend. Deshalb übst du sie in den Kapiteln 13 und 14 an einem gesunden Cluster, bevor es ernst wird.

## 1.3 Phase, State und Reason

Die Spalte `STATUS` von `kubectl get pods` ist eine Zusammenfassung, die `kubectl` aus mehreren Feldern baut. Wer sie liest, als wäre sie ein einziges Feld, zieht falsche Schlüsse. Dahinter stehen drei Ebenen:

| Ebene | Feld | Werte |
|---|---|---|
| Pod-Phase | `status.phase` | `Pending`, `Running`, `Succeeded`, `Failed`, `Unknown`. Nur diese fünf. |
| Container-State | `status.containerStatuses[].state` | Genau einer von `waiting`, `running`, `terminated` |
| Reason | `state.waiting.reason`, `state.terminated.reason` | `ContainerCreating`, `ImagePullBackOff`, `CrashLoopBackOff`, `CreateContainerConfigError`, `Completed`, `Error`, `OOMKilled` und weitere |

Ein Pod in `CrashLoopBackOff` hat die Phase `Running`. Das überrascht viele. Die Phase sagt nur, dass der Pod einem Knoten zugewiesen ist und mindestens ein Container gestartet wurde oder gerade neu startet. Der Container selbst steht in `waiting` mit dem Reason `CrashLoopBackOff`, und `kubectl` zeigt diesen Reason in der Spalte `STATUS`. Auch `Terminating` ist keine Phase. `kubectl` zeigt es an, sobald der Pod einen `deletionTimestamp` hat.

Für die Ursache wichtiger als der aktuelle State ist oft `lastState`. Dort steht, wie der vorige Lauf des Containers endete:

MAC Schritt 1

```
kubectl --context rke2 get pod <pod> -n dbg-01 \
  -o jsonpath='{.status.containerStatuses[0].lastState.terminated}'
```

Zeigt Reason, Exit-Code, Start- und Endzeit des letzten beendeten Laufs. `describe` zeigt dasselbe unter `Last State`, aber hier bekommst du es maschinenlesbar, etwa für ein Skript.

## 1.4 Exit-Codes

Endet ein Prozess im Container, meldet die Runtime seinen Exit-Code an das kubelet. Wurde der Prozess durch ein Signal beendet, lautet der Code nach Unix-Konvention 128 plus Signalnummer. Diese sieben Codes solltest du auswendig kennen:

| Code | Bedeutung | Typische Ursache |
|---|---|---|
| `0` | Regulär beendet | Bei einem Job richtig. Bei einem Deployment ein Fehler: Der Hauptprozess darf nicht enden, sonst startet Kubernetes ihn neu. |
| `1` | Allgemeiner Fehler | Die Anwendung hat selbst abgebrochen. Die Antwort steht fast immer im Log, oft im vorherigen. |
| `126` | Nicht ausführbar | Datei gefunden, aber keine Ausführungsrechte, oder ein Skript ohne passenden Interpreter. |
| `127` | Befehl nicht gefunden | Tippfehler im `command`, Programm fehlt im Image. Die Shell im Container meldet `not found`. |
| `137` | SIGKILL (128 + 9) | Mit Reason `OOMKilled`: Speicherlimit überschritten. Ohne: hart beendet, etwa weil der Prozess nach einer gescheiterten Liveness-Probe nicht auf SIGTERM reagierte. |
| `139` | SIGSEGV (128 + 11) | Speicherzugriffsfehler, meist ein Fehler in nativem Code, etwa in einer C-Bibliothek der Anwendung. |
| `143` | SIGTERM (128 + 15) | Geordnet beendet, zum Beispiel beim Rollout oder durch `kubectl delete`. Selten ein Fehler. |

**STOLPERFALLE**

Ein Container, dessen Programm gar nicht erst gestartet werden kann, etwa weil der Pfad im `command` nicht existiert und keine Shell dazwischen sitzt, endet nicht mit 127. containerd meldet dann den Reason `StartError` mit Exit-Code `128` und einer Meldung wie `executable file not found in $PATH`. Erst wenn eine Shell den Befehl sucht, wie bei `sh -c`, bekommst du die 127.

## 1.5 Die Grundwerkzeuge

Alle Befehle in diesem Abschnitt laufen gegen beide Cluster gleich. Die Beispiele zeigen `--context rke2`. Für Talos tauschst du nur den Kontext.

### get

MAC — Schritt 2

```
kubectl --context rke2 get pods -n dbg-01 -o wide
```

`-o wide` ergänzt Pod-IP und Knoten. Damit siehst du sofort, ob alle kaputten Pods auf demselben Knoten liegen. Dann liegt die Ursache eher am Knoten als an der Anwendung.

Achte in der Ausgabe auf `READY` (wie viele Container sind bereit), `RESTARTS` (mit Zeitpunkt des letzten Neustarts) und `AGE`. Ein Pod mit 40 Neustarts in zwei Stunden hat ein anderes Problem als einer, der vor fünf Minuten zum ersten Mal abgestürzt ist. Mit `-w` beobachtest du Änderungen live, mit `-o yaml` siehst du das vollständige Objekt samt `status`.

### describe

MAC — Schritt 3

```
kubectl --context rke2 describe pod <pod> -n dbg-01
```

Fasst Spezifikation, Status aller Container, Conditions und die Events des Pods zusammen. Lies von unten nach oben: zuerst die Events, dann `State` und `Last State` der Container.

`describe` zeigt nur Events, die direkt an diesem Objekt hängen. Lehnt etwa Pod Security einen Pod ab, entsteht der Pod nie. Das Event hängt dann am ReplicaSet, und ein `describe` auf den Pod ist unmöglich, weil es ihn nicht gibt. Darum gehört der nächste Befehl immer dazu.

### events

MAC — Schritt 4

```
kubectl --context rke2 events -n dbg-01
kubectl --context rke2 events -n dbg-01 --for pod/<pod>
kubectl --context rke2 events -A --types=Warning
```

`kubectl events` listet Events zeitlich sortiert, das neueste unten. `--for` filtert auf ein Objekt, `--types=Warning` blendet die normalen Meldungen aus. `-A` sucht in allen Namespaces.

Das ältere `kubectl get events` sortiert nicht nach Zeit. Brauchst du es trotzdem, etwa für ein bestimmtes Ausgabeformat, ergänze `--sort-by=.lastTimestamp`.

**STOLPERFALLE**

Events sind kurzlebig. Der API-Server löscht sie standardmäßig nach einer Stunde. Ein Fehler von gestern Abend hat heute Morgen keine Events mehr. Dann bleiben dir nur `status`, Logs und, falls vorhanden, ein zentrales Log-System.

### logs

MAC — Schritt 5

```
kubectl --context rke2 logs <pod> -n dbg-01
kubectl --context rke2 logs <pod> -n dbg-01 --previous
kubectl --context rke2 logs <pod> -n dbg-01 -c <container>
kubectl --context rke2 logs <pod> -n dbg-01 --all-containers --since=10m
kubectl --context rke2 logs deploy/app -n dbg-01 --tail=50
```

Der erste Befehl zeigt das Log des laufenden Containers. Startet der Container ständig neu, ist das oft leer oder zeigt nur den Anfang. Die Ursache steht dann im vorherigen Lauf: `--previous`. Bei mehreren Containern wählst du mit `-c` einen aus oder nimmst `--all-containers`. `deploy/app` sucht sich einen Pod des Deployments heraus.

Das kubelet hält vom vorherigen Lauf genau einen Stand vor. Wird der Pod gelöscht und neu erstellt, etwa durch einen Rollout, ist auch dieser weg. Das gilt für `kubectl` und k9s gleichermaßen.

### debug

Viele Images enthalten keine Shell und keine Werkzeuge. `kubectl exec` hilft dann nicht. `kubectl debug` hängt einen zusätzlichen Container mit deinem Werkzeug-Image an:

MAC — Schritt 6

```
kubectl --context rke2 debug -it <pod> -n dbg-01 \
  --image=nicolaka/netshoot:v0.15 --profile=baseline --target=app
```

Startet einen Ephemeral Container mit netshoot im laufenden Pod. Er teilt das Netzwerk des Pods. Mit `--target` sieht er auch die Prozesse des Containers `app`, etwa mit `ps`.

| Variante | Wofür |
|---|---|
| `kubectl debug <pod> --image=...` | Ephemeral Container im laufenden Pod. Netzwerk und mit `--target` auch Prozesse teilen. |
| `kubectl debug <pod> --copy-to=<neu> ...` | Kopie des Pods, optional mit anderem Image (`--set-image`) oder anderem Befehl. Gut, wenn der Original-Pod sofort abstürzt. |
| `kubectl debug node/<knoten> ...` | Pod mit Host-Namespaces auf einem Knoten, das Dateisystem des Knotens liegt unter `/host`. |

**STOLPERFALLE**

Ohne `--profile` nimmt `kubectl debug` das Profil `general`. Es fügt dem Debug-Container die Capability `SYS_PTRACE` hinzu. Pod Security im Level `baseline` erlaubt diese Capability nicht, und Talos erzwingt `baseline` in jedem Namespace außer `kube-system`. Dort lehnt der API-Server den Debug-Container ab. Mit `--profile=baseline` klappt es auf beiden Clustern. `kubectl debug node/` braucht dagegen Host-Namespaces und scheitert auf Talos in einem normalen Namespace. Kapitel 13 zeigt, wie du damit umgehst.

### explain, auth can-i und der API-Server

Drei Befehle ergänzen das Werkzeug. Du brauchst sie seltener, aber sie beantworten Fragen, die sonst lange dauern:

MAC — Schritt 7

```
kubectl --context rke2 explain pod.spec.containers.startupProbe
```

Zeigt die Dokumentation eines Felds direkt aus dem Schema des API-Servers, passend zur Version des Clusters. Schneller als jede Suche, und es gilt genau für diesen Cluster, auch für CRDs.

MAC — Schritt 8

```
kubectl --context rke2 auth can-i list pods -n dbg-01 \
  --as=system:serviceaccount:dbg-01:default
```

Fragt, ob ein bestimmter Benutzer oder ServiceAccount eine Aktion ausführen darf. Die Antwort ist `yes` oder `no`. Mit `--list` siehst du alle Rechte. Mehr dazu in Kapitel 10.

MAC — Schritt 9

```
kubectl --context rke2 get --raw '/readyz?verbose'
```

Fragt den API-Server nach seinen internen Prüfungen. Jede Zeile beginnt mit `[+]` (in Ordnung) oder `[-]` (fehlgeschlagen), am Ende steht `readyz check passed`. Wenn hier etcd oder ein Informer hängt, hilft keine Suche in deinen Pods.

## 1.6 Ein Beispiel von Anfang bis Ende

Ein kleiner Fall zeigt den Ablauf. Eine Anwendung erwartet eine Umgebungsvariable `DB_HOST`. Fehlt sie, bricht sie mit einer Meldung ab. Das Labor aus Kapitel 2 muss dafür laufen. Lies das Beispiel jetzt trotzdem, und spiel es nach, sobald die Cluster stehen.

DATEI `szenarien/01-methode/kaputt.yaml (Ausschnitt)`

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: app
  namespace: dbg-01
spec:
  # ...
      containers:
        - name: app
          image: busybox:1.37
          command:
            - sh
            - -c
            - |
              if [ -z "$DB_HOST" ]; then
                echo "FEHLER: DB_HOST ist nicht gesetzt" >&2
                exit 1
              fi
              echo "verbinde mit $DB_HOST"
              sleep 3600
```

Die Datei legt zuerst den Namespace `dbg-01` an. Pod und Container laufen ohne Root und ohne Capabilities. So entspricht der Pod dem Level `restricted`, und Talos gibt beim Anwenden keine Warnungen aus.

**MAC** Schritt 10

```
kubectl --context rke2 apply -f szenarien/01-methode/kaputt.yaml
```

Legt Namespace und Deployment an.

**SYMPTOM**

```
NAME                   READY   STATUS             RESTARTS      AGE
app-6b8f9c7d54-x7kqp   0/1     CrashLoopBackOff   3 (21s ago)   88s
```

Der Pod ist nie bereit, die Neustarts steigen, und die Abstände dazwischen werden länger.

Der Status `CrashLoopBackOff` sagt nur, dass der Container wiederholt endet und das kubelet mit dem nächsten Start wartet. Warum er endet, steht woanders. Erste Frage: Wie endete der letzte Lauf?

**MAC** Schritt 11

```
kubectl --context rke2 describe pod -n dbg-01 -l app=app
```

`-l` wählt den Pod über sein Label, du musst den Namen nicht abtippen.

```
    State:          Waiting
      Reason:       CrashLoopBackOff
    Last State:     Terminated
      Reason:       Error
      Exit Code:    1
...
Events:
  Type     Reason   Age                From     Message
  ----     ------   ----               ----     -------
```

```
  Warning  BackOff  12s (x6 over 85s)  kubelet  Back-off restarting failed container
app in pod ...
```

Exit-Code `1`, Reason `Error`, kein `OOMKilled`. Das Image wurde gezogen, der Container gestartet. Damit sind Scheduling, Image und Runtime ausgeschlossen. Der Prozess selbst hat aufgegeben. Nächste Frage: Was hat er gesagt?

MAC — Schritt 12

```
kubectl --context rke2 logs -n dbg-01 deploy/app --previous
```

Das Log des vorherigen, abgestürzten Laufs.

```
FEHLER: DB_HOST ist nicht gesetzt
```

**URSACHE**

Die Anwendung prüft beim Start, ob `DB_HOST` gesetzt ist, und beendet sich sonst mit Exit-Code 1. Das Deployment setzt die Variable nicht. Weil ein Deployment `restartPolicy: Always` hat, startet das kubelet den Container immer wieder und verlängert die Pause zwischen den Versuchen bis auf fünf Minuten.

Die Lösung ist die fehlende Variable, nicht ein Neustart des Pods. Die Datei `fix.yaml` enthält dasselbe Deployment mit einem `env`-Block:

DATEI `szenarien/01-methode/fix.yaml (Ausschnitt)`

```
          env:
            - name: DB_HOST
              value: db.dbg-01.svc.cluster.local
```

MAC — Schritt 13

```
kubectl --context rke2 apply -f szenarien/01-methode/fix.yaml
kubectl --context rke2 rollout status deploy/app -n dbg-01
```

Die Änderung am Pod-Template löst einen Rollout aus. `rollout status` wartet, bis der neue Pod bereit ist.

**DAS SOLLTEST DU SEHEN**

- `rollout status` meldet `deployment "app" successfully rolled out.`
- `get pods -n dbg-01` zeigt einen neuen Pod: `1/1`, `Running`, `0` Neustarts.
- `logs -n dbg-01 deploy/app` gibt `verbinde mit db.dbg-01.svc.cluster.local` aus.

Auf Talos verläuft das Beispiel genauso. Wiederhole es mit `--context talos`, wenn du magst. Danach räumst du auf:

MAC Schritt 14

```
kubectl --context rke2 delete ns dbg-01
kubectl --context talos delete ns dbg-01 --ignore-not-found
```

Löscht den Namespace samt Deployment und Pods.

Die Untersuchung hat drei Befehle gebraucht. Jeder hat eine Frage beantwortet und eine Schicht ausgeschlossen. Genau so laufen alle Szenarien in diesem Buch, nur dass die Fragen schwerer werden.

**PROBIER ES SELBST**

Ändere in einer Kopie von `kaputt.yaml` den Befehl so, dass statt `exit 1` die Zeile `slep 3600` ausgeführt wird, und wende sie an. Welchen Exit-Code erwartest du, bevor du nachsiehst? Prüfe deine Vermutung mit `describe` und `logs --previous`. Hinweis: Die Shell meldet `sh: slep: not found` und endet mit 127. Lösche den Namespace danach wieder.

---

**WAS DU GELERNT HAST**

- Jeder Fehler durchläuft fünf Schritte: reproduzieren, untersuchen, verstehen, beheben, prüfen.
- Die Landkarte ordnet ein Symptom einer Schicht zu: Scheduling, Image und Runtime, Workload, Netzwerk, Storage, Policy, Knoten, Control Plane.
- Die Untersuchung geht von außen nach innen: `get`, `describe`, `events`, `logs`, `debug`, dann Knoten und Control Plane.
- `STATUS` ist eine Zusammenfassung. Pod-Phase, Container-State und Reason sind drei verschiedene Ebenen, und `lastState` erklärt den vorigen Lauf.
- Exit-Codes über 128 bedeuten ein Signal. 137 mit `OOMKilled` ist das Speicherlimit, 137 ohne ist ein hartes Beenden.
- `kubectl debug` braucht auf Talos `--profile=baseline`, weil das Standardprofil eine Capability hinzufügt, die Pod Security `baseline` verbietet.

---

KAPITEL 2

# Das Labor

*Aus vier leeren VMs werden zwei Cluster: RKE2 auf Ubuntu und Talos Linux. Dein Mac steuert sie über die Kontexte `rke2` und `talos`, beide haben eine StorageClass `local-path`, und k9s ist mit den Plugins dieses Buchs eingerichtet.*

**Dauer:** 120 Minuten **Voraussetzung:** Vier VMs (zwei mit Ubuntu 24.04 und SSH, zwei leer), ein Mac mit Homebrew, das geklonte Repo

## 2.1 Was du aufbaust

Jeder Fehler in diesem Buch läuft auf zwei Clustern. Beide nutzen dieselbe Kubernetes-Version, unterscheiden sich aber im Betriebsmodell: RKE2 ist ein Paket auf einem gewöhnlichen Linux, in das du dich per SSH einloggst. Talos ist ein eigenes Betriebssystem ohne Shell und ohne SSH, das du nur über eine API steuerst. Gerade diese Unterschiede machen das Debuggen interessant.

| Knoten | IP | Kontext | System | Rolle |
|---|---|---|---|---|
| `rke2-cp` | `172.16.0.160` | `rke2` | Ubuntu 24.04 | Server: Control Plane, etcd, nimmt auch Workloads |
| `rke2-w1` | `172.16.0.161` | `rke2` | Ubuntu 24.04 | Agent |
| `talos-cp` | `172.16.0.170` | `talos` | Talos v1.14.5 | Control Plane, Scheduling erlaubt |
| `talos-w1` | `172.16.0.171` | `talos` | Talos v1.14.5 | Worker |

Die Control-Plane-Knoten brauchen 4 CPUs, 8 GB RAM und 60 GB Platte, die beiden anderen 2 CPUs, 4 GB und 40 GB. Gib jeder VM eine feste IP oder eine DHCP-Reservierung. Die Adressen stehen in Konfigurationen und Zertifikaten und dürfen sich später nicht ändern. Hast du andere Adressen, ersetze sie in den Dateien unter `labor/` und in den Befehlen.

Beide Cluster laufen mit Kubernetes 1.36.6. Was sie mitbringen, unterscheidet sich deutlich:

| | RKE2 v1.36.6+rke2r1 | Talos v1.14.5 |
|---|---|---|
| CNI | Canal (Flannel plus Calico für Network-Policies) | Flannel, setzt **keine** NetworkPolicies durch |
| Ingress | Traefik ( `rke2-traefik` ) | keiner |
| metrics-server | ja | nein |

| | RKE2 v1.36.6+rke2r1 | Talos v1.14.5 |
|---|---|---|
| StorageClass | keine | keine |
| Pod Security | keine Vorgabe | cluster-weit `enforce: baseline`, `warn` und `audit` auf `restricted` |
| Zugang zum Knoten | SSH, `journalctl`, `crictl` | nur `talosctl` über Port 50000 |

Die fehlenden Teile ergänzen spätere Kapitel, wenn sie gebraucht werden. Nur die StorageClass legst du schon hier an.

**WARUM DAS WICHTIG IST**

Mach von allen vier VMs einen Snapshot, sobald dieses Kapitel durch ist. Du wirst die Cluster in den nächsten Kapiteln absichtlich beschädigen. Meistens behebst du den Fehler danach selbst, aber wenn ein Experiment schiefgeht, bist du mit dem Snapshot in einer Minute wieder am Ausgangspunkt.

## 2.2 Werkzeuge auf dem Mac

MAC — Schritt 1

```
brew install kubernetes-cli helm siderolabs/tap/talosctl derailed/k9s/k9s
```

Installiert `kubectl`, `helm`, `talosctl` und k9s. Homebrew installiert jeweils die neueste Version. Das Buch beschreibt kubectl 1.36, talosctl 1.14.5, helm 4 und k9s 0.51.0.

MAC — Schritt 2

```
kubectl version --client
talosctl version --client
k9s version
```

Zeigt die installierten Versionen.

**STOLPERFALLE**

`talosctl` sollte zur Talos-Version der Knoten passen. Ein deutlich neueres `talosctl` erzeugt mit `gen config` Konfigurationen für seine eigene Talos-Version und trägt das passende Installer-Image ein. Liegt deine Version nicht bei 1.14, lade das Binary für v1.14.5 von der Release-Seite des Projekts auf GitHub.

MAC — Schritt 3

```
git clone https://github.com/thomaszachmann/kubernetes-debuggen.git
cd kubernetes-debuggen
```

Alle Pfade im Buch sind relativ zu diesem Verzeichnis. Befehle mit dem Etikett **Mac** führst du hier aus.

## 2.3 RKE2: die Konfiguration

RKE2 liest beim Start die Datei `/etc/rancher/rke2/config.yaml`. Jeder Schlüssel entspricht einer Option des Befehls `rke2 server` oder `rke2 agent`. Die Dateien für beide Knoten liegen im Repo:

DATEI `labor/rke2/config-cp.yaml`

```yaml
node-name: rke2-cp
node-ip: 172.16.0.160
token: TOKEN-HIER-EINSETZEN
tls-san:
  - 172.16.0.160
  - rke2-cp
write-kubeconfig-mode: "0600"
cni: canal
ingress-controller: traefik
```

`token` ist das gemeinsame Geheimnis, mit dem sich der Agent beim Server ausweist. `tls-san` nimmt IP und Namen in das Zertifikat des API-Servers auf, sonst lehnt `kubectl` vom Mac die Verbindung mit einem `x509`-Fehler ab. `write-kubeconfig-mode` lässt die Admin-Kubeconfig nur für root lesbar. `cni` und `ingress-controller` entsprechen den Standardwerten von RKE2 1.36. Sie stehen trotzdem hier, damit die Datei zeigt, womit das Labor arbeitet.

DATEI `labor/rke2/config-w1.yaml`

```yaml
server: https://172.16.0.160:9345
node-name: rke2-w1
node-ip: 172.16.0.161
token: TOKEN-HIER-EINSETZEN
```

Der Agent meldet sich über den Supervisor-Port 9345 beim Server an, nicht über den API-Port 6443. Über diese Verbindung holt er sich Zertifikate und die Adresse des API-Servers.

Das Repo ist öffentlich, deshalb steht dort kein echtes Token. Du erzeugst es jetzt und setzt es beim Kopieren ein:

MAC — Schritt 4

```
RKE2_TOKEN=$(openssl rand -hex 32)
sed "s/TOKEN-HIER-EINSETZEN/$RKE2_TOKEN/" labor/rke2/config-cp.yaml \
  | ssh <user>@172.16.0.160 'umask 077; cat > ~/rke2-config.yaml'
sed "s/TOKEN-HIER-EINSETZEN/$RKE2_TOKEN/" labor/rke2/config-w1.yaml \
  | ssh <user>@172.16.0.161 'umask 077; cat > ~/rke2-config.yaml'
```

Erzeugt ein zufälliges Token und legt jedem Knoten seine Konfiguration ins Home-Verzeichnis, nur für deinen Benutzer lesbar. Ersetze `<user>` durch deinen SSH-Benutzer. Das Token brauchst du später nicht mehr. Der Server legt es zusätzlich unter `/var/lib/rancher/rke2/server/node-token` ab.

## 2.4 RKE2: der Server

MAC Schritt 5

```
ssh <user>@172.16.0.160
```

Öffnet die Sitzung auf dem Server. Alle Schritte mit dem Etikett **rke2-cp** laufen hier.

RKE2-CP Schritt 6

```
sudo hostnamectl set-hostname rke2-cp
sudo install -D -m 600 -o root -g root \
  ~/rke2-config.yaml /etc/rancher/rke2/config.yaml
rm ~/rke2-config.yaml
```

Setzt den Hostnamen passend zum `node-name` und legt die Konfiguration an ihren Platz. Sie muss dort liegen, **bevor** RKE2 zum ersten Mal startet. `-D` legt das Verzeichnis an.

RKE2-CP Schritt 7

```
curl -sfL https://get.rke2.io | \
  sudo INSTALL_RKE2_VERSION=v1.36.6+rke2r1 sh -
sudo systemctl enable --now rke2-server.service
```

Das Skript installiert genau die Version dieses Buchs. Ohne `INSTALL_RKE2_VERSION` bekämst du die neueste Version aus dem Kanal `stable`. Der erste Start dauert einige Minuten, weil RKE2 die Images der Control Plane lädt.

RKE2-CP Schritt 8

```
sudo journalctl -u rke2-server -f
```

Verfolgt das Log des Dienstes. Beenden mit `Ctrl-C`. Fehler in der `config.yaml` stehen hier ganz am Anfang.

Für die Arbeit direkt auf dem Knoten brauchst du in späteren Kapiteln immer dieselbe Umgebung. RKE2 legt `kubectl`, `crictl` und `ctr` in ein eigenes Verzeichnis und schreibt seine Kubeconfig an einen eigenen Ort:

RKE2-CP Schritt 9

```
sudo -i
export KUBECONFIG=/etc/rancher/rke2/rke2.yaml
export PATH=$PATH:/var/lib/rancher/rke2/bin
export CRI_CONFIG_FILE=/var/lib/rancher/rke2/agent/etc/crictl.yaml
kubectl get pods -n kube-system
```

Wechselt zu root und setzt die drei Variablen. Ohne `CRI_CONFIG_FILE` findet `crictl` den Socket von containerd nicht. Wenn spätere Kapitel auf einem RKE2-Knoten arbeiten, setzen sie genau diese vier Zeilen voraus.

**DAS SOLLTEST DU SEHEN**

In `kube-system` laufen `etcd-rke2-cp`, `kube-apiserver-rke2-cp` und die anderen Static Pods der Control Plane, dazu `rke2-canal`, `rke2-coredns`, `rke2-metrics-server` und `rke2-traefik`. Mehrere `helm-install-rke2-…`-Pods stehen auf `Completed`. Das ist richtig: Sie haben die mitgelieferten Charts installiert und sind fertig.

Die Admin-Kubeconfig gehört root. Für den Mac legst du eine Kopie an, die dein Benutzer holen darf:

RKE2-CP — Schritt 10

```
install -m 600 -o <user> /etc/rancher/rke2/rke2.yaml /tmp/rke2.yaml
exit
exit
```

Läuft noch als root aus der `sudo -i`-Sitzung. Das erste `exit` beendet die root-Shell, das zweite die SSH-Sitzung.

## 2.5 RKE2: der Agent

MAC — Schritt 11

```
ssh <user>@172.16.0.161
```

Öffnet die Sitzung auf dem Agent.

RKE2-W1 — Schritt 12

```
sudo hostnamectl set-hostname rke2-w1
sudo install -D -m 600 -o root -g root \
  ~/rke2-config.yaml /etc/rancher/rke2/config.yaml
rm ~/rke2-config.yaml
curl -sfL https://get.rke2.io | \
  sudo INSTALL_RKE2_VERSION=v1.36.6+rke2r1 INSTALL_RKE2_TYPE=agent sh -
sudo systemctl enable --now rke2-agent.service
```

Wie beim Server, aber mit `INSTALL_RKE2_TYPE=agent`. Der Dienst heißt jetzt `rke2-agent`, sein Log liest du mit `sudo journalctl -u rke2-agent -f`.

**STOLPERFALLE**

Bleibt der Agent hängen und das Log meldet wiederholt Verbindungsfehler zu `172.16.0.160:9345`, blockiert meist eine Firewall. Ubuntu Server hat `ufw` standardmäßig inaktiv, `sudo ufw status` zeigt es. Ist sie aktiv, braucht RKE2 mindestens TCP 6443 und 9345 zum Server, TCP 10250 zwischen allen Knoten und für Canal UDP 8472 (VXLAN) und TCP 9099. Lass Port 22 offen, sonst sperrst du dich aus.

## 2.6 Die Kubeconfig von RKE2

Die Datei vom Server zeigt auf `127.0.0.1` und nennt Cluster, Benutzer und Kontext `default`. Beides änderst du, bevor sie in deine `~/.kube/config` wandert.

MAC — Schritt 13

```
mkdir -p ~/.kube
[ -f ~/.kube/config ] && cp -p ~/.kube/config ~/.kube/config.bak
scp <user>@172.16.0.160:/tmp/rke2.yaml ~/.kube/rke2.raw
ssh <user>@172.16.0.160 'rm /tmp/rke2.yaml'
```

Sichert eine vorhandene Kubeconfig, holt die Datei vom Server und löscht die Kopie dort. Sie enthält volle Admin-Rechte.

MAC — Schritt 14

```
sed -e 's/127\.0\.0\.1/172.16.0.160/' -e 's/: default$/: rke2/' \
  ~/.kube/rke2.raw > ~/.kube/rke2.yaml
KUBECONFIG=~/.kube/config:~/.kube/rke2.yaml \
  kubectl config view --flatten > ~/.kube/config.neu
chmod 600 ~/.kube/config.neu
mv ~/.kube/config.neu ~/.kube/config
rm ~/.kube/rke2.raw ~/.kube/rke2.yaml
```

`sed` setzt die IP ein und benennt alles in `rke2` um. Mehrere Dateien in `KUBECONFIG` führt `kubectl` zusammen, `--flatten` schreibt die Zertifikate direkt in die Ausgabe. Gibt es schon einen Kontext `rke2`, entferne ihn vorher mit `kubectl config delete-context rke2`, sonst gewinnt der alte Eintrag.

MAC — Schritt 15

```
kubectl --context rke2 get nodes
```

Erster Zugriff vom Mac.

**DAS SOLLTEST DU SEHEN**

Zwei Knoten, `rke2-cp` und `rke2-w1`, beide `Ready`, Version `v1.36.6+rke2r1`. Fehlt `rke2-w1`, prüfe das Log des Agents.

## 2.7 Talos: booten

Talos installierst du nicht per Paket, sondern bootest es von einem ISO. Hol dir das ISO aus der Image Factory unter `factory.talos.dev`: Version v1.14.5, Plattform Bare Metal, Architektur amd64. Erweiterungen brauchst du für das Labor keine. Läuft dein Hypervisor mit einem Gast-Agenten, etwa Proxmox mit `qemu-guest-agent`, kannst du die passende Erweiterung hinzufügen.

Boote beide Talos-VMs von diesem ISO. Auf der Konsole erscheint das Talos-Dashboard mit der IP-Adresse. Der Knoten ist jetzt im **Maintenance-Modus**: Er läuft im RAM, hat noch keine Konfi-

guration und wartet auf eine. Die Maintenance-API nimmt in diesem Zustand Konfigurationen ohne Anmeldung an. Das ist so gewollt, deshalb gehört das Labor in ein Netz, das nur du erreichst.

MAC Schritt 16

```
talosctl get disks --insecure --nodes 172.16.0.170
talosctl get disks --insecure --nodes 172.16.0.171
```

Listet die Platten der Knoten. `--insecure` spricht die Maintenance-API an. Merk dir den Namen der Systemplatte, etwa `sda` oder `vda`.

Entferne danach das ISO aus den virtuellen Laufwerken. Der Knoten läuft weiter, weil Talos komplett im RAM ist. So kann Talos nach der Installation nicht versehentlich wieder vom ISO starten.

## 2.8 Talos: die Patches

Eine Talos-Konfiguration ist ein langes YAML-Dokument, das `talosctl gen config` erzeugt. Du änderst es nicht von Hand, sondern beschreibst deine Abweichungen in kleinen Patches. Drei liegen im Repo:

DATEI `labor/talos/patch-cp.yaml`

```yaml
machine:
  install:
    disk: /dev/sda
cluster:
  allowSchedulingOnControlPlanes: true
---
apiVersion: v1alpha1
kind: HostnameConfig
hostname: talos-cp
auto: off
```

`machine.install.disk` ist die Platte, auf die Talos sich installiert. Hieß sie bei `get disks` anders, passe sie hier und in `patch-w1.yaml` an. `allowSchedulingOnControlPlanes` entfernt den Taint, mit dem Talos normale Pods von der Control Plane fernhält. Das zweite Dokument setzt den Hostnamen. `auto: off` schaltet den automatisch erzeugten Namen wie `talos-2ld-dvw` ab.

DATEI `labor/talos/patch-w1.yaml`

```yaml
machine:
  install:
    disk: /dev/sda
---
apiVersion: v1alpha1
kind: HostnameConfig
hostname: talos-w1
auto: off
```

DATEI `labor/talos/patch-local-path.yaml`

```
apiVersion: v1alpha1
kind: UserVolumeConfig
name: local-path-provisioner
volumeType: directory
```

Ein **User Volume** vom Typ `directory` legt das Verzeichnis `/var/mnt/local-path-provisioner` auf der Partition `EPHEMERAL` an und reicht es in den Container des kubelet durch. Dort legt der local-path-provisioner später die Volumes ab. Ein eigenes Verzeichnis ist nötig, weil das Standardverzeichnis `/opt/local-path-provisioner` auf Talos schreibgeschützt ist und das kubelet nicht jeden Pfad unter `/var` sieht.

## 2.9 Talos: Konfiguration erzeugen

MAC — Schritt 17

```
talosctl gen secrets -o labor/talos/secrets.yaml
```

Erzeugt die Zertifizierungsstellen und Schlüssel des Clusters. Mit dieser Datei kannst du die Konfiguration jederzeit neu erzeugen, ohne dass sich die Zertifikate ändern. Sie steht wie alle erzeugten Dateien in `.gitignore`.

MAC — Schritt 18

```
talosctl gen config talos https://172.16.0.170:6443 \
  --with-secrets labor/talos/secrets.yaml \
  --kubernetes-version 1.36.6 \
  --config-patch @labor/talos/patch-local-path.yaml \
  --config-patch-control-plane @labor/talos/patch-cp.yaml \
  --config-patch-worker @labor/talos/patch-w1.yaml \
  --output labor/talos/
```

`talos` ist der Name des Clusters, die URL der Endpunkt des API-Servers. `--config-patch` gilt für alle Knoten, die beiden anderen nur für Control Plane bzw. Worker. Ohne `--kubernetes-version` nimmt `talosctl` seine eigene Vorgabe, und die passt nicht unbedingt zum RKE2-Cluster.

MAC — Schritt 19

```
talosctl validate --config labor/talos/controlplane.yaml --mode metal
talosctl validate --config labor/talos/worker.yaml --mode metal
```

Prüft beide Dateien, bevor sie auf einen Knoten gehen.

**DAS SOLLTEST DU SEHEN**

Im Verzeichnis `labor/talos/` liegen jetzt `controlplane.yaml`, `worker.yaml` und `talosconfig`. `validate` meldet für beide Dateien `is valid for metal mode`.

## 2.10 Talos: anwenden und bootstrappen

MAC Schritt 20

```
talosctl apply-config --insecure --nodes 172.16.0.170 \
  --file labor/talos/controlplane.yaml
talosctl apply-config --insecure --nodes 172.16.0.171 \
  --file labor/talos/worker.yaml
```

Schickt jedem Knoten seine Konfiguration. Talos installiert sich auf die Platte und startet neu. Ab jetzt ist der Knoten nur noch mit den Zertifikaten aus `talosconfig` erreichbar, `--insecure` funktioniert nicht mehr.

MAC Schritt 21

```
export TALOSCONFIG=$PWD/labor/talos/talosconfig
talosctl config endpoint 172.16.0.170
echo "export TALOSCONFIG=$PWD/labor/talos/talosconfig" >> ~/.zshrc
```

`TALOSCONFIG` sagt `talosctl`, wo Zertifikate und Kontext liegen. Der Endpunkt ist der Knoten, über den `talosctl` alle Anfragen schickt. Die letzte Zeile trägt die Variable dauerhaft in deine Shell ein. Alle späteren Kapitel setzen sie voraus.

MAC Schritt 22

```
talosctl --nodes 172.16.0.170 bootstrap
```

Startet etcd auf der Control Plane. Der Befehl läuft genau **einmal** und nur auf einem Knoten. Wartet der Knoten noch auf den Neustart, wiederhole ihn nach einer Minute.

MAC Schritt 23

```
talosctl --nodes 172.16.0.170 health
```

Prüft etcd, kubelet, die Control Plane und ob alle Knoten im Cluster angekommen sind. Der Befehl wartet bis zu 20 Minuten und meldet jeden Schritt.

**STOLPERFALLE**

Wer `bootstrap` ein zweites Mal auf einem anderen Control-Plane-Knoten ausführt, erzeugt einen zweiten, unabhängigen etcd-Cluster. Im Labor gibt es nur einen Control-Plane-Knoten, die Gefahr ist also gering. In echten Clustern mit drei Knoten ist das ein häufiger Fehler.

MAC Schritt 24

```
talosctl --nodes 172.16.0.170 kubeconfig --force-context-name talos
kubectl --context talos get nodes
```

Lädt die Admin-Kubeconfig und fügt sie in `~/.kube/config` ein. Ohne `--force-context-name` hieße der Kontext `admin@talos`.

**DAS SOLLTEST DU SEHEN**

Zwei Knoten, `talos-cp` und `talos-w1`, beide `Ready`, Version `v1.36.6`. `kubectl config get-contexts` zeigt `rke2` und `talos` nebeneinander.

**RKE2**

Du verwaltest den Knoten wie jedes Linux. Konfiguration ist eine Datei unter `/etc/rancher/rke2/`, Änderungen wirken nach `systemctl restart rke2-server`. Logs liest du mit `journalctl`.

**TALOS**

Der Knoten hat keine Shell. Die gesamte Konfiguration ist die Machine-Config, die du mit `talosctl patch machineconfig` änderst. Logs liest du mit `talosctl logs kubelet` oder `talosctl dmesg`, immer mit `--nodes`.

## 2.11 Storage: local-path-provisioner

Beide Cluster haben noch keine StorageClass. Ein PVC bliebe für immer `Pending`. Der local-path-provisioner von Rancher legt für jedes PVC ein Verzeichnis auf dem Knoten an, auf dem der Pod läuft. Für ein Labor ist das genau richtig: kein Netzwerkspeicher, keine Zusatzplatten.

Die StorageClass heißt auf beiden Clustern `local-path`. Sie wird bewusst **nicht** zur Default-StorageClass. Ein PVC muss sie ausdrücklich mit `storageClassName: local-path` anfordern. Ein PVC ohne diese Angabe bleibt `Pending`, und genau diesen Fehler untersuchen die Kapitel 4 und 9.

MAC — Schritt 25

```
kubectl --context rke2 apply -f \
  https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.37/deploy/local-path-storage.
yaml
```

Installiert Version v0.0.37 im Namespace `local-path-storage`. Auf Ubuntu ist das Standardverzeichnis `/opt/local-path-provisioner` beschreibbar, eine Anpassung ist nicht nötig.

Für Talos braucht es zwei Änderungen am selben Manifest. Eine Kustomization im Repo setzt sie:

DATEI `werkzeuge/local-path-talos/kustomization.yaml (Ausschnitt)`

```
resources:
  - https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.37/deploy/local-path-storage.
yaml
patches:
  - patch: |-
      kind: ConfigMap
      # ... local-path-config: "paths": ["/var/mnt/local-path-provisioner"]
  - patch: |-
      apiVersion: v1
      kind: Namespace
      metadata:
        name: local-path-storage
```

```
        labels:
          pod-security.kubernetes.io/enforce: privileged
```

Der erste Patch lenkt die Volumes in das User Volume aus `patch-local-path.yaml`. Der zweite erlaubt im Namespace privilegierte Pods. Der Provisioner startet für jedes Volume einen kurzlebigen Helper-Pod mit `hostPath`, und den würde Pod Security `baseline` auf Talos ablehnen.

**MAC** Schritt 26

```
kubectl --context talos apply -k werkzeuge/local-path-talos/
```

`-k` baut die Kustomization und wendet das Ergebnis an. Das Manifest lädt `kubectl` dabei direkt von GitHub.

**MAC** Schritt 27

```
kubectl --context rke2 get storageclass
kubectl --context talos get storageclass
```

Zeigt die StorageClasses beider Cluster.

**DAS SOLLTEST DU SEHEN**

Auf jedem Cluster genau eine StorageClass:

```
NAME         PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE
local-path   rancher.io/local-path   Delete          WaitForFirstConsumer
```

Die Ausgabe hat noch die Spalten `ALLOWVOLUMEEXPANSION` und `AGE`. Wichtig ist der Name: Dahinter steht **kein** `(default)`.

Ob wirklich ein Volume entsteht, zeigt ein kleiner Test: ein PVC und ein Pod, der eine Datei hineinschreibt.

**MAC** Schritt 28

```
for c in rke2 talos; do
  kubectl --context $c apply -f labor/pvc-test.yaml
  kubectl --context $c wait pod/schreiber -n dbg-02 \
    --for=condition=Ready --timeout=120s
  kubectl --context $c get pvc,pod -n dbg-02 -o wide
done
```

Legt Namespace `dbg-02`, PVC `daten` und Pod `schreiber` auf beiden Clustern an und wartet, bis der Pod läuft. Wegen `WaitForFirstConsumer` entsteht das Volume erst, wenn der Pod einem Knoten zugeteilt ist.

**DAS SOLLTEST DU SEHEN**

Das PVC steht auf `Bound`, der Pod auf `Running`, und `logs -n dbg-02 schreiber` zeigt Datum und Uhrzeit. Auf Talos siehst du das Volume auch auf dem Knoten. Nimm die IP des Knotens aus der Spalte `NODE`:

```
talosctl --nodes 172.16.0.171 ls /var/mnt/local-path-provisioner
```

MAC — Schritt 29

```
kubectl --context rke2 delete ns dbg-02
kubectl --context talos delete ns dbg-02
```

Räumt auf. Wegen `reclaimPolicy: Delete` löscht der Provisioner auch die Verzeichnisse auf den Knoten.

## 2.12 k9s einrichten

k9s hast du schon installiert. Es fehlen die Plugins dieses Buchs. k9s sucht sie in seinem Konfigurationsverzeichnis, und das liegt auf macOS nicht unter `~/.config`:

MAC — Schritt 30

```
k9s info
```

Zeigt, wo k9s Konfiguration, Plugins, Skins und Logs erwartet.

**DAS SOLLTEST DU SEHEN**

Die Zeile `Plugins` nennt `~/Library/Application Support/k9s/plugins.yaml`, ausgeschrieben mit deinem Home-Verzeichnis. Unter Linux steht dort `~/.config/k9s/plugins.yaml`.

MAC — Schritt 31

```
mkdir -p "$HOME/Library/Application Support/k9s"
cp k9s/plugins.yaml "$HOME/Library/Application Support/k9s/plugins.yaml"
```

Kopiert die Plugins an ihren Platz. Hast du dort schon eine `plugins.yaml`, führe beide Dateien von Hand zusammen, statt sie zu überschreiben. Was die Plugins tun, zeigt Kapitel 3.

## 2.13 Abschluss: beide Cluster im Überblick

MAC — Schritt 32

```
kubectl --context rke2 get nodes -o wide
kubectl --context talos get nodes -o wide
```

Der Abschluss des Kapitels. Diese beiden Befehle stehen auch am Anfang jeder Fehlersuche, wenn du nicht weißt, in welchem Zustand das Labor gerade ist.

**DAS SOLLTEST DU SEHEN**

Vier Knoten, alle `Ready`. Bei RKE2 zeigt `VERSION` `v1.36.6+rke2r1` und `OS-IMAGE` Ubuntu 24.04, bei Talos `v1.36.6` und `Talos (v1.14.5)`. Die `INTERNAL-IP` stimmt mit der Tabelle am Anfang des Kapitels überein. Jetzt ist der richtige Moment für die Snapshots.

**PROBIER ES SELBST**

Vergleiche, was in `kube-system` der beiden Cluster läuft: `kubectl --context rke2 get pods -n kube-system` und dasselbe für `talos`. Welcher Pod ist auf welchem Cluster für DNS zuständig, und wie heißt er? Welcher für das Pod-Netz? Hinweis: Auf RKE2 tragen die mitgelieferten Komponenten das Präfix `rke2-`, auf Talos heißen sie schlicht `coredns` und `kube-flannel`. Kapitel 7 braucht genau diese Namen.

---

**WAS DU GELERNT HAST**

- RKE2 liest `/etc/rancher/rke2/config.yaml`. Server und Agent teilen sich ein Token, der Agent meldet sich über Port 9345 an.
- Talos wird über eine Machine-Config gesteuert, die `talosctl gen config` aus Secrets und kleinen Patches erzeugt. `bootstrap` läuft genau einmal.
- Die Kontexte heißen `rke2` und `talos`, `TALOSCONFIG` zeigt auf `labor/talos/talosconfig`.
- Beide Cluster haben die StorageClass `local-path`, aber keine Default-StorageClass. Auf Talos liegen die Volumes unter `/var/mnt/local-path-provisioner`, und der Namespace des Provisioners braucht Pod Security `privileged`.
- k9s erwartet seine Plugins auf macOS unter `~/Library/Application Support/k9s/`. `k9s info` zeigt den genauen Pfad.

---

KAPITEL 3

# k9s als Cockpit

*Du bewegst dich in k9s ohne Maus durch beide Cluster, filterst, liest Logs auch vom vorherigen Lauf, öffnest Shells und Port-Forwards und startest mit einem Tastendruck einen Debug-Container. Du weißt auch, wann `kubectl` die bessere Wahl ist.*

**Dauer:** 45 Minuten **Voraussetzung:** Kapitel 2

## 3.1 Warum ein zweites Werkzeug

`kubectl` zeigt dir einen Zustand zu einem Zeitpunkt. Für die Fehlersuche ist das oft zu wenig. Du willst sehen, wie ein Pod neu startet, wie ein Rollout voranschreitet, und dabei schnell zwischen Pod, Logs, Events und Deployment springen. Genau dafür ist k9s gebaut: eine Terminal-Oberfläche, die alle zwei Sekunden aktualisiert und jede Ansicht mit wenigen Tasten erreichbar macht.

k9s spricht dieselbe API wie `kubectl` und nutzt dieselbe Kubeconfig. Es sieht also nichts, was `kubectl` nicht auch sehen könnte, und darf nichts, was dein Benutzer nicht darf. Der Vorteil ist allein die Geschwindigkeit. Dieses Buch zeigt deshalb in jedem Szenario beide Wege: `kubectl` für Befehle, die du dokumentieren und wiederholen willst, k9s für die schnelle Runde durch den Cluster.

Damit du etwas zum Anschauen hast, legst du zuerst ein paar Übungsobjekte an:

DATEI `szenarien/03-k9s/demo.yaml (Ausschnitt)`

```yaml
kind: Deployment
metadata:
  name: web
  namespace: dbg-03
spec:
  replicas: 2
  # ... busybox httpd auf Port 8080, dazu Service web (80 -> 8080)
---
kind: Deployment
metadata:
  name: wackel
  namespace: dbg-03
# ...
          command:
            - sh
            - -c
            - |
```

```
            echo "start"
            sleep 15
            echo "FEHLER: Cache voll, gebe auf" >&2
            exit 3
```

`web` ist ein kleiner Webserver mit zwei Replikas und einem Service. `wackel` stürzt nach 15 Sekunden mit Exit-Code 3 ab und landet nach ein paar Runden in `CrashLoopBackOff`. Beide laufen ohne Root, damit Talos keine Warnungen ausgibt.

MAC Schritt 1

```
kubectl --context rke2 apply -f szenarien/03-k9s/demo.yaml
```

Legt Namespace `dbg-03`, beide Deployments und den Service an.

## 3.2 Starten und Orientieren

MAC Schritt 2

```
k9s --context rke2 -n dbg-03
```

Startet k9s im Kontext `rke2` und direkt im Namespace `dbg-03`. Ohne `-n` öffnet k9s den zuletzt benutzten Namespace dieses Kontexts. `-A` startet in allen Namespaces, `-c deploy` mit einer anderen Ansicht als Pods.

Der Bildschirm hat drei Bereiche. Oben links stehen Kontext, Cluster, Benutzer und die Versionen von k9s und Kubernetes, daneben die Tasten, die in der aktuellen Ansicht gelten. Die Mitte zeigt die Tabelle, darüber den Titel mit Ressource, Namespace und Anzahl der Objekte. Unten zeigen die **Crumbs**, wie du hierher gekommen bist.

**STOLPERFALLE**

Schau vor jeder ändernden Aktion auf den Kontext oben links. Wer mit k9s in zwei Terminals arbeitet, eines pro Cluster, löscht sonst leicht im falschen. Eine Hilfe: k9s kann pro Kontext einen eigenen Skin laden, etwa rot für das Labor, das du gerade kaputt machst.

Die wichtigsten Tasten sind wenige. `?` zeigt alle Tasten der aktuellen Ansicht. Das ist die zuverlässigste Quelle, denn die Belegung hängt von Ressource und Version ab.

| Taste | Wirkung |
|---|---|
| `:` | Befehlszeile: Ressource öffnen, etwa `:deploy⏎`, `:svc⏎`, `:events⏎` |
| `/` | Filter auf die aktuelle Tabelle |
| `Esc` | Filter oder Befehlszeile abbrechen, eine Ansicht zurück |
| `Enter` | Objekt öffnen: Pod zeigt Container, Deployment zeigt Pods |

| Taste | Wirkung |
|---|---|
| `?` | Hilfe zur aktuellen Ansicht |
| `Ctrl-a` | Alle Ressourcen und ihre Aliase |
| `-` | Zurück zum vorigen Befehl, wie `cd -` |
| `:q⏎` oder `Ctrl-c` | k9s beenden |

## 3.3 Ressourcen, Namespaces und Filter

Hinter `:` akzeptiert k9s jeden Namen, den auch `kubectl` versteht: Singular, Plural und Kurzname. `:po`, `:pod` und `:pods` öffnen dieselbe Ansicht. Auch CRDs funktionieren, sobald sie im Cluster sind, etwa `:policyreports` in Kapitel 11.

K9S Schritt 3

```
:deploy dbg-03⏎
:pods kube-system⏎
:pods app=web⏎
:pods /wackel⏎
```

Ressource mit Namespace öffnen, Pods mit einem Label-Selektor anzeigen und die Liste gleich beim Öffnen filtern.

Den Namespace wechselst du auf drei Wegen. `0` zeigt alle Namespaces. Die Ziffern `1` bis `9` springen zu den zuletzt benutzten Namespaces, k9s blendet sie oben in der Tastenliste ein. Und `:ns⏎` listet alle Namespaces, `u` macht den markierten zum aktiven.

Der Filter mit `/` ist das Werkzeug, das du am häufigsten brauchst:

| Eingabe | Wirkung |
|---|---|
| `/web⏎` | Nur Zeilen, die `web` enthalten. Reguläre Ausdrücke sind erlaubt, etwa `web\|wackel`. |
| `/!Running⏎` | Umgekehrt: alle Zeilen, die **nicht** `Running` enthalten |
| `/-l app=web⏎` | Filter über Labels |
| `/-f wkl⏎` | Unscharfe Suche |
| `Ctrl-z` | Nur Objekte mit Fehlern zeigen, ein zweites `Ctrl-z` schaltet zurück |

K9S Schritt 4

```
:pods⏎
0
/!Running|Completed⏎
```

Alle Pods in allen Namespaces, die weder laufen noch fertig sind. Das ist eine gute erste Runde, wenn du nicht weißt, was in einem Cluster kaputt ist.

Sortiert wird mit Großbuchstaben: `Shift-s` nach Status, `Shift-a` nach Alter, `Shift-n` nach Name. `Ctrl-w` blendet zusätzliche Spalten ein, bei Pods etwa IP und Knoten. Das entspricht `kubectl get -o wide`.

## 3.4 Ein Pod im Detail

Markiere in der Pod-Ansicht einen Pod von `wackel`. Alle folgenden Tasten wirken auf die markierte Zeile:

| Taste | Wirkung | kubectl-Gegenstück |
|---|---|---|
| `d` | Describe | `describe pod` |
| `y` | YAML des Objekts samt `status` | `get -o yaml` |
| `e` | Im Editor ändern | `edit` |
| `l` | Logs | `logs -f` |
| `p` | Logs des vorherigen Laufs | `logs --previous` |
| `s` | Shell im Container | `exec -it ... -- sh` |
| `a` | An den Container anhängen | `attach` |
| `o` | Zum Knoten des Pods springen | `get node` |
| `Shift-j` | Zum Besitzer springen, etwa zum ReplicaSet | — |
| `Shift-f` | Port-Forward einrichten | `port-forward` |
| `Ctrl-d` | Löschen, mit Bestätigung | `delete` |
| `Ctrl-k` | Sofort löschen, **ohne** Bestätigung | `delete --now` |

K9S — Schritt 5

```
:pods dbg-03⏎
/wackel⏎
d
```

Describe des abstürzenden Pods. Mit den Pfeiltasten scrollst du, `Esc` führt zurück zur Liste.

**DAS SOLLTEST DU SEHEN**

Unter `Last State` steht `Terminated`, `Reason: Error`, `Exit Code: 3`. Ganz unten wiederholt sich das Event `BackOff` mit `Back-off restarting failed container wackel`.

`Enter` auf einem Pod öffnet die Container-Ansicht. Dort gelten `l`, `p` und `s` für genau einen Container. Das brauchst du bei Pods mit Sidecars oder Init-Containern. In der Pod-Ansicht wählt k9s sonst den ersten Container oder fragt nach.

**STOLPERFALLE**

`s` startet `bash`, wenn das Image eines hat, sonst `sh`. Hat das Image gar keine Shell, wie bei distroless-Images, scheitert das mit einer Fehlermeldung von `exec`. Dann hilft nur ein Debug-Container. Dafür gibt es weiter unten das Plugin auf `Shift-d`.

## 3.5 Logs lesen

K9S Schritt 6

```
:pods dbg-03⏎
/wackel⏎
p
```

Zeigt das Log des vorherigen Laufs. Dort steht die letzte Zeile vor dem Absturz:

`FEHLER: Cache voll, gebe auf.`

In der Log-Ansicht gelten eigene Tasten. Sie unterscheiden sich von der Tabelle, zum Beispiel heißt `w` hier Zeilenumbruch und nicht Namespace-Wechsel:

| Taste | Wirkung |
|---|---|
| `0` | Ab jetzt mitlaufen (tail) |
| `1` | Von Anfang an (head) |
| `2` bis `6` | Zeitfenster: 1, 5, 15, 30 Minuten, 1 Stunde |
| `s` | Autoscroll an oder aus |
| `w` | Lange Zeilen umbrechen |
| `t` | Zeitstempel einblenden |
| `f` | Vollbild |
| `/` | Filter auf die Logzeilen |
| `Ctrl-s` | Log in eine Datei speichern |

Zeitstempel sind beim Debuggen fast immer hilfreich. Sie zeigen, ob ein Fehler gerade erst auftrat oder schon seit Stunden. Wenn du sie immer sehen willst, setzt du in der `config.yaml` von k9s unter `k9s.logger` den Wert `showTime: true`. Dort legst du auch mit `tail` fest, wie viele Zeilen k9s beim Öffnen lädt, und mit `textWrap` den Umbruch.

## 3.6 Events, XRay und Pulses

K9S Schritt 7

```
:events dbg-03⏎
```

Events des Namespace als Tabelle, die sich laufend aktualisiert. `Ctrl-z` zeigt nur Warnungen und Fehler.

Zwei Ansichten gibt es nur in k9s. **XRay** zeigt Objekte als Baum: Deployment, darunter ReplicaSet, darunter Pods mit ihren Containern, ConfigMaps, Secrets und dem ServiceAccount. Ein rotes Element im Baum führt dich direkt zur Ursache. Das hilft besonders, wenn ein Deployment angenommen wurde, aber keine Pods entstehen.

K9S Schritt 8

```
:xray deploy dbg-03⏎
```

Baum aller Deployments in `dbg-03`. Erlaubt sind unter anderem `po`, `svc`, `dp`, `rs`, `sts` und `ds`.

**Pulses** (`:pulses⏎`) zeigt kleine Verlaufsgrafiken für Knoten, Namespaces, Services, Events, Pods, Deployments, StatefulSets, DaemonSets, Jobs, PVCs und weitere Objekte, jeweils mit der Zahl der gesunden und fehlerhaften, dazu CPU und Speicher, sofern ein metrics-server läuft. Das ist kein Werkzeug für eine einzelne Ursache, aber ein guter Blick darauf, ob ein Rollout gerade Wellen schlägt.

Ältere Anleitungen erwähnen `:popeye` für eine Prüfung des ganzen Clusters. In k9s 0.51 ist diese Ansicht nicht mehr enthalten. Wer Popeye nutzen will, startet es als eigenes Programm.

## 3.7 Port-Forward

K9S Schritt 9

```
:svc dbg-03⏎
/web⏎
Shift-f
```

Öffnet den Dialog für einen Port-Forward auf den Service `web`. Trage unter `Local Port` den Wert `8080` ein und bestätige mit `OK`.

MAC Schritt 10

```
curl -s localhost:8080
```

Ein zweites Terminal fragt den Webserver über den Port-Forward ab.

**DAS SOLLTEST DU SEHEN**

Die Antwort lautet `hallo aus web-…` mit dem Namen eines der beiden Pods. Die Taste `f` in der Service- oder Pod-Ansicht zeigt alle aktiven Port-Forwards, ebenso `:pf⏎`. Dort beendest du einen mit `Ctrl-d`. Port-Forwards leben nur so lange wie k9s.

## 3.8 Kontexte und Read-only

`:ctx⏎` listet alle Kontexte aus deiner Kubeconfig, `Enter` wechselt. Schneller geht `:ctx talos⏎`. k9s merkt sich für jeden Kontext Namespace und Ansicht getrennt.

Für dieses Buch empfehle ich trotzdem zwei Terminals mit je einem k9s: `k9s --context rke2` und `k9s --context talos`. Dann siehst du beide Cluster nebeneinander und musst nie überlegen, wo du gerade bist.

Wer in einem fremden oder produktiven Cluster nur schauen will, startet k9s schreibgeschützt:

MAC — Schritt 11

```
k9s --context talos --readonly
```

Schaltet alle ändernden Aktionen ab: Edit, Delete, Kill, Shell, Attach und alle Plugins, die als `dangerous` markiert sind. Dauerhaft geht das mit `readOnly: true` in der `config.yaml`, und `--write` hebt es für einen Start wieder auf.

**WARUM DAS WICHTIG IST**

Read-only schützt nicht vor Neugier, sondern vor Tippfehlern. `Ctrl-k` löscht ohne Rückfrage. In einem Cluster, in dem du nur Fehler suchst, gibt es keinen Grund, dieses Risiko zu tragen.

## 3.9 Die Plugins dieses Buchs

Plugins hängen eigene Befehle an eine Taste. k9s ersetzt dabei Variablen wie `$NAME`, `$NAMESPACE`, `$CONTEXT` und in der Container-Ansicht `$POD` durch die Werte der markierten Zeile. Die Datei `k9s/plugins.yaml` hast du in Kapitel 2 nach `~/Library/Application Support/k9s/` kopiert. Sie definiert drei Plugins:

| Taste | Ansicht | Wirkung |
|---|---|---|
| `Shift-d` | Pods | Plugin `netshoot`: startet `nicolaka/netshoot:v0.15` als Eph- |

| Taste | Ansicht | Wirkung |
|---|---|---|
| | | emeral Container im markierten Pod. Teilt das Netzwerk des Pods. |
| `Shift-d` | Container | Plugin `netshoot-target`: wie oben, zusätzlich mit `--target` auf den markierten Container. Sieht dessen Prozesse. |
| `Shift-e` | Pods, Deployments, ReplicaSets, StatefulSets, DaemonSets, Jobs, Services, PVCs | Plugin `events`: `kubectl events --for` genau dieses Objekts, chronologisch, in `less`. Beenden mit `q`. |

DATEI `k9s/plugins.yaml (Ausschnitt)`

```yaml
plugins:
  netshoot:
    shortCut: Shift-D
    description: netshoot debug
    scopes:
      - pods
    dangerous: true
    confirm: true
    background: false
    command: kubectl
    args:
      - debug
      - -it
```

```
      - --context
      - $CONTEXT
      - -n
      - $NAMESPACE
      - $NAME
      - --image=nicolaka/netshoot:v0.15
      - --profile=baseline
  # ... netshoot-target (containers) und events
```

`scopes` legt fest, in welchen Ansichten die Taste gilt. `confirm: true` zeigt den fertigen Befehl vor dem Start und fragt nach. `dangerous: true` blendet das Plugin im Read-only-Modus aus, denn ein Ephemeral Container verändert den Pod dauerhaft. `--profile=baseline` ist nötig, weil Talos sonst den Debug-Container ablehnt (Kapitel 1). `background: false` übergibt das Terminal an `kubectl`, bis du die Shell mit `exit` verlässt.

Die Tasten sind so gewählt, dass sie mit keiner Belegung von k9s 0.51 kollidieren. Das ist wichtig: Belegt ein Plugin eine Taste, die k9s in dieser Ansicht schon nutzt, lädt k9s das Plugin nicht, zeigt oben rechts `Plugins load failed!` und schreibt den Grund (`duplicate plugin key`) ins Log. Erst mit `override: true` ersetzt das Plugin die eingebaute Funktion. Wenn ein Plugin fehlt, schau mit `?` nach, ob die Taste erscheint, und in die Logdatei, deren Pfad `k9s info` zeigt.

**K9S** Schritt 12

```
:pods dbg-03⏎
/web⏎
Shift-d
```

Markiere einen der `web`-Pods und drücke `Shift-d`. Bestätige den angezeigten Befehl. Nach ein paar Sekunden landest du in einer Shell von netshoot.

**POD** Schritt 13

```
ss -ltn
curl -s localhost:8080
exit
```

`ss` zeigt den Webserver, der im **anderen** Container auf Port 8080 lauscht. Beide teilen das Netzwerk des Pods, deshalb erreicht `curl` ihn über `localhost`. Mit `exit` kehrst du zu k9s zurück.

**DAS SOLLTEST DU SEHEN**

`ss -ltn` listet eine Zeile mit `*:8080` im Zustand `LISTEN`, `curl` liefert `hallo aus web-…`. Ein `describe` des Pods (`d`) zeigt danach einen Abschnitt `Ephemeral Containers` mit dem Debug-Container im Zustand `Terminated`. Ephemeral Container lassen sich nicht entfernen, sie verschwinden erst mit dem Pod.

Probier zum Schluss `Shift-e` auf dem Deployment `wackel` in der Ansicht `:deploy⏎`. Du siehst die Events des Deployments selbst, also die Skalierung des ReplicaSets, nicht die der Pods. Auf dem

Pod zeigt dieselbe Taste die `BackOff`-Events. Diese Trennung ist beabsichtigt und hilft dir in Kapitel 10 und 11, wenn Pods gar nicht erst entstehen.

## 3.10 Wo kubectl besser ist

k9s ersetzt `kubectl` nicht. Für einige Aufgaben ist die Kommandozeile klar überlegen:

- **Dokumentieren und teilen.** Ein Befehl mit Ausgabe lässt sich in ein Ticket kopieren und von anderen wiederholen. Eine Folge von Tastendrücken nicht.
- **Skripte und Schleifen.** Alles, was du über mehrere Cluster oder Namespaces wiederholst, gehört in `kubectl` mit `-o jsonpath` oder `-o json`.
- **Fragen an die API.** `kubectl explain`, `kubectl auth can-i --as`, `kubectl get --raw '/readyz?verbose'` und `kubectl debug --copy-to` haben in k9s kein Gegenstück.
- **Zustand nach dem Neustart.** `p` zeigt wie `--previous` genau einen vorherigen Lauf. Kombinationen wie `--previous --all-containers --since=10m` gehen nur mit `kubectl`.
- **Kurze Ereignisse.** k9s aktualisiert in Intervallen von zwei Sekunden. Ein Pod, der nur eine Sekunde existiert, kann dazwischen verschwinden. `kubectl get pods -w` und Events zeigen jede Änderung.

Und k9s sieht nur, was dein Benutzer sehen darf. Bleibt eine Ansicht leer, obwohl `kubectl` mit Admin-Rechten Objekte zeigt, liegt es an RBAC, nicht an k9s.

MAC — Schritt 14

```
kubectl --context rke2 delete ns dbg-03
```

Räumt auf. In k9s ginge das mit `:ns⏎`, Namespace markieren und `Ctrl-d`.

**PROBIER ES SELBST**

Wende `szenarien/03-k9s/demo.yaml` auf `talos` an und arbeite ausschließlich in `k9s --context talos`. Finde heraus, auf welchem Knoten der Pod von `wackel` läuft (`o`), wie oft er schon neu gestartet ist, und lies die letzte Logzeile vor dem Absturz. Starte dann `Shift-d` auf einem `web`-Pod und prüfe mit `dig web.dbg-03.svc.cluster.local`, ob der Service-Name aufgelöst wird. Hinweis: Ohne `--profile=baseline` im Plugin würde Talos den Debug-Container mit `violates PodSecurity "baseline:latest"` ablehnen. Lösche den Namespace danach.

**WAS DU GELERNT HAST**

- k9s nutzt dieselbe API und Kubeconfig wie `kubectl`. Sein Vorteil ist Geschwindigkeit, nicht zusätzliches Wissen.
- `:` öffnet Ressourcen, `/` filtert (auch `/!`, `/-l`), `0` zeigt alle Namespaces, `Ctrl-z` nur Fehler, `?` die Tasten der Ansicht.
- `d`, `y`, `l` und `p` decken die meisten Untersuchungen ab. `p` entspricht `kubectl logs --previous`.
- `--readonly` schaltet ändernde Aktionen und gefährliche Plugins ab.
- Die Plugins dieses Buchs: `Shift-d` startet netshoot im Pod oder mit `--target` im Container, `Shift-e` zeigt die Events eines Objekts.
- Für Dokumentation, Skripte und API-Fragen wie `auth can-i` ist `kubectl` die bessere Wahl.

KAPITEL 4

# Pods bleiben Pending

*Du liest die Meldungen des Schedulers, erkennst, ob ein Pod an Ressourcen, Labels, Taints, Volumes oder einer Quota scheitert, und behebst jeden dieser Fehler auf RKE2 und Talos.*

**Dauer:** 60 Minuten **Voraussetzung:** Kapitel 2 und 3

## 4.1 Was in diesem Kapitel kaputtgeht

Ein Pod in `Pending` hat noch keinen Knoten. Kein Container läuft, es gibt keine Logs, `kubectl logs` bleibt leer. Die Antwort steht fast immer an einer von zwei Stellen: in den Events des Pods, geschrieben vom Scheduler, oder, wenn es den Pod gar nicht gibt, in den Events des ReplicaSets.

| Szenario | Symptom | Wo die Antwort steht |
|---|---|---|
| Zu große Requests | Pod `Pending`, Event `FailedScheduling` | `Insufficient cpu` im Pod-Event |
| nodeSelector und Taint | Pod `Pending` | `didn't match Pod's node affinity/selector`, `had untolerated taint(s)` |
| PVC ohne StorageClass | Pod und PVC `Pending` | `unbound immediate PersistentVolumeClaims`, Events der PVC |
| ResourceQuota | Deployment `0/2`, **kein** Pod | Event `FailedCreate` am ReplicaSet |

Alle Szenarien laufen im Namespace `dbg-04`, den jede `kaputt-*.yaml` selbst anlegt. Die Quota aus dem letzten Szenario gilt für den ganzen Namespace, spiele sie deshalb zuletzt. Auf Talos meldet `kubectl apply` bei diesen Manifesten `Warning: would violate PodSecurity "restricted:latest": ...`. Talos erzwingt cluster-weit nur `baseline` und warnt bei `restricted`. Die Warnung blockiert nichts, mehr dazu in Kapitel 10.

## 4.2 Der Scheduler erklärt sich selbst

Findet der Scheduler keinen Knoten, schreibt er ein Event mit dem Reason `FailedScheduling`. Der Text folgt immer dem Muster `0/2 nodes are available: <Gründe je Knoten>. preemption: <Ergebnis>`. Der erste Teil zählt für jeden Grund, an wie vielen Knoten er zutraf. `2 Insufficient cpu` heißt: beide Knoten haben zu wenig freie CPU. Stehen mehrere Gründe da, ergeben die Zahlen zusammen die Knotenzahl, denn der Scheduler meldet je Knoten den ersten Filter, der scheitert. Der zweite Teil sagt, ob der Scheduler andere Pods verdrängen könnte, um Platz zu schaffen.

`Preemption is not helpful for scheduling` bedeutet: Verdrängen löst dieses Problem nicht, etwa weil ein Label fehlt. Dann musst du selbst etwas ändern.

## 4.3 Zu große Requests

Requests sind Reservierungen. Der Scheduler rechnet nicht mit der tatsächlichen Last eines Knotens, sondern mit der Summe der Requests aller Pods darauf. Ein Pod, der mehr anfordert, als ein Knoten zuteilen kann, findet nie einen Platz, auch wenn der Cluster gerade nichts tut.

DATEI `szenarien/04-pending/kaputt-ressourcen.yaml (Ausschnitt)`

```
      containers:
        - name: app
          image: busybox:1.37
          command: ["sleep", "infinity"]
          resources:
            requests:
              cpu: "6"
              memory: 256Mi
```

Sechs CPU-Kerne als Request. Der größte Knoten im Labor hat vier.

MAC Schritt 1

```
kubectl --context rke2 apply -f szenarien/04-pending/kaputt-ressourcen.yaml
kubectl --context talos apply -f szenarien/04-pending/kaputt-ressourcen.yaml
```

Legt Namespace und Deployment `gross` auf beiden Clustern an.

**SYMPTOM**

```
$ kubectl --context rke2 get pods -n dbg-04 -o wide
NAME                     READY   STATUS    RESTARTS   AGE   IP       NODE     NOMINATED
NODE   READINESS GATES
gross-6d4b9c7f8d-q2xkp   0/1     Pending   0          48s   <none>   <none>   <none>
<none>
```

Kein Knoten, keine IP, keine Neustarts. Der Pod wartet, und das bleibt so.

### Untersuchen

MAC Schritt 2

```
kubectl --context rke2 describe pod -n dbg-04 -l app=gross
```

`describe` mit Label-Selektor erspart dir das Abtippen des Pod-Namens. Wichtig ist der Block `Events` ganz unten.

```
Events:
  Type     Reason            Age   From               Message
  ----     ------            ----  ----               -------
  Warning  FailedScheduling  51s   default-scheduler  0/2 nodes are available: 2
```

```
Insufficient cpu.
  preemption: 0/2 nodes are available: 2 No preemption victims found for incoming pod.
```

Beide Knoten haben zu wenig CPU, und auch das Verdrängen anderer Pods würde nicht helfen. Was ein Knoten vergeben kann (`Allocatable`) und was davon schon vergeben ist, zeigt `describe node`:

MAC Schritt 3

```
kubectl --context rke2 describe node rke2-cp | grep -A 8 "Allocated resources"
```

Die Spalte `Requests` ist entscheidend, nicht `Limits`. Limits dürfen in Summe über 100 Prozent liegen, Requests nicht. Den Block `Allocatable` weiter oben in derselben Ausgabe siehst du ohne `grep`.

```
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource           Requests     Limits
  --------           --------     ------
  cpu                1150m (28%)  200m (5%)
  memory             1004Mi (12%) 662Mi (8%)
  ephemeral-storage  0 (0%)       0 (0%)
```

In k9s kommst du auf demselben Weg zum Ziel, nur schneller zwischen den Ansichten:

K9S Schritt 4

```
:pods dbg-04⏎   Pod wählen, d   (describe, Events unten)
                oder Shift-E    (Plugin: kubectl events --for)
:nodes⏎         Knoten wählen, d
```

Mit `Esc` geht es zurück zur Liste, mit `/` filterst du die Describe-Ansicht nach `FailedScheduling`. `Shift-E` ist das Plugin aus Kapitel 3.

**URSACHE**

Der Pod fordert 6 CPU an, kein Knoten kann mehr als 4 zuteilen. Der Scheduler prüft für jeden Knoten, ob `allocatable` minus Summe der bestehenden Requests für den neuen Request reicht. Das ist reine Buchhaltung: Ob die CPU gerade ausgelastet ist, spielt keine Rolle. Ein Request größer als jeder einzelne Knoten bleibt deshalb für immer `Pending`.

### Beheben

Realistische Requests orientieren sich am gemessenen Verbrauch. Im Labor reicht ein Viertel Kern: `fix-ressourcen.yaml` setzt `requests.cpu: 250m`, sonst ist alles gleich.

MAC Schritt 5

```
kubectl --context rke2 apply -f szenarien/04-pending/fix-ressourcen.yaml
kubectl --context talos apply -f szenarien/04-pending/fix-ressourcen.yaml
```

Das Deployment erzeugt ein neues ReplicaSet, der alte Pending-Pod verschwindet.

**DAS SOLLTEST DU SEHEN**

```
$ kubectl --context rke2 get pods -n dbg-04 -l app=gross
NAME                     READY   STATUS    RESTARTS   AGE
gross-7f5c8b9d64-mz7wd   1/1     Running   0          9s
```

Im Event-Verlauf (`kubectl --context rke2 events -n dbg-04 --for pod/<name>`) steht jetzt `Successfully assigned dbg-04/... to rke2-cp` oder `to rke2-w1`.

**WARUM DAS WICHTIG IST**

Gefährlicher ist der Teil-Erfolg: Drei Replikas zu je 1500m passen im Labor nur zweimal, der dritte Pod bleibt `Pending`, und das Deployment zeigt ein halb gesundes `2/3`.

## 4.4 nodeSelector und Taint

Ein `nodeSelector` ist eine harte Bedingung: Der Pod darf nur auf Knoten, die alle genannten Labels tragen. Ein Taint ist das Gegenstück: Der Knoten weist Pods ab, die ihn nicht ausdrücklich tolerieren. Typisch sind Knoten für Spezialaufgaben, Knoten in Wartung und Control-Plane-Knoten. Im Labor machst du `rke2-w1` und `talos-w1` zu einem Batch-Knoten mit Label **und** Taint.

MAC — Schritt 6

```
kubectl --context rke2 label node rke2-w1 dbg-pool=batch
kubectl --context rke2 taint node rke2-w1 dbg-pool=batch:NoSchedule
kubectl --context talos label node talos-w1 dbg-pool=batch
kubectl --context talos taint node talos-w1 dbg-pool=batch:NoSchedule
```

`NoSchedule` verhindert nur neue Pods. Was schon auf dem Knoten läuft, bleibt. `NoExecute` würde auch laufende Pods ohne Toleration vertreiben.

DATEI `szenarien/04-pending/kaputt-taint.yaml (Ausschnitt)`

```yaml
  spec:
    nodeSelector:
      dbg-pool: batch
    containers:
      - name: app
        image: busybox:1.37
```

Der Pod will auf den Batch-Knoten, hat aber keine Toleration.

MAC — Schritt 7

```
kubectl --context rke2 apply -f szenarien/04-pending/kaputt-taint.yaml
kubectl --context talos apply -f szenarien/04-pending/kaputt-taint.yaml
```

Legt das Deployment `batch` an.

**SYMPTOM**

Der Pod `batch-...` steht auf `Pending`, obwohl es einen Knoten mit dem passenden Label gibt.

## Untersuchen

MAC — Schritt 8

```
kubectl --context rke2 describe pod -n dbg-04 -l app=batch | tail -n 4
```

Die letzten Zeilen von `describe` sind die Events.

```
  Warning  FailedScheduling  20s   default-scheduler  0/2 nodes are available: 1
node(s) didn't
  match Pod's node affinity/selector, 1 node(s) had untolerated taint(s). preemption:
0/2 nodes
  are available: 2 Preemption is not helpful for scheduling.
```

Zwei Gründe, und ihre Zahlen ergeben zusammen die zwei Knoten: `rke2-cp` scheitert am Selektor, `rke2-w1` am Taint. Hätte gar kein Knoten das Label, stünde dort `2 node(s) didn't match Pod's node affinity/selector`. Der Filter `NodeAffinity` prüft `nodeSelector` und `requiredDuringScheduling...` gemeinsam, daher der Doppelname. Welches Label und welcher Taint gemeint ist, verrät die Meldung nicht. Ältere Versionen nannten den Taint noch, Kubernetes 1.36 nicht mehr. Du vergleichst selbst:

MAC — Schritt 9

```
kubectl --context rke2 get nodes \
  -o custom-columns=NAME:.metadata.name,POOL:.metadata.labels.dbg-pool,TAINTS:.spec.taints
```

Zeigt Label und Taints je Knoten. `<none>` heißt: nicht vorhanden. Was der Pod verlangt und toleriert, steht in `describe pod` unter `Node-Selectors` und `Tolerations`.

```
NAME      POOL    TAINTS
rke2-cp   <none>  <none>
rke2-w1   batch   [map[effect:NoSchedule key:dbg-pool value:batch]]
```

Der Pod hat nur die beiden Tolerations `node.kubernetes.io/not-ready` und `unreachable` für 300 Sekunden, die jeder Pod automatisch bekommt. `dbg-pool` ist nicht darunter. In k9s siehst du die Taints mit `:nodes⏎`, Knoten auswählen, `y` für das YAML, dort unter `spec.taints`.

**URSACHE**

Der Selektor schickt den Pod auf genau einen Knoten, und dieser Knoten weist ihn per Taint ab. Selektor und Toleration lösen verschiedene Fragen: Der Selektor sagt, wohin der Pod **will**, die Toleration sagt, wohin er **darf**. Ein Spezialknoten braucht beides, sonst landen entweder fremde Pods darauf oder die eigenen nicht.

### Beheben

`fix-taint.yaml` ergänzt das Pod-Template um die passende Toleration:

```
      tolerations:
        - {key: dbg-pool, operator: Equal, value: batch, effect: NoSchedule}
```

MAC Schritt 10

```
kubectl --context rke2 apply -f szenarien/04-pending/fix-taint.yaml
kubectl --context talos apply -f szenarien/04-pending/fix-taint.yaml
```

Das neue ReplicaSet erzeugt einen Pod mit Toleration.

**DAS SOLLTEST DU SEHEN**

`kubectl --context rke2 get pods -n dbg-04 -l app=batch -o wide` zeigt `Running` auf `rke2-w1`, auf Talos entsprechend `talos-w1`.

Entferne danach Taint und Label, sonst landen künftige Pods nicht mehr auf dem Worker:

MAC Schritt 11

```
kubectl --context rke2 taint node rke2-w1 dbg-pool=batch:NoSchedule-
kubectl --context rke2 label node rke2-w1 dbg-pool-
kubectl --context talos taint node talos-w1 dbg-pool=batch:NoSchedule-
kubectl --context talos label node talos-w1 dbg-pool-
```

Das angehängte Minus entfernt Taint bzw. Label.

**RKE2**

Der RKE2-Server nimmt im Labor Workloads an und hat keinen Taint. Wer Server-Knoten frei halten will, setzt in `/etc/rancher/rke2/config.yaml` zum Beispiel `node-taint: ["CriticalAddonsOnly=true:NoExecute"]`. Dauerhafte Labels gehören dort unter `node-label:`.

**TALOS**

Ohne `cluster.allowSchedulingOnControlPlanes: true` trägt `talos-cp` den Taint `node-role.kubernetes.io/control-plane:NoSchedule`. In einem Cluster mit nur einem Worker landet dann jeder Pod dort, und ist der Worker voll oder weg, bleibt alles `Pending` mit `had untolerated taint(s)`. Im Labor ist das Scheduling auf der Control Plane erlaubt. Dauerhafte Labels gehören in `machine.nodeLabels`.

## 4.5 PVC ohne StorageClass

Ein Pod mit einem PersistentVolumeClaim kann erst starten, wenn der Claim an ein Volume gebunden ist oder gebunden werden kann. Im Labor gibt es auf beiden Clustern die StorageClass `local-path`, aber sie ist bewusst **nicht** Standard. Eine PVC ohne `storageClassName` bekommt deshalb keine Klasse, und niemand legt ein Volume an.

DATEI szenarien/04-pending/kaputt-pvc.yaml (Ausschnitt)

```
kind: PersistentVolumeClaim
metadata:
  name: daten
  namespace: dbg-04
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 1Gi
# ... Deployment "datenbank" bindet die PVC unter /daten ein
```

Kein `storageClassName`. Das sieht man in vielen Helm-Charts, die sich auf eine Default-StorageClass verlassen.

MAC Schritt 12

```
kubectl --context rke2 apply -f szenarien/04-pending/kaputt-pvc.yaml
kubectl --context talos apply -f szenarien/04-pending/kaputt-pvc.yaml
```

Legt PVC `daten` und Deployment `datenbank` an.

**SYMPTOM**

```
$ kubectl --context rke2 get pvc -n dbg-04
NAME    STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
VOLUMEATTRIBUTESCLASS   AGE
daten   Pending                                                 <unset>
31s
```

Die PVC ist `Pending`, die Spalte `STORAGECLASS` ist leer. Der Pod `datenbank-...` ist ebenfalls `Pending`.

## Untersuchen

MAC Schritt 13

```
kubectl --context rke2 describe pod -n dbg-04 -l app=datenbank | tail -n 3
kubectl --context rke2 describe pvc -n dbg-04 daten | tail -n 4
```

Erst der Pod, dann die PVC. Die eigentliche Erklärung steht bei der PVC.

```
  Warning  FailedScheduling  40s   default-scheduler  0/2 nodes are available: pod has
unbound
  immediate PersistentVolumeClaims. preemption: 0/2 nodes are available: 2 Preemption
is not
  helpful for scheduling.

  Type    Reason         Age              From                         Message
  ----    ------         ----             ----                         -------
  Normal  FailedBinding  5s (x4 over 40s) persistentvolume-controller  no persistent
volumes
  available for this claim and no storage class is set
```

Beachte die Form der Scheduler-Meldung: keine Zählung je Knoten, sondern ein einziger Satz. Das Volume-Binding scheitert schon vor der Prüfung der einzelnen Knoten. Und die PVC-Meldung ist vom Typ `Normal`, nicht `Warning`. Wer mit `--types=Warning` filtert, übersieht sie.

`kubectl --context rke2 get storageclass` zeigt `local-path` ohne den Zusatz `(default)` hinter dem Namen. In k9s: `:pvc dbg-04⏎`, PVC auswählen, `d`. Die Spalte `STORAGECLASS` ist schon in der Liste leer.

**URSACHE**

Ohne `storageClassName` und ohne Default-StorageClass gibt es niemanden, der ein Volume bereitstellt. Der PV-Controller sucht ein vorhandenes, passendes PersistentVolume ohne Klasse, findet keines und wartet. Der Scheduler wiederum setzt keinen Pod mit ungebundener PVC im Modus `Immediate` auf einen Knoten. Beide warten aufeinander, für immer.

### Beheben

Die PVC nennt die Klasse ausdrücklich. Die Spezifikation einer bestehenden PVC lässt sich nur eingeschränkt ändern, deshalb löschst du Deployment und PVC und legst beide neu an:

DATEI `szenarien/04-pending/fix-pvc.yaml (Ausschnitt)`

```yaml
spec:
  storageClassName: local-path
  accessModes: ["ReadWriteOnce"]
```

MAC — Schritt 14

```
kubectl --context rke2 -n dbg-04 delete deployment datenbank
kubectl --context rke2 -n dbg-04 delete pvc daten
kubectl --context rke2 apply -f szenarien/04-pending/fix-pvc.yaml
```

Auf Talos dieselben drei Befehle mit `--context talos`. Die PVC zuerst zu löschen hilft nicht: Solange ein Pod sie verwendet, hält der Finalizer `kubernetes.io/pvc-protection` sie in `Terminating`.

**DAS SOLLTEST DU SEHEN**

```
$ kubectl --context rke2 get pvc -n dbg-04
NAME    STATUS   VOLUME                                     CAPACITY   ACCESS MODES
STORAGECLASS   ...
daten   Bound    pvc-3c1e6a0b-8f5d-4c47-9b1e-2d7a4f0c9e18   1Gi        RWO
local-path     ...
```

Direkt nach dem Anlegen steht die PVC kurz auf `Pending` mit dem Event `WaitForFirstConsumer`: `local-path` legt das Volume erst an, wenn der Pod einen Knoten hat. Das ist kein Fehler.

**STOLPERFALLE**

Alternativ machst du `local-path` per Annotation `storageclass.kubernetes.io/is-default-class: "true"` zur Standardklasse. Seit Kubernetes 1.28 bekommen dann auch bestehende PVCs ohne Klasse nachträglich diese Klasse. Im Labor bleibt sie absichtlich ohne Default.

**RKE2**

RKE2 bringt keine StorageClass mit. `local-path` aus Kapitel 2 legt die Daten auf dem Knoten unter `/opt/local-path-provisioner` ab.

**TALOS**

Talos bringt ebenfalls keine StorageClass mit. `local-path` schreibt unter `/var/mnt/local-path-provisioner`, und der Helper-Pod läuft im Namespace `local-path-storage`, der deshalb `enforce=privileged` trägt. Fehlt das Label, scheitert der Helper-Pod, und die PVC bleibt trotz Klasse `Pending`. Diesen Fall behandelt Kapitel 9.

## 4.6 ResourceQuota: Der Pod, den es nicht gibt

Die ersten drei Szenarien hatten einen Pod in `Pending`. Dieses hat gar keinen Pod. Eine ResourceQuota begrenzt, was ein Namespace insgesamt anfordern darf. Sobald sie CPU oder Speicher begrenzt, muss außerdem **jeder** Container Requests und Limits für diese Ressourcen angeben. Die Quota-Admission prüft das beim Anlegen des Pods im API-Server, und wer es nicht erfüllt, wird abgelehnt.

Lösche vorher mit `kubectl --context rke2 -n dbg-04 delete deployment --all` (und ebenso auf Talos) die Deployments der anderen Szenarien, damit die Zahlen unten übersichtlich bleiben.

DATEI `szenarien/04-pending/kaputt-quota.yaml (Ausschnitt)`

```yaml
kind: ResourceQuota
metadata:
  name: dbg-quota
  namespace: dbg-04
spec:
  hard:
    pods: "10"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
---
# ... Deployment "web" mit nginx:1.29, 2 Replikas, ohne resources
```

MAC Schritt 15

```
kubectl --context rke2 apply -f szenarien/04-pending/kaputt-quota.yaml
kubectl --context talos apply -f szenarien/04-pending/kaputt-quota.yaml
```

`apply` meldet `deployment.apps/web created`. Kein Fehler, keine Warnung.

**SYMPTOM**

```
$ kubectl --context rke2 get deploy,pods -n dbg-04 -l app=web
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/web   0/2     0            0           25s
```

Das Deployment existiert, aber es gibt keinen einzigen Pod, auch keinen in `Pending`.

### Untersuchen

Die Kette lautet Deployment → ReplicaSet → Pod. Fehlt das letzte Glied, liegt der Fehler beim vorletzten:

MAC Schritt 16

```
kubectl --context rke2 get rs -n dbg-04 -l app=web
kubectl --context rke2 describe rs -n dbg-04 -l app=web | tail -n 4
```

Das ReplicaSet will zwei Pods (`DESIRED`), hat aber keinen (`CURRENT`). In seinen Events steht, warum.

```
NAME             DESIRED   CURRENT   READY   AGE
web-5d8c6b7f9c   2         0         0       31s

  Warning  FailedCreate  12s (x5 over 31s)  replicaset-controller  Error creating:
pods
  "web-5d8c6b7f9c-x8k2p" is forbidden: failed quota: dbg-quota: must specify
limits.cpu for: web;
  limits.memory for: web; requests.cpu for: web; requests.memory for: web
```

Auch das Deployment kennt das Problem: `kubectl describe deploy` zeigt unter `Conditions` die Zeile `ReplicaFailure True FailedCreate`. In k9s ist der schnellste Weg `:deploy dbg-04⏎`, dann `Enter` auf `web`. Das führt zum ReplicaSet, dort `d` für die Events. Den Füllstand der Quota zeigt `:quota dbg-04⏎` mit `d`, oder auf der Kommandozeile `kubectl --context rke2 describe quota -n dbg-04`.

**URSACHE**

Die Quota begrenzt `requests.cpu`, `requests.memory`, `limits.cpu` und `limits.memory`. Damit verlangt die Admission von jedem Container in `dbg-04` genau diese vier Angaben. Der nginx-Container hat keine, also lehnt der API-Server jeden Pod ab. Das ReplicaSet versucht es mit wachsenden Abständen immer wieder und protokolliert jeden Versuch als `FailedCreate`. Weil kein Pod entsteht, gibt es auch kein Pod-Event, und `kubectl get pods` bleibt leer.

### Beheben

DATEI szenarien/04-pending/fix-quota.yaml (Ausschnitt)

```
      resources:
        requests:
          cpu: 100m
          memory: 64Mi
        limits:
          cpu: 250m
          memory: 128Mi
```

MAC Schritt 17

```
kubectl --context rke2 apply -f szenarien/04-pending/fix-quota.yaml
kubectl --context talos apply -f szenarien/04-pending/fix-quota.yaml
```

Neues Pod-Template, neues ReplicaSet. Das alte wird auf null skaliert.

**DAS SOLLTEST DU SEHEN**

```
$ kubectl --context rke2 describe quota -n dbg-04
Name:            dbg-quota
Namespace:       dbg-04
Resource         Used   Hard
--------         ----   ----
limits.cpu       500m   2
limits.memory    256Mi  2Gi
pods             2      10
requests.cpu     200m   1
requests.memory  128Mi  1Gi
```

Zwei Pods laufen, und die Quota zählt genau deren Requests und Limits.

Ist die Quota ausgeschöpft, lautet die Meldung `exceeded quota: dbg-quota, requested: ..., used: ..., limited: ...`. Der Ort bleibt derselbe, die Events am ReplicaSet. Dort landen auch Ablehnungen durch Pod Security (Kapitel 10) und Webhooks wie Kyverno (Kapitel 11).

**PROBIER ES SELBST**

Wende `szenarien/04-pending/aufgabe-limitrange.yaml` auf einen der beiden Cluster an. Das Deployment `rechner` erreicht nie `1/1`. Finde heraus, warum, ohne die Datei vorher zu lesen, und behebe es mit einer eigenen Änderung.

Hinweis: Auch hier gibt es keinen Pod. Die Events am ReplicaSet nennen eine LimitRange, deren `max` kleiner ist als das Limit des Containers. Senke `limits.cpu` auf höchstens `500m`. Requests brauchst du nicht zu ergänzen: Gibt ein Container nur Limits an, setzt Kubernetes die Requests auf denselben Wert, und damit ist auch die Quota zufrieden.

Zum Schluss räumst du auf. Damit verschwinden Quota, LimitRange, PVC und alle Deployments:

MAC — Schritt 18

```
kubectl --context rke2 delete namespace dbg-04
kubectl --context talos delete namespace dbg-04
```

Das local-path-Volume wird wegen `RECLAIMPOLICY Delete` mit gelöscht.

---

**WAS DU GELERNT HAST**

- `Pending` heißt: kein Knoten. Logs gibt es keine, die Antwort steht in den Events des Pods (`describe`, `kubectl events --for`).
- Die Scheduler-Meldung zählt Gründe je Knoten. Mehrere Gründe ergeben zusammen die Knotenzahl, „Preemption is not helpful“ heißt: Du musst selbst etwas ändern.
- Requests sind Reservierungen gegen `allocatable`, nicht gegen die tatsächliche Last.
- Selektor sagt, wohin ein Pod will, Toleration, wohin er darf. Welches Label fehlt und welcher Taint stört, zeigst du mit `custom-columns` an den Knoten.
- Beide Labor-Distributionen haben keine Default-StorageClass. PVCs brauchen dort `storageClassName`.
- Fehlt der Pod ganz, liegt die Antwort am ReplicaSet (`FailedCreate`): Quota, LimitRange, Pod Security oder Webhook.

---

KAPITEL 5

# Images und Container-Start

*Du unterscheidest die Pull-Fehler an ihrer Meldung, prüfst Images direkt auf dem Knoten, richtest einen Registry-Mirror auf RKE2 und Talos ein und findest Fehler, die zwischen Pull und Start eines Containers passieren.*

**Dauer:** 60 Minuten **Voraussetzung:** Kapitel 2 und 4

## 5.1 Was in diesem Kapitel kaputtgeht

Nach dem Scheduling ist der Kubelet am Zug. Er lässt containerd das Image ziehen, baut die Konfiguration des Containers aus Umgebungsvariablen, Secrets und Volumes zusammen und startet ihn. An jedem dieser drei Schritte kann es scheitern, und jeder Schritt hat seinen eigenen Status.

| Schritt | Status in `get pods` | Szenario |
|---|---|---|
| Image ziehen | `ErrImagePull`, `ImagePullBackOff` | Tippfehler im Tag, Registry nicht erreichbar, privates Repository, Rate-Limit |
| Konfiguration bauen | `CreateContainerConfigError` | ConfigMap fehlt, Schlüssel im Secret fehlt |
| Prozess starten | `CrashLoopBackOff`, Log `exec format error` | Skript ohne Shebang, falsche Architektur |

Alle Szenarien laufen im Namespace `dbg-05`. Den Pod hat der Scheduler in diesem Kapitel immer schon platziert. Die Spalte `NODE` in `kubectl get pods -o wide` sagt dir, auf welchem Knoten du nachsehen musst.

## 5.2 Drei Pull-Fehler, drei Meldungen

`ImagePullBackOff` ist kein Fehler, sondern ein Wartezustand: Der Kubelet hat einen Pull versucht, er ist gescheitert, und jetzt wartet er mit wachsendem Abstand (bis zu fünf Minuten) auf den nächsten Versuch. Die Ursache steht in der Meldung des **ersten** Fehlschlags. Das Manifest erzeugt drei kaputte Deployments auf einmal:

DATEI `szenarien/05-images/kaputt-pull.yaml (Ausschnitt)`

```
  name: tippfehler
# ...
        - name: web
```

```
          image: nginx:1.29-alpnie
---
  name: unerreichbar
# ...
          image: registry.labor.invalid:5000/web:1.0
---
  name: privat
# ...
          image: docker.io/dbglabor/privat-app:1.0
```

Ein vertauschter Buchstabe im Tag, eine Registry, die es nicht gibt (die Domain `.invalid` ist dafür reserviert), und ein Repository, für das du keine Zugangsdaten hast.

**MAC** Schritt 1

```
kubectl --context rke2 apply -f szenarien/05-images/kaputt-pull.yaml
kubectl --context talos apply -f szenarien/05-images/kaputt-pull.yaml
```

Legt Namespace und drei Deployments an.

**SYMPTOM**

```
$ kubectl --context rke2 get pods -n dbg-05
NAME                            READY   STATUS             RESTARTS   AGE
privat-6c7d9f8b5-4kq9m          0/1     ImagePullBackOff   0          58s
tippfehler-7b9c5d4f6-p8x2l      0/1     ErrImagePull       0          58s
unerreichbar-5f6d8c7b9-r3n7v    0/1     ImagePullBackOff   0          58s
```

Der Status wechselt zwischen `ErrImagePull` (gerade gescheitert) und `ImagePullBackOff` (wartet). Am Status allein erkennst du die drei Fälle nicht.

### Untersuchen

**MAC** Schritt 2

```
kubectl --context rke2 describe pod -n dbg-05 -l app=tippfehler | tail -n 6
```

Die Events zeigen den ganzen Ablauf eines Pull-Versuchs.

```
  Normal   Pulling    40s (x3 over 82s)  kubelet  Pulling image "nginx:1.29-alpnie"
  Warning  Failed     39s (x3 over 81s)  kubelet  Failed to pull image "nginx:1.29-
alpnie": rpc
  error: code = NotFound desc = failed to pull and unpack image "docker.io/library/
nginx:1.29-
  alpnie": failed to resolve reference "docker.io/library/nginx:1.29-alpnie":
docker.io/library/
  nginx:1.29-alpnie: not found
  Warning  Failed     39s (x3 over 81s)  kubelet  Error: ErrImagePull
  Normal   BackOff    12s (x4 over 80s)  kubelet  Back-off pulling image "nginx:1.29-
alpnie"
  Warning  Failed     12s (x4 over 80s)  kubelet  Error: ImagePullBackOff
```

Wichtig ist nur die Zeile `Failed to pull image`. Lies sie von hinten: Das letzte Glied ist die eigentliche Antwort von containerd bzw. der Registry. Davor steht, was containerd gerade tun wollte.

Beachte auch, dass der Kubelet den kurzen Namen `nginx` zu `docker.io/library/nginx` ergänzt. Für die beiden anderen Pods holst du dir nur diese Zeile:

MAC Schritt 3

```
kubectl --context rke2 events -n dbg-05 --types=Warning | grep "Failed to pull"
```

Filtert alle Pull-Fehler im Namespace. Auf Talos sieht die Ausgabe gleich aus, nur die Adresse des DNS-Resolvers kann sich unterscheiden.

Die drei Meldungen enden so:

| Deployment | Ende der Meldung (gekürzt) |
|---|---|
| `tippfehler` | `docker.io/library/nginx:1.29-alpnie: not found` |
| `unerreichbar` | `dial tcp: lookup registry.labor.invalid ...: no such host` |
| `privat` | `pull access denied, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed` |

In k9s: `:pods dbg-05⏎`, Pod wählen, `d` (Events unten) oder `Shift-E` (Plugin aus Kapitel 3).

**URSACHE**

Drei verschiedene Fehler unter demselben Status. `not found` heißt: Die Registry kennt das Repository, aber nicht diesen Tag. `no such host` heißt: Der **Knoten** kann den Namen der Registry nicht auflösen. Den Pull macht containerd auf dem Knoten, nicht der Pod, deshalb zählt das DNS des Knotens und nicht CoreDNS. `pull access denied` heißt: Die Registry verweigert den Zugriff. Docker Hub unterscheidet dabei absichtlich nicht zwischen „gibt es nicht“ und „darfst du nicht sehen“.

Weitere Enden, die du kennen solltest:

| Meldung endet mit | Wahrscheinliche Ursache |
|---|---|
| `i/o timeout` | Firewall oder Proxy zwischen Knoten und Registry |
| `x509: certificate signed by unknown authority` | Registry mit eigener CA, der Knoten vertraut ihr nicht |
| `401 Unauthorized` | Zugangsdaten vorhanden, aber falsch oder abgelaufen |
| `429 Too Many Requests`, `toomanyrequests` | Rate-Limit von Docker Hub, siehe unten |
| `no match for platform in manifest` | Image gibt es nicht für die Architektur des Knotens |

### Beheben

Tippfehler und falsche Registry behebt nur ein korrigiertes Manifest:

MAC Schritt 4

```
kubectl --context rke2 apply -f szenarien/05-images/fix-pull.yaml
kubectl --context talos apply -f szenarien/05-images/fix-pull.yaml
```

Setzt bei `tippfehler` und `unerreichbar` das Image `nginx:1.29-alpine`.

Für ein privates Repository braucht der Pod Zugangsdaten in einem Secret vom Typ `kubernetes.io/dockerconfigjson` im **selben** Namespace. Du legst es mit `kubectl -n dbg-05 create secret docker-registry regcred --docker-server=docker.io --docker-username=... --docker-password=...` an, am besten mit einem Token, das nur lesen darf, und referenzierst es im Pod:

DATEI `szenarien/05-images/fix-pull-privat.yaml (Ausschnitt)`

```yaml
    spec:
      imagePullSecrets:
        - name: regcred
      containers:
        - name: web
          image: docker.io/dbglabor/privat-app:1.0
```

Das Repository `dbglabor/privat-app` gibt es nicht, im Labor bleibt dieser Pod deshalb auch mit Secret hängen. Ersetze das Image durch ein eigenes privates Repository, um den Weg durchzuspielen. Alternativ hängst du das Secret an den ServiceAccount: `kubectl -n dbg-05 patch serviceaccount default -p '{"imagePullSecrets":[{"name":"regcred"}]}'`. Dann bekommen alle neuen Pods dieses ServiceAccounts es automatisch.

**DAS SOLLTEST DU SEHEN**

`tippfehler` und `unerreichbar` stehen auf `Running`. In den Events erscheint `Successfully pulled image "nginx:1.29-alpine" in ...`. Den Pod `privat` löschst du, wenn du kein eigenes privates Repository hast: `kubectl --context rke2 -n dbg-05 delete deployment privat`.

## 5.3 Auf dem Knoten nachsehen

Manchmal willst du wissen, ob ein Pull am Knoten selbst funktioniert, ohne Kubelet dazwischen: etwa um Zugangsdaten, Mirror oder Proxy zu prüfen. Auf RKE2 geht das mit `crictl`, auf Talos mit `talosctl image`. Beide sprechen mit derselben containerd-Instanz, die auch der Kubelet benutzt.

RKE2-W1 Schritt 5

```
sudo -i
export PATH=$PATH:/var/lib/rancher/rke2/bin
export CRI_CONFIG_FILE=/var/lib/rancher/rke2/agent/etc/crictl.yaml
crictl pull nginx:1.29-alpnie
crictl pull nginx:1.29-alpine
crictl images | grep nginx
```

Der erste Pull scheitert mit derselben Meldung wie im Event. Der zweite endet mit `Image is up to date for sha256:...`. `crictl images` listet die Spalten `IMAGE`, `TAG`, `IMAGE ID` und `SIZE`.

MAC — Schritt 6

```
talosctl -n 172.16.0.171 image pull docker.io/library/nginx:1.29-alpine
talosctl -n 172.16.0.171 image list | grep nginx
```

`image pull` meldet `172.16.0.171: pulled image ...`. `image list` zeigt die Spalten `NODE`, `IMAGE`, `DIGEST`, `SIZE` und mehr. Ohne Angabe arbeitet `talosctl image` im Namespace `cri`, also mit den Images der Kubernetes-Workloads.

**RKE2**

Ausführlichere Pull-Fehler stehen im containerd-Log `/var/lib/rancher/rke2/agent/containerd/containerd.log`.

**TALOS**

Das Log der Workload-Laufzeit liest du mit `talosctl -n 172.16.0.171 logs cri`.

## 5.4 Rate-Limit und Registry-Mirror

Docker Hub begrenzt anonyme Pulls auf 100 je sechs Stunden und IP-Adresse. Hinter einem NAT teilen sich alle Knoten und alle Kollegen diese eine Adresse. Wird das Limit erreicht, scheitern Pulls von `docker.io` mit `429 Too Many Requests`, und die Meldung von Docker enthält `toomanyrequests: You have reached your pull rate limit`. Im Labor provozierst du das nicht absichtlich. Du richtest aber die Abhilfe ein, die auch gegen eine langsame oder ausgefallene Registry hilft: einen Mirror auf Knotenebene.

Ein Mirror wird nicht im Pod konfiguriert, sondern in containerd auf jedem Knoten. Der Pod schreibt weiter `nginx:1.29-alpine`, containerd fragt zuerst den Mirror. Im Labor ist das `mirror.gcr.io`, ein öffentlicher Cache von Google für beliebte Images von Docker Hub.

### RKE2: registries.yaml

DATEI `labor/rke2/registries-mirror.yaml (Ausschnitt)`

```
mirrors:
  docker.io:
    endpoint:
      - "https://mirror.gcr.io"
# Zugangsdaten für eine eigene Registry (nicht ins Repo einchecken):
# configs:
#   "registry.example.com:5000":
#     auth:
#       username: xxxxxx
#       password: xxxxxx
```

Das Format ist RKE2-eigen. RKE2 erzeugt daraus beim Start die containerd-Konfiguration. Zugangsdaten stehen unter `configs`, mit dem Hostnamen der Registry als Schlüssel.

Kopiere die Datei mit `scp` und deinem SSH-Benutzer aus Kapitel 2 nach `/tmp/registries.yaml` auf jeden Knoten und installiere sie dort:

RKE2-W1 Schritt 7

```
sudo install -m 600 /tmp/registries.yaml /etc/rancher/rke2/registries.yaml
sudo systemctl restart rke2-agent
sudo cat /var/lib/rancher/rke2/agent/etc/containerd/certs.d/docker.io/hosts.toml
```

RKE2 liest `registries.yaml` nur beim Start, deshalb der Neustart (auf `rke2-cp` : `rke2-server` ). Laufende Pods überstehen ihn. In der erzeugten `hosts.toml` muss ein Eintrag für `https://mirror.gcr.io` stehen.

**STOLPERFALLE**

RKE2 probiert die Original-Registry immer als letzten Ausweg, auch wenn ein Mirror eingetragen ist. Ein kaputter Mirror fällt deshalb nicht sofort auf, Pulls werden nur langsamer und zählen wieder gegen das Limit. Willst du das Fallback abschalten, setzt du in `/etc/rancher/rke2/config.yaml` die Option `disable-default-registry-endpoint: true` .

### Talos: RegistryMirrorConfig

Talos 1.14 beschreibt Mirrors in einem eigenen Konfigurationsdokument. Der alte Weg über `machine.registries` in der Machine-Config funktioniert noch, gilt aber als veraltet.

DATEI `labor/talos/patch-registry.yaml (Ausschnitt)`

```
apiVersion: v1alpha1
kind: RegistryMirrorConfig
name: docker.io
endpoints:
  - url: https://mirror.gcr.io
```

`name` ist die Registry, die gespiegelt wird. Zugangsdaten für einen Mirror kommen in ein weiteres Dokument vom Typ `RegistryAuthConfig` mit `name` , `username` und `password` , eigene CAs in `RegistryTLSConfig` . Mit `skipFallback: true` fragt Talos die Original-Registry gar nicht mehr.

MAC Schritt 8

```
talosctl -n 172.16.0.170,172.16.0.171 patch machineconfig \
  --patch @labor/talos/patch-registry.yaml
talosctl -n 172.16.0.171 ls /etc/cri/conf.d/hosts
```

Der Patch fügt das Dokument auf beiden Knoten hinzu. Talos schreibt daraus die containerd-Konfiguration unter `/etc/cri/conf.d/hosts` , mit einem Eintrag für `docker.io` . `talosctl patch` meldet, ob die Änderung ohne Neustart angewendet wurde.

**DAS SOLLTEST DU SEHEN**

Ein Pull eines noch nicht vorhandenen Images von Docker Hub gelingt weiter, auf RKE2 mit `crictl pull busybox:1.37`, auf Talos mit `talosctl -n 172.16.0.171 image pull docker.io/library/busybox:1.37`. Das Image erscheint unter seinem Originalnamen `docker.io/library/busybox`, nicht unter dem des Mirrors. Ob der Mirror tatsächlich gefragt wurde, zeigt auf RKE2 das containerd-Log, auf Talos `talosctl logs cri`.

## 5.5 CreateContainerConfigError

Das Image ist da, aber der Kubelet kann den Container nicht konfigurieren. Fehlt eine ConfigMap oder ein Secret, das in `env` oder `envFrom` referenziert wird, startet der Container gar nicht erst.

DATEI `szenarien/05-images/kaputt-config.yaml (Ausschnitt)`

```yaml
kind: Secret
metadata:
  name: app-secret
stringData:
  db-password: geheim-nur-im-labor
---
# ... Deployment "app"
          envFrom:
            - configMapRef:
                name: app-config
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: app-secret
                  key: DB_PASSWORD
```

Zwei Fehler: Die ConfigMap `app-config` existiert nicht, und das Secret hat den Schlüssel `db-password`, nicht `DB_PASSWORD`.

**MAC** Schritt 9

```
kubectl --context rke2 apply -f szenarien/05-images/kaputt-config.yaml
```

Auf Talos verhält sich dieses Szenario genauso.

**SYMPTOM**

```
$ kubectl --context rke2 get pods -n dbg-05 -l app=app
NAME                   READY   STATUS                       RESTARTS   AGE
app-6f9d7c8b5d-w2xkq   0/1     CreateContainerConfigError   0          21s
```

### Untersuchen

MAC — Schritt 10

```
kubectl --context rke2 events -n dbg-05 --types=Warning
```

Alle Warnungen im Namespace, die neueste unten. In k9s: Pod wählen, `Shift-E`.

```
LAST SEEN   TYPE      REASON    OBJECT                      MESSAGE
3s          Warning   Failed    Pod/app-6f9d7c8b5d-w2xkq    Error: configmap "app-
config" not found
```

Der Kubelet meldet immer nur den ersten Fehler, auf den er trifft. Lege die ConfigMap an und sieh, was danach kommt:

MAC — Schritt 11

```
kubectl --context rke2 -n dbg-05 create configmap app-config \
  --from-literal=DB_HOST=db.dbg-05.svc.cluster.local
```

Der Kubelet versucht es von selbst wieder. Nach einigen Sekunden steht ein neues Event da.

```
Warning   Failed    Pod/app-6f9d7c8b5d-w2xkq   Error: couldn't find key DB_PASSWORD in
Secret
dbg-05/app-secret
```

Welche Schlüssel das Secret wirklich hat, zeigt `kubectl --context rke2 describe secret app-secret -n dbg-05` unter `Data`, ohne die Werte auszugeben: nur `db-password`.

**URSACHE**

Umgebungsvariablen löst der Kubelet unmittelbar vor dem Start des Containers auf. Fehlt das Objekt oder der Schlüssel, und ist die Referenz nicht `optional: true`, bricht er ab, setzt `CreateContainerConfigError` und versucht es periodisch wieder. Der Pod ist dabei schon geplant und das Image gezogen, deshalb sieht der Fehler auf den ersten Blick wie ein Image-Problem aus.

### Beheben

MAC — Schritt 12

```
kubectl --context rke2 apply -f szenarien/05-images/fix-config.yaml
```

Legt die ConfigMap deklarativ an und referenziert den richtigen Schlüssel `db-password`.

**DAS SOLLTEST DU SEHEN**

Der Pod ist `Running`, und `kubectl --context rke2 logs -n dbg-05 deploy/app` zeigt `Verbinde mit db.dbg-05.svc.cluster.local`.

**STOLPERFALLE**

Ist die fehlende ConfigMap nicht als Umgebungsvariable, sondern als **Volume** eingebunden, sieht der Fehler anders aus: Der Pod hängt in `ContainerCreating`, und das Event heißt `FailedMount` mit `MountVolume.SetUp failed for volume "config" : configmap "app-config" not found`. Dieselbe Ursache, ein anderer Status.

## 5.6 exec format error

Dieser Fehler entsteht, wenn der Kernel die Datei, die als Prozess gestartet werden soll, nicht ausführen kann. Der klassische Fall ist ein Image für die falsche CPU-Architektur, etwa auf einem Mac mit Apple Silicon für `arm64` gebaut und auf `amd64`-Knoten gestartet. Im Labor reproduzierst du ihn mit einem zweiten, ebenso häufigen Auslöser: einem Startskript ohne Shebang-Zeile.

DATEI `szenarien/05-images/kaputt-exec.yaml (Ausschnitt)`

```yaml
kind: ConfigMap
data:
  start.sh: |
    echo "Starte Dienst"
    exec sleep infinity
---
# ... Deployment "skript"
          command: ["/scripts/start.sh"]
          volumeMounts:
            - name: skript
              mountPath: /scripts
      volumes:
        - name: skript
          configMap:
            name: start-skript
            defaultMode: 0755
```

Das Skript ist ausführbar, aber die erste Zeile `#!/bin/sh` fehlt.

MAC Schritt 13

```
kubectl --context rke2 apply -f szenarien/05-images/kaputt-exec.yaml
kubectl --context talos apply -f szenarien/05-images/kaputt-exec.yaml
```

Legt ConfigMap und Deployment `skript` an.

**SYMPTOM**

`kubectl --context rke2 get pods -n dbg-05 -l app=skript` zeigt nach kurzer Zeit `CrashLoopBackOff` mit steigender Zahl in `RESTARTS`.

### Untersuchen

MAC Schritt 14

```
kubectl --context rke2 logs -n dbg-05 deploy/skript --previous
```

`--previous` liest das Log des letzten beendeten Containers. In k9s: Pod wählen, `p`.

```
exec /scripts/start.sh: exec format error
```

Die Zeile stammt nicht vom Skript, sondern von der Container-Runtime (runc), die den Prozess starten wollte. Ob Architektur oder Shebang schuld ist, klärst du in zwei Schritten:

MAC Schritt 15

```
kubectl --context rke2 get nodes -L kubernetes.io/arch
kubectl --context rke2 get cm start-skript -n dbg-05 -o jsonpath='{.data.start\.sh}' | head -n 1
```

Die erste Zeile zeigt die Architektur jedes Knotens in der Spalte `ARCH`. Die zweite zeigt die erste Zeile des Skripts: Hier fehlt `#!/bin/sh`.

**URSACHE**

runc startet den Befehl des Containers direkt mit dem Systemaufruf `execve`, ohne Shell. Der Kernel erkennt ausführbare Dateien an ihren ersten Bytes: ELF-Binaries für die eigene Architektur oder Skripte mit `#!`. Alles andere lehnt er mit `ENOEXEC` ab, und das erscheint als `exec format error`. In einer interaktiven Shell würde dasselbe Skript laufen, weil die Shell es in diesem Fall selbst interpretiert.

### Beheben

MAC Schritt 16

```
kubectl --context rke2 apply -f szenarien/05-images/fix-exec.yaml
kubectl --context rke2 -n dbg-05 rollout restart deployment skript
```

Die neue ConfigMap beginnt mit `#!/bin/sh`. Der Neustart sorgt dafür, dass der Pod die neue Fassung sofort sieht. Auf Talos dieselben Befehle mit `--context talos`.

**DAS SOLLTEST DU SEHEN**

`kubectl --context rke2 logs -n dbg-05 deploy/skript` zeigt `Starte Dienst`, der Pod ist `Running` mit `RESTARTS 0`.

**WARUM DAS WICHTIG IST**

Bei falscher Architektur scheitert oft schon der Pull (`no match for platform in manifest`), wenn das Image die Architektur des Knotens gar nicht anbietet. Stimmen die Metadaten, aber das Binary nicht, kommt `exec format error`. Welche Plattformen ein Image hat, zeigt `docker buildx imagetools inspect nginx:1.29`.

**PROBIER ES SELBST**

Wende `szenarien/05-images/aufgabe-pullpolicy.yaml` auf `--context talos` an. Das Deployment `offline` startet nicht. Finde die Ursache über die Events und bringe den Pod zum Laufen, **ohne** das Manifest zu ändern.

Hinweis: Der Status heißt `ErrImageNeverPull`, die Meldung `Container image "nginx:1.29-alpine-slim" is not present with pull policy of Never`. Der Kubelet zieht bei `imagePullPolicy: Never` nie selbst. Lade das Image auf dem Knoten, auf dem der Pod liegt, mit `talosctl image pull` vor. Der Pod startet beim nächsten Versuch des Kubelets.

Aufräumen auf beiden Clustern:

MAC — Schritt 17

```
kubectl --context rke2 delete namespace dbg-05
kubectl --context talos delete namespace dbg-05
```

Der Registry-Mirror bleibt aktiv. Er schadet nicht und schont das Rate-Limit in den folgenden Kapiteln.

---

**WAS DU GELERNT HAST**

- `ImagePullBackOff` ist ein Wartezustand. Die Ursache steht im Event `Failed to pull image`, am Ende der Meldung.
- `not found`, `no such host`, `pull access denied`, `x509`, `429`: Jede Endung zeigt auf eine andere Ursache. Den Pull macht der Knoten, nicht der Pod.
- `crictl` auf RKE2 und `talosctl image` auf Talos prüfen Pulls direkt an containerd, ohne Kubelet.
- Mirrors und Registry-Zugangsdaten auf Knotenebene: RKE2 `/etc/rancher/rke2/registries.yaml` plus Neustart, Talos `RegistryMirrorConfig` per `talosctl patch`.
- `CreateContainerConfigError` meldet fehlende ConfigMaps, Secrets und Schlüssel, immer nur den ersten Fehler.
- `exec format error` heißt: Der Kernel kann die Datei nicht ausführen. Falsche Architektur oder fehlender Shebang.

---

KAPITEL 6

# Abstürze und Neustarts

*Du liest Exit-Codes und das Log des vorigen Containers, erkennst OOMKilled und Probe-Fehler und untersuchst Container ohne Shell mit* `kubectl debug`.

**Dauer:** 75 Minuten **Voraussetzung:** Kapitel 1, 3 und 5

## 6.1 Was in diesem Kapitel kaputtgeht

Jetzt läuft der Container, aber nicht lange oder nicht richtig. Die zwei wichtigsten Werkzeuge dafür sind der Block `Last State` in `kubectl describe pod` und das Log des **vorigen** Containers. Nach einem Neustart zeigt `kubectl logs` nur den neuen Container, die Fehlermeldung des alten steht in `kubectl logs --previous`.

| Szenario | Symptom | Erster Hinweis |
|---|---|---|
| Falscher Befehl, App-Fehler | `CrashLoopBackOff` | Exit-Code 127 bzw. 1, `logs --previous` |
| Speicherlimit | `OOMKilled`, dann `CrashLoopBackOff` | Exit-Code 137, Reason `OOMKilled` |
| Liveness zu streng, Readiness falsch | Neustarts bzw. `0/1` ohne Neustart | Events `Unhealthy` |
| `readOnlyRootFilesystem` | `CrashLoopBackOff` | `Read-only file system` im Log |
| Distroless-Image | `exec` scheitert, keine Shell | `kubectl debug` |

Alle Szenarien laufen im Namespace `dbg-06`. Die Exit-Codes aus Kapitel 1 brauchst du hier ständig: 1 ist ein Fehler der Anwendung, 126 und 127 bedeuten „nicht ausführbar“ und „nicht gefunden“ in einer Shell, 137 ist 128 + 9, also `SIGKILL`.

## 6.2 CrashLoopBackOff: Befehl und Anwendung

`CrashLoopBackOff` ist wie `ImagePullBackOff` ein Wartezustand. Der Container ist beendet, der Kubelet startet ihn neu und verdoppelt die Wartezeit nach jedem Absturz, von 10 Sekunden bis höchstens 5 Minuten. Die Ursache steht nicht im Status, sondern im Exit-Code und im Log.

DATEI `szenarien/06-crash/kaputt-crash.yaml (Ausschnitt)`

```
  name: tippfehler
# ...
```

```
          command: ["sh", "-c", "serv -p 8080 -h /www"]
---
  name: appfehler
# ...
          command:
            - sh
            - -c
            - |
              echo "Konfiguration wird geladen"
              if [ -z "$DATABASE_URL" ]; then
                echo "FEHLER: DATABASE_URL ist nicht gesetzt" >&2
                exit 1
              fi
              exec sleep infinity
```

`tippfehler` soll `httpd` starten, aber da steht `serv`. `appfehler` bricht ab, weil eine Umgebungsvariable fehlt, wie viele echte Anwendungen beim Start.

**MAC** Schritt 1

```
kubectl --context rke2 apply -f szenarien/06-crash/kaputt-crash.yaml
kubectl --context talos apply -f szenarien/06-crash/kaputt-crash.yaml
```

Legt Namespace und zwei Deployments an.

**SYMPTOM**

```
$ kubectl --context rke2 get pods -n dbg-06
NAME                          READY   STATUS             RESTARTS      AGE
appfehler-5c8d7b6f4-n4w8k     0/1     CrashLoopBackOff   3 (25s ago)   71s
tippfehler-6b9f8c7d5-k2v7x    0/1     CrashLoopBackOff   3 (28s ago)   71s
```

### Untersuchen

**MAC** Schritt 2

```
kubectl --context rke2 describe pod -n dbg-06 -l app=tippfehler | grep -A 9 "State:"
```

Der Ausschnitt um `State` und `Last State` zeigt, wie der letzte Container geendet hat.

```
  State:          Waiting
    Reason:       CrashLoopBackOff
  Last State:     Terminated
    Reason:       Error
    Exit Code:    127
    Started:      Mon, 09 Nov 2026 10:12:41 +0100
    Finished:     Mon, 09 Nov 2026 10:12:41 +0100
  Ready:          False
  Restart Count:  3
```

Started und Finished in derselben Sekunde, Exit-Code 127: Die Shell hat einen Befehl nicht gefunden. Das Log bestätigt es:

MAC Schritt 3

```
kubectl --context rke2 logs -n dbg-06 deploy/tippfehler --previous
kubectl --context rke2 logs -n dbg-06 deploy/appfehler --previous
```

`--previous` liest den vorigen, abgestürzten Container. Ohne die Option erwischst du je nach Zeitpunkt einen gerade gestarteten Container mit leerem Log.

```
sh: serv: not found
Konfiguration wird geladen
FEHLER: DATABASE_URL ist nicht gesetzt
```

`appfehler` endet mit Exit-Code 1, die Meldung hat die Anwendung selbst geschrieben. In k9s ist der Weg kürzer: `:pods dbg-06⏎`, Pod wählen, `p` öffnet die Logs des vorigen Containers (`l` die des aktuellen), `d` zeigt `Last State`.

**URSACHE**

`tippfehler`: Die Shell findet `serv` nicht und endet mit 127. `appfehler`: Die Anwendung prüft ihre Konfiguration und beendet sich mit 1. In beiden Fällen ist der Container fertig, und weil `restartPolicy` eines Deployments immer `Always` ist, startet der Kubelet ihn neu. Wiederholt sich das, wartet er zunehmend länger: `CrashLoopBackOff`.

## Beheben

MAC Schritt 4

```
kubectl --context rke2 apply -f szenarien/06-crash/fix-crash.yaml
kubectl --context talos apply -f szenarien/06-crash/fix-crash.yaml
```

`tippfehler` startet jetzt `httpd`, `appfehler` bekommt `DATABASE_URL` als Umgebungsvariable.

**DAS SOLLTEST DU SEHEN**

Beide Pods sind `Running` mit `RESTARTS 0`, denn das Deployment hat neue Pods angelegt.

**STOLPERFALLE**

Steht der falsche Befehl direkt in `command` statt hinter `sh -c`, also `command: ["serv"]`, gibt es keine Shell, die 127 melden könnte. Dann kann die Runtime den Prozess gar nicht starten. `Last State` zeigt `Reason: StartError` mit Exit-Code 128, und die Meldung enthält `exec: "serv": executable file not found in $PATH`. Ein Log gibt es in diesem Fall nicht.

## 6.3 OOMKilled

Ein Speicherlimit ist eine harte Grenze. Überschreitet ein Container sie, beendet der Kernel ihn mit `SIGKILL`. Es gibt keine Warnung, keine Log-Zeile der Anwendung und keinen geordneten Shutdown.

DATEI `szenarien/06-crash/kaputt-oom.yaml (Ausschnitt)`

```
        - name: app
          image: polinux/stress:1.0.4
          command: ["/usr/local/bin/stress"]
          args: ["--vm", "1", "--vm-bytes", "200M", "--vm-hang", "0"]
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              memory: 128Mi
```

`stress` belegt 200 MB und hält sie. Das Limit erlaubt 128 MiB.

MAC — Schritt 5

```
kubectl --context rke2 apply -f szenarien/06-crash/kaputt-oom.yaml
kubectl --context talos apply -f szenarien/06-crash/kaputt-oom.yaml
```

Legt das Deployment `speicher` an.

**SYMPTOM**

`kubectl --context rke2 get pods -n dbg-06 -l app=speicher` zeigt kurz `OOMKilled` in der Spalte `STATUS`, danach `CrashLoopBackOff`. Wer nur gelegentlich hinsieht, sieht nur Letzteres.

### Untersuchen

MAC — Schritt 6

```
kubectl --context rke2 get pod -n dbg-06 -l app=speicher \
  -o jsonpath='{.items[0].status.containerStatuses[0].lastState.terminated}' | jq .
```

Liest den letzten Endzustand direkt aus dem Status. Ohne `jq` kommt dieselbe Information in einer Zeile.

```
{
  "containerID": "containerd://9f2c4e1b7a...",
  "exitCode": 137,
  "finishedAt": "2026-11-09T09:31:07Z",
  "reason": "OOMKilled",
  "startedAt": "2026-11-09T09:31:06Z"
}
```

`reason: OOMKilled` und Exit-Code 137 sind eindeutig. Das Log hilft hier kaum: `logs --previous` zeigt nur `stress: info: [1] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd`, danach nichts mehr. Den Beweis liefert der Kernel des Knotens:

**RKE2**

Auf dem Knoten des Pods (Spalte `NODE` in `get pods -o wide`), per SSH:

```
sudo journalctl -k --since "10 min ago" \
  | grep -i "out of memory"
```

**TALOS**

Ohne SSH, vom Mac aus:

```
talosctl -n 172.16.0.171 dmesg \
  | grep -i "out of memory"
```

Beide liefern eine Zeile wie `Memory cgroup out of memory: Killed process 23817 (stress) total-vm:208712kB, anon-rss:130816kB, ...`.

**URSACHE**

Das Speicherlimit wird als cgroup-Grenze im Kernel durchgesetzt. Überschreitet der Container sie, wählt der OOM-Killer des Kernels einen Prozess in dieser cgroup und beendet ihn mit `SIGKILL` (Signal 9, daher 128 + 9 = 137). Der Kubelet erfährt davon über die Runtime und setzt den Reason `OOMKilled`. Requests helfen nicht: Sie sind nur Buchhaltung für den Scheduler.

### Beheben

Entweder braucht die Anwendung wirklich mehr Speicher, dann hebst du das Limit an. Oder sie verbraucht zu viel, dann ist es ein Fehler der Anwendung, etwa ein Leck oder ein zu großer Cache. Im Labor bekommt sie mehr:

MAC — Schritt 7

```
kubectl --context rke2 apply -f szenarien/06-crash/fix-oom.yaml
kubectl --context talos apply -f szenarien/06-crash/fix-oom.yaml
```

Setzt Request und Limit für Speicher auf `256Mi`.

**DAS SOLLTEST DU SEHEN**

Der Pod bleibt `Running`. Auf RKE2 zeigt `kubectl --context rke2 top pod -n dbg-06 -l app=speicher` etwa 200 MiB. Auf Talos fehlt noch der metrics-server (Kapitel 15), dort meldet `kubectl top` nur `error: Metrics API not available`.

**STOLPERFALLE**

`OOMKilled` betrifft einen Container, der sein **eigenes** Limit sprengt. Wird der **Knoten** knapp, räumt der Kubelet ganze Pods, sie stehen dann auf `Failed` mit Reason `Evicted` (Kapitel 13).

## 6.4 Probes: zu streng und falsch gezielt

Probes sind die Fragen, die der Kubelet regelmäßig an einen Container stellt. Die Liveness-Probe fragt „lebst du noch?“ und startet den Container bei wiederholtem Nein neu. Die Readiness-Probe fragt „kannst du Anfragen annehmen?“ und nimmt den Pod bei Nein aus dem Service. Beide falsch zu setzen, erzeugt zwei völlig verschiedene Symptome.

DATEI `szenarien/06-crash/kaputt-probes.yaml (Ausschnitt)`

```
  name: langsam
# ...
              echo "Lade Daten (dauert 40 Sekunden) ..."
              sleep 40
              # ... danach httpd auf Port 8080
          livenessProbe:
            httpGet: {path: /healthz, port: 8080}
            initialDelaySeconds: 10
            periodSeconds: 5
            failureThreshold: 3
---
  name: niemeready
# ... httpd auf Port 8080, liefert nur /healthz
          readinessProbe:
            httpGet: {path: /ready, port: 8080}
# ... dazu ein Service "niemeready"
```

`langsam` braucht 40 Sekunden bis zum ersten Lebenszeichen, die Liveness-Probe gibt ihm nur etwa 20. `niemeready` fragt einen Pfad ab, den es nicht gibt.

MAC — Schritt 8

```
kubectl --context rke2 apply -f szenarien/06-crash/kaputt-probes.yaml
kubectl --context talos apply -f szenarien/06-crash/kaputt-probes.yaml
```

Zwei Deployments und ein Service.

**SYMPTOM**

```
$ kubectl --context rke2 get pods -n dbg-06 -l 'app in (langsam,niemeready)'
NAME                         READY   STATUS    RESTARTS      AGE
langsam-7d6c9b8f5-x4q2m      1/1     Running   3 (12s ago)   2m5s
niemeready-5f7b8d9c6-j6t3p   0/1     Running   0             2m5s
```

`langsam` läuft, startet aber ständig neu. `niemeready` läuft ohne Neustart, wird aber nie bereit.

### Untersuchen

MAC — Schritt 9

```
kubectl --context rke2 events -n dbg-06 | grep -E "Unhealthy|Killing"
```

Probe-Fehler haben den Reason `Unhealthy` (Typ `Warning`). Den Neustart meldet ein Event vom Typ `Normal` mit Reason `Killing`, deshalb hier kein Filter `--types=Warning`.

```
Warning   Unhealthy   Pod/langsam-7d6c9b8f5-x4q2m    Liveness probe failed: Get
"http://10.42.1.23:8080/healthz": dial tcp 10.42.1.23:8080: connect: connection
refused
Warning   Unhealthy   Pod/nieready-5f7b8d9c6-j6t3p   Readiness probe failed: HTTP
probe failed
with statuscode: 404
Normal    Killing     Pod/langsam-7d6c9b8f5-x4q2m    Container app failed liveness
probe, will be
restarted
```

Bei `langsam` lohnt noch ein Blick auf `Last State` und das vorige Log: Exit-Code 137 und als letzte Zeile `Lade Daten (dauert 40 Sekunden) ...`. Der Container wurde mitten im Laden beendet. Exit-Code 137 entsteht, weil die Shell das `SIGTERM` des Kubelets ignoriert und nach der Grace Period `SIGKILL` folgt. Bei `nieready` interessiert, was der Service sieht:

**MAC** Schritt 10

```
kubectl --context rke2 get endpointslices -n dbg-06 -l kubernetes.io/service-name=nieready \
  -o jsonpath='{range .items[*].endpoints[*]}{.addresses[0]} ready={.conditions.ready}{"\n"}{end}'
```

Listet jede Adresse der EndpointSlice mit ihrem Zustand.

```
10.42.1.24 ready=false
```

In k9s zeigt `d` auf dem Pod die Probe-Konfiguration unter `Liveness` bzw. `Readiness` samt `delay`, `period` und `#failure`.

**URSACHE**

`langsam`: Die Liveness-Probe beginnt nach 10 Sekunden und gibt nach drei Fehlschlägen im Abstand von 5 Sekunden auf, also nach rund 20 Sekunden. Die Anwendung braucht 40. Der Kubelet tötet sie also jedes Mal vor dem Ziel, ein Kreislauf ohne Ausweg. `nieready`: Die Readiness-Probe fragt `/ready`, httpd liefert 404. Der Pod bleibt in der EndpointSlice, aber mit `ready=false`, und der Service schickt keinen Verkehr hin. Readiness startet nie neu, deshalb gibt es hier keinen Absturz, nur Stille.

### Beheben

Für langsame Starts gibt es die Startup-Probe. Solange sie nicht erfolgreich war, setzt der Kubelet Liveness und Readiness aus. Danach übernimmt die strenge Liveness-Probe wie gewohnt.

DATEI `szenarien/06-crash/fix-probes.yaml (Ausschnitt)`

```
        startupProbe:
          httpGet: {path: /healthz, port: 8080}
          periodSeconds: 5
          failureThreshold: 24
        livenessProbe:
          httpGet: {path: /healthz, port: 8080}
          periodSeconds: 5
          failureThreshold: 3
```

```
# ... bei nieready:
          readinessProbe:
            httpGet: {path: /healthz, port: 8080}
```

Die Startup-Probe gibt dem Container bis zu $24 \times 5 = 120$ Sekunden. `initialDelaySeconds` der Liveness-Probe entfällt.

**MAC** Schritt 11

```
kubectl --context rke2 apply -f szenarien/06-crash/fix-probes.yaml
kubectl --context talos apply -f szenarien/06-crash/fix-probes.yaml
```

Beide Deployments rollen neu aus.

**DAS SOLLTEST DU SEHEN**

Nach gut 40 Sekunden zeigt `get pods` beide mit `1/1 Running` und `RESTARTS 0`. Die EndpointSlice meldet `ready=true`, und `kubectl --context rke2 run test -n dbg-06 --rm -it --image=busybox:1.37 --restart=Never -- wget -qO- nieready/healthz` antwortet mit `ok`.

## 6.5 readOnlyRootFilesystem

Ein schreibgeschütztes Root-Dateisystem ist eine verbreitete Härtung, auch wenn Pod Security `restricted` sie nicht verlangt. Viele Images schreiben aber beim Start an Stellen, die der Entwickler nie bewusst gewählt hat.

DATEI `szenarien/06-crash/kaputt-readonly.yaml (Ausschnitt)`

```
      - name: web
        image: nginx:1.29
        securityContext:
          readOnlyRootFilesystem: true
```

**MAC** Schritt 12

```
kubectl --context rke2 apply -f szenarien/06-crash/kaputt-readonly.yaml
```

Auf Talos verhält es sich gleich.

**SYMPTOM**

`webro` steht nach wenigen Sekunden auf `CrashLoopBackOff`, `Last State` zeigt Exit-Code 1.

**MAC** Schritt 13

```
kubectl --context rke2 logs -n dbg-06 deploy/webro --previous
```

Das Log des abgestürzten Containers.

```
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf
(read-only file system?)
...
2026/11/09 09:44:12 [emerg] 1#1: mkdir() "/var/cache/nginx/client_temp" failed (30:
Read-only file system)
nginx: [emerg] mkdir() "/var/cache/nginx/client_temp" failed (30: Read-only file
system)
```

Die `info`-Zeile ist harmlos, erst `[emerg]` beendet nginx.

**URSACHE**

Mit `readOnlyRootFilesystem: true` wird das Dateisystem des Images nur lesbar eingehängt. nginx braucht beim Start schreibbare Verzeichnisse für temporäre Dateien ( `/var/cache/nginx` ) und die PID-Datei (unter `/run` ). Der Fehlercode 30 ist `EROFS` , „Read-only file system“. Volumes bleiben auch bei schreibgeschütztem Root schreibbar.

DATEI `szenarien/06-crash/fix-readonly.yaml (Ausschnitt)`

```yaml
          volumeMounts:
            - name: cache
              mountPath: /var/cache/nginx
            - name: run
              mountPath: /run
      volumes:
        - name: cache
          emptyDir: {}
        - name: run
          emptyDir:
            medium: Memory
            sizeLimit: 16Mi
```

Zwei `emptyDir`-Volumes genau an den Stellen, an denen nginx schreibt. Alles andere bleibt geschützt.

**MAC** Schritt 14

```
kubectl --context rke2 apply -f szenarien/06-crash/fix-readonly.yaml
```

Ebenso auf Talos.

**DAS SOLLTEST DU SEHEN**

`webro` ist `Running` , das Log endet mit `Configuration complete; ready for start up` und den Startmeldungen der Worker-Prozesse. `kubectl --context rke2 exec -n dbg-06 deploy/webro -- touch /test` schlägt weiterhin mit `Read-only file system` fehl, wie gewünscht.

## 6.6 Distroless: Debuggen ohne Shell

Distroless-Images enthalten nur die Anwendung, keine Shell, kein `ls`. `kubectl exec ... -- sh` endet mit einer Meldung, die `exec: "sh": executable file not found in $PATH` enthält. Als Beispiel dient CoreDNS, dessen offizielles Image distroless ist.

DATEI `szenarien/06-crash/kaputt-distroless.yaml (Ausschnitt)`

```
kind: ConfigMap
data:
  corefile: |
    .:1053 {
        whoami
        health :8080
        log
    }
---
# ... Deployment "dns-test"
          image: registry.k8s.io/coredns/coredns:v1.14.7
          args: ["-conf", "/etc/coredns/Corefile"]
          volumeMounts:
            - name: config
              mountPath: /etc/coredns
```

MAC Schritt 15

```
kubectl --context rke2 apply -f szenarien/06-crash/kaputt-distroless.yaml
kubectl --context rke2 logs -n dbg-06 deploy/dns-test --previous
```

Ebenso auf Talos. Der Pod stürzt sofort ab.

```
open /etc/coredns/Corefile: no such file or directory
```

**SYMPTOM**

`dns-test` in `CrashLoopBackOff`, Exit-Code 1. Die Datei fehlt angeblich, obwohl die ConfigMap eingehängt ist. Nachsehen per `exec` geht nicht: keine Shell, und der Container läuft kaum.

### Untersuchen mit einem Ephemeral Container

`kubectl debug` fügt dem Pod einen zusätzlichen Container mit einem Image deiner Wahl hinzu. Er teilt das Netzwerk des Pods, Volumes bekommt er nur über `--custom`:

DATEI `szenarien/06-crash/debug-custom.yaml (Ausschnitt)`

```
securityContext:
  runAsUser: 65532
  runAsGroup: 65532
volumeMounts:
  - name: config
    mountPath: /etc/coredns
```

Kein vollständiges Manifest, nur die Felder, die `kubectl debug` in den Debug-Container übernimmt. Benutzer 65532 ist der Benutzer `nonroot` des CoreDNS-Images.

MAC Schritt 16

```
POD=$(kubectl --context rke2 get pod -n dbg-06 -l app=dns-test -o name)
kubectl --context rke2 debug -it -n dbg-06 $POD --image=busybox:1.37 \
  --profile=baseline --custom=szenarien/06-crash/debug-custom.yaml -- sh
```

Startet busybox als Ephemeral Container im kaputten Pod und öffnet eine Shell. `--profile=baseline` ist auf Talos Pflicht, siehe unten.

POD Schritt 17

```
ls -l /etc/coredns
lrwxrwxrwx    1 root     root            15 Nov  9 10:31 corefile -> ..data/corefile
```

Im Debug-Container, hier mit Ausgabe. Mit `exit` verlässt du ihn, er bleibt beendet in der Pod-Spezifikation stehen.

Da ist sie: `corefile` klein geschrieben. Bevor du die ConfigMap änderst, prüfst du die Vermutung an einer Kopie des Pods mit geändertem Befehl:

MAC Schritt 18

```
kubectl --context rke2 debug -n dbg-06 $POD --copy-to=dns-test-kopie --profile=baseline \
  --container=coredns -- /coredns -conf /etc/coredns/corefile
kubectl --context rke2 logs -n dbg-06 dns-test-kopie -c coredns
```

`--copy-to` legt einen neuen Pod an, der dem Original gleicht, aber ohne Labels, also ohne Service und Deployment. `--container` mit Befehl ersetzt dessen Startbefehl. Das Log beginnt mit `.:1053` und `CoreDNS-1.14.7`: Mit dem richtigen Dateinamen läuft es.

**URSACHE**

Der Schlüssel einer ConfigMap wird beim Einhängen zum Dateinamen, mit Groß- und Kleinschreibung. Die ConfigMap liefert `corefile`, CoreDNS sucht `Corefile`. Ohne Shell im Image bringst du das Werkzeug zum Nachsehen mit `kubectl debug` selbst mit.

### Beheben und gegenprüfen

MAC Schritt 19

```
kubectl --context rke2 -n dbg-06 delete pod dns-test-kopie
kubectl --context rke2 apply -f szenarien/06-crash/fix-distroless.yaml
kubectl --context rke2 -n dbg-06 rollout restart deployment dns-test
```

Kopie löschen, ConfigMap mit dem Schlüssel `Corefile` anwenden, Pods neu starten, damit sie die geänderte ConfigMap sofort sehen.

Jetzt läuft der Container, und du kannst dich mit `--target` in seinen Prozess-Namensraum hängen:

**MAC** Schritt 20

```
POD=$(kubectl --context rke2 get pod -n dbg-06 -l app=dns-test -o name)
kubectl --context rke2 debug -it -n dbg-06 $POD --image=busybox:1.37 --target=coredns \
  --profile=baseline --custom=szenarien/06-crash/debug-custom.yaml -- sh
```

`--target=coredns` teilt die Prozesse dieses Containers mit dem Debug-Container.

**POD** Schritt 21

```
ps
ls /proc/1/root/etc/coredns/
```

PID 1 ist jetzt CoreDNS. Über `/proc/1/root` siehst du dessen Dateisystem, weil der Debug-Container als derselbe Benutzer 65532 läuft.

**DAS SOLLTEST DU SEHEN**

```
PID   USER     TIME  COMMAND
    1 65532     0:00 /coredns -conf /etc/coredns/Corefile
   14 65532     0:00 sh
   21 65532     0:00 ps
Corefile
```

`kubectl get pods -n dbg-06 -l app=dns-test` zeigt `1/1 Running`.

**RKE2**

RKE2 erzwingt ohne CIS-Profil keinen Pod-Security-Standard. Das Standardprofil `general` von `kubectl debug` funktioniert. Es fügt die Capability `SYS_PTRACE` hinzu, mit der ein Debug-Container als root auch in fremde Prozesse schauen darf.

**TALOS**

Talos erzwingt cluster-weit `baseline`. Mit dem Profil `general` lehnt der API-Server den Debug-Container ab: `violates PodSecurity "baseline:latest": non-default capabilities (container "debugger-..." must not include "SYS_PTRACE" in securityContext.capabilities.add)`. Deshalb `--profile=baseline` und für `/proc/1/root` derselbe Benutzer wie der Zielprozess. Das k9s-Plugin `Shift-D` aus Kapitel 3 nutzt ebenfalls `baseline`.

**PROBIER ES SELBST**

Wende `szenarien/06-crash/aufgabe-probe.yaml` an. Der Pod `waechter` läuft, die Anwendung antwortet auf Port 8080, trotzdem steigt `RESTARTS` stetig. Finde die Ursache und korrigiere sie mit `kubectl edit` (in k9s: `e`).

Hinweis: Die Events `Unhealthy` nennen `dial tcp ...:80: connect: connection refused`. Die Liveness-Probe prüft per `tcpSocket` Port 80, httpd lauscht auf 8080.

Aufräumen:

MAC Schritt 22

```
kubectl --context rke2 delete namespace dbg-06
kubectl --context talos delete namespace dbg-06
```

Ephemeral Container lassen sich nicht einzeln entfernen. Sie verschwinden erst mit ihrem Pod.

---

**WAS DU GELERNT HAST**

- `CrashLoopBackOff` ist ein Wartezustand. Ursache: Exit-Code in `Last State` und das Log von `kubectl logs --previous` (k9s: `p`).
- 127 heißt Befehl nicht gefunden (in einer Shell), `StartError` mit 128 heißt: Die Runtime konnte nicht starten. 1 ist ein Fehler der Anwendung.
- `OOMKilled` mit 137 ist ein Kernel-Eingriff an der cgroup-Grenze. Beweis: `dmesg` bzw. `journalctl -k`.
- Liveness startet neu, Readiness nimmt aus dem Service. Langsame Starts gehören in eine Startup-Probe.
- `readOnlyRootFilesystem` braucht `emptyDir`-Volumes an den Stellen, an denen die Anwendung schreibt.
- Ohne Shell: `kubectl debug` mit Ephemeral Container, `--target`, `--custom` und `--copy-to`. Auf Talos immer mit `--profile=baseline`.

---

KAPITEL 7

# Services, DNS und NetworkPolicies

*Du verfolgst eine Anfrage vom Client-Pod über den Namen, den Service und die EndpointSlice bis zum Ziel-Pod. Du erkennst, an welcher Station sie hängen bleibt, und weißt, warum eine NetworkPolicy auf RKE2 wirkt und auf Talos nicht.*

**Dauer:** 75 Minuten **Voraussetzung:** Kapitel 2 und 3

## 7.1 Was in diesem Kapitel kaputtgeht

Eine Anfrage an `http://backend` passiert vier Stationen: Der Name wird per DNS zur IP des Service, kube-proxy leitet die Service-IP auf eine Pod-IP aus der EndpointSlice um, das Paket läuft durch das CNI zum Ziel-Pod, und dort muss ein Prozess auf dem Port lauschen. Jede Station hat ihren eigenen typischen Fehler.

| Szenario | Symptom | Station |
|---|---|---|
| Selektor passt nicht | `Connection refused` oder Timeout, Service ohne Endpoints | EndpointSlice |
| Falscher `targetPort` | `Could not connect to server`, obwohl Endpoints da sind | Ziel-Pod |
| Falscher Name | `Could not resolve host`, `NXDOMAIN` | DNS |
| Default-Deny ohne DNS | Name löst nicht auf, IP funktioniert | NetworkPolicy |
| Policy ohne Wirkung | Alles geht, obwohl es verboten sein sollte | CNI |

Alle Tests laufen aus einem Werkzeug-Pod `client` mit dem Image `nicolaka/netshoot:v0.15`. Darin stecken `curl`, `dig`, `nslookup`, `ss` und `tcpdump`. Als Ziel dient `traefik/whoami`, ein winziger Webserver auf Port 80, der in seiner Antwort den eigenen Pod-Namen nennt. So siehst du sofort, welcher Pod geantwortet hat.

## 7.2 Der Service findet keine Pods

DATEI `szenarien/07-netzwerk/kaputt-selektor.yaml (Ausschnitt)`

```yaml
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 2
  template:
    metadata:
      labels:
        app: backend
# ...
---
kind: Service
metadata:
  name: backend
spec:
  selector:
    app: backend-api        # FEHLER: die Pods heißen app=backend
  ports:
    - name: http
      port: 80
      targetPort: 80
```

Die vollständige Datei legt den Namespace `dbg-07`, den Pod `client`, das Deployment und den Service an. Der Selektor des Service sucht nach einem Label, das kein Pod trägt.

**MAC** Schritt 1

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/kaputt-selektor.yaml
kubectl --context rke2 -n dbg-07 exec client -- curl -sS -m 5 http://backend
```

Legt alles an und ruft den Service aus dem Client-Pod auf. `-m 5` bricht nach fünf Sekunden ab, `-sS` unterdrückt den Fortschrittsbalken, zeigt aber Fehler.

**SYMPTOM**

```
curl: (7) Failed to connect to backend port 80 after 3 ms: Could not connect to server
command terminated with exit code 7
```

Der Name löst auf, sonst käme `Could not resolve host`. Die Verbindung scheitert sofort. Je nach Proxy-Modus und CNI siehst du statt der sofortigen Ablehnung auch einen Timeout nach fünf Sekunden. Beides heißt: Hinter der Service-IP wartet niemand.

Die erste Frage lautet nicht „Was ist mit dem Netzwerk?“, sondern „Hat der Service überhaupt Ziele?“. Die Antwort steht in der EndpointSlice, die der EndpointSlice-Controller für jeden Service mit Selektor pflegt.

**MAC** Schritt 2

```
kubectl --context rke2 -n dbg-07 get endpointslices -l kubernetes.io/service-name=backend
```

Jede EndpointSlice trägt das Label `kubernetes.io/service-name` mit dem Namen ihres Service. Das ältere `kubectl get endpoints` funktioniert noch, gibt seit Kubernetes 1.33 aber eine Deprecation-Warnung aus.

```
NAME            ADDRESSTYPE   PORTS     ENDPOINTS   AGE
backend-7xqzl   IPv4          <unset>   <unset>     41s
```

`<unset>` in der Spalte `ENDPOINTS` heißt: Kein einziger Pod passt. Jetzt vergleichst du Selektor und Labels direkt:

**MAC** Schritt 3

```
kubectl --context rke2 -n dbg-07 get svc backend -o jsonpath='{.spec.selector}{"\n"}'
kubectl --context rke2 -n dbg-07 get pods --show-labels
```

Die erste Zeile zeigt, wonach der Service sucht, die zweite, was die Pods tragen.

```
{"app":"backend-api"}
NAME                       READY   STATUS    RESTARTS   AGE   LABELS
backend-5b8d9f6c7d-4kq2m   1/1     Running   0          58s   app=backend,pod-
template-hash=...
backend-5b8d9f6c7d-xv7jt   1/1     Running   0          58s   app=backend,pod-
template-hash=...
client                     1/1     Running   0          58s   app=client
```

Die Gegenprobe mit dem Selektor des Service liefert eine leere Liste: `kubectl --context rke2 -n dbg-07 get pods -l app=backend-api` antwortet mit `No resources found in dbg-07 namespace.` In k9s siehst du dasselbe als Baum:

**K9S** Schritt 4

```
:xray svc dbg-07⏎
```

XRay zeigt jeden Service mit den Pods darunter, die er erreicht. Unter `backend` hängt nichts. Mit `Esc` und `:pods dbg-07⏎` gelangst du zur Pod-Liste, `y` auf einem Pod zeigt die Labels im YAML.

**URSACHE**

Ein Service ist nur ein Selektor plus eine virtuelle IP. Der EndpointSlice-Controller sucht laufend alle Pods im Namespace, deren Labels zum Selektor passen und die ready sind, und trägt sie in die EndpointSlice ein. Passt kein Pod, bleibt die Slice leer, und kube-proxy hat kein Ziel für die Service-IP. Kubernetes meldet das nirgends als Fehler, es gibt kein Event und keinen Status. Dieselbe leere Slice entsteht, wenn der Selektor stimmt, die Pods aber nicht ready sind (Kapitel 6). `0/1` in `kubectl get pods` spricht für die Probe, `1/1` für den Selektor.

MAC Schritt 5

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/fix-service.yaml
kubectl --context rke2 -n dbg-07 get endpointslices -l kubernetes.io/service-name=backend
```

Ersetzt den Selektor durch `app: backend`. Die Datei verweist außerdem mit `targetPort: http` auf den benannten Port aus dem Pod-Template statt auf eine Zahl.

**DAS SOLLTEST DU SEHEN**

```
NAME            ADDRESSTYPE   PORTS   ENDPOINTS               AGE
backend-7xqzl   IPv4          80      10.42.0.21,10.42.1.17   2m
```

Zwei Pod-IPs, Port 80. `kubectl --context rke2 -n dbg-07 exec client -- curl -s http://backend` antwortet mit `Hostname: backend-5b8d9f6c7d-...`. Rufst du mehrmals auf, wechselt der Name zwischen beiden Pods.

## 7.3 Endpoints da, Verbindung trotzdem abgelehnt

DATEI `szenarien/07-netzwerk/kaputt-port.yaml (Ausschnitt)`

```
spec:
  selector:
    app: backend
  ports:
    - name: http
      port: 80
      targetPort: 8080      # FEHLER: whoami lauscht auf Port 80
```

MAC Schritt 6

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/kaputt-port.yaml
kubectl --context rke2 -n dbg-07 exec client -- curl -sS -m 5 http://backend
```

Der Selektor stimmt diesmal, nur der Zielport ist falsch.

**SYMPTOM**

Wieder `curl: (7) Failed to connect to backend port 80 ... Could not connect to server`. Diesmal ist die EndpointSlice aber gefüllt:

```
NAME            ADDRESSTYPE   PORTS   ENDPOINTS               AGE
backend-7xqzl   IPv4          8080    10.42.0.21,10.42.1.17   4m
```

Die Spalte `PORTS` verrät den Fehler bereits: Die Slice zeigt den Port, auf dem der **Pod** angesprochen wird, nicht den Service-Port. Um sicherzugehen, fragst du den Pod direkt, am Service vorbei:

MAC Schritt 7

```
kubectl --context rke2 -n dbg-07 exec client -- curl -s -o /dev/null -w '%{http_code}\n'
http://10.42.0.21:80
```

Fragt eine Pod-IP aus der Slice direkt auf Port 80 ab und gibt nur den HTTP-Status aus: `200`. Derselbe Aufruf auf Port 8080 wird abgelehnt.

Welche Ports ein Container wirklich geöffnet hat, siehst du mit `ss`. Das whoami-Image enthält keine Shell und kein `ss`. Ein Ephemeral Container mit netshoot teilt sich aber den Netzwerk-Namespace des Pods und sieht dieselben Sockets:

MAC — Schritt 8

```
POD=$(kubectl --context rke2 -n dbg-07 get pod -l app=backend -o name | head -n1)
kubectl --context rke2 -n dbg-07 debug -it "$POD" \
  --image=nicolaka/netshoot:v0.15 --profile=baseline --target=whoami -- ss -tlnp
```

`--target=whoami` hängt den Debug-Container zusätzlich an den Prozess-Namespace des Containers `whoami`, so zeigt `ss` auch den Prozessnamen. `--profile=baseline` ist auf RKE2 nicht nötig, auf Talos aber Pflicht: Das Standardprofil `general` fügt `SYS_PTRACE` hinzu, und das lehnt Talos' PSA `baseline` ab. In k9s erledigt das Plugin aus Kapitel 3 denselben Schritt: In der Container-Ansicht des Pods `Shift-D`.

```
State  Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0      4096               *:80              *:*     users:
(("whoami",pid=1,fd=3))
```

**URSACHE**

`targetPort` ist der Port im Pod, an den kube-proxy die Pakete weiterreicht. Dort lauscht niemand, also antwortet der Kernel des Pods mit einem TCP-Reset. Der Service, die EndpointSlice und die Pods sind alle „gesund", nur ihre Verabredung über den Port stimmt nicht. Benannte Ports (`targetPort: http`) vermeiden diesen Fehler, weil der Port dann nur an einer Stelle steht: im Pod-Template.

MAC — Schritt 9

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/fix-service.yaml
```

Setzt `targetPort` wieder auf den benannten Port `http`.

**DAS SOLLTEST DU SEHEN**

Die EndpointSlice zeigt Port `80`, und `curl -s http://backend` aus dem Client-Pod liefert die whoami-Antwort.

## 7.4 Namen, Namespaces und ndots

DNS-Fehler sehen anders aus als Verbindungsfehler: `curl: (6) Could not resolve host`. Bevor du CoreDNS verdächtigst, schau dir an, wie der Pod Namen auflöst.

MAC Schritt 10

```
kubectl --context rke2 -n dbg-07 exec client -- cat /etc/resolv.conf
```

Diese Datei schreibt das Kubelet beim Start des Pods.

```
search dbg-07.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.43.0.10
options ndots:5
```

Mit `ndots:5` gilt jeder Name mit weniger als fünf Punkten als relativ. Der Resolver hängt dann nacheinander die Suchdomänen an. Aus `backend` wird zuerst `backend.dbg-07.svc.cluster.local`, und das existiert. Aus `backend.dbg07` (Tippfehler, Bindestrich fehlt) wird `backend.dbg07.dbg-07.svc.cluster.local`, dann `backend.dbg07.svc.cluster.local`, dann `backend.dbg07.cluster.local` und zuletzt `backend.dbg07.` als absoluter Name. Alle vier Anfragen enden mit `NXDOMAIN`.

MAC Schritt 11

```
kubectl --context rke2 -n dbg-07 exec client -- nslookup backend.dbg07
kubectl --context rke2 -n dbg-07 exec client -- nslookup backend.dbg-07
```

Der erste Name ist falsch geschrieben, der zweite ist die richtige Kurzform `<service>.<namespace>`.

```
** server can't find backend.dbg07: NXDOMAIN
# ...
Name:   backend.dbg-07.svc.cluster.local
Address: 10.43.118.204
```

Der häufigste Fall in der Praxis ist kein Tippfehler, sondern ein fehlender Namespace: Eine Anwendung in `dbg-07` ruft `postgres` auf, der Service liegt aber in `datenbank`. Der Kurzname funktioniert nur im eigenen Namespace, über Namespace-Grenzen hinweg brauchst du mindestens `postgres.datenbank`. Das lässt sich im Labor gut an CoreDNS selbst zeigen, denn sein Service heißt auf beiden Clustern unterschiedlich:

MAC Schritt 12

```
kubectl --context rke2 -n kube-system get svc -l k8s-app=kube-dns
kubectl --context talos -n kube-system get svc -l k8s-app=kube-dns
```

Beide Distributionen setzen das Label `k8s-app=kube-dns`, die Namen unterscheiden sich.

```
NAME                         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)         AGE
rke2-coredns-rke2-coredns    ClusterIP   10.43.0.10   <none>        53/UDP,53/TCP   3d
NAME       TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)                  AGE
kube-dns   ClusterIP   10.96.0.10   <none>        53/UDP,53/TCP,9153/TCP   3d
```

Ein Skript, das `kube-dns.kube-system` abfragt, funktioniert also auf Talos und scheitert auf RKE2 mit `NXDOMAIN`. Von solchen Unterschieden handelt dieses Buch.

### CoreDNS selbst untersuchen

Erst wenn auch `kubernetes.default` nicht auflöst, liegt das Problem bei CoreDNS oder auf dem Weg dorthin.

MAC — Schritt 13

```
kubectl --context rke2 -n dbg-07 exec client -- dig +short kubernetes.default.svc.cluster.local
kubectl --context rke2 -n kube-system get pods -l k8s-app=kube-dns -o wide
kubectl --context rke2 -n kube-system logs deploy/rke2-coredns-rke2-coredns --tail=20
```

Erst die Funktion, dann die Pods, dann ihre Logs. `dig +short` liefert nur die Adresse, auf RKE2 `10.43.0.1`.

In k9s: `:pods kube-system⏎`, dann `/coredns⏎`, `l` für die Logs und `d` für Restarts und Probes.

Meldungen wie `[ERROR] plugin/errors: 2 example.org. A: read udp ...: i/o timeout` heißen: CoreDNS erreicht seinen Upstream-Server nicht. Das betrifft nur externe Namen. Interne Namen wie `backend.dbg-07.svc.cluster.local` beantwortet CoreDNS selbst aus der Kubernetes-API.

**RKE2**

Deployment und Service heißen `rke2-coredns-rke2-coredns`, die Corefile liegt in der gleichnamigen ConfigMap. Änderungen machst du nicht dort, sondern über eine `HelmChartConfig` namens `rke2-coredns` in `kube-system`, sonst überschreibt der Helm-Controller sie. Service-IP `10.43.0.10`, Upstream ist `/etc/resolv.conf` des Knotens.

**TALOS**

Deployment `coredns`, Service `kube-dns`, ConfigMap `coredns`, Service-IP `10.96.0.10`. Talos aktiviert für neue Cluster `forwardKubeDNSToHost`: CoreDNS schickt externe Anfragen an den DNS-Cache des Knotens unter `169.254.116.108`. Upstream-Fehler suchst du dann auf dem Knoten: `talosctl -n 172.16.0.171 get dnsupstream` zeigt, ob die Upstream-Server erreichbar sind, `talosctl -n 172.16.0.171 logs dns-resolve-cache` die Logs des Caches.

## 7.5 Default-Deny blockiert DNS

Eine NetworkPolicy, die mit `podSelector: {}` alles erfasst und `Egress` nennt, verbietet jeden ausgehenden Verkehr, für den keine andere Policy eine Ausnahme macht. Das schließt DNS ein.

DATEI `szenarien/07-netzwerk/kaputt-policy.yaml (Ausschnitt)`

```
kind: NetworkPolicy
metadata:
  name: default-deny
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

```
---
kind: NetworkPolicy
metadata:
  name: client-zu-backend
spec:
  podSelector:
    matchLabels:
      app: client
  policyTypes:
    - Egress
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: backend
      ports:
        - protocol: TCP
          port: 80
# ... dazu backend-von-client: Ingress auf backend nur von client
```

Gedacht war: Alles ist zu, nur der Client darf zum Backend. Vergessen wurde, dass der Client vorher den Namen `backend` auflösen muss.

**MAC** Schritt 14

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/kaputt-policy.yaml
kubectl --context rke2 -n dbg-07 exec client -- curl -sS -m 10 http://backend
```

Wendet die drei Policies an und testet wieder mit `curl`.

**SYMPTOM**

```
curl: (6) Could not resolve host: backend
command terminated with exit code 6
```

Nach einigen Sekunden Wartezeit kommt die Meldung. Der Aufruf über die Service-IP, etwa `curl -s http://10.43.118.204`, funktioniert dagegen sofort.

Dieses Muster ist eindeutig: Die IP geht, der Name nicht. Zur Bestätigung fragst du den DNS-Server direkt:

**MAC** Schritt 15

```
kubectl --context rke2 -n dbg-07 exec client -- dig +time=2 +tries=1 backend.dbg-07.svc.cluster.local
kubectl --context rke2 -n dbg-07 get networkpolicy
```

Ohne Antwort meldet `dig` einen Timeout. Danach listest du alle Policies im Namespace.

```
;; communications error to 10.43.0.10#53: timed out
;; no servers could be reached

NAME                 POD-SELECTOR   AGE
backend-von-client   app=backend    1m
client-zu-backend    app=client     1m
default-deny         <none>         1m
```

`<none>` bei `default-deny` bedeutet: leerer Selektor, die Policy gilt für jeden Pod im Namespace. Mit `kubectl describe networkpolicy client-zu-backend` siehst du die erlaubten Ziele ausgeschrieben. Port 53 taucht nirgends auf. In k9s öffnest du dafür `:netpol dbg-07⏎` und drückst `d` auf der markierten Policy.

**URSACHE**

NetworkPolicies sind Whitelists und wirken additiv. Sobald irgendeine Policy einen Pod für `Egress` auswählt, ist für ihn jeder ausgehende Verkehr verboten, den keine Policy ausdrücklich erlaubt. Die Anfrage an CoreDNS ist ausgehender Verkehr zu einem Pod in `kube-system` auf UDP- und TCP-Port 53. Weil die Policy nach der Umsetzung der Service-IP auf die Pod-IP greift, muss die Regel die CoreDNS-Pods auswählen, nicht die Service-IP.

DATEI `szenarien/07-netzwerk/fix-policy.yaml (Ausschnitt)`

```yaml
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
```

`namespaceSelector` und `podSelector` stehen im **selben** Listeneintrag, also gilt beides zugleich: CoreDNS-Pods in `kube-system`. Mit einem Bindestrich vor `podSelector` wären es zwei Einträge, und die Regel würde alle Pods in `kube-system` **oder** alle Pods mit `k8s-app=kube-dns` in `dbg-07` erlauben. Das Label `kubernetes.io/metadata.name` setzt Kubernetes auf jedem Namespace automatisch.

MAC — Schritt 16

```
kubectl --context rke2 apply -f szenarien/07-netzwerk/fix-policy.yaml
kubectl --context rke2 -n dbg-07 exec client -- curl -s -m 10 http://backend
```

Ergänzt eine vierte Policy, die DNS für alle Pods im Namespace freigibt.

**DAS SOLLTEST DU SEHEN**

`curl` liefert sofort die whoami-Antwort. Die Gegenprobe zeigt, dass der Rest zu bleibt: `kubectl --context rke2 -n dbg-07 exec client -- curl -sS -m 5 https://kubernetes.io` endet mit einem Timeout, weil `client` nur `backend` erreichen darf.

**STOLPERFALLE**

TCP-Port 53 nicht vergessen. DNS weicht auf TCP aus, wenn eine Antwort nicht in ein UDP-Paket passt. Mit nur UDP funktionieren die meisten Abfragen, einzelne große Antworten scheitern dann sporadisch, und die Fehlersuche wird mühsam.

## 7.6 Die Policy, die nichts tut

Jetzt dasselbe auf Talos. Wende die kaputte Policy an, diesmal mit der Erwartung, dass DNS scheitert:

MAC — Schritt 17

```
kubectl --context talos apply -f szenarien/07-netzwerk/kaputt-selektor.yaml \
  -f szenarien/07-netzwerk/fix-service.yaml -f szenarien/07-netzwerk/kaputt-policy.yaml
kubectl --context talos -n dbg-07 exec client -- curl -s -m 5 http://backend
kubectl --context talos -n dbg-07 exec client -- curl -sS -m 5 -o /dev/null -w '%{http_code}\n' https://
kubernetes.io
```

Baut die Ausgangslage aus diesem Kapitel nach und testet sowohl den erlaubten als auch einen verbotenen Weg. Beim `apply` siehst du Warnungen `would violate PodSecurity "restricted:latest"`. Das sind nur Warnungen, Talos erzwingt `baseline`.

**SYMPTOM**

Beide Aufrufe funktionieren. `curl http://backend` liefert die whoami-Antwort, der Aufruf ins Internet einen HTTP-Status statt eines Timeouts. Die Policies existieren, `kubectl get networkpolicy -n dbg-07` listet alle drei. Es gibt keinen Fehler und keine Warnung.

Die Kubernetes-API speichert NetworkPolicies, setzt sie aber nicht selbst durch. Das macht das CNI-Plugin. Welches CNI läuft, zeigt `kubectl --context talos -n kube-system get ds`: Auf Talos findest du `kube-flannel`, auf RKE2 dagegen `rke2-canal`.

**URSACHE**

Talos installiert standardmäßig Flannel. Flannel baut nur das Overlay-Netz zwischen den Knoten und kennt keine NetworkPolicies. Die API nimmt die Objekte trotzdem an, weil sie zum Kern von Kubernetes gehören. Niemand wertet sie aus. Canal auf RKE2 kombiniert Flannel für das Overlay mit dem Policy-Teil von Calico (Felix), der die Regeln als iptables-Regeln auf jedem Knoten umsetzt.

Die Lösung ist hier kein Patch, sondern eine Architekturentscheidung. Auf Talos setzt du dafür beim Erzeugen der Machine-Config `cluster.network.cni.name: none` und installierst danach Cilium oder Calico selbst. Für einen laufenden Cluster ist der Wechsel des CNI ein Umbau mit Ausfall, kein Fix für zwischendurch. Im Labor bleibt Talos deshalb bei Flannel, und dieses Szenario bleibt eine Demonstration.

**RKE2**

Canal setzt NetworkPolicies durch. Felix schreibt die Regeln in iptables-Ketten mit dem Präfix `cali-`. Auf dem Knoten zählt `sudo iptables-save | grep -c cali-` sie grob durch. Eine neue Policy wirkt nach wenigen Sekunden.

**TALOS**

Flannel ignoriert NetworkPolicies vollständig. Wer auf Talos Policies braucht, muss das CNI beim Aufbau ersetzen (`cni.name: none` plus Cilium oder Calico). Die Talos-Doku beschreibt beide Wege.

**WARUM DAS WICHTIG IST**

Teste jede Policy mit einer Gegenprobe, die scheitern **muss**. Ein grüner Test des erlaubten Weges beweist nur, dass die Policy nicht zu viel verbietet. Ob sie überhaupt etwas verbietet, zeigt dir erst der verbotene Weg. Auf Talos wäre dir der Fehler sonst nie aufgefallen.

MAC — Schritt 18

```
kubectl --context rke2 delete ns dbg-07
kubectl --context talos delete ns dbg-07
```

Räumt beide Cluster auf. Policies, Services und Pods verschwinden mit dem Namespace.

**PROBIER ES SELBST**

Wende `kaputt-selektor.yaml` und `fix-service.yaml` auf RKE2 erneut an. Ergänze dann im Service `backend` den Selektor um ein zweites Label `version: v2` (`kubectl edit svc backend` oder `e` in k9s). Was zeigt die EndpointSlice, was `curl`? Wie bekommst du den Service wieder gefüllt, ohne den Selektor zurückzunehmen?

Hinweis: Die Labels eines Selektors sind UND-verknüpft. Kein Pod trägt `version: v2`, also wird die Slice leer. Ergänze das Label im Pod-Template des Deployments (nicht im `selector` des Deployments, der ist unveränderlich). Nach dem Rollout füllt sich die Slice wieder.

---

**WAS DU GELERNT HAST**

- Zuerst die EndpointSlice prüfen: Ein Service ohne Endpoints ist ein Selektor- oder Readiness-Problem, kein Netzwerkproblem.
- Die Spalte `PORTS` der EndpointSlice zeigt den Port im Pod. Benannte Ports verhindern Abweichungen zwischen Service und Pod.
- `ndots:5` und die Suchdomänen erklären, warum Kurznamen nur im eigenen Namespace funktionieren. CoreDNS heißt auf RKE2 `rke2-coredns-rke2-coredns`, auf Talos `coredns` bzw. `kube-dns`.
- Eine Default-Deny-Policy mit `Egress` braucht eine Ausnahme für UDP und TCP 53 zu den CoreDNS-Pods.
- NetworkPolicies wirken nur, wenn das CNI sie umsetzt: Canal auf RKE2 ja, Flannel auf Talos nein. Jede Policy braucht eine Gegenprobe, die scheitern muss.

---

KAPITEL 8

# Ingress, Gateway und TLS

*Du findest den Ingress-Controller auf beiden Clustern, liest seine Logs und unterscheidest 404, 503 und das falsche Zertifikat nach ihrer Ursache. Auf Talos installierst du Traefik selbst, so dass sich beide Cluster gleich verhalten.*

**Dauer:** 75 Minuten **Voraussetzung:** Kapitel 2 und 7

## 8.1 Was in diesem Kapitel kaputtgeht

Ein Ingress ist nur eine Beschreibung. Erst ein Ingress-Controller macht daraus Routing-Regeln. In diesem Buch ist das auf beiden Clustern Traefik. Zwischen deinem `curl` und dem Pod stehen damit drei Fragen: Fühlt sich der Controller für diesen Ingress zuständig? Findet er Host und Pfad? Hat das Ziel bereite Pods auf dem richtigen Port? Dazu kommt bei HTTPS das Zertifikat.

| Szenario | Symptom | Traefik-Log |
|---|---|---|
| Falsche `ingressClassName` | `404 page not found` | nichts |
| Falscher Service-Port im Ingress | `404 page not found` | `Cannot create service` |
| Keine bereiten Pods | `503 Service Unavailable` | Access-Log mit 503 |
| TLS-Secret mit falschen Keys | Zertifikat `TRAEFIK DEFAULT CERT` | `Error configuring TLS` |
| Abgelaufenes Zertifikat | `certificate has expired` im Client | nichts |

Die Spalte rechts ist der Schlüssel: Manche Fehler meldet Traefik laut, andere gar nicht. Gerade das Schweigen ist ein Hinweis.

## 8.2 Traefik auf beiden Clustern

### RKE2: mitgeliefert

Seit RKE2 1.36 ist Traefik der Standard-Ingress-Controller für neue Cluster. Im Labor steht zusätzlich `ingress-controller: traefik` ausdrücklich in `config-cp.yaml` (Kapitel 2). ingress-nginx ist in 1.36 noch als Option enthalten, aber abgekündigt und fällt mit 1.37 weg. RKE2 installiert Traefik über seinen Helm-Controller als Chart `rke2-traefik` in `kube-system`.

MAC Schritt 1

```
kubectl --context rke2 -n kube-system get helmchart rke2-traefik
kubectl --context rke2 -n kube-system get ds rke2-traefik -o wide
kubectl --context rke2 get ingressclass
```

Zeigt das HelmChart-Objekt, das DaemonSet und die IngressClass.

```
NAME           DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR
AGE
rke2-traefik   2         2         2       2            2           kubernetes.io/
os=linux   3d
NAME      CONTROLLER                       PARAMETERS   AGE
traefik   traefik.io/ingress-controller    <none>       3d
```

Das RKE2-Chart weicht in drei Punkten vom Upstream-Chart ab: Traefik läuft als **DaemonSet**, bindet auf jedem Knoten die **hostPorts** 80 und 443, und der Service ist nur vom Typ `ClusterIP`. Du erreichst Traefik also direkt über die Knoten-IPs `172.16.0.160` und `172.16.0.161`, ohne Load Balancer. Die IngressClass heißt `traefik` und ist die Standardklasse, solange Traefik der gewählte Controller ist.

Eigene Werte für ein mitgeliefertes Chart setzt du nie mit `helm upgrade`, sondern über eine `HelmChartConfig` gleichen Namens. Für die Fehlersuche in diesem Kapitel schaltest du die Access-Logs und das Dashboard ein:

DATEI `werkzeuge/rke2-traefik-config.yaml`

```yaml
apiVersion: helm.cattle.io/v1
kind: HelmChartConfig
metadata:
  name: rke2-traefik
  namespace: kube-system
spec:
  valuesContent: |-
    accessLog:
      enabled: true
    ingressRoute:
      dashboard:
        enabled: true
```

Der Helm-Controller führt diese Werte mit den RKE2-Vorgaben zusammen und rollt das DaemonSet neu aus. Dasselbe erreichst du, wenn du die Datei auf `rke2-cp` nach `/var/lib/rancher/rke2/server/manifests/` legst.

MAC Schritt 2

```
kubectl --context rke2 apply -f werkzeuge/rke2-traefik-config.yaml
kubectl --context rke2 -n kube-system rollout status ds/rke2-traefik
```

Der Helm-Controller startet einen Job `helm-install-rke2-traefik`. Scheitert der, etwa wegen eines YAML-Fehlers in `valuesContent`, steht der Grund in `kubectl logs job/helm-install-rke2-traefik -n kube-system`.

### Talos: selbst installieren

Talos bringt keinen Ingress-Controller mit. Du installierst Traefik aus dem offiziellen Chart, mit Werten, die das RKE2-Verhalten nachbauen.

DATEI `werkzeuge/traefik-values-talos.yaml (Ausschnitt)`

```yaml
deployment:
  kind: DaemonSet
ingressClass:
  enabled: true
  isDefaultClass: true
  name: traefik
ports:
  web:
    hostPort: 80
  websecure:
    hostPort: 443
service:
  spec:
    type: ClusterIP
ingressRoute:
  dashboard:
    enabled: true
accessLog:
  enabled: true
```

Gleiche Klasse, gleiche Ports, gleiches Verhalten wie auf RKE2. Intern lauscht Traefik auf 8000 und 8443 und läuft als UID 65532. Die Abbildung auf 80 und 443 übernimmt das `portmap`-Plugin des CNI.

MAC — Schritt 3

```
kubectl --context talos create namespace traefik
kubectl --context talos label namespace traefik pod-security.kubernetes.io/enforce=privileged
helm --kube-context talos upgrade --install traefik traefik/traefik \
  --version 42.3.1 -n traefik -f werkzeuge/traefik-values-talos.yaml --wait
```

Das Label ist nötig, weil Talos cluster-weit `baseline` erzwingt und `baseline` keine hostPorts erlaubt. Ohne Label nimmt die API das DaemonSet an, aber der Controller kann keine Pods anlegen.

**STOLPERFALLE**

Fehlt das Label, meldet `helm --wait` nur einen Timeout. Den Grund findest du am DaemonSet, nicht am Pod, denn es gibt keinen Pod: `kubectl --context talos -n traefik describe ds traefik` zeigt das Event `FailedCreate` mit `violates PodSecurity "baseline:latest": hostPort (container "traefik" uses hostPorts 80, 443)`. Mehr dazu in Kapitel 10.

**DAS SOLLTEST DU SEHEN**

`curl -s http://172.16.0.171/` und `curl -s http://172.16.0.161/` antworten beide mit `404 page not found`. Das ist hier ein gutes Zeichen: Traefik läuft und antwortet, es gibt nur noch keine Route.

## 8.3 404: Der Controller fühlt sich nicht zuständig

DATEI `szenarien/08-ingress/kaputt-klasse.yaml (Ausschnitt)`

```yaml
kind: Ingress
metadata:
  name: shop
  namespace: dbg-08
spec:
  ingressClassName: nginx     # FEHLER: aus einer alten Anleitung kopiert
  rules:
    - host: shop.dbg08.example
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: shop
                port:
                  number: 80
```

Die Datei legt außerdem den Namespace `dbg-08`, das Deployment `shop` mit zwei whoami-Pods und den Service `shop` an. Der Fehler ist realistisch: Unzählige Anleitungen und Charts setzen noch `nginx` als Klasse.

MAC Schritt 4

```
kubectl --context rke2 apply -f szenarien/08-ingress/kaputt-klasse.yaml
curl -s --resolve shop.dbg08.example:80:172.16.0.161 http://shop.dbg08.example/
```

`--resolve` legt fest, dass `shop.dbg08.example` für diesen Aufruf auf `172.16.0.161` zeigt. So brauchst du keinen DNS-Eintrag, und `curl` schickt trotzdem den richtigen `Host`-Header und bei HTTPS den richtigen SNI-Namen.

**SYMPTOM**

```
404 page not found
```

Die Pods laufen, der Service hat Endpoints, der Ingress existiert. Trotzdem antwortet Traefik mit seiner Standard-404-Seite.

Eine 404 von Traefik heißt: Keine Route passt auf diese Anfrage. Ob das an Host, Pfad oder an der fehlenden Route selbst liegt, verrät der Ingress:

MAC Schritt 5

```
kubectl --context rke2 -n dbg-08 get ingress shop \
  -o custom-columns=NAME:.metadata.name,CLASS:.spec.ingressClassName,HOSTS:.spec.rules[*].host
kubectl --context rke2 get ingressclass
```

Vergleicht die Klasse des Ingress mit den Klassen, die es im Cluster gibt.

```
NAME   CLASS   HOSTS
shop   nginx   shop.dbg08.example
NAME      CONTROLLER                      PARAMETERS   AGE
traefik   traefik.io/ingress-controller   <none>       3d
```

Eine IngressClass `nginx` gibt es nicht. Die Traefik-Logs bleiben leer, kein Event am Ingress. In k9s prüfst du das mit `:ingress dbg-08⏎` und `d`: Die Zeile `Ingress Class:` zeigt `nginx`, unter `Events` steht `<none>`.

**URSACHE**

Traefik verarbeitet nur Ingresses, deren `ingressClassName` auf eine IngressClass mit dem Controller `traefik.io/ingress-controller` zeigt, oder Ingresses ohne Klasse, wenn seine Klasse die Standardklasse ist. Einen Ingress mit fremder Klasse überspringt Traefik still, denn er könnte ja einem anderen Controller gehören. Deshalb gibt es weder Log noch Event, und für die Anfrage existiert einfach keine Route.

MAC Schritt 6

```
kubectl --context rke2 apply -f szenarien/08-ingress/fix-ingress.yaml
curl -s --resolve shop.dbg08.example:80:172.16.0.161 http://shop.dbg08.example/ | head -n2
```

Setzt `ingressClassName: traefik`. Die Klasse ist ein normales Feld und lässt sich ändern, der Ingress muss nicht neu angelegt werden.

**DAS SOLLTEST DU SEHEN**

```
Hostname: shop-6c9b7d5f48-r8wzn
IP: 127.0.0.1
```

Ein Aufruf ohne `--resolve`, also `curl http://172.16.0.161/`, liefert weiterhin 404. Der Ingress verlangt den Host `shop.dbg08.example`, und ohne ihn passt die Regel nicht. Auch das ist eine häufige Ursache für eine 404.

## 8.4 404 trotz richtiger Klasse

DATEI `szenarien/08-ingress/kaputt-port.yaml (Ausschnitt)`

```
spec:
  ingressClassName: traefik
```

```
  rules:
    - host: shop.dbg08.example
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: shop
                port:
                  number: 8080   # FEHLER: der Service hat nur Port 80
```

MAC

Schritt 7

```
kubectl --context rke2 apply -f szenarien/08-ingress/kaputt-port.yaml
curl -s --resolve shop.dbg08.example:80:172.16.0.161 http://shop.dbg08.example/
```

Wieder kommt `404 page not found`. Diesmal ist Traefik aber zuständig.

**SYMPTOM**

Gleiche Antwort wie im vorigen Szenario. Von außen sind beide Fehler nicht zu unterscheiden. Erst das Log von Traefik trennt sie.

MAC

Schritt 8

```
kubectl --context rke2 -n kube-system logs -l app.kubernetes.io/name=rke2-traefik \
  --prefix --since=5m | grep -v '"GET'
```

`-l` liest die Logs aller Traefik-Pods, `--prefix` setzt den Pod-Namen davor, `grep -v` blendet die Access-Log-Zeilen aus. Auf Talos lautet das Label `app.kubernetes.io/name=traefik` im Namespace `traefik`.

```
[pod/rke2-traefik-8kq4d/rke2-traefik] 2026-11-09T19:42:07Z ERR Cannot create service
  error="service port not found" ingress=shop namespace=dbg-08 providerName=kubernetes
  serviceName=shop servicePort=&ServiceBackendPort{Name:,Number:8080,}
```

Die Zeile wiederholt sich bei jeder Konfigurationsänderung im Cluster. In k9s findest du sie mit `:pods kube-system⏎`, `/traefik⏎`, `l` und dann `/` mit dem Suchbegriff `ERR` im Log-Fenster.

**URSACHE**

Traefik sucht im Service `shop` einen Port mit der Nummer 8080 oder einem passenden Namen. Den gibt es nicht, also kann Traefik keinen Backend-Service bauen und lässt den ganzen Pfad weg. Ohne Pfad gibt es keine Route, und die Anfrage endet bei der Standard-404. Die Kubernetes-API prüft diesen Querverweis beim Anlegen des Ingress nicht. Merke: `service.port` im Ingress meint den **Service**-Port, nicht den Container-Port.

MAC

Schritt 9

```
kubectl --context rke2 apply -f szenarien/08-ingress/fix-ingress.yaml
```

Port 80, wie im Service.

**DAS SOLLTEST DU SEHEN**

`curl` mit `--resolve` liefert wieder die whoami-Antwort, und im Log erscheinen keine neuen `ERR`-Zeilen mehr.

## 8.5 503: Die Route steht, aber niemand ist bereit

DATEI `szenarien/08-ingress/kaputt-bereit.yaml (Ausschnitt)`

```yaml
      containers:
        - name: whoami
          image: traefik/whoami:v1.11.0
          readinessProbe:
            tcpSocket:
              port: 8080         # FEHLER: whoami lauscht auf Port 80
            periodSeconds: 5
```

MAC — Schritt 10

```
kubectl --context rke2 -n dbg-08 delete deploy shop
kubectl --context rke2 apply -f szenarien/08-ingress/kaputt-bereit.yaml
kubectl --context rke2 -n dbg-08 get pods
curl -s -i --resolve shop.dbg08.example:80:172.16.0.161 http://shop.dbg08.example/ | head -n1
```

Löscht zuerst das alte Deployment. Bei einem normalen Update blieben die alten, bereiten Pods stehen, weil der Rollout an den neuen hängen bliebe (Kapitel 15), und der Fehler wäre von außen nicht zu sehen.

**SYMPTOM**

```
NAME                    READY   STATUS    RESTARTS   AGE
shop-7f4d8c9b6d-2xkqv   0/1     Running   0          2m
shop-7f4d8c9b6d-tn5wz   0/1     Running   0          2m
HTTP/1.1 503 Service Unavailable
```

Traefik kennt die Route und den Service, aber der Service hat keine bereiten Endpoints. Mit `allowEmptyServices`, das beide Charts einschalten, antwortet Traefik dann mit 503 statt mit 404. Das Access-Log bestätigt es:

```
172.16.0.1 - - [09/Nov/2026:19:51:33 +0000] "GET / HTTP/1.1" 503 19 "-" "curl/8.7.1"
41
  "dbg-08-shop-shop-dbg08-example@kubernetes" "-" 0ms
```

Das vorletzte Feld zeigt die Server-URL, an die Traefik weitergeleitet hat. `"-"` heißt: Es gab keinen Server. Die Ursache liegt also nicht im Ingress, sondern eine Ebene tiefer. Ab hier gilt die Kette aus Kapitel 7: EndpointSlice ansehen, dann die Readiness der Pods.

MAC Schritt 11

```
kubectl --context rke2 -n dbg-08 events --types=Warning
```

Die Probe-Events hängen an den Pods, nicht am Deployment. Sie nennen `Readiness probe failed: dial tcp 10.42.1.31:8080: connect: connection refused`. In k9s: `:events dbg-08`.

**URSACHE**

Die Readiness-Probe fragt einen Port ab, auf dem niemand lauscht. Die Pods laufen, gelten aber nie als bereit. Der EndpointSlice-Controller trägt sie mit `ready: false` ein, Traefik übernimmt nur bereite Endpoints und hat damit keinen Server für den Service.

MAC Schritt 12

```
kubectl --context rke2 apply -f szenarien/08-ingress/fix-bereit.yaml
kubectl --context rke2 -n dbg-08 rollout status deploy/shop
```

Die Probe fragt jetzt per HTTP den Pfad `/health` auf dem benannten Port `http` ab. whoami beantwortet ihn mit `200`.

**DAS SOLLTEST DU SEHEN**

Beide Pods stehen auf `1/1`, `curl -i` liefert `HTTP/1.1 200 OK`, und das Access-Log zeigt eine Server-URL wie `"http://10.42.1.34:80"`.

| Status | Bedeutung bei Traefik |
|---|---|
| 404 | Keine Route: falsche Klasse, falscher Host oder Pfad, Backend-Service nicht baubar |
| 503 | Route da, aber keine bereiten Endpoints |
| 502 | Endpoint da, aber die Verbindung zum Pod scheitert, etwa falscher `targetPort` (Kapitel 7) |
| 504 | Pod nimmt an, antwortet aber nicht rechtzeitig |

## 8.6 TLS: das falsche Zertifikat

Für HTTPS braucht der Ingress ein Secret mit Zertifikat und Schlüssel. Weil Schlüssel nicht ins Repo gehören, erzeugst du sie mit einem Skript.

Das Skript `szenarien/08-ingress/zertifikat.sh` legt einmalig eine Labor-CA an und signiert damit ein Zertifikat für `shop.dbg08.example`, wahlweise gültig (`gueltig`, 30 Tage) oder seit Februar 2025 abgelaufen (`abgelaufen`). Die Dateien landen in `szenarien/08-ingress/tls/`, das per `.gitignore` aus dem Repo herausgehalten wird. Für das abgelaufene Zertifikat nutzt das Skript

`openssl x509 -not_before/-not_after`, das es erst ab OpenSSL 3.4 gibt. Das LibreSSL von macOS reicht nicht, nimm OpenSSL aus Homebrew.

### Secret mit falschen Schlüsselnamen

MAC — Schritt 13

```
szenarien/08-ingress/zertifikat.sh gueltig
kubectl --context rke2 -n dbg-08 create secret generic shop-tls \
  --from-file=cert.pem=szenarien/08-ingress/tls/shop.crt \
  --from-file=key.pem=szenarien/08-ingress/tls/shop.key
kubectl --context rke2 apply -f szenarien/08-ingress/ingress-tls.yaml
```

Ein generisches Secret mit den Dateinamen als Keys. Die API prüft bei Typ `Opaque` nichts. `ingress-tls.yaml` ist der Ingress aus `fix-ingress.yaml` plus ein Abschnitt `tls` mit `secretName: shop-tls`.

MAC — Schritt 14

```
curl -sS --cacert szenarien/08-ingress/tls/ca.crt \
  --resolve shop.dbg08.example:443:172.16.0.161 https://shop.dbg08.example/
openssl s_client -connect 172.16.0.161:443 -servername shop.dbg08.example </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -issuer
```

Erst der Aufruf mit Prüfung gegen die Labor-CA, dann der Blick auf das Zertifikat, das Traefik wirklich ausliefert.

**SYMPTOM**

```
curl: (60) SSL certificate problem: self-signed certificate
subject=CN=TRAEFIK DEFAULT CERT
issuer=CN=TRAEFIK DEFAULT CERT
```

Traefik liefert sein selbst erzeugtes Standardzertifikat aus, nicht das aus dem Secret. Mit `curl -k` funktioniert die Seite, und genau deshalb fällt der Fehler im Browser oft erst spät auf.

MAC — Schritt 15

```
kubectl --context rke2 -n kube-system logs -l app.kubernetes.io/name=rke2-traefik \
  --since=5m | grep 'configuring TLS'
kubectl --context rke2 -n dbg-08 get secret shop-tls -o jsonpath='{.type}{"\n"}'
```

Traefik nennt den Fehler wörtlich. Das Secret verrät seinen Typ, `kubectl describe secret shop-tls` zeigt die Keys `cert.pem` und `key.pem`.

```
2026-11-09T20:03:12Z ERR Error configuring TLS error="secret dbg-08/shop-tls is
missing
  the following TLS data entries: tls.crt, tls.key" ingress=shop namespace=dbg-08
providerName=kubernetes
Opaque
```

**URSACHE**

Ingress-Controller erwarten ein Secret im Format `kubernetes.io/tls` mit genau den Keys `tls.crt` und `tls.key`. Traefik findet die Keys nicht, verwirft das Zertifikat und nimmt für den Host das Standardzertifikat. Die Route selbst bleibt aktiv, deshalb funktioniert HTTPS mit `-k` weiter.

MAC — Schritt 16

```
kubectl --context rke2 -n dbg-08 delete secret shop-tls
kubectl --context rke2 -n dbg-08 create secret tls shop-tls \
  --cert=szenarien/08-ingress/tls/shop.crt --key=szenarien/08-ingress/tls/shop.key
```

`create secret tls` setzt Typ und Keys richtig. Bei diesem Typ lehnt die API ein Secret ohne `tls.crt` oder `tls.key` gleich ab. Traefik bemerkt das neue Secret von selbst, ein Neustart ist nicht nötig.

**DAS SOLLTEST DU SEHEN**

`curl --cacert ... https://shop.dbg08.example/` liefert die whoami-Antwort ohne `-k`, und `openssl s_client` zeigt `subject=CN=shop.dbg08.example` und `issuer=CN=dbg-08 Labor-CA`.

### Abgelaufenes Zertifikat

MAC — Schritt 17

```
szenarien/08-ingress/zertifikat.sh abgelaufen
kubectl --context rke2 -n dbg-08 create secret tls shop-tls \
  --cert=szenarien/08-ingress/tls/shop.crt --key=szenarien/08-ingress/tls/shop.key \
  --dry-run=client -o yaml | kubectl --context rke2 apply -f -
curl -sS --cacert szenarien/08-ingress/tls/ca.crt \
  --resolve shop.dbg08.example:443:172.16.0.161 https://shop.dbg08.example/
```

Ersetzt das Secret durch das abgelaufene Zertifikat. `--dry-run=client -o yaml | apply` ist das übliche Muster, um ein bestehendes Secret zu überschreiben.

**SYMPTOM**

```
curl: (60) SSL certificate problem: certificate has expired
```

Traefik loggt nichts. Für ihn ist das Secret formal in Ordnung, und er liefert das Zertifikat aus.

Hier hilft nur der Blick auf das ausgelieferte Zertifikat selbst. `openssl s_client ... | openssl x509 -noout -subject -dates` zeigt `notAfter=Feb  1 00:00:00 2025 GMT`. Dasselbe Datum steckt im Secret, das du ohne Netzwerk prüfen kannst:

MAC — Schritt 18

```
kubectl --context rke2 -n dbg-08 get secret shop-tls -o jsonpath='{.data.tls\.crt}' \
  | base64 -d | openssl x509 -noout -enddate -checkend 0
```

`-checkend 0` prüft, ob das Zertifikat jetzt noch gilt, und antwortet mit `Certificate will expire` und Exit-Code 1. Mit `-checkend 1209600` prüfst du die nächsten 14 Tage. Das eignet sich für einen Cronjob oder eine Alarmregel.

**URSACHE**

Weder Kubernetes noch Traefik prüfen die Laufzeit eines Zertifikats. Das ist Sache des Clients. Ohne cert-manager oder einen anderen Erneuerungsmechanismus läuft jedes Zertifikat irgendwann still ab, und der erste Hinweis ist ein Fehler beim Nutzer.

Zur Behebung erzeugst du mit `zertifikat.sh gueltig` ein neues Zertifikat von derselben CA und ersetzt das Secret mit demselben `--dry-run=client -o yaml | apply`-Muster.

**DAS SOLLTEST DU SEHEN**

`curl --cacert ...` liefert wieder die whoami-Antwort, `-checkend 0` meldet `Certificate will not expire`.

**RKE2**

Alle Szenarien laufen wie gezeigt. Traefik-Pods im Namespace `kube-system`, Label `app.kubernetes.io/name=rke2-traefik`. Werte änderst du über die `HelmChartConfig` `rke2-traefik`, nie mit `helm upgrade`, sonst überschreibt der Helm-Controller deine Änderung.

**TALOS**

Ersetze `--context rke2` durch `--context talos` und `172.16.0.161` durch `172.16.0.171`. Traefik-Pods im Namespace `traefik`, Label `app.kubernetes.io/name=traefik`. Werte änderst du mit `helm upgrade` und `werkzeuge/traefik-values-talos.yaml`.

## 8.7 Das Dashboard und die Gateway API

Das Traefik-Dashboard zeigt alle Router, Services und Middlewares, die Traefik aus den Ingresses gebaut hat, mit Fehlern und Warnungen. Es läuft auf dem internen Entrypoint `traefik` (Port 8080) und ist von außen nicht erreichbar:

MAC — Schritt 19

```
kubectl --context rke2 -n kube-system port-forward ds/rke2-traefik 8080:8080
```

Danach öffnest du `http://localhost:8080/dashboard/` im Browser. Auf Talos lautet der Befehl `kubectl --context talos -n traefik port-forward ds/traefik 8080:8080`. In k9s erreichst du dasselbe mit `Shift-f` auf einem Traefik-Pod.

Ein Ingress, der im Dashboard unter **HTTP Routers** fehlt, wurde nicht verarbeitet.

Die Gateway API ist der Nachfolger von Ingress: `GatewayClass` und `Gateway` gehören dem Plattform-Team, `HTTPRoute` dem Anwendungsteam. RKE2 1.36 bringt die Gateway-API-CRDs mit Traefik mit, der Gateway-Provider von Traefik ist aber aus. Du schaltest ihn mit `providers.kubernetesGateway.enabled: true` in der `HelmChartConfig` ein. Ab RKE2 1.37 kommen die CRDs aus einem eigenen Chart, auf Talos installierst du sie selbst. Für die Fehlersuche ist die Gateway API angenehmer, weil sie Status-Conditions setzt: `kubectl describe httproute` zeigt `Accepted` und `ResolvedRefs` mit Gründen wie `BackendNotFound` oder `NotAllowedByListeners` statt einer stillen 404.

MAC — Schritt 20

```
kubectl --context rke2 delete ns dbg-08
rm -f szenarien/08-ingress/tls/*.crt szenarien/08-ingress/tls/*.key szenarien/08-ingress/tls/*.srl
```

Räumt den Namespace auf und löscht die erzeugten Zertifikate und Schlüssel.

**PROBIER ES SELBST**

Spiele das Szenario aus „404: Der Controller fühlt sich nicht zuständig“ auf Talos nach, aber entferne die Zeile `ingressClassName` ganz, statt sie auf `traefik` zu setzen. Funktioniert der Ingress? Warum?

Hinweis: Ein Ingress ohne Klasse bekommt beim Anlegen die Standardklasse, also die IngressClass mit der Annotation `ingressclass.kubernetes.io/is-default-class: "true"`. In `traefik-values-talos.yaml` setzt `isDefaultClass: true` genau diese Annotation. Gibt es keine Standardklasse, bleibt das Feld leer, und ob sich ein Controller zuständig fühlt, hängt von seiner Konfiguration ab. Setz die Klasse deshalb immer ausdrücklich.

---

**WAS DU GELERNT HAST**

- RKE2 1.36 bringt Traefik als DaemonSet mit hostPorts 80/443 und IngressClass `traefik` mit. Werte setzt du per `HelmChartConfig` `rke2-traefik`. Auf Talos installierst du dasselbe Verhalten per Helm, im Namespace mit PSA `privileged`.
- `curl --resolve` testet Host-Regeln und SNI ohne DNS-Eintrag.
- 404 ohne Log: Traefik ist nicht zuständig (Klasse) oder Host und Pfad passen nicht. 404 mit `Cannot create service`: Der Ingress zeigt auf einen Port, den der Service nicht hat.
- 503 heißt bei Traefik: Route da, aber keine bereiten Endpoints. Weiter geht es mit der EndpointSlice und der Readiness.
- `TRAEFIK DEFAULT CERT` deutet auf ein unbrauchbares TLS-Secret hin. Abgelaufene Zertifikate meldet nur der Client, prüfen kannst du sie mit `openssl x509 -checkend`.

---

KAPITEL 9

# Storage

*Du liest die Events von PVC, PV und Provisioner, erkennst, warum ein Volume nicht entsteht oder nicht eingebunden wird, und findest die Daten eines Volumes auf dem Knoten wieder, auf RKE2 per SSH und auf Talos per* `talosctl`.

**Dauer:** 75 Minuten **Voraussetzung:** Kapitel 2 (StorageClass `local-path`) und Kapitel 4

## 9.1 Was in diesem Kapitel kaputtgeht

Beide Labor-Cluster nutzen den local-path-provisioner aus Kapitel 2. Er legt für jeden PersistentVolumeClaim ein Verzeichnis auf genau einem Knoten an und erzeugt dazu ein PersistentVolume vom Typ `hostPath`, das an diesen Knoten gebunden ist. Das ist einfach und schnell, hat aber Eigenheiten, die in Produktion mit Netzwerk-Storage anders aussehen. Beides kommt in diesem Kapitel vor.

| Szenario | Symptom | Zu sehen an |
|---|---|---|
| Falscher StorageClass-Name | PVC und Pod `Pending` | Events am PVC |
| Provisioner ohne `privileged` (Talos) | PVC `Pending`, `ProvisioningFailed` | Events am PVC |
| RWO-Volume, zwei Knoten | zweite Replika `Pending` | Events am Pod |
| Falsche Rechte im Volume | `Permission denied`, CrashLoopBackOff | Logs des Pods |
| emptyDir läuft über | Pod wird geräumt, neuer startet | Status und Events |

Die Reihenfolge der Fehlersuche ist immer gleich: Pod, dann PVC, dann PV, dann Provisioner, dann Knoten. Die ersten drei Stationen zeigt `kubectl get pod,pvc,pv`, die vierte das Log des Provisioners in `local-path-storage`.

**WARUM DAS WICHTIG IST**

Die StorageClass `local-path` nutzt `volumeBindingMode: WaitForFirstConsumer`. Ein neuer PVC steht deshalb auf `Pending`, bis ein Pod ihn nutzt, und das Event lautet `waiting for first consumer to be created before binding`. Das ist kein Fehler. Erst wenn der Scheduler einen Knoten für den Pod gewählt hat, legt der Provisioner das Volume genau dort an. Ein `Pending`-PVC ist also nur dann verdächtig, wenn schon ein Pod auf ihn wartet.

## 9.2 Der PVC findet seine StorageClass nicht

DATEI `szenarien/09-storage/kaputt-klasse.yaml (Ausschnitt)`

```
kind: PersistentVolumeClaim
metadata:
  name: daten
  namespace: dbg-09
spec:
  storageClassName: local-paht    # FEHLER: Tippfehler
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Die Datei legt außerdem den Namespace `dbg-09` und einen Pod `app` an, der den PVC unter `/data` einbindet.

MAC — Schritt 1

```
kubectl --context rke2 apply -f szenarien/09-storage/kaputt-klasse.yaml
kubectl --context rke2 -n dbg-09 get pod,pvc
```

Legt PVC und Pod an und zeigt beide.

**SYMPTOM**

```
NAME      READY   STATUS    RESTARTS   AGE
pod/app   0/1     Pending   0          35s

NAME                           STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
AGE
persistentvolumeclaim/daten   Pending                                       local-paht
35s
```

Beide hängen. Der Pod wartet auf den PVC, also beginnt die Suche beim PVC:

MAC — Schritt 2

```
kubectl --context rke2 -n dbg-09 describe pvc daten | tail -n 4
kubectl --context rke2 get storageclass
```

Die letzten Zeilen von `describe` sind die Events. Danach die Liste aller StorageClasses.

```
  Type     Reason              Age               From                         Message
  ----     ------              ----              ----                         -------
  Warning  ProvisioningFailed  4s (x4 over 49s)  persistentvolume-controller
storageclass.storage.k8s.io "local-paht" not found
NAME         PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION   AGE
local-path   rancher.io/local-path   Delete          WaitForFirstConsumer   false
3d
```

Am Pod steht das Event `0/2 nodes are available: pod has unbound immediate PersistentVolumeClaims.` Der Scheduler behandelt einen PVC mit unbekannter Klasse wie einen mit sofortiger Bindung, der eben noch nicht gebunden ist. In k9s findest du dieselben Events mit `:pvc dbg-09⏎` und `d` auf dem PVC.

**URSACHE**

Der PersistentVolume-Controller sucht die StorageClass `local-paht`, um den passenden Provisioner zu finden. Die gibt es nicht, also wird nie ein Volume angelegt, und der PVC bleibt `Pending`. Fehlt `storageClassName` ganz, gibt es eine ähnliche Sackgasse: Beide Labor-Cluster haben keine Default-StorageClass, der PVC wartet dann auf ein passendes, manuell angelegtes PV (Kapitel 4).

Der naheliegende Fix ist ein `kubectl edit`. Die API lehnt ihn ab:

```
error: persistentvolumeclaims "daten" is invalid: spec: Forbidden: spec is immutable
after
creation except resources.requests and volumeAttributesClassName for bound claims
```

Die Spec eines PVC ist nach dem Anlegen unveränderlich. Du löschst deshalb Pod und PVC und legst beide neu an. Im Labor gehen dabei keine Daten verloren, es gab ja nie ein Volume.

**MAC** Schritt 3

```
kubectl --context rke2 -n dbg-09 delete pod app
kubectl --context rke2 -n dbg-09 delete pvc daten
kubectl --context rke2 apply -f szenarien/09-storage/fix-klasse.yaml
kubectl --context rke2 -n dbg-09 get pvc daten
```

Erst den Pod löschen, sonst hält der Finalizer `kubernetes.io/pvc-protection` den PVC fest. `fix-klasse.yaml` enthält PVC und Pod, diesmal mit `storageClassName: local-path`.

**DAS SOLLTEST DU SEHEN**

```
NAME    STATUS   VOLUME                                     CAPACITY   ACCESS MODES
STORAGECLASS   AGE
daten   Bound    pvc-3f1c2a9e-8b7d-4e21-9c55-1d0a6b2e7f43   1Gi        RWO
local-path     12s
```

`kubectl --context rke2 -n dbg-09 events --for pvc/daten` zeigt den Ablauf: `WaitForFirstConsumer`, `ExternalProvisioning`, `Provisioning` und `ProvisioningSucceeded` vom Provisioner `rancher.io/local-path`.

## 9.3 Talos: Der Helper-Pod darf nicht starten

Der local-path-provisioner legt das Verzeichnis nicht selbst an. Er startet dafür kurz einen Helper-Pod im Namespace `local-path-storage`, der das Verzeichnis des Knotens per `hostPath` einbindet. Genau das verbietet der Pod-Security-Standard `baseline`, den Talos cluster-weit erzwingt. In

Kapitel 2 hast du den Namespace deshalb mit `pod-security.kubernetes.io/enforce=privileged` gekennzeichnet. Was passiert ohne dieses Label, zeigt dieses Szenario.

MAC Schritt 4

```
kubectl --context talos label namespace local-path-storage pod-security.kubernetes.io/enforce-
kubectl --context talos apply -f szenarien/09-storage/pvc-test.yaml
```

Das Minus am Ende entfernt das Label. Danach gilt wieder der Talos-Standard `baseline`. `pvc-test.yaml` enthält nur einen PVC `probe` und einen Pod, der ihn nutzt.

**SYMPTOM**

Pod und PVC bleiben `Pending`. Diesmal existiert die StorageClass, und das Event `WaitForFirstConsumer` ist längst vorbei, denn der Pod hat einen Knoten bekommen.

MAC Schritt 5

```
kubectl --context talos -n dbg-09 events --for pvc/probe
```

Die Events des PVC, chronologisch sortiert.

```
LAST SEEN   TYPE      REASON                 OBJECT                         MESSAGE
40s         Normal    WaitForFirstConsumer   PersistentVolumeClaim/probe    waiting for
first consumer to be created before binding
38s         Normal    Provisioning           PersistentVolumeClaim/probe    External
provisioner is provisioning volume for claim "dbg-09/probe"
38s         Warning   ProvisioningFailed     PersistentVolumeClaim/probe    failed to
provision volume with StorageClass "local-path": failed to create volume pvc-8e2d...:
pods "helper-pod-create-pvc-8e2d..." is forbidden: violates PodSecurity
"baseline:latest": hostPath volumes (volume "data")
```

Die Meldung nennt alles: den Helper-Pod, den Standard `baseline` und das verbotene Feld. Der Provisioner wiederholt den Versuch mit wachsendem Abstand. Im Log des Provisioners siehst du jeden Versuch:

MAC Schritt 6

```
kubectl --context talos -n local-path-storage logs deploy/local-path-provisioner --tail=20
```

In k9s: `:pods local-path-storage⏎`, Pod markieren, `l`. Die Zeile `create the helper pod helper-pod-create-pvc-... into local-path-storage` steht direkt vor der Fehlermeldung.

**URSACHE**

Pod Security Admission prüft jeden Pod beim Anlegen gegen den Standard des Namespace. Der Helper-Pod braucht ein `hostPath`-Volume, um auf dem Knoten ein Verzeichnis anzulegen, und `baseline` verbietet `hostPath` vollständig. Die API lehnt den Pod ab, der Provisioner kann das Volume nicht anlegen. Die Anwendung selbst ist nicht betroffen: Ihr Pod bindet nur einen PVC ein, kein `hostPath`, und darf deshalb in einem `baseline`-Namespace laufen.

**MAC** Schritt 7

```
kubectl --context talos label namespace local-path-storage \
  pod-security.kubernetes.io/enforce=privileged
kubectl --context talos -n dbg-09 get pvc probe -w
```

Setzt das Label wieder. Beim nächsten Versuch des Provisioners klappt es, `-w` zeigt den Wechsel auf `Bound`. Beenden mit `Ctrl-C`.

**DAS SOLLTEST DU SEHEN**

Der PVC ist `Bound`, der Pod `Running`. Auf dem Knoten liegt jetzt das Verzeichnis, das der Provisioner angelegt hat:

```
$ talosctl -n 172.16.0.171 ls /var/mnt/local-path-provisioner
NODE           NAME
172.16.0.171   .
172.16.0.171   pvc-8e2d4b1a-77c3-4f0e-a9d2-5b6c0e1f3a88_dbg-09_probe
```

Steht der Pod auf `talos-cp`, fragst du `-n 172.16.0.170`. Den Knoten nennt `kubectl get pod probe -o wide`.

**RKE2**

RKE2 ohne CIS-Profil erzwingt keinen Pod-Security-Standard. Der Helper-Pod startet auch ohne Label. Mit `profile: cis` gilt dagegen cluster-weit `restricted`, und du brauchst eine Ausnahme für `local-path-storage` (Kapitel 10). Die Volumes liegen unter `/opt/local-path-provisioner`, dem Standardpfad des Provisioners.

**TALOS**

Talos erzwingt `baseline` in allen Namespaces außer `kube-system`. Der Namespace des Provisioners braucht `enforce=privileged`. Die Volumes liegen im User-Volume `local-path-provisioner` unter `/var/mnt/local-path-provisioner` auf der EPHEMERAL-Partition. Den Zustand zeigt `talosctl -n 172.16.0.171 get volumestatus u-local-path-provisioner`.

## 9.4 Zwei Replikas, ein Volume, zwei Knoten

DATEI szenarien/09-storage/kaputt-rwo.yaml (Ausschnitt)

```yaml
kind: Deployment
metadata:
  name: cache
spec:
  replicas: 2
  template:
    spec:
      affinity:
        podAntiAffinity:         # jede Replika auf einen anderen Knoten
          requiredDuringSchedulingIgnoredDuringExecution:
            - topologyKey: kubernetes.io/hostname
              labelSelector:
                matchLabels:
                  app: cache
      # ...
      volumes:
        - name: daten
          persistentVolumeClaim:
            claimName: cache     # FEHLER: beide Replikas teilen ein RWO-Volume
```

Der PVC `cache` hat `accessModes: [ReadWriteOnce]`. Die Anti-Affinity soll die Replikas auf beide Knoten verteilen, damit der Ausfall eines Knotens nicht beide trifft.

MAC — Schritt 8

```
kubectl --context rke2 apply -f szenarien/09-storage/kaputt-rwo.yaml
kubectl --context rke2 -n dbg-09 get pods -l app=cache -o wide
```

`-o wide` zeigt, auf welchem Knoten jeder Pod gelandet ist.

**SYMPTOM**

```
NAME                     READY   STATUS    RESTARTS   AGE   IP           NODE      ...
cache-6f8d7b9c5d-hx2lp   1/1     Running   0          40s   10.42.1.41   rke2-w1   ...
cache-6f8d7b9c5d-q9m4z   0/1     Pending   0          40s   <none>       <none>    ...
```

MAC — Schritt 9

```
kubectl --context rke2 -n dbg-09 describe pod cache-6f8d7b9c5d-q9m4z | grep -A3 Events
```

Die Events des wartenden Pods. In k9s: `:pods dbg-09⏎`, den Pending-Pod markieren, `d`.

```
  Warning  FailedScheduling  25s  default-scheduler  0/2 nodes are available: 1
node(s) didn't match
  PersistentVolume's node affinity, 1 node(s) didn't match pod anti-affinity rules.
preemption: 0/2
  nodes are available: 2 Preemption is not helpful for scheduling.
```

Lies die Meldung Knoten für Knoten: `rke2-w1` scheidet aus, weil dort schon eine Replika läuft (Anti-Affinity). `rke2-cp` scheidet aus, weil das PV nur auf `rke2-w1` existiert. Die Node-Affinity

des PV bestätigt das: `kubectl --context rke2 get pv -o yaml` zeigt unter `spec.nodeAffinity` den Knoten `rke2-w1`.

**URSACHE**

Ein local-path-Volume ist ein Verzeichnis auf genau einem Knoten. Das PV trägt deshalb eine Node-Affinity, und jeder Pod, der es nutzt, muss auf diesen Knoten. Die Anti-Affinity verlangt das Gegenteil. Beides zugleich ist unmöglich. `ReadWriteOnce` heißt übrigens „von einem **Knoten** beschreibbar“, nicht „von einem Pod“. Ohne Anti-Affinity liefen beide Replikas auf `rke2-w1` und teilten sich das Verzeichnis, was für die meisten Anwendungen ebenso falsch wäre.

**WARUM DAS WICHTIG IST**

Mit Netzwerk-Storage wie Longhorn, Ceph RBD oder Cloud-Disks sieht derselbe Fehler anders aus. Der Scheduler setzt die zweite Replika auf den anderen Knoten, das Volume ist aber schon am ersten Knoten angehängt. Der Pod bleibt in `ContainerCreating`, und das Event lautet `Multi-Attach error for volume "pvc-..." Volume is already used by pod(s) ...`. Ursache und Lösung sind dieselben. Wer garantieren will, dass wirklich nur ein Pod schreibt, nutzt `ReadWriteOncePod`.

Die Lösung: Jede Replika bekommt ein eigenes Volume. Genau dafür gibt es das StatefulSet mit `volumeClaimTemplates`.

DATEI `szenarien/09-storage/fix-rwo.yaml (Ausschnitt)`

```yaml
kind: StatefulSet
metadata:
  name: cache
spec:
  serviceName: cache
  replicas: 2
  # ... gleiche Anti-Affinity und gleicher Container
  volumeClaimTemplates:
    - metadata:
        name: daten
      spec:
        storageClassName: local-path
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 1Gi
```

Das StatefulSet erzeugt die PVCs `daten-cache-0` und `daten-cache-1`. Jeder wird erst angelegt, wenn sein Pod einen Knoten hat, und landet dank `WaitForFirstConsumer` auf diesem Knoten. Die Datei enthält außerdem den Headless-Service, den `serviceName` verlangt.

MAC Schritt 10

```
kubectl --context rke2 -n dbg-09 delete deploy cache
kubectl --context rke2 -n dbg-09 delete pvc cache
kubectl --context rke2 apply -f szenarien/09-storage/fix-rwo.yaml
kubectl --context rke2 -n dbg-09 get pods,pvc -l app=cache -o wide
```

Altes Deployment und alten PVC entfernen, StatefulSet anlegen. Der StatefulSet-Controller kopiert die `matchLabels` des Selektors auf jeden PVC aus `volumeClaimTemplates`. Deshalb findet `-l app=cache` Pods und PVCs zugleich.

**DAS SOLLTEST DU SEHEN**

`cache-0` und `cache-1` laufen auf verschiedenen Knoten, `kubectl get pvc -n dbg-09` zeigt `daten-cache-0` und `daten-cache-1` als `Bound`. Die PVCs bleiben stehen, wenn du das StatefulSet löschst. So überleben die Daten auch ein `delete sts`.

## 9.5 Permission denied auf dem Volume

DATEI `szenarien/09-storage/kaputt-rechte.yaml (Ausschnitt)`

```yaml
      initContainers:
        - name: vorbereiten       # läuft als root und legt das Verzeichnis an
          image: busybox:1.37
          command: ["sh", "-c", "mkdir -p /data/notizen"]
      containers:
        - name: app
          image: busybox:1.37
          command: ["sh", "-c", "date >> /data/notizen/log.txt && sleep infinity"]
          securityContext:
            runAsUser: 1000
            runAsGroup: 1000
```

Ein typisches Muster nach dem Härten: Die Anwendung läuft nicht mehr als root, ein älterer Init-Container schon.

MAC Schritt 11

```
kubectl --context rke2 apply -f szenarien/09-storage/kaputt-rechte.yaml
kubectl --context rke2 -n dbg-09 get pods -l app=notizen
kubectl --context rke2 -n dbg-09 logs deploy/notizen -c app --previous
```

`--previous` zeigt die Ausgabe des letzten abgestürzten Containers. In k9s: Pod markieren, `l`, dann `p` für die vorherigen Logs.

**SYMPTOM**

```
NAME                       READY   STATUS             RESTARTS      AGE
notizen-7c8f9d6b5f-l4wqk   0/1     CrashLoopBackOff   3 (25s ago)   80s
sh: can't create /data/notizen/log.txt: Permission denied
```

Wem gehört das Verzeichnis? Das zeigt ein Blick auf den Knoten. Wo das Volume liegt, steht im PV:

MAC Schritt 12

```
PV=$(kubectl --context rke2 -n dbg-09 get pvc notizen -o jsonpath='{.spec.volumeName}')
kubectl --context rke2 get pv "$PV" -o jsonpath='{.spec.hostPath.path}{"\n"}'
kubectl --context rke2 -n dbg-09 get pod -l app=notizen -o wide
```

Der Pfad des Verzeichnisses auf dem Knoten und der Knoten selbst.

RKE2-W1 Schritt 13

```
sudo ls -ln /opt/local-path-provisioner/pvc-5a0e..._dbg-09_notizen/
```

Auf RKE2 schaust du direkt per SSH nach. `-n` zeigt numerische UIDs.

```
drwxr-xr-x 2 0 0 4096 Nov  9 20:31 notizen
```

Auf Talos fragst du dasselbe mit `talosctl -n 172.16.0.171 ls -l /var/mnt/local-path-provisioner/pvc-..._dbg-09_notizen`. Die Spalten `UID` und `GID` zeigen ebenfalls `0`.

**URSACHE**

Das Wurzelverzeichnis des Volumes legt der Provisioner mit den Rechten `0777` an, dort darf jeder schreiben. Das Unterverzeichnis `notizen` hat aber der Init-Container als root mit `0755` angelegt. Die Anwendung läuft als UID 1000 und darf darin nichts anlegen. Der Fehler liegt also nicht in Kubernetes, sondern in den Dateirechten auf dem Volume.

Der Reflex wäre `fsGroup: 1000` im `securityContext` des Pods. Bei vielen Volume-Typen ändert der Kubelet damit beim Einbinden die Gruppe aller Dateien. Hier nicht:

**STOLPERFALLE**

local-path legt standardmäßig PVs vom Typ `hostPath` an, und für `hostPath` wendet der Kubelet `fsGroup` nicht an. Der Pod startet mit `fsGroup`, die Rechte auf dem Volume bleiben aber unverändert, und der Fehler bleibt. Das kostet viel Zeit, wenn man es nicht weiß. Bei den meisten CSI-Treibern funktioniert `fsGroup` dagegen.

Die zuverlässige Lösung: Wer das Verzeichnis anlegt, übergibt es dem Benutzer der Anwendung.

MAC Schritt 14

```
kubectl --context rke2 apply -f szenarien/09-storage/fix-rechte.yaml
kubectl --context rke2 -n dbg-09 rollout status deploy/notizen
```

Der Init-Container führt jetzt `mkdir -p /data/notizen && chown -R 1000:1000 /data/notizen` aus. Er läuft weiterhin als root. Das ist für einen Einmal-Schritt in Ordnung, verletzt aber den Standard `restricted` (Kapitel 10).

**DAS SOLLTEST DU SEHEN**

Der Pod ist `1/1 Running`, und `kubectl --context rke2 -n dbg-09 exec deploy/notizen -c app -- cat /data/notizen/log.txt` zeigt einen Zeitstempel. `ls -ln` auf dem Knoten zeigt jetzt `1000 1000` für `notizen`.

## 9.6 Wenn der Platz ausgeht

Nicht jeder Speicher ist ein PVC. Ein `emptyDir` liegt auf der Platte des Knotens unter `/var/lib/kubelet/pods/`, und der Kubelet überwacht seine Größe. Überschreitet ein Pod sein Limit, räumt der Kubelet ihn: Er wird **evicted**.

DATEI `szenarien/09-storage/kaputt-platte.yaml (Ausschnitt)`

```yaml
      containers:
        - name: export
          image: busybox:1.37
          command: ["sh", "-c", "dd if=/dev/zero of=/cache/export.tmp bs=1M count=80 && sleep infinity"]
          volumeMounts:
            - name: cache
              mountPath: /cache
      volumes:
        - name: cache
          emptyDir:
            sizeLimit: 50Mi        # FEHLER: kleiner als die Zwischendaten
```

MAC — Schritt 15

```
kubectl --context talos apply -f szenarien/09-storage/kaputt-platte.yaml
kubectl --context talos -n dbg-09 get pods -l app=export -w
```

Diesmal auf Talos, das Verhalten ist auf beiden Clustern gleich. Nach einigen Sekunden wird der erste Pod geräumt, und das Deployment startet einen neuen.

**SYMPTOM**

Nach kurzer Zeit verschwindet der laufende Pod, ein neuer startet, und die alten bleiben mit `0/1` stehen. Ihr Status heißt je nach Zeitpunkt `Error`, `ContainerStatusUnknown` oder `Evicted`. `RESTARTS` steht überall auf 0, denn es ist kein Neustart, sondern ein neuer Pod.

MAC

Schritt 16

```
kubectl --context talos -n dbg-09 get pods -l app=export \
  -o custom-
columns=NAME:.metadata.name,PHASE:.status.phase,REASON:.status.reason,MESSAGE:.status.message
```

`status.reason` und `status.message` erklären jede Räumung. In k9s: `:events dbg-09⏎`, dort steht der Grund `Evicted`.

```
NAME                      PHASE     REASON    MESSAGE
export-6d5c8f7b9d-9kq2x   Failed    Evicted   Usage of EmptyDir volume "cache" exceeds
the limit "50Mi".
export-6d5c8f7b9d-x7wlm   Running   <none>    <none>
```

**URSACHE**

`sizeLimit` begrenzt ein `emptyDir` nicht wie ein Dateisystem. Der Prozess kann weiterschreiben, `dd` meldet keinen Fehler. Der Kubelet misst den Verbrauch aber regelmäßig, und sobald er über dem Limit liegt, beendet er den Pod. Das Deployment ersetzt ihn, der neue Pod schreibt dieselben 80 MiB, und der Kreislauf beginnt von vorn.

MAC

Schritt 17

```
kubectl --context talos apply -f szenarien/09-storage/fix-platte.yaml
kubectl --context talos -n dbg-09 delete pod --field-selector=status.phase=Failed
```

Hebt `sizeLimit` auf 200 MiB und setzt Request und Limit für `ephemeral-storage`, damit der Scheduler den Platz einplant. Der zweite Befehl räumt die geräumten Pods weg, die sonst als Leichen stehen bleiben.

**DAS SOLLTEST DU SEHEN**

Ein Pod `1/1 Running`, der auch nach Minuten noch läuft. `kubectl describe pod` zeigt unter `Requests` den Eintrag `ephemeral-storage: 200Mi`.

Wird nicht ein einzelner Pod, sondern der ganze Knoten knapp, setzt der Kubelet die Condition `DiskPressure`, versieht den Knoten mit dem Taint `node.kubernetes.io/disk-pressure` und räumt Pods nach Priorität. Wie du das untersuchst, zeigt Kapitel 13. Für Storage bleibt eine Erkenntnis: local-path-Volumes liegen auf derselben Platte wie Images, Logs und `emptyDir`.

**RKE2**

Volumes unter `/opt/local-path-provisioner` auf der Root-Partition. Prüfen per SSH mit `df -h /opt /var/lib/kubelet` und `sudo du -sh /opt/local-path-provisioner/*`. Ein volles Volume füllt die Systemplatte von Ubuntu.

**TALOS**

Volumes unter `/var/mnt/local-path-provisioner` auf der EPHEMERAL-Partition, zusammen mit Images und Kubelet-Daten. Prüfen mit `talosctl -n 172.16.0.171 mounts` (Belegung je Mount) und `talosctl -n 172.16.0.171 usage /var/mnt/local-path-provisioner`.

**MAC** Schritt 18

```
kubectl --context rke2 delete ns dbg-09
kubectl --context talos delete ns dbg-09
```

Räumt beide Cluster auf. Weil `local-path` die `reclaimPolicy: Delete` hat, löscht der Provisioner mit den PVCs auch die PVs und die Verzeichnisse auf den Knoten. Dafür startet er wieder Helper-Pods, `helper-pod-delete-...`.

**PROBIER ES SELBST**

Lege auf RKE2 den PVC aus `fix-klasse.yaml` an, ohne einen Pod. Warte eine Minute. Welchen Status hat der PVC, und welches Event siehst du? Lege dann einen Pod an, der den PVC mit `nodeSelector: kubernetes.io/hostname: rke2-cp` nutzt. Auf welchem Knoten liegt das Volume danach?

Hinweis: Ohne Pod bleibt der PVC `Pending` mit `WaitForFirstConsumer`, das ist korrekt. Sobald der Pod auf `rke2-cp` geplant ist, legt der Provisioner das Verzeichnis dort an. Die Node-Affinity im PV zeigt `rke2-cp`. Jeder spätere Pod mit diesem PVC muss ebenfalls auf `rke2-cp` laufen.

**WAS DU GELERNT HAST**

- Bei Storage-Problemen gilt die Reihenfolge Pod, PVC, PV, Provisioner, Knoten. Die Events am PVC sind fast immer der schnellste Weg zur Ursache.
- `WaitForFirstConsumer` ist kein Fehler. Die Spec eines PVC ist unveränderlich, eine falsche StorageClass heißt neu anlegen.
- Auf Talos braucht der local-path-provisioner einen Namespace mit PSA `privileged`, sonst lehnt die API seine Helper-Pods ab (`hostPath volumes`).
- local-path-Volumes sind an einen Knoten gebunden. `ReadWriteOnce` gilt pro Knoten. Mehrere Replikas mit eigenen Daten brauchen ein StatefulSet mit `volumeClaimTemplates`.
- Für `hostPath`-PVs wirkt `fsGroup` nicht. Rechte im Volume setzt du beim Anlegen, etwa im Init-Container. Ein `emptyDir` über seinem `sizeLimit` führt zur Räumung, nicht zu einem Schreibfehler.

KAPITEL 10

# RBAC und Pod Security

*Du liest `Forbidden`-Meldungen wie ein Protokoll, prüfst Rechte mit `kubectl auth can-i`, findest falsch gebundene Rollen und erkennst, wann Pod Security Admission einen Pod ablehnt, obwohl das Deployment angenommen wurde.*

**Dauer:** 60 Minuten **Voraussetzung:** Kapitel 2 und 3

## 10.1 Was in diesem Kapitel kaputtgeht

Zwei Mechanismen sagen im Cluster „nein“, und beide antworten mit `Forbidden`. RBAC entscheidet, **wer** eine Aktion auf der API ausführen darf. Pod Security Admission (PSA) entscheidet, **welche** Pods überhaupt entstehen dürfen, egal wer sie anlegt. Die Meldungen sehen sich ähnlich, die Fehlersuche ist aber völlig verschieden.

| Szenario | Symptom | Wo die Antwort steht |
|---|---|---|
| RoleBinding im falschen Namespace | Pod in `CrashLoopBackOff`, Log: `cannot list resource "pods"` | `kubectl auth can-i --as`, `get rolebindings -A` |
| Role statt ClusterRole | Log: `... at the cluster scope` | `kubectl api-resources --namespaced=false` |
| hostPath auf Talos | DaemonSet `0/2`, kein Pod | Event `FailedCreate` am DaemonSet |
| Namespace mit `restricted` | Deployment `0/2`, kein Pod | Event `FailedCreate` am ReplicaSet |

Alle Szenarien laufen im Namespace `dbg-10`. Spiele sie in der Reihenfolge des Kapitels. Die PSA-Szenarien setzen Labels am Namespace, die für alles darin gelten.

## 10.2 Eine Forbidden-Meldung lesen

Jede RBAC-Ablehnung folgt demselben Satzbau. Wer ihn kennt, muss nicht raten:

```
Error from server (Forbidden): pods is forbidden: User
"system:serviceaccount:dbg-10:app"
cannot list resource "pods" in API group "" in the namespace "dbg-10"
```

| Teil | Bedeutung |
|---|---|
| `User "system:serviceaccount:dbg-10:app"` | Die Identität. Bei ServiceAccounts immer `system:serviceaccount:<namespace>:<name>`. |
| `cannot list` | Das Verb: `get`, `list`, `watch`, `create`, `update`, `patch`, `delete`. |
| `resource "pods"` | Die Ressource im Plural, bei Unterressourcen etwa `pods/log` oder `pods/exec`. |
| `in API group ""` | Die API-Gruppe. `""` ist die Kerngruppe (Pods, Services, ConfigMaps), Deployments liegen in `apps`. |
| `in the namespace "dbg-10"` | Der Geltungsbereich. Bei cluster-weiten Ressourcen steht hier `at the cluster scope`. |

Diese fünf Angaben übersetzt du eins zu eins in eine Regel einer Role. Fehlt eine davon in der Rolle, oder bindet die Rolle die falsche Identität, kommt `Forbidden`.

## 10.3 RoleBinding im falschen Namespace

Eine Role gilt nur in ihrem Namespace, eine RoleBinding ebenso. Ein häufiger Kopierfehler: Die RoleBinding stammt aus einem anderen Projekt, und ihr `metadata.namespace` wurde nicht angepasst.

DATEI `szenarien/10-rbac-psa/kaputt-binding.yaml (Ausschnitt)`

```yaml
kind: RoleBinding
metadata:
  name: pod-reader
  namespace: default        # Fehler: kopiert, Namespace nicht angepasst
subjects:
  - kind: ServiceAccount
    name: app
    namespace: dbg-10
roleRef:
  kind: Role
  name: pod-reader
# ...
      containers:
        - name: kubectl
          image: registry.k8s.io/kubectl:v1.36.6
          args: ["get", "pods", "-n", "dbg-10", "--watch"]
```

Der Container führt `kubectl get pods --watch` mit dem Token des ServiceAccounts `app` aus. Klappt das, läuft er dauerhaft. Scheitert es, beendet sich `kubectl` mit Exit-Code 1. Der Pod erfüllt `restricted`, damit er dir später nicht mit PSA dazwischenfunkt.

MAC Schritt 1

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/kaputt-binding.yaml
kubectl --context talos apply -f szenarien/10-rbac-psa/kaputt-binding.yaml
```

Legt Namespace, ServiceAccount, Role, die falsche RoleBinding und das Deployment `pod-lister` an.

**SYMPTOM**

```
$ kubectl --context rke2 get pods -n dbg-10
NAME                          READY   STATUS             RESTARTS      AGE
pod-lister-7d9f6c8b5-k2x8w    0/1     CrashLoopBackOff   3 (28s ago)   75s
```

Ein CrashLoop wie aus Kapitel 6. Erst das Log zeigt, dass es kein Programmfehler ist.

### Untersuchen

MAC Schritt 2

```
kubectl --context rke2 logs -n dbg-10 deploy/pod-lister
```

Bei einem Neustart-Zyklus kann der Container gerade weg sein. Dann hilft `--previous`.

```
Error from server (Forbidden): pods is forbidden: User
"system:serviceaccount:dbg-10:app"
cannot list resource "pods" in API group "" in the namespace "dbg-10"
```

Jetzt fragst du den API-Server selbst, ob diese Identität das darf. `--as` lässt dich als jemand anderes auftreten, ohne dessen Token zu kennen. Dein Admin-Konto darf das, normale Benutzer nicht.

MAC Schritt 3

```
kubectl --context rke2 auth can-i list pods -n dbg-10 \
  --as=system:serviceaccount:dbg-10:app
```

Antwortet mit `yes` oder `no`. Das ist dieselbe Prüfung, die der API-Server bei echten Anfragen macht.

```
no
```

Was darf der ServiceAccount überhaupt? `--list` zählt alle Regeln auf, die für ihn in diesem Namespace greifen:

MAC Schritt 4

```
kubectl --context rke2 auth can-i --list -n dbg-10 \
  --as=system:serviceaccount:dbg-10:app
```

Hier stehen nur die Grundrechte, die jeder authentifizierte Benutzer hat (`selfsubjectaccessreviews`, `/api/*` und Ähnliches), aber keine Pods.

Die Role existiert aber. Also suchst du die Bindung, und zwar in allen Namespaces:

MAC — Schritt 5

```
kubectl --context rke2 get rolebindings -A -o wide | grep -E 'NAMESPACE|pod-reader'
```

`-o wide` zeigt Rolle und Subjekte. So siehst du, wo die Bindung wirklich liegt.

```
NAMESPACE   NAME          ROLE                AGE   USERS   GROUPS   SERVICEACCOUNTS
default     pod-reader    Role/pod-reader     3m                     dbg-10/app
```

Die Bindung liegt in `default` und verweist auf eine Role `pod-reader` in `default`. Die gibt es dort nicht. Der API-Server prüft das beim Anlegen nicht, eine RoleBinding auf eine fehlende Rolle ist erlaubt und wirkungslos.

K9S — Schritt 6

```
:sa dbg-10⏎        app auswählen, Enter = wirksame Regeln
:rolebindings⏎     0 = alle Namespaces, d = describe
```

In der ServiceAccount-Ansicht zeigt `Enter` die Regeln, die über Bindungen auf den Account wirken. Bleibt die Liste leer, greift keine Bindung. `u` zeigt, welche Pods den Account benutzen.

**URSACHE**

Role und RoleBinding sind namespaced, und eine RoleBinding verweist immer auf eine Role im **eigenen** Namespace. Die Bindung in `default` sucht dort eine Role `pod-reader`, findet keine und vergibt deshalb nichts. Der ServiceAccount in `dbg-10` bleibt ohne Rechte.

## Beheben

DATEI `szenarien/10-rbac-psa/fix-binding.yaml (Ausschnitt)`

```yaml
kind: RoleBinding
metadata:
  name: pod-reader
  namespace: dbg-10
```

MAC — Schritt 7

```
kubectl --context rke2 delete rolebinding pod-reader -n default
kubectl --context rke2 apply -f szenarien/10-rbac-psa/fix-binding.yaml
kubectl --context rke2 rollout restart deploy/pod-lister -n dbg-10
```

Löscht die verirrte Bindung und legt die richtige an. RBAC wird bei jeder Anfrage neu geprüft, der Neustart verkürzt nur das Warten auf das Ende des Backoffs. Auf Talos dieselben Befehle mit `--context talos`.

**DAS SOLLTEST DU SEHEN**

`kubectl --context rke2 auth can-i list pods -n dbg-10 --as=system:serviceaccount:dbg-10:app` antwortet `yes`. Der Pod läuft, und sein Log zeigt die Pod-Liste:

```
NAME                          READY   STATUS    RESTARTS   AGE
pod-lister-6b8d7f9c4-q7tlm    1/1     Running   0          5s
```

## 10.4 Role statt ClusterRole

Manche Ressourcen gehören keinem Namespace: Nodes, PersistentVolumes, Namespaces selbst, StorageClasses, ClusterRoles. Eine Role kann darauf keine Rechte vergeben. Der API-Server nimmt eine solche Role trotzdem an, denn er prüft Regeln nicht auf Sinn.

DATEI `szenarien/10-rbac-psa/kaputt-clusterrole.yaml (Ausschnitt)`

```yaml
kind: Role
metadata:
  name: node-reader
  namespace: dbg-10
rules:
  - apiGroups: [""]
    resources: ["nodes"]
    verbs: ["get", "list", "watch"]
```

Dazu ein ServiceAccount `node-reader`, eine passende RoleBinding in `dbg-10` und ein Deployment, das `kubectl get nodes --watch` ausführt.

MAC Schritt 8

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/kaputt-clusterrole.yaml
```

Diesmal ist die Bindung im richtigen Namespace.

**SYMPTOM**

Der Pod `node-lister` geht in `CrashLoopBackOff`. Das Log:

```
Error from server (Forbidden): nodes is forbidden: User
"system:serviceaccount:dbg-10:node-reader"
cannot list resource "nodes" in API group "" at the cluster scope
```

### Untersuchen

Der entscheidende Teil ist `at the cluster scope`. Die Anfrage hat keinen Namespace, also können nur cluster-weite Bindungen helfen. Ob eine Ressource namespaced ist, verrät `api-resources`:

MAC Schritt 9

```
kubectl --context rke2 api-resources --namespaced=false | grep -E 'NAME|^nodes'
```

Listet nur cluster-weite Ressourcen. Mit `--namespaced=true` bekommst du die Gegenliste.

```
NAME    SHORTNAMES   APIVERSION   NAMESPACED   KIND
nodes   no           v1           false        Node
```

`kubectl auth can-i list nodes -n dbg-10 --as=system:serviceaccount:dbg-10:node-reader` antwortet `no`, obwohl die Role genau dieses Recht nennt. Ein `-n` ändert nichts, Nodes kennen keinen Namespace.

**URSACHE**

RBAC wertet eine Role nur für Anfragen **innerhalb** ihres Namespace aus. `list nodes` ist eine Anfrage ohne Namespace, dafür zählen ausschließlich ClusterRoleBindings. Auch eine RoleBinding, die auf eine ClusterRole zeigt, hilft nicht: Sie vergibt deren Rechte nur innerhalb des Namespace der Bindung.

### Beheben

DATEI `szenarien/10-rbac-psa/fix-clusterrole.yaml (Ausschnitt)`

```yaml
kind: ClusterRole
metadata:
  name: dbg-10-node-reader
rules:
  - apiGroups: [""]
    resources: ["nodes"]
    verbs: ["get", "list", "watch"]
---
kind: ClusterRoleBinding
metadata:
  name: dbg-10-node-reader
subjects:
  - kind: ServiceAccount
    name: node-reader
    namespace: dbg-10
```

MAC — Schritt 10

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/fix-clusterrole.yaml
kubectl --context rke2 rollout restart deploy/node-lister -n dbg-10
```

Vergibt das Leserecht auf Nodes cluster-weit, aber nur an diesen einen ServiceAccount.

**DAS SOLLTEST DU SEHEN**

`auth can-i list nodes --as=system:serviceaccount:dbg-10:node-reader` antwortet `yes`, und `kubectl --context rke2 logs -n dbg-10 deploy/node-lister` zeigt `rke2-cp` und `rke2-w1`.

**STOLPERFALLE**

ClusterRole und ClusterRoleBinding gehören keinem Namespace, `kubectl delete ns dbg-10` entfernt sie nicht. Vergessene ClusterRoleBindings sind ein Klassiker bei Audits: Legt jemand später einen ServiceAccount mit gleichem Namen an, erbt er die Rechte.

## 10.5 Pod Security: Was die Distributionen vorgeben

PSA prüft Pods gegen eine von drei Stufen: `privileged` (alles erlaubt), `baseline` (keine bekannten Ausbrüche wie `privileged`, `hostPath`, Host-Namespaces) und `restricted` (zusätzlich kein root, alle Capabilities weg, Seccomp). Pro Namespace legen Labels die Stufe für drei Modi fest: `enforce` lehnt ab, `warn` gibt `kubectl` eine Warnung zurück, `audit` schreibt ins Audit-Log. Fehlt ein Label, gilt die cluster-weite Vorgabe des API-Servers. Und genau die unterscheidet sich im Labor:

**RKE2**

Ohne `profile` in `/etc/rancher/rke2/config.yaml` schreibt RKE2 nach `/etc/rancher/rke2/rke2-pss.yaml` nur `enforce: privileged`. Es gibt keine Vorgabe für `warn` und `audit`, also auch keine Warnungen. Mit `profile: cis` wird daraus `restricted` für alle drei Modi, ausgenommen sind `kube-system`, `compliance-operator-system` und `tigera-operator`. Eine eigene Datei setzt du mit `pod-security-admission-config-file`.

**TALOS**

Talos setzt in der Machine-Config unter `cluster.apiServer.admissionControl` cluster-weit `enforce: baseline`, `audit: restricted` und `warn: restricted`. Ausgenommen ist nur `kube-system`. Deshalb siehst du auf Talos bei vielen Manifesten Warnungen `would violate PodSecurity "restricted:latest"`, die nichts blockieren. Ändern kannst du die Vorgabe mit einem Machine-Config-Patch.

Dasselbe Manifest kann also auf RKE2 laufen und auf Talos scheitern. Prüfe die Vorgabe deines Clusters, bevor du ein Problem anderswo suchst:

MAC — Schritt 11

```
talosctl -n 172.16.0.170 get machineconfig -o yaml | grep -A 14 'name: PodSecurity'
```

Zeigt den PSA-Block aus der aktiven Machine-Config des Control-Plane-Knotens. Auf RKE2 liest du stattdessen auf `rke2-cp` die Datei `/etc/rancher/rke2/rke2-pss.yaml`.

## 10.6 hostPath: läuft auf RKE2, nicht auf Talos

Ein kleiner Log-Sammler soll auf jedem Knoten `/var/log` lesen. Typischer Kandidat für ein DaemonSet mit `hostPath`.

DATEI `szenarien/10-rbac-psa/kaputt-hostpath.yaml (Ausschnitt)`

```
kind: DaemonSet
metadata:
  name: log-sammler
  namespace: dbg-10
# ...
      volumes:
        - name: varlog
          hostPath:
            path: /var/log
```

MAC Schritt 12

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/kaputt-hostpath.yaml
kubectl --context talos apply -f szenarien/10-rbac-psa/kaputt-hostpath.yaml
```

Auf RKE2 kommt keine Rückfrage. Auf Talos gibt `kubectl` eine Warnung aus.

```
Warning: would violate PodSecurity "restricted:latest": allowPrivilegeEscalation !=
false
(container "sammler" must set securityContext.allowPrivilegeEscalation=false), ...
restricted volume types (volume "varlog" uses restricted volume type "hostPath"), ...
daemonset.apps/log-sammler created
```

**SYMPTOM**

```
$ kubectl --context talos get ds -n dbg-10
NAME          DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
log-sammler   2         0         0       0            0           <none>          40s
```

Zwei Pods gewünscht, null angelegt. Auf RKE2 steht in derselben Zeile `2 2 2`.

### Untersuchen

Wo kein Pod ist, gibt es kein `describe pod`. Die Meldung hängt am Objekt, das den Pod anlegen wollte:

MAC Schritt 13

```
kubectl --context talos events -n dbg-10 --for daemonset/log-sammler
```

Die Events des DaemonSets. Bei Deployments schaust du eine Ebene tiefer, beim ReplicaSet.

```
LAST SEEN   TYPE      REASON         OBJECT                  MESSAGE
12s         Warning   FailedCreate   DaemonSet/log-sammler   Error creating: pods
"log-sammler-8kq2d" is
forbidden: violates PodSecurity "baseline:latest": hostPath volumes (volume "varlog")
```

K9S Schritt 14

```
:ds dbg-10⏎       log-sammler auswählen, d = describe
:events⏎          / FailedCreate⏎
```

Die Warnung steht am Ende der Describe-Ansicht des DaemonSets. In der Event-Ansicht filterst du mit `/` nach dem Reason.

`kubectl --context talos get ns dbg-10 --show-labels` bestätigt: kein Label `pod-security.kubernetes.io/...`, also gilt die Talos-Vorgabe.

**URSACHE**

Die Warnung beim `apply` kam aus dem Modus `warn` (Talos: `restricted`) und hat das DaemonSet nicht aufgehalten. PSA prüft im Modus `enforce` nur Pods, nicht ihre Controller. Deshalb wurde das DaemonSet angenommen, und erst der DaemonSet-Controller bekam beim Anlegen jedes Pods die Ablehnung gegen `baseline`. RKE2 ohne CIS-Profil erzwingt `privileged` und lässt den Pod durch.

### Beheben

Zwei saubere Wege: Der Pod verzichtet auf `hostPath`, oder der Namespace bekommt bewusst eine Ausnahme. Ein Log-Sammler braucht den Knoten, also das Zweite. Dabei setzt du `warn` und `audit` auf `baseline`, damit die Ausnahme sichtbar bleibt.

DATEI `szenarien/10-rbac-psa/fix-hostpath.yaml`

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dbg-10
  labels:
    pod-security.kubernetes.io/enforce: privileged
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/warn: baseline
    pod-security.kubernetes.io/warn-version: latest
    pod-security.kubernetes.io/audit: baseline
    pod-security.kubernetes.io/audit-version: latest
```

MAC Schritt 15

```
kubectl --context talos apply -f szenarien/10-rbac-psa/fix-hostpath.yaml
```

Der DaemonSet-Controller versucht es nach einer Wartezeit erneut, du musst nichts neu starten.

**DAS SOLLTEST DU SEHEN**

`kubectl --context talos get ds -n dbg-10` zeigt `2 2 2`. Jede Änderung am DaemonSet bringt weiterhin die Warnung `would violate PodSecurity "baseline:latest": hostPath volumes ...`. Das ist gewollt: Die Ausnahme erinnert an sich selbst.

**STOLPERFALLE**

Dasselbe trifft `kubectl debug` auf Talos. Das Standardprofil `general` fügt die Capability `SYS_PTRACE` hinzu, und `baseline` lehnt den Debug-Container ab. Nutze dort immer `--profile=baseline`, wie es auch das k9s-Plugin aus Kapitel 3 tut.

## 10.7 restricted: Deployment ja, Pods nein

Für das letzte Szenario setzt du den Namespace neu auf, diesmal mit der strengsten Stufe. Das ist der Normalfall für Anwendungs-Namespaces.

MAC — Schritt 16

```
kubectl --context rke2 delete ns dbg-10
kubectl --context talos delete ns dbg-10
```

Räumt die bisherigen Szenarien ab. Denk an die ClusterRole aus dem zweiten Szenario, sie bleibt stehen.

DATEI `szenarien/10-rbac-psa/kaputt-restricted.yaml (Ausschnitt)`

```yaml
kind: Namespace
metadata:
  name: dbg-10
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/warn: restricted
# ...
      containers:
        - name: web
          image: nginx:1.29-alpine
```

Ein gewöhnliches nginx-Image ohne `securityContext`. Es läuft als root.

MAC — Schritt 17

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/kaputt-restricted.yaml
```

Hier zeigt sich RKE2 genauso streng wie Talos, denn das Label am Namespace schlägt jede clusterweite Vorgabe.

```
namespace/dbg-10 created
Warning: would violate PodSecurity "restricted:latest": allowPrivilegeEscalation !=
false
(container "web" must set securityContext.allowPrivilegeEscalation=false), ...
deployment.apps/web created
```

**SYMPTOM**

```
$ kubectl --context rke2 get deploy,rs,pods -n dbg-10
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/web   0/2     0            0           30s

NAME                             DESIRED   CURRENT   READY   AGE
replicaset.apps/web-5d7c9b8f6d   2         0         0       30s
```

`created` und trotzdem kein einziger Pod. In Pipelines geht die Warnung oben gern unter, dann ist dieses Bild alles, was du siehst.

### Untersuchen

MAC — Schritt 18

```
kubectl --context rke2 describe rs -n dbg-10 -l app=web | tail -4
```

Das ReplicaSet legt die Pods an, also hat es die Ablehnung bekommen.

```
  Warning  FailedCreate  12s (x14 over 52s)  replicaset-controller  Error creating:
pods
"web-5d7c9b8f6d-x7k2p" is forbidden: violates PodSecurity "restricted:latest":
allowPrivilegeEscalation != false (container "web" must set securityContext.
allowPrivilegeEscalation=false), unrestricted capabilities (container "web" must set
securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or container
"web"
must set securityContext.runAsNonRoot=true), seccompProfile (pod or container "web"
must
set securityContext.seccompProfile.type to "RuntimeDefault" or "Localhost")
```

Die Meldung ist eine Checkliste. Jede Klammer nennt das Feld, das fehlt. Bevor du einen bestehenden Namespace verschärfst, prüft `kubectl label --dry-run=server --overwrite ns <name> pod-security.kubernetes.io/enforce=restricted` alle laufenden Pods gegen die neue Stufe, ohne das Label zu setzen.

**URSACHE**

`enforce` gilt nur für Pods. Ein Deployment enthält zwar eine Pod-Vorlage, wird aber nur im Modus `warn` und `audit` geprüft. Das Deployment entsteht, das ReplicaSet ebenfalls, und erst die Pods scheitern. Ohne `warn`-Label (auf RKE2 ohne CIS der Normalfall) gäbe es beim `apply` nicht einmal die Warnung.

### Beheben

Nicht die Stufe senken, sondern den Pod reparieren. nginx bietet ein Image, das ohne root auf Port 8080 läuft.

DATEI szenarien/10-rbac-psa/fix-restricted.yaml (Ausschnitt)

```
    spec:
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: web
          image: nginxinc/nginx-unprivileged:1.29-alpine
          ports:
            - containerPort: 8080
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
```

MAC

Schritt 19

```
kubectl --context rke2 apply -f szenarien/10-rbac-psa/fix-restricted.yaml
kubectl --context talos apply -f szenarien/10-rbac-psa/kaputt-restricted.yaml \
  -f szenarien/10-rbac-psa/fix-restricted.yaml
```

Auf Talos legst du Namespace und Fix in einem Schritt an. Diesmal kommt keine Warnung mehr.

**DAS SOLLTEST DU SEHEN**

```
$ kubectl --context rke2 get pods -n dbg-10
NAME                   READY   STATUS    RESTARTS   AGE
web-6f8b7d5c9b-4tq8z   1/1     Running   0          8s
web-6f8b7d5c9b-n2wlx   1/1     Running   0          8s
```

**PROBIER ES SELBST**

`szenarien/10-rbac-psa/aufgabe-job.yaml` legt einen Job an, der ConfigMaps in `dbg-10` lesen soll. Er scheitert mit `Forbidden`, obwohl Role und RoleBinding im richtigen Namespace liegen und den richtigen ServiceAccount nennen. Finde den Fehler nur mit dem Log und `kubectl auth can-i`.

Hinweis: Lies in der Meldung den Teil `in API group "..."` und vergleiche ihn mit `apiGroups` der Role.

MAC

Schritt 20

```
kubectl --context rke2 delete ns dbg-10
kubectl --context talos delete ns dbg-10
kubectl --context rke2 delete -f szenarien/10-rbac-psa/fix-clusterrole.yaml
```

Räumt das Kapitel auf, einschließlich der cluster-weiten Objekte.

**WAS DU GELERNT HAST**

- Eine `Forbidden`-Meldung nennt Identität, Verb, Ressource, API-Gruppe und Geltungsbereich, genau das, was eine Rolle enthalten muss.
- `kubectl auth can-i ... --as=system:serviceaccount:<ns>:<name>` beantwortet Rechtefragen, ohne den Pod zu bemühen. `--list` zeigt alles.
- RoleBindings wirken nur in ihrem Namespace. Cluster-weite Ressourcen brauchen ClusterRole und ClusterRoleBinding.
- PSA erzwingt Stufen nur für Pods. Deployments und DaemonSets werden angenommen, die Ablehnung steht als `FailedCreate` am Controller.
- RKE2 ohne CIS erzwingt `privileged`, Talos `baseline`. Labels am Namespace schlagen beide Vorgaben.

KAPITEL 11

# Fehler durch Kyverno

*Du erkennst, wann Kyverno hinter einer Ablehnung oder einer unerwarteten Änderung steckt, liest seine Meldungen, findest die verantwortliche Policy, kommst bei einem ausgefallenen Webhook wieder in den Cluster und testest Policies vorab ohne Cluster.*

**Dauer:** 75 Minuten **Voraussetzung:** Kapitel 2, 3 und 10

## 11.1 Was in diesem Kapitel kaputtgeht

Kyverno ist ein Admission-Webhook. Der API-Server schickt ihm jede passende Anfrage, bevor er sie speichert. Kyverno kann ablehnen (`ValidatingPolicy`), ändern (`MutatingPolicy`) oder, wenn es nicht antwortet, den Cluster lahmlegen. Alle drei Fälle sehen für Anwender zunächst aus wie „Kubernetes spinnt“.

| Szenario | Symptom | Wo die Antwort steht |
|---|---|---|
| Policy lehnt Deployment ab | `apply` scheitert sofort | `admission webhook "vpol..." denied the request` |
| Policy lehnt nur Pods ab | Deployment `0/1`, kein Pod | Event `FailedCreate` am ReplicaSet |
| Mutation ändert den Pod | `CreateContainerConfigError` | Live-Objekt gegen Manifest, `kubectl get mpol` |
| Webhook nicht erreichbar | **Jede** Änderung in `dbg-11` scheitert | `failed calling webhook`, `failurePolicy` |
| PolicyException wirkungslos | Ausnahme angelegt, Pods trotzdem abgelehnt | Warnung beim `apply`, Startparameter |

Alle Policies dieses Kapitels liegen unter `policies/` und wirken nur im Namespace `dbg-11`. Lege jede Policy zu Beginn ihres Szenarios an und lösche sie am Ende wieder, damit sich die Szenarien nicht gegenseitig stören.

## 11.2 Kyverno installieren

DATEI `werkzeuge/kyverno-values.yaml (Ausschnitt)`

```
admissionController:
  replicas: 1
backgroundController:
  replicas: 1
# ... cleanupController, reportsController ebenso
```

```
features:
  policyExceptions:
    enabled: false
    namespace: ""
```

Je Controller eine Instanz spart im Labor Speicher. Der Preis: Fällt der Admission-Controller aus, antwortet niemand mehr auf Webhook-Anfragen. PolicyExceptions bleiben vorerst aus, wie im Chart vorgegeben. Das brauchst du im letzten Szenario.

MAC — Schritt 1

```
helm repo add kyverno https://kyverno.github.io/kyverno/
helm repo update
for ctx in rke2 talos; do
  helm --kube-context $ctx upgrade --install kyverno kyverno/kyverno \
    -n kyverno --create-namespace --version 3.9.2 \
    -f werkzeuge/kyverno-values.yaml --wait --timeout 5m
done
```

Chart 3.9.2 bringt Kyverno 1.19.2 mit. Die Schleife installiert auf beiden Clustern dieselbe Version.

MAC — Schritt 2

```
kubectl --context rke2 get deploy -n kyverno
```

Vier Deployments: `kyverno-admission-controller`, `kyverno-background-controller`, `kyverno-cleanup-controller`, `kyverno-reports-controller`, jeweils `1/1`.

Die Kyverno-Pods erfüllen `restricted`, auf Talos gibt es also keine PSA-Warnungen. Von Kyverno ausgenommen sind auf beiden Clustern `kube-system` und `kyverno` selbst.

**STOLPERFALLE**

Viele Anleitungen nutzen noch `kind: ClusterPolicy` aus `kyverno.io/v1`. In Kyverno 1.19 ist das veraltet, `kubectl apply` warnt mit `kyverno.io/v1 ClusterPolicy is deprecated and will be removed in a future release`. Dieses Kapitel nutzt die CEL-basierten Policies aus `policies.kyverno.io/v1`, deren Meldungen anders aussehen.

## 11.3 Policy lehnt das Deployment ab

DATEI `policies/require-team-label.yaml (Ausschnitt)`

```
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: require-team-label
spec:
  validationActions: [Deny]
  matchConstraints:
    namespaceSelector:
      matchExpressions:
```

```
        - key: kubernetes.io/metadata.name
          operator: In
          values: [dbg-11]
    resourceRules:
      - apiGroups: [""]
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: Pods brauchen das Label team.
      expression: >-
        object.metadata.?labels.team.orValue('') != ''
```

Die Policy prüft nur Pods. Kyverno erzeugt daraus aber automatisch dieselbe Regel für Deployments, StatefulSets, DaemonSets, Jobs und CronJobs. Das heißt Autogen und ist für CEL-Policies der Standard, wenn genau eine Regel auf `pods` ohne `objectSelector` passt.

**MAC** Schritt 3

```
kubectl --context rke2 apply -f policies/require-team-label.yaml
kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-label.yaml
```

Das Deployment `web` hat im Pod-Template nur das Label `app`.

**SYMPTOM**

```
namespace/dbg-11 created
Error from server: error when creating "szenarien/11-kyverno/kaputt-label.yaml":
admission webhook
"vpol.validate.kyverno.svc-fail" denied the request: Policy require-team-label failed:
Pods
brauchen das Label team.
```

### Untersuchen

Die Meldung trägt bereits alles: Der Webhook-Name beginnt mit `vpol` (ValidatingPolicy, bei MutatingPolicy `mpol`, bei alten ClusterPolicies nur `validate.kyverno.svc`), die Endung `-fail` steht für `failurePolicy: Fail`. Danach folgen der Name der Policy und wörtlich ihr `message`-Text.

**MAC** Schritt 4

```
kubectl --context rke2 get vpol
kubectl --context rke2 get vpol require-team-label -o jsonpath='{.status.autogen}{"\n"}'
```

`vpol` ist die Kurzform von `validatingpolicies`, die Spalte `READY` sollte `True` zeigen. Unter `status.autogen` steht, für welche Controller Kyverno die Regel mitgeneriert hat.

**K9S** Schritt 5

```
:vpol⏎              require-team-label auswählen, y = YAML
:polr dbg-11⏎       Policy Reports im Namespace
```

k9s kennt die Kyverno-CRDs über ihre Kurznamen. In der YAML-Ansicht suchst du mit `/` nach `message`.

**STOLPERFALLE**

Alte ClusterPolicies antworten mit `resource Deployment/dbg-11/web was blocked due to the following policies`, darunter Policy, Regel und `validation error: ... failed at path ...`. Liegen mehrere CEL-Policies mit unterschiedlichen Selektoren im Cluster, hängt Kyverno an den Webhook-Namen einen kurzen Hash, etwa `vpol.validate.kyverno.svc-fail-1a2b3c4d`.

**URSACHE**

Das Pod-Template enthält kein Label `team`. Durch Autogen prüft Kyverno schon das Deployment und nicht erst den Pod. Der Ausdruck aus der Policy wurde dafür auf `object.spec.template.metadata` umgeschrieben. Deshalb scheitert `apply` sofort und sichtbar.

### Beheben

DATEI `szenarien/11-kyverno/fix-label.yaml (Ausschnitt)`

```yaml
  template:
    metadata:
      labels:
        app: web
        team: shop
```

MAC — Schritt 6

```
kubectl --context rke2 apply -f szenarien/11-kyverno/fix-label.yaml
```

Diesmal kommt `deployment.apps/web created`.

**DAS SOLLTEST DU SEHEN**

`kubectl --context rke2 get pods -n dbg-11 -l app=web` zeigt einen Pod in `Running`. Danach aufräumen:

`kubectl --context rke2 delete -f policies/require-team-label.yaml`.

## 11.4 Policy lehnt nur die Pods ab

Gefährlicher ist der Fall, in dem Kyverno das Deployment durchlässt und erst die Pods ablehnt. Dann sieht der Entwickler `created`, die Pipeline ist grün, und trotzdem läuft nichts.

DATEI `policies/require-memory-limit.yaml (Ausschnitt)`

```yaml
  matchConstraints:
    objectSelector:
      matchExpressions:
        - key: policy.nyrvex.com/skip
```

```
          operator: DoesNotExist
    resourceRules:
      - apiGroups: [""]
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: Jeder Container braucht ein Memory-Limit.
      expression: >-
        object.spec.containers.all(c,
          c.?resources.?limits.?memory.orValue('') != '')
```

Jemand wollte einen Ausstieg per Label einbauen und hat einen `objectSelector` ergänzt. Mit einem `objectSelector` schaltet Kyverno Autogen ab, denn Labels am Deployment und am Pod sind verschiedene Dinge.

**MAC** Schritt 7

```
kubectl --context rke2 apply -f policies/require-memory-limit.yaml
kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-limit.yaml
```

Das Deployment `api` setzt kein Memory-Limit.

**SYMPTOM**

```
$ kubectl --context rke2 get deploy,rs -n dbg-11 -l app=api
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/api   0/1     0            0           25s

NAME                              DESIRED   CURRENT   READY   AGE
replicaset.apps/api-6c8d9f7b54    1         0         0       25s
```

### Untersuchen

Dasselbe Muster wie bei PSA in Kapitel 10: kein Pod, also Events am ReplicaSet.

**MAC** Schritt 8

```
kubectl --context rke2 events -n dbg-11 --for replicaset/api-6c8d9f7b54
```

Setze den Namen deines ReplicaSets ein, oder nutze in k9s das Plugin `Shift-E` auf dem ReplicaSet.

```
LAST SEEN   TYPE      REASON         OBJECT                      MESSAGE
8s          Warning   FailedCreate   ReplicaSet/api-6c8d9f7b54   Error creating:
admission webhook
"vpol.validate.kyverno.svc-fail" denied the request: Policy require-memory-limit
failed: Jeder
Container braucht ein Memory-Limit.
```

Ob eine Policy auch Controller prüft, verrät ihr Status. Bei `require-memory-limit` bleibt `status.autogen` leer:

MAC Schritt 9

```
kubectl --context rke2 get vpol require-memory-limit -o jsonpath='{.status.autogen}{"\n"}'
```

Leer oder `{}` heißt: Diese Policy sieht nur Pods. Vergleiche mit `require-team-label` aus dem ersten Szenario.

**URSACHE**

Die Policy gilt nur für Pods und hat wegen des `objectSelector` keine Autogen-Regeln. Das Deployment und das ReplicaSet passieren Kyverno, erst die Pod-Erstellung durch den ReplicaSet-Controller wird abgelehnt. Die Meldung landet deshalb nicht bei dir im Terminal, sondern als Event am ReplicaSet. Für Anwender ist eine Ablehnung schon beim `apply` viel freundlicher, Autogen sollte deshalb nur aus gutem Grund fehlen.

### Beheben

DATEI `szenarien/11-kyverno/fix-limit.yaml (Ausschnitt)`

```
        resources:
          limits:
            memory: 128Mi
```

MAC Schritt 10

```
kubectl --context rke2 apply -f szenarien/11-kyverno/fix-limit.yaml
```

Neues Pod-Template, also neues ReplicaSet. Das alte bleibt mit `0` Replikas stehen.

**DAS SOLLTEST DU SEHEN**

`kubectl --context rke2 get pods -n dbg-11 -l app=api` zeigt `1/1 Running`. Danach:

`kubectl --context rke2 delete -f policies/require-memory-limit.yaml`.

## 11.5 Mutation ändert den Pod unerwartet

Das Plattform-Team rollt eine „sichere Voreinstellung“ aus: Jeder Pod in `dbg-11` bekommt `runAsNonRoot: true`. Gut gemeint, nur weiß das Anwendungsteam nichts davon.

DATEI `policies/add-run-as-non-root.yaml (Ausschnitt)`

```
apiVersion: policies.kyverno.io/v1
kind: MutatingPolicy
metadata:
  name: add-run-as-non-root
spec:
  # ... matchConstraints wie oben, nur CREATE
  mutations:
```

```
    - patchType: ApplyConfiguration
      applyConfiguration:
        expression: >-
          Object{
            spec: Object.spec{
              securityContext: Object.spec.securityContext{
                runAsNonRoot: true
              }
            }
          }
```

MAC Schritt 11

```
kubectl --context rke2 apply -f policies/add-run-as-non-root.yaml
kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-mutation.yaml
```

Das Deployment `shop` nutzt das normale Image `nginx:1.29-alpine`, das als root startet. Im Manifest steht kein `securityContext`.

**SYMPTOM**

```
$ kubectl --context rke2 get pods -n dbg-11 -l app=shop
NAME                    READY   STATUS                       RESTARTS   AGE
shop-7b9c6d5f8-2kx9p    0/1     CreateContainerConfigError   0          20s
```

### Untersuchen

MAC Schritt 12

```
kubectl --context rke2 describe pod -n dbg-11 -l app=shop | tail -3
```

Die Kubelet-Meldung am Ende erklärt den Status.

```
  Warning  Failed     4s (x3 over 20s)  kubelet  Error: container has runAsNonRoot and
image will run as
root (pod: "shop-7b9c6d5f8-2kx9p_dbg-11(5c1e9a7d-...)", container: shop)
```

Das Team schwört, nie `runAsNonRoot` gesetzt zu haben. Also vergleichst du Manifest und Live-Objekt:

MAC Schritt 13

```
grep -c runAsNonRoot szenarien/11-kyverno/kaputt-mutation.yaml
kubectl --context rke2 get deploy shop -n dbg-11 \
  -o jsonpath='{.spec.template.spec.securityContext}{"\n"}'
```

Im Manifest: `0` Treffer. Im Cluster: `{"runAsNonRoot":true}`. Zwischen `apply` und Speichern hat jemand das Objekt verändert.

Sogar das Deployment ist verändert, nicht nur der Pod: Autogen gilt auch für MutatingPolicies. Objekte beim Anlegen verändern nur Mutating-Webhooks. `kubectl --context rke2 get mpol` listet

die Kyverno-Mutationen, `kubectl --context rke2 get mutatingwebhookconfigurations` alle anderen Kandidaten.

Den Verdacht bestätigst du offline mit der Kyverno-CLI. Sie zeigt das Objekt so, wie es nach der Mutation aussähe:

MAC Schritt 14

```
kyverno apply policies/add-run-as-non-root.yaml \
  --resource szenarien/11-kyverno/kaputt-mutation.yaml \
  --values-file policies/values-dbg-11.yaml
```

Offline kennt die CLI keine Namespace-Labels. `values-dbg-11.yaml` sagt ihr, dass `dbg-11` das Label `kubernetes.io/metadata.name=dbg-11` trägt.

```
policy add-run-as-non-root applied to dbg-11/Deployment/shop:
# ...
    template:
        spec:
            containers:
                - image: nginx:1.29-alpine
                  name: shop
            securityContext:
                runAsNonRoot: true
```

**URSACHE**

Die MutatingPolicy setzt `runAsNonRoot: true` in jede neue Pod-Vorlage. Das Image `nginx:1.29-alpine` startet als UID 0. Das Kubelet prüft vor dem Start, ob die effektive UID zu `runAsNonRoot` passt, und verweigert den Container mit `CreateContainerConfigError`. Kyverno schreibt erfolgreiche Mutationen in der Standardkonfiguration nicht als Event, der Eingriff ist deshalb unsichtbar.

### Beheben

Die Policy ist hier sinnvoll, falsch ist das Image. nginx bietet eine Variante ohne root:

DATEI `szenarien/11-kyverno/fix-mutation.yaml (Ausschnitt)`

```
        - name: shop
          image: nginxinc/nginx-unprivileged:1.29-alpine
          ports:
            - containerPort: 8080
```

MAC Schritt 15

```
kubectl --context rke2 apply -f szenarien/11-kyverno/fix-mutation.yaml
```

Das Image läuft als UID 101 und lauscht auf 8080, weil Ports unter 1024 root brauchen.

**DAS SOLLTEST DU SEHEN**

Der neue Pod ist `Running`. Danach: `kubectl --context rke2 delete -f policies/add-run-as-non-root.yaml`.

**STOLPERFALLE**

`kubectl diff` hilft hier nicht: Es schickt das Manifest als Server-Side-Dry-Run durch die Admission, die Mutation greift auch dort, und der Unterschied verschwindet. Willst du Mutationen als Event sehen, setze im Chart `config.generateSuccessEvents: true` und entferne `PolicyApplied` aus `features.omitEvents.eventTypes`.

## 11.6 Webhook nicht erreichbar

Jetzt fällt Kyverno selbst aus. Im echten Leben durch einen OOMKill, einen ausgefallenen Knoten oder eine NetworkPolicy. Im Labor simulierst du das mit `scale`.

MAC — Schritt 16

```
kubectl --context rke2 apply -f policies/require-team-label.yaml
kubectl --context rke2 scale deploy kyverno-admission-controller -n kyverno --replicas=0
kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-webhook.yaml
```

Das Deployment `notfall` erfüllt die Policy. Es dürfte also angelegt werden.

**SYMPTOM**

```
namespace/dbg-11 unchanged
Error from server (InternalError): error when creating "szenarien/11-kyverno/kaputt-
webhook.yaml":
Internal error occurred: failed calling webhook "vpol.validate.kyverno.svc-fail": failed
to call
webhook: Post "https://kyverno-svc.kyverno.svc:443/vpol/require-team-label?timeout=10s":
no
endpoints available for service "kyverno-svc"
```

Dasselbe trifft jeden Pod, den ein ReplicaSet in `dbg-11` neu anlegen will. Ein gelöschter Pod kommt nicht wieder.

### Untersuchen

Die Meldung nennt Webhook, Service und Grund. Prüfe zuerst, ob hinter dem Service überhaupt jemand steht:

MAC — Schritt 17

```
kubectl --context rke2 get deploy kyverno-admission-controller -n kyverno
kubectl --context rke2 get endpointslices -n kyverno -l kubernetes.io/service-name=kyverno-svc
```

`READY 0/0` und eine EndpointSlice ohne Endpunkte. Im Ernstfall siehst du hier `CrashLoopBackOff` oder `Pending`, dann geht es mit den Kapiteln 4 und 6 weiter.

Dann die Frage, warum ein fehlender Webhook blockiert, statt übersprungen zu werden:

MAC — Schritt 18

```
kubectl --context rke2 get validatingwebhookconfiguration kyverno-resource-validating-webhook-cfg \
  -o jsonpath='{range .webhooks[*]}{.name}{"\t"}{.failurePolicy}{"\n"}{end}' | grep vpol
```

Listet jeden Webhook der Konfiguration mit seiner `failurePolicy`.

```
vpol.validate.kyverno.svc-fail     Fail
```

In k9s findest du dasselbe unter `:validatingwebhookconfigurations`, dann `y` für die YAML-Ansicht und `/failurePolicy`.

**URSACHE**

CEL-Policies haben ohne Angabe `failurePolicy: Fail`. Kann der API-Server den Webhook nicht erreichen, lehnt er die Anfrage ab, statt sie durchzulassen. Mit nur einer Replika des Admission-Controllers reicht dafür ein einziger ausgefallener Pod. Betroffen sind alle Ressourcen, auf die eine Policy passt. `kube-system` und `kyverno` sind ausgenommen, deshalb kann Kyverno selbst überhaupt wieder starten.

### Beheben

Die Reihenfolge im Notfall:

1. Den Admission-Controller wieder starten und die Ursache seines Ausfalls beheben.
2. Kommt er nicht hoch, die Webhook-Konfiguration löschen. Kyverno legt sie beim nächsten Start selbst wieder an. Bis dahin gelten keine Policies.
3. Langfristig: zwei oder drei Replikas und für unkritische Policies `failurePolicy: Ignore`.

MAC — Schritt 19

```
kubectl --context rke2 scale deploy kyverno-admission-controller -n kyverno --replicas=1
kubectl --context rke2 rollout status deploy kyverno-admission-controller -n kyverno
```

Schritt 1. Der Notausgang aus Schritt 2 wäre `kubectl --context rke2 delete validatingwebhookconfiguration kyverno-resource-validating-webhook-cfg`.

**DAS SOLLTEST DU SEHEN**

`kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-webhook.yaml` meldet `deployment.apps/notfall created`.

Danach: `kubectl --context rke2 delete -f policies/require-team-label.yaml`.

**RKE2**

Der API-Server läuft als Static Pod `kube-apiserver-rke2-cp`. Seine Meldungen zu Webhooks siehst du mit `kubectl --context rke2 logs -n kube-system kube-apiserver-rke2-cp | grep 'failed calling webhook'`.

**TALOS**

Auch hier ein Static Pod, `kube-apiserver-talos-cp`, mit demselben `kubectl logs`. Fällt der API-Server selbst aus, hilft `talosctl -n 172.16.0.170 containers --namespace cri` und `talosctl logs --namespace cri` mit der Container-ID.

## 11.7 PolicyException ohne Wirkung

Eine Altanwendung läuft noch ohne Memory-Limit. Statt die Policy aufzuweichen, bekommt sie eine `PolicyException`.

DATEI `policies/exceptions/legacy-batch.yaml`

```yaml
apiVersion: policies.kyverno.io/v1
kind: PolicyException
metadata:
  name: legacy-batch
  namespace: dbg-11
spec:
  policyRefs:
    - name: require-memory-limit
      kind: ValidatingPolicy
  matchConditions:
    - name: nur-legacy-batch
      expression: >-
        object.metadata.?labels.app.orValue('') == 'legacy-batch'
```

MAC

Schritt 20

```
kubectl --context rke2 apply -f policies/require-memory-limit.yaml
kubectl --context rke2 apply -f szenarien/11-kyverno/kaputt-ausnahme.yaml
kubectl --context rke2 apply -f policies/exceptions/legacy-batch.yaml
```

Policy, Deployment `legacy-batch` ohne Limit, Ausnahme. Achte auf die Ausgabe des letzten Befehls.

**SYMPTOM**

```
Warning: PolicyException resources would not be processed until it is enabled.
policyexception.policies.kyverno.io/legacy-batch created
```

Das Deployment bleibt bei `0/1`, am ReplicaSet steht dieselbe `FailedCreate`-Meldung wie im zweiten Szenario.

### Untersuchen

Die Warnung kommt von Kyverno selbst. Welche Einstellung gemeint ist, zeigen die Startparameter des Admission-Controllers:

MAC Schritt 21

```
kubectl --context rke2 get deploy kyverno-admission-controller -n kyverno -o yaml \
  | grep -E 'enablePolicyException|exceptionNamespace'
```

Die Helm-Werte unter `features.policyExceptions` werden zu diesen Argumenten.

```
- --enablePolicyException=false
```

Zum Auflisten schreibst du die Gruppe aus, `kubectl get policyexceptions.policies.kyverno.io -n dbg-11`. Auch die alte Gruppe `kyverno.io` kennt eine `PolicyException`, und ohne Gruppe wählt `kubectl` still eine der beiden.

**URSACHE**

PolicyExceptions sind in Kyverno ein Opt-in. Ohne `--enablePolicyException=true` nimmt der API-Server die Ausnahme zwar an, Kyverno beachtet sie aber nicht. Zusätzlich legt `--exceptionNamespace` fest, aus welchem Namespace Ausnahmen gelten. Das verhindert, dass jeder, der in seinem Namespace schreiben darf, sich selbst von Policies befreit.

## Beheben

DATEI `szenarien/11-kyverno/fix-ausnahme-values.yaml (Ausschnitt)`

```
features:
  policyExceptions:
    enabled: true
    namespace: dbg-11
```

Im Labor liegen die Ausnahmen direkt in `dbg-11`. In Produktion nimmst du einen eigenen Namespace, in den nur das Plattform-Team schreiben darf.

MAC Schritt 22

```
helm --kube-context rke2 upgrade kyverno kyverno/kyverno -n kyverno --version 3.9.2 \
  -f werkzeuge/kyverno-values.yaml -f szenarien/11-kyverno/fix-ausnahme-values.yaml --wait
```

Die zweite Datei überschreibt nur den Block `features.policyExceptions`. Die Controller starten mit den neuen Argumenten neu.

**DAS SOLLTEST DU SEHEN**

Nach kurzer Zeit legt das ReplicaSet den Pod an, `kubectl --context rke2 get pods -n dbg-11 -l app=legacy-batch` zeigt `Running`. Im Policy Report steht die Ausnahme als `SKIP`:

```
$ kubectl --context rke2 get polr -n dbg-11
NAME                                   KIND   NAME                          PASS   FAIL
WARN   ERROR   SKIP   AGE
9b2f4c1e-7a3d-4e8b-b6f0-2d5c8a1e9f37   Pod    legacy-batch-7f6d9c8b5-wq4zr   0      0
0      0       1      40s
```

## 11.8 Policies testen, bevor sie stören

Fast alle Fehler dieses Kapitels hättest du ohne Cluster finden können. Zu jeder Policy unter `policies/` gibt es Testfälle in `policies/tests/kyverno-test.yaml`: Ressourcen, die bestehen, scheitern oder per Ausnahme übersprungen werden müssen, und für die Mutation das erwartete Ergebnis.

DATEI `policies/tests/kyverno-test.yaml (Ausschnitt)`

```yaml
apiVersion: cli.kyverno.io/v1alpha1
kind: Test
metadata:
  name: kapitel-11
policies:
  - ../require-team-label.yaml
  - ../require-memory-limit.yaml
  - ../add-run-as-non-root.yaml
exceptions:
  - ../exceptions/legacy-batch.yaml
# ... resources, values
results:
  - isValidatingPolicy: true
    policy: require-team-label
    kind: Deployment
    resources: [web-ohne-team]
    result: fail
  # ... weitere Fälle
  - isMutatingPolicy: true
    policy: add-run-as-non-root
    kind: Pod
    resources: [root-web]
    patchedResources: patched/pod-root.yaml
    result: pass
```

MAC Schritt 23

```
kyverno test policies/tests
```

Lokal mit der Kyverno-CLI 1.19 (`brew install kyverno`). Im Labor wurde die Datei mit der CLI 1.19.1 geprüft.

**DAS SOLLTEST DU SEHEN**

```
Test Summary: 13 tests passed and 0 tests failed
```

Auch das zweite Szenario zeigt sich offline: `kyverno apply` mit beiden Policies auf `kaputt-limit.yaml` meldet `pass: 1, fail: 0`. Das eine `pass` kommt von `require-team-label`, das Deployment prüft `require-memory-limit` gar nicht. Ein fehlendes `fail` heißt also nicht „in Ordnung“, sondern womöglich „nicht zuständig“.

Der zweite Sicherheitsgurt läuft im Cluster: Führe neue Policies mit `validationActions: [Audit]` ein. Sie blockieren dann nichts, Kyverno prüft aber im Hintergrund alle bestehenden Ressourcen und schreibt das Ergebnis in Policy Reports. `kubectl --context rke2 get polr -A` zeigt in der Spalte `FAIL`, wen ein späteres `Deny` treffen würde.

**PROBIER ES SELBST**

Schreibe einen weiteren Testfall: einen Pod mit dem Label `policy.nyrvex.com/skip: "true"` und ohne Memory-Limit. Welches Ergebnis erwartest du für `require-memory-limit`, und warum ist dieser Ausstieg per Label ein Sicherheitsproblem?

Hinweis: Der `objectSelector` filtert schon im API-Server. Jeder, der Pods anlegen darf, darf auch ihre Labels setzen. Eine PolicyException in einem geschützten Namespace ist der sauberere Weg.

MAC — Schritt 24

```
kubectl --context rke2 delete -f policies/require-memory-limit.yaml -f policies/exceptions/legacy-
batch.yaml
kubectl --context rke2 delete ns dbg-11
```

Räumt das Kapitel auf. Willst du die Szenarien auf Talos wiederholen, nimm dieselben Befehle mit `--context talos`.

**WAS DU GELERNT HAST**

- Kyverno-Ablehnungen nennen Webhook, Policy und Meldung. `vpol` steht für ValidatingPolicy, `-fail` für `failurePolicy: Fail`.
- Autogen entscheidet, ob schon das Deployment oder erst der Pod abgelehnt wird. Ohne Autogen steht die Meldung als `FailedCreate` am ReplicaSet.
- Mutationen sind standardmäßig unsichtbar. Vergleiche Manifest und Live-Objekt und prüfe `kubectl get mpol` und die Mutating-Webhooks.
- Ist der Webhook weg, blockiert `failurePolicy: Fail` alles Passende. Notausgang: Controller starten oder Webhook-Konfiguration löschen.
- PolicyExceptions wirken nur mit `features.policyExceptions.enabled` und passendem Namespace.
- `kyverno test` prüft Policies samt Ausnahmen und Mutationen ohne Cluster.

KAPITEL 12

# Falco: Alarme und Störungen

*Du bringst Falco auf beiden Clustern zum Laufen, führst einen Alarm auf Pod und Container zurück, stellst einen Fehlalarm gezielt ab und erkennst, wenn Falco keine Kubernetes-Metadaten mehr liefert.*

**Dauer:** 60 Minuten **Voraussetzung:** Kapitel 2, 3 und 10

## 12.1 Was in diesem Kapitel kaputtgeht

Falco beobachtet per eBPF die Systemaufrufe auf jedem Knoten und vergleicht sie mit Regeln. Es blockiert nichts, es meldet nur. Fehler mit Falco sind deshalb selten Ausfälle der Anwendung, sondern Fragen an den Betrieb: Warum startet Falco nicht? Woher kommt dieser Alarm? Warum meldet es jede Nacht denselben Job? Und warum steht im Alarm kein Pod-Name?

| Szenario | Symptom | Wo die Antwort steht |
|---|---|---|
| Falco startet nicht auf Talos | DaemonSet `0/2` | `FailedCreate` am DaemonSet, PSA `baseline` |
| Alarm nach `kubectl exec` | `Terminal shell in container` | `output_fields`, `hostname`, Container-ID |
| Fehlalarm durch Job | Alle zehn Minuten derselbe Alarm | Regel, Bedingung, `exceptions` |
| Keine Kubernetes-Metadaten | `k8s_pod=<NA>` in jedem Alarm | Socket-Liste und Mounts des Charts |

Die Workloads laufen im Namespace `dbg-12`, Falco selbst im Namespace `falco`.

## 12.2 Falco installieren

Für RKE2 und Talos gibt es je eine Werte-Datei. Sie unterscheiden sich nur in einer Zeile, und genau diese Zeile ist Thema des letzten Szenarios.

DATEI `werkzeuge/falco-values-rke2.yaml (Ausschnitt)`

```
tty: true
driver:
  kind: modern_ebpf
falco:
  json_output: true
  json_include_output_property: true
  append_output:
```

```
    - suggested_output: true
    - match:
        source: syscall
      extra_output: "k8s_ns=%k8s.ns.name k8s_pod=%k8s.pod.name"
collectors:
  containerEngine:
    engines:
      docker:
        enabled: false
      # ... podman, containerd ebenso aus
      cri:
        enabled: true
        sockets: ["/run/k3s/containerd/containerd.sock"]
```

`modern_ebpf` braucht kein Kernel-Modul, das ist auf Talos auch gar nicht ladbar. `suggested_output` hängt Container-ID und -Name an jede Meldung, der zweite Eintrag Namespace und Pod. Die Metadaten holt sich das Plugin `container` über die CRI-Schnittstelle von containerd. In `werkzeuge/falco-values-talos.yaml` steht dort `/run/containerd/containerd.sock`.

MAC — Schritt 1

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm repo update
helm --kube-context rke2 upgrade --install falco falcosecurity/falco \
  -n falco --create-namespace --version 9.2.2 \
  -f werkzeuge/falco-values-rke2.yaml --wait --timeout 5m
```

Chart 9.2.2 bringt Falco 0.45.2 mit. Auf RKE2 läuft das ohne Zwischenfall durch.

MAC — Schritt 2

```
kubectl --context rke2 logs -n falco -l app.kubernetes.io/name=falco -c falco --tail=20
```

Mit `-l` liest `kubectl` aus allen Falco-Pods, je Knoten einer. Bei Selektoren zeigt `kubectl` standardmäßig nur zehn Zeilen pro Pod, daher `--tail`.

```
Mon Nov  9 14:02:11 2026: Falco version: 0.45.2 (x86_64)
Mon Nov  9 14:02:11 2026: Loading rules from:
Mon Nov  9 14:02:11 2026:    /etc/falco/falco_rules.yaml | schema validation: ok
Mon Nov  9 14:02:12 2026: Opening 'syscall' source with modern BPF probe.
```

Wichtig sind `schema validation: ok` für jede Regeldatei und die Zeile mit `modern BPF probe`. Fehlt sie, kam der Treiber nicht hoch.

## 12.3 Falco startet nicht auf Talos

Dieselbe Installation auf Talos, aber ohne `--wait`, damit du zusiehst:

MAC — Schritt 3

```
helm --kube-context talos upgrade --install falco falcosecurity/falco \
  -n falco --create-namespace --version 9.2.2 \
  -f werkzeuge/falco-values-talos.yaml
```

`--create-namespace` legt `falco` ohne Labels an. Helm meldet `STATUS: deployed`, denn ohne `--wait` prüft es nur, ob die Objekte angelegt wurden.

**SYMPTOM**

```
$ kubectl --context talos get ds -n falco
NAME    DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
falco   2         0         0       0            0           <none>          1m
```

### Untersuchen

Kein Pod, also Events am DaemonSet. Das Muster kennst du aus Kapitel 10:

MAC — Schritt 4

```
kubectl --context talos events -n falco --for daemonset/falco
```

Oder in k9s: `:ds falco⏎`, dann `d` und ganz nach unten.

```
LAST SEEN   TYPE      REASON         OBJECT            MESSAGE
5s          Warning   FailedCreate   DaemonSet/falco   Error creating: pods
"falco-7xk2p" is forbidden:
violates PodSecurity "baseline:latest": hostPath volumes (volumes "container-engine-
socket-0",
"boot-fs", "lib-modules", "usr-fs", "etc-fs", "sys-fs", "proc-fs"), privileged
(container "falco"
must not set securityContext.privileged=true)
```

**URSACHE**

Talos erzwingt cluster-weit `baseline`. Falco braucht einen privilegierten Container, um den eBPF-Treiber zu laden, und `hostPath`-Mounts für `/proc`, `/etc` und den containerd-Socket. Beides verbietet `baseline`. Das DaemonSet selbst wird angenommen, denn `enforce` gilt nur für Pods.

### Beheben

DATEI `szenarien/12-falco/namespace-falco.yaml`

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: falco
  labels:
    pod-security.kubernetes.io/enforce: privileged
    pod-security.kubernetes.io/enforce-version: latest
```

MAC — Schritt 5

```
kubectl --context talos apply -f szenarien/12-falco/namespace-falco.yaml
kubectl --context talos rollout status ds/falco -n falco --timeout=5m
```

`kubectl apply` warnt, dass der von Helm angelegte Namespace die Annotation `kubectl.kubernetes.io/last-applied-configuration` nicht hat, und ergänzt sie. Das ist harmlos. Bei einer Neuinstallation legst du den Namespace vor Helm an.

**DAS SOLLTEST DU SEHEN**

`kubectl --context talos get ds -n falco` zeigt `2 2 2`, und das Log enthält `Opening 'syscall' source with modern BPF probe.`

**RKE2**

RKE2 ohne CIS-Profil erzwingt `privileged`, Falco startet sofort. Mit `profile: cis` gilt `restricted`, dann brauchst du dasselbe Label wie auf Talos.

**TALOS**

Statt des Namespace-Labels kannst du `falco` auch in der Machine-Config unter `cluster.apiServer.admissionControl` bei `exemptions.namespaces` eintragen. Das Label ist sichtbarer und braucht keinen Patch der Knoten. `driver.kind: kmod` funktioniert auf Talos nicht, Kernel-Module lassen sich dort nicht nachladen.

## 12.4 Alarm nach `kubectl exec`

Ein Kollege öffnet eine Shell in einem Pod. Falco schlägt an. Deine Aufgabe ist, aus dem Alarm herauszulesen, wo das passiert ist.

DATEI `szenarien/12-falco/kaputt-shell.yaml (Ausschnitt)`

```yaml
    containers:
      - name: web
        image: busybox:1.37
        command: ["sh", "-c", "while true; do sleep 3600; done"]
```

MAC — Schritt 6

```
kubectl --context rke2 apply -f szenarien/12-falco/kaputt-shell.yaml
kubectl --context rke2 exec -it -n dbg-12 deploy/web -- sh -c 'id; exit'
```

`-it` gibt der Shell ein Terminal. Genau darauf achtet die Regel `Terminal shell in container`.

**SYMPTOM**

In den Falco-Logs erscheint eine JSON-Zeile. Den Text unter `output` kürzt dieser Auszug:

```
{"hostname":"rke2-w1","output":"14:21:37.482915063: Notice A shell was spawned in a
container
with an attached terminal | evt_type=execve user=root user_uid=0 user_loginuid=-1
process=sh
proc_exepath=/bin/sh ... command=sh -c id; exit terminal=34816 ...
container_id=8f2c41d9a7b3
container_name=web k8s_ns=dbg-12 k8s_pod=web-5b9d8c7f6-tx4lp","priority":"Notice",
"rule":"Terminal shell in container","source":"syscall", ...}
```

### Untersuchen

Bei vielen Meldungen hilft `jq`. Die Felder unter `output_fields` tragen die Namen der Falco-Felder:

**MAC** Schritt 7

```
kubectl --context rke2 logs -n falco -l app.kubernetes.io/name=falco -c falco --tail=500 \
  | jq -cR 'fromjson? | select(.rule=="Terminal shell in container")
      | {hostname, priority,
         ns: .output_fields["k8s.ns.name"], pod: .output_fields["k8s.pod.name"],
         id: .output_fields["container.id"], cmd: .output_fields["proc.cmdline"],
         user: .output_fields["user.name"]}'
```

`fromjson?` überspringt die Textzeilen vom Start. Übrig bleibt pro Alarm eine kompakte Zeile.

```
{"hostname":"rke2-w1","priority":"Notice","ns":"dbg-12","pod":"web-5b9d8c7f6-tx4lp",
 "id":"8f2c41d9a7b3","cmd":"sh -c id; exit","user":"root"}
```

| Feld | Was es dir sagt |
|---|---|
| `hostname` | Der Knoten, auf dem Falco den Aufruf gesehen hat. Im Chart ist das der Knotenname. |
| `k8s.ns.name`, `k8s.pod.name` | Namespace und Pod. Beide kommen über die Container-Runtime, nicht von der Kubernetes-API. |
| `container.id` | Die ersten zwölf Zeichen der Container-ID. Falco liest sie direkt aus den cgroups, sie ist auch ohne Runtime-Zugriff da. |
| `proc.cmdline` | Die ausgeführte Befehlszeile. |
| `user.name`, `user.loginuid` | Benutzer **im** Container. `-1` heißt: kein Login, typisch für `kubectl exec`. |

Die Container-ID ist der sichere Anker, falls Pod-Namen fehlen oder der Pod längst weg ist:

**MAC** Schritt 8

```
kubectl --context rke2 get pods -A \
  -o custom-
columns=NS:.metadata.namespace,POD:.metadata.name,ID:.status.containerStatuses[*].containerID \
  | grep 8f2c41d9a7b3
```

`containerID` hat die Form `containerd://8f2c41d9a7b3...`, der Präfix aus Falco reicht für `grep`.

RKE2-W1 Schritt 9

```
crictl ps -a | grep 8f2c41d9a7b3
```

Auf dem Knoten aus `hostname`, nach den Exporten aus Kapitel 2 (`CRI_CONFIG_FILE`). Für Talos: `talosctl -n 172.16.0.171 containers --namespace cri | grep 8f2c41d9a7b3`.

K9S Schritt 10

```
:ds falco⏎      Enter, Pod auf dem richtigen Knoten wählen, l = Logs
/Terminal⏎      Logs auf die Regel filtern
```

In der Log-Ansicht von k9s filtert `/` die Zeilen. Für die Auswertung über mehrere Pods ist `kubectl` mit `jq` bequemer.

**URSACHE**

Kein Fehler im eigentlichen Sinn: Die Regel `Terminal shell in container` meldet jeden Prozess einer Shell mit Terminal, der als Einstiegspunkt in einen Container kommt. `kubectl exec -it` erfüllt genau das. **Wer** den Befehl abgesetzt hat, sieht Falco nicht. Dafür brauchst du das Audit-Log des API-Servers, abgeglichen über Zeit, Namespace und Pod.

**DAS SOLLTEST DU SEHEN**

Du kannst zu jedem Alarm Knoten, Namespace, Pod und Container nennen. Ein `kubectl exec` **ohne** `-it`, etwa `kubectl exec -n dbg-12 deploy/web -- id`, löst diese Regel nicht aus.

## 12.5 Fehlalarm durch einen legitimen Job

Ein Aufräum-Job löscht alle zehn Minuten fertige Pods. Dazu spricht er mit `kubectl` die Kubernetes-API an.

DATEI `szenarien/12-falco/kaputt-cronjob.yaml (Ausschnitt)`

```yaml
kind: CronJob
metadata:
  name: aufraeumer
  namespace: dbg-12
spec:
  schedule: "*/10 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: aufraeumer
          containers:
            - name: kubectl
              image: registry.k8s.io/kubectl:v1.36.6
              args: ["delete", "pods", "-n", "dbg-12", "--field-selector=status.phase==Succeeded"]
```

In der Datei steht `args` als Liste über mehrere Zeilen. Role und RoleBinding geben dem ServiceAccount genau `get`, `list` und `delete` auf Pods.

MAC Schritt 11

```
kubectl --context rke2 apply -f szenarien/12-falco/kaputt-cronjob.yaml
kubectl --context rke2 create job -n dbg-12 --from=cronjob/aufraeumer aufraeumer-jetzt
```

Der zweite Befehl startet den Job sofort, statt auf die nächste volle zehnte Minute zu warten.

**SYMPTOM**

```
14:40:02.118204551: Notice Unexpected connection to K8s API Server from container |
connection=10.42.1.23:51734->10.43.0.1:443 lport=51734 rport=443 fd_type=ipv4
fd_proto=tcp
evt_type=connect user=root ... process=kubectl ... command=kubectl delete pods -n dbg-12
--field-selector=status.phase==Succeeded ... container_name=kubectl k8s_ns=dbg-12
k8s_pod=aufraeumer-jetzt-q8w2n
```

Ab jetzt sechsmal pro Stunde. Wer so etwas eine Woche lang sieht, liest Falco-Alarme nicht mehr.

### Untersuchen

Welche Bedingung hat angeschlagen? Die Regel steht in den Standardregeln des Pods:

MAC Schritt 12

```
kubectl --context rke2 exec -n falco ds/falco -c falco -- \
  grep -A 12 'rule: Contact K8S API Server From Container' /etc/falco/falco_rules.yaml
```

Zeigt die Regel, wie sie geladen wurde. Die Regeldateien installiert der Init-Container `falcoctl-artifact-install`.

```
- rule: Contact K8S API Server From Container
  desc: > ...
  condition: >
    evt.type=connect
    and (fd.typechar=4 or fd.typechar=6)
    and container
    and k8s_api_server
    and not k8s_containers
    and not user_known_contact_k8s_api_server_activities
```

`k8s_api_server` prüft, ob das Ziel `kubernetes.default.svc.cluster.local` ist. `k8s_containers` ist eine feste Liste bekannter Images, unser Job steht nicht darin.

**URSACHE**

Die Regel ist bewusst laut: Ein Container, der die API anspricht, kann ein Angreifer mit gestohlenem Token sein. Unser Job tut das aber legitim und regelmäßig. Ohne Ausnahme erzeugt er Rauschen, das echte Treffer derselben Regel verdeckt.

### Beheben

Nicht die Regel abschalten, sondern eine enge Ausnahme ergänzen: dieser Namespace, dieses Image, dieser Prozess.

DATEI `szenarien/12-falco/fix-ausnahme-values.yaml`

```yaml
customRules:
  dbg-ausnahmen.yaml: |-
    - rule: Contact K8S API Server From Container
      exceptions:
        - name: dbg12_aufraeumer
          fields: [k8s.ns.name, container.image.repository, proc.name]
          comps: [=, =, =]
          values:
            - [dbg-12, registry.k8s.io/kubectl, kubectl]
      override:
        exceptions: append
```

`override: exceptions: append` hängt die Ausnahme an die bestehende Regel an, ohne Bedingung oder Ausgabe anzufassen. Jede Zeile unter `values` ist ein Tupel, passend zu `fields` und `comps`. Das Chart legt `customRules` nach `/etc/falco/rules.d`, das Falco nach den Standardregeln lädt. Das alte `append: true` ist veraltet.

MAC — Schritt 13

```
helm --kube-context rke2 upgrade falco falcosecurity/falco -n falco --version 9.2.2 \
  -f werkzeuge/falco-values-rke2.yaml -f szenarien/12-falco/fix-ausnahme-values.yaml --wait
```

Die Falco-Pods starten mit der zusätzlichen Regeldatei neu. Lass bei künftigen Upgrades die zweite Datei nicht weg, sonst ist die Ausnahme wieder verschwunden.

**DAS SOLLTEST DU SEHEN**

Im Start-Log steht `/etc/falco/rules.d/dbg-ausnahmen.yaml | schema validation: ok`. Ein neuer Lauf mit `kubectl --context rke2 create job -n dbg-12 --from=cronjob/aufraeumer aufraeumer-2` bleibt still. Die Regel selbst lebt noch:

```
kubectl --context rke2 run apitest -n dbg-12 --image=busybox:1.37 --restart=Never -- wget -T 2 -q -O /dev/
null https://kubernetes.default.svc
```

löst sie weiter aus.

**STOLPERFALLE**

Fehlt in einer neuen Ausnahme `fields` oder `values`, verweigert Falco das Laden der Regeln. Der Pod startet dann nicht, im Log steht `Rule exception must have fields property with a list of fields`. Prüfe nach jeder Regeländerung das Start-Log. Für einfache Fälle gibt es einen zweiten Weg: das leere Makro `user_known_contact_k8s_api_server_activities` mit `override: condition: replace` füllen.

## 12.6 Keine Kubernetes-Metadaten

Jemand „vereinheitlicht“ die Falco-Werte für beide Cluster und setzt den Socket auf den Standardpfad. Auf Talos stimmt der, auf RKE2 nicht.

DATEI `szenarien/12-falco/kaputt-values-sockets.yaml`

```yaml
collectors:
  containerEngine:
    engines:
      cri:
        enabled: true
        sockets: ["/run/containerd/containerd.sock"]
```

MAC — Schritt 14

```
helm --kube-context rke2 upgrade falco falcosecurity/falco -n falco --version 9.2.2 \
  -f werkzeuge/falco-values-rke2.yaml -f szenarien/12-falco/kaputt-values-sockets.yaml --wait
kubectl --context rke2 exec -it -n dbg-12 deploy/web -- sh -c 'exit'
```

Die Werte werden in der Reihenfolge der `-f` zusammengeführt, die letzte Datei gewinnt. Danach löst du den Shell-Alarm erneut aus.

**SYMPTOM**

```
14:58:10.903311277: Notice A shell was spawned in a container with an attached terminal
| ...
container_id=3c7e90b1f2a4 container_name=<NA> k8s_ns=<NA> k8s_pod=<NA>
```

Falco läuft, alle Pods sind `Running`, die Alarme kommen. Nur der Pod fehlt, und eine Ausnahme mit `k8s.ns.name` wie im letzten Szenario greift nicht mehr.

### Untersuchen

`container_id` ist gefüllt, sie stammt aus dem Kernel. Alles, was von der Runtime kommt, fehlt. Also fragst du: Welchen Socket fragt Falco, und gibt es den auf dem Knoten?

MAC — Schritt 15

```
kubectl --context rke2 get ds falco -n falco -o yaml \
  | yq '.spec.template.spec.volumes[] | select(.hostPath) | .hostPath.path'
```

Das Chart mountet genau die Verzeichnisse der konfigurierten Sockets unter `/host` in den Container.

```
/run/containerd
/boot
/lib/modules
/usr
/etc
/sys/kernel
/proc
```

RKE2-W1 Schritt 16

```
grep runtime-endpoint /var/lib/rancher/rke2/agent/etc/crictl.yaml
ls -l /run/containerd/ /run/k3s/containerd/containerd.sock
```

Die crictl-Konfiguration von RKE2 nennt den echten Socket. `/run/containerd/` ist leer oder fehlt.

```
runtime-endpoint: unix:///run/k3s/containerd/containerd.sock
/run/containerd/:
total 0
srw-rw---- 1 root root 0 Nov  9 08:11 /run/k3s/containerd/containerd.sock
```

**URSACHE**

RKE2 bringt ein eigenes containerd mit, dessen Socket unter `/run/k3s/containerd/` liegt. Das Plugin `container` fragt nur den konfigurierten Pfad und erreicht dort keine Runtime. Ohne Runtime-Antwort bleiben alle Container- und Pod-Felder leer, Falco zeigt `<NA>`. Der Kernel liefert weiter Systemaufrufe und die Container-ID aus den cgroups, deshalb wirkt alles gesund.

## Beheben

MAC Schritt 17

```
helm --kube-context rke2 upgrade falco falcosecurity/falco -n falco --version 9.2.2 \
  -f werkzeuge/falco-values-rke2.yaml -f szenarien/12-falco/fix-ausnahme-values.yaml --wait
```

Ohne die kaputte Datei gilt wieder `/run/k3s/containerd/containerd.sock` aus der RKE2-Werte-Datei. Die Ausnahme bleibt erhalten.

**DAS SOLLTEST DU SEHEN**

Ein neues `kubectl exec -it` erzeugt wieder `container_name=web k8s_ns=dbg-12 k8s_pod=web-...`. Im DaemonSet steht `/run/k3s/containerd` unter den `hostPath`-Volumes.

**RKE2**

Socket der Kubelet-Runtime: `/run/k3s/containerd/containerd.sock`. Nachlesen in `/var/lib/rancher/rke2/agent/etc/crictl.yaml`. Die Standardwerte des Charts enthalten diesen Pfad bereits. Der Fehler entsteht erst, wenn jemand die Liste überschreibt.

**TALOS**

Talos betreibt zwei containerd-Instanzen: eine für Systemdienste unter `/system/run/containerd/` und eine für die Kubelet-Pods unter `/run/containerd/containerd.sock`. Falco braucht die zweite. Prüfen kannst du sie mit `talosctl -n 172.16.0.171 containers --namespace cri`.

**WARUM DAS WICHTIG IST**

Falco schreibt im Labor auf die Standardausgabe. In Produktion schickst du Alarme an ein zentrales System, etwa mit Falcosidekick, das im Chart als Option enthalten ist. Für die Fehlersuche ändert das nichts: Die Felder sind dieselben, und fehlende Metadaten fallen dort genauso auf, nur später.

**PROBIER ES SELBST**

Öffne auf Talos eine Shell im Pod `web` (Szenario aus diesem Kapitel, `--context talos`). Lies aus dem Alarm Knoten, Pod und Container-ID ab und finde den Container mit `talosctl ... containers --namespace cri`. Welcher der beiden Talos-Knoten war es?

Hinweis: `hostname` im JSON ist der Knotenname, `talosctl -n` braucht die IP. Die Zuordnung zeigt `kubectl --context talos get nodes -o wide`.

**MAC** Schritt 18

```
kubectl --context rke2 delete ns dbg-12
kubectl --context talos delete ns dbg-12
```

Räumt die Workloads auf. Falco kann installiert bleiben. Entfernen würdest du es mit `helm --kube-context rke2 uninstall falco -n falco`.

**WAS DU GELERNT HAST**

- Falco braucht `privileged` und `hostPath`. Auf Talos (Vorgabe `baseline`) gehört der Namespace `falco` deshalb auf `privileged`, sonst bleibt das DaemonSet bei `0`.
- `hostname`, `k8s.ns.name`, `k8s.pod.name` und `container.id` führen jeden Alarm auf Knoten, Pod und Container zurück. Den auslösenden Benutzer kennt nur das Audit-Log.
- Fehlalarme stellst du mit einer engen Ausnahme ab: `exceptions` mit `override: exceptions: append` in `customRules`.
- `<NA>` bei Pod und Namespace heißt: Falco erreicht die Container-Runtime nicht. Auf RKE2 liegt der Socket unter `/run/k3s/containerd/`.

KAPITEL 13

# Knoten

*Du erkennst, wann ein Fehler nicht im Pod, sondern im Knoten steckt, untersuchst Kubelet und Container-Runtime auf RKE2 per SSH und auf Talos per `talosctl`, und verstehst, warum Pods nach einem Knotenausfall minutenlang hängen.*

**Dauer:** 75 Minuten **Voraussetzung:** Kapitel 2 und 3

## 13.1 Was in diesem Kapitel kaputtgeht

Bisher lag die Ursache immer in einem Manifest. In diesem Kapitel liegt sie eine Ebene tiefer: beim Kubelet, das auf jedem Knoten die Pods startet, überwacht und an den API-Server meldet. Fällt es aus, sieht der Cluster nur noch Schweigen. Und Schweigen interpretiert Kubernetes vorsichtig, mit Fristen, die man kennen muss.

| Szenario | Symptom | Wo die Antwort steht |
|---|---|---|
| Kubelet läuft nicht | Knoten `NotReady` | `describe node`, dann Kubelet-Log auf dem Knoten |
| Knotenausfall | Pods hängen in `Terminating`, Ersatz erst nach 5 Minuten | Taints des Knotens, Tolerations des Pods |
| Lokaler Speicher voll | Pods `Error`, Reason `Evicted`, immer neue Pods | `.status.message` des Pods, Events |
| Drain hängt | `kubectl drain` wiederholt endlos | `kubectl get pdb`, Spalte `ALLOWED DISRUPTIONS` |

Alle Szenarien laufen im Namespace `dbg-13`. Er trägt das Label `pod-security.kubernetes.io/enforce: privileged`. Das braucht nur der Debug-Pod aus dem Eviction-Szenario auf Talos, die Anwendungen selbst halten `restricted` ein.

## 13.2 Der Werkzeugkasten für Knoten

Auf einem RKE2-Knoten meldest du dich per SSH an und arbeitest mit den gewohnten Linux-Werkzeugen. Auf Talos gibt es keine Shell. Jede Frage an den Knoten geht als API-Aufruf über `talosctl`. Die Tabelle stellt die Befehle gegenüber, die du in diesem Kapitel brauchst.

| Frage | RKE2 (auf dem Knoten, als root) | Talos (auf dem Mac) |
|---|---|---|
| Läuft das Kubelet? | `systemctl status rke2-agent` (Server: `rke2-server`) | `talosctl -n <IP> service kubelet` |
| Kubelet-Log | `/var/lib/rancher/rke2/agent/logs/kubelet.log` | `talosctl -n <IP> logs kubelet` |
| Log des Dienstes | `journalctl -u rke2-agent` | `talosctl -n <IP> services`, `logs <dienst>` |
| Kernel-Meldungen | `dmesg -T` | `talosctl -n <IP> dmesg` |
| Container des Knotens | `crictl ps -a`, `crictl logs <id>` | `talosctl -n <IP> containers --namespace cri` |
| Überblick live | `top`, `df -h` | `talosctl -n <IP> dashboard` |
| Alles für den Support | `journalctl` und Logs von Hand sammeln | `talosctl -n <IP> support` |

Für RKE2 gilt die Umgebung aus Kapitel 2: `sudo -i`, dann `KUBECONFIG`, `PATH` und `CRI_CONFIG_FILE` setzen. Ohne `CRI_CONFIG_FILE` findet `crictl` den containerd-Socket von RKE2 nicht.

**STOLPERFALLE**

Seit Talos 1.14 heißt der Schalter für Kubernetes-Container bei `talosctl containers` und `talosctl logs` `--namespace cri`. Das ältere `-k` funktioniert noch, ist aber als veraltet markiert. Außerdem verschlüsselt `talosctl support` das Archiv ab 1.14 standardmäßig für das Support-Team von Sidero Labs. Für die eigene Analyse brauchst du `--no-encryption` oder eigene Empfänger über `--encryption-recipients`.

## 13.3 Knoten wird NotReady

Das Kubelet meldet sich regelmäßig beim API-Server: über ein Lease-Objekt im Namespace `kube-node-lease` und über den Status des Node-Objekts. Bleibt diese Meldung aus, wartet der Node-Lifecycle-Controller im Controller-Manager eine Frist ab, die `node-monitor-grace-period`. Sie beträgt in Kubernetes 1.36 standardmäßig 50 Sekunden (früher 40). Danach setzt er alle Conditions des Knotens auf `Unknown`.

Auf RKE2 hältst du das Kubelet an, indem du den Agenten stoppst. Auf Talos geht das nicht so einfach, also baust du einen Fehler, der in der Praxis häufig vorkommt: einen Tippfehler in den Kubelet-Argumenten.

DATEI `labor/talos/patch-kubelet-tippfehler.yaml`

```yaml
apiVersion: v1alpha1
kind: KubeletConfig
extraArgs:
  max-pod: "150"
```

Gemeint war `max-pods`. Seit Talos 1.14 steht die Kubelet-Konfiguration im eigenen Dokument `KubeletConfig`, das alte Feld `machine.kubelet` ist veraltet.

MAC Schritt 1

```
kubectl --context rke2 apply -f szenarien/13-nodes/web.yaml
kubectl --context talos apply -f szenarien/13-nodes/web.yaml
ssh <user>@172.16.0.161
```

Legt die Testanwendung `web` auf beiden Clustern an und öffnet eine Sitzung auf `rke2-w1`.

RKE2-W1 Schritt 2

```
sudo systemctl stop rke2-agent
```

Stoppt den RKE2-Agenten und damit das Kubelet. Die Container der Pods laufen in der Regel weiter, nur meldet sie niemand mehr.

MAC Schritt 3

```
talosctl -n 172.16.0.171 patch mc --patch @labor/talos/patch-kubelet-tippfehler.yaml
```

Talos übernimmt die Änderung ohne Neustart des Knotens und startet das Kubelet mit dem neuen Argument neu.

**SYMPTOM**

```
$ kubectl --context rke2 get nodes
NAME      STATUS     ROLES                       AGE   VERSION
rke2-cp   Ready      control-plane,etcd,master   9d    v1.36.6+rke2r1
rke2-w1   NotReady   <none>                      9d    v1.36.6+rke2r1
```

Nach etwa einer Minute steht `rke2-w1` auf `NotReady`, ebenso `talos-w1` im Cluster `talos`. Die Pods darauf zeigen weiterhin `Running`.

### Untersuchen

Der erste Blick gilt dem Node-Objekt. Dort steht, seit wann und warum der Knoten nicht mehr bereit ist.

MAC Schritt 4

```
kubectl --context rke2 describe node rke2-w1 | sed -n '/^Taints/,/^Addresses/p'
```

Zeigt Taints und Conditions. Der Rest der Ausgabe (Kapazität, Pods, Events) ist hier zweitrangig.

```
Taints:             node.kubernetes.io/unreachable:NoExecute
                    node.kubernetes.io/unreachable:NoSchedule
Unschedulable:      false
...
Conditions:
Type             Status    ... Reason              Message
----             ------    ... ------              -------
```

```
MemoryPressure  Unknown  ... NodeStatusUnknown   Kubelet stopped posting node status.
DiskPressure    Unknown  ... NodeStatusUnknown   Kubelet stopped posting node status.
PIDPressure     Unknown  ... NodeStatusUnknown   Kubelet stopped posting node status.
Ready           Unknown  ... NodeStatusUnknown   Kubelet stopped posting node status.
```

`Unknown` heißt: Der Controller weiß es nicht, weil das Kubelet schweigt. Steht dort dagegen `False` mit einer Meldung des Kubelets, läuft das Kubelet und meldet selbst ein Problem, etwa ein fehlendes CNI. Diese Unterscheidung spart viel Zeit. Der Taint `unreachable` gehört zum Status `Unknown`, bei `False` setzt der Controller `node.kubernetes.io/not-ready`.

In k9s siehst du dasselbe in der Knotenansicht:

**K9S** Schritt 5

```
:nodes⏎           Knoten auswählen, d = describe
```

Die Spalte `STATUS` zeigt `NotReady`. In der Describe-Ansicht findest du mit `/` und `Conditions` schnell die richtige Stelle.

Weil das Kubelet nicht antwortet, hilft `kubectl` hier nicht weiter. Auch `kubectl debug node/rke2-w1` scheitert: Den Debug-Pod müsste genau das Kubelet starten, das ausgefallen ist. Du musst auf den Knoten.

**RKE2-W1** Schritt 6

```
systemctl status rke2-agent --no-pager
journalctl -u rke2-agent --since "15 min ago" --no-pager | tail -n 20
tail -n 20 /var/lib/rancher/rke2/agent/logs/kubelet.log
```

Zustand des Dienstes, seine letzten Meldungen und das Ende des Kubelet-Logs. `inactive (dead)` mit einem `Stopped`-Eintrag im Journal heißt: Jemand hat den Dienst angehalten. Ein Absturz sähe anders aus, mit Exit-Code und Neustartversuchen.

**URSACHE**

Auf `rke2-w1` läuft das Kubelet nicht, weil der Dienst `rke2-agent` gestoppt wurde. Ohne Kubelet erneuert niemand das Lease des Knotens. Nach 50 Sekunden setzt der Node-Lifecycle-Controller die Conditions auf `Unknown` und den Taint `node.kubernetes.io/unreachable`. Neue Pods landen nicht mehr auf dem Knoten, die alten bleiben vorerst, wo sie sind.

### Dasselbe auf Talos

Auf Talos fragst du nicht `systemd`, sondern die Dienstverwaltung von Talos. Das Kubelet ist dort ein eigener Dienst, genau wie `etcd`, `containerd` oder `apid`.

MAC Schritt 7

```
talosctl -n 172.16.0.171 services
talosctl -n 172.16.0.171 service kubelet
talosctl -n 172.16.0.171 logs kubelet --tail 20
```

Liste aller Dienste mit Zustand und Health, dann die Details zum Kubelet mit den letzten Ereignissen, zuletzt sein Log.

Im Dienststatus wechseln sich Starts und Fehlschläge ab, Talos startet das Kubelet immer wieder neu. Den Grund liefert das Log, sinngemäß:

```
Error: failed to parse kubelet flag: unknown flag: --max-pod
```

**URSACHE**

Das Kubelet bricht beim Parsen seiner Argumente ab, weil es `--max-pod` nicht kennt. Talos reicht `extraArgs` ungeprüft durch. Es lehnt nur Argumente ab, die es selbst verwaltet, etwa `kubeconfig` oder `container-runtime-endpoint`. Dasselbe passiert mit Flags, die eine neue Kubernetes-Version entfernt hat. Nach einem Upgrade ist das eine typische Ursache für `NotReady`.

### Beheben

RKE2-W1 Schritt 8

```
sudo systemctl start rke2-agent
```

Startet den Agenten wieder. Das Kubelet meldet sich innerhalb weniger Sekunden.

MAC Schritt 9

```
talosctl -n 172.16.0.171 edit mc
```

Öffnet die Machine-Config im Editor. Lösche im Dokument `KubeletConfig` die Zeile `max-pod: "150"` (und `extraArgs:`, falls es leer wird), speichere und schließe. Talos wendet die Änderung sofort an.

**DAS SOLLTEST DU SEHEN**

`kubectl --context rke2 get nodes` und `kubectl --context talos get nodes` zeigen wieder zweimal `Ready`. Der Taint `unreachable` ist verschwunden, `talosctl -n 172.16.0.171 service kubelet` meldet `STATE Running` und `HEALTH OK`.

**RKE2**

Kubelet, containerd und die Steuerung laufen als Kindprozesse von `rke2-agent` (auf dem Server: `rke2-server`). Logs liegen als Dateien unter `/var/lib/rancher/rke2/agent/logs/` und im Journal. Kubelet-Argumente setzt du mit `kubelet-arg` in `/etc/rancher/rke2/config.yaml`, danach Neustart des Dienstes.

**TALOS**

Das Kubelet ist ein Talos-Dienst, kein Prozess, den du anfassen kannst. Status und Log kommen nur über `talosctl service` und `talosctl logs`. Argumente stehen im Dokument `KubeletConfig`. Ist sogar die Talos-API weg, bleibt nur die Konsole der VM mit dem Dashboard.

## 13.4 Pods hängen nach einem Knotenausfall

Ein `NotReady`-Knoten verliert seine Pods nicht sofort. Jeder Pod bekommt beim Anlegen vom Admission-Plugin `DefaultTolerationSeconds` zwei Tolerations: Er duldet die Taints `not-ready` und `unreachable` für 300 Sekunden. Erst danach räumt der Taint-Eviction-Controller ihn ab. Diesmal lässt du den Knoten lange genug ausgefallen.

MAC — Schritt 10

```
kubectl --context rke2 get pod -n dbg-13 -l app=web \
  -o jsonpath='{.items[0].spec.tolerations}' | jq
```

Zeigt die Tolerations eines `web`-Pods. Im Manifest steht keine einzige, beide stammen vom API-Server.

Zwei Einträge, für `node.kubernetes.io/not-ready` und `node.kubernetes.io/unreachable`, beide mit `"effect": "NoExecute"` und `"tolerationSeconds": 300`.

RKE2-W1 — Schritt 11

```
sudo systemctl stop rke2-agent
```

Derselbe Ausfall wie eben. Diesmal wartest du sechs Minuten, bevor du weitermachst.

**SYMPTOM**

```
$ kubectl --context rke2 get pods -n dbg-13 -o wide
NAME                   READY   STATUS        RESTARTS   AGE   IP           NODE
web-7c9d8f6b5d-4kq2m   1/1     Running       0          31m   10.42.0.37   rke2-cp
web-7c9d8f6b5d-r8vtn   1/1     Terminating   0          31m   10.42.1.12   rke2-w1
web-7c9d8f6b5d-x7lpd   1/1     Running       0          47s   10.42.0.41   rke2-cp
```

Fünf Minuten lang stand der Pod auf `rke2-w1` scheinbar gesund auf `Running`. Jetzt hängt er in `Terminating`, und erst jetzt gibt es Ersatz auf `rke2-cp`.

### Untersuchen

MAC Schritt 12

```
kubectl --context rke2 events -n dbg-13 --for pod/web-7c9d8f6b5d-r8vtn
```

Die Events des hängenden Pods. Den Namen übernimmst du aus deiner Ausgabe.

```
LAST SEEN   TYPE     REASON                 OBJECT                      MESSAGE
5m10s       Normal   TaintManagerEviction   pod/web-7c9d8f6b5d-r8vtn    Marking ...
```

Die vollständige Message lautet `Marking for deletion Pod dbg-13/web-7c9d8f6b5d-r8vtn`. Der Controller hat den Pod zum Löschen markiert. Gelöscht ist er erst, wenn das Kubelet bestätigt, dass die Container weg sind. Dieses Kubelet läuft aber nicht. Der Pod bleibt deshalb in `Terminating`, bis der Knoten zurückkommt oder jemand das Node-Objekt entfernt.

K9S Schritt 13

```
:pods dbg-13⏎      Spalte NODE beachten, Pod wählen, d = describe
:nodes⏎            rke2-w1 wählen, d = describe, Taints prüfen
```

In der Pod-Liste fällt der Ersatz-Pod mit kleinem `AGE` sofort auf.

**URSACHE**

Der Knoten war nicht erreichbar, also hat der Controller den Taint `node.kubernetes.io/unreachable:NoExecute` gesetzt. Die Pods dulden ihn 300 Sekunden lang. Danach markiert der Controller sie zum Löschen. Bestätigen kann das nur das Kubelet des ausgefallenen Knotens, also hängen sie in `Terminating`. Der Ersatz entsteht trotzdem, weil der ReplicaSet-Controller Pods mit gesetztem `deletionTimestamp` nicht mehr mitzählt.

**WARUM DAS WICHTIG IST**

Die Wartezeit ist Absicht. Kubernetes kann einen toten Knoten nicht von einer Netzwerkstörung unterscheiden. Vielleicht laufen die Container noch und schreiben in eine Datenbank. Bei einem StatefulSet würde ein vorschneller Ersatz zwei Pods mit derselben Identität erzeugen. Deshalb entsteht dort auch nach 300 Sekunden kein Ersatz, solange der alte Pod nicht wirklich gelöscht ist.

### Beheben

Der akute Fix ist, den Knoten zurückzuholen. Sobald das Kubelet wieder läuft, beendet es die markierten Pods und bestätigt das Löschen.

RKE2-W1 Schritt 14

```
sudo systemctl start rke2-agent
```

Der hängende Pod verschwindet nach wenigen Sekunden.

Ist der Knoten dauerhaft verloren, löschst du das Node-Objekt mit `kubectl --context rke2 delete node rke2-w1`. Dann räumt der Garbage Collector die Pods ab, die an den nicht mehr vorhandenen Knoten gebunden waren. Für zustandslose Anwendungen kannst du die Frist außerdem selbst festlegen:

DATEI `szenarien/13-nodes/fix-toleranz.yaml (Ausschnitt)`

```yaml
      tolerations:
        - key: node.kubernetes.io/unreachable
          operator: Exists
          effect: NoExecute
          tolerationSeconds: 30
        # ... dasselbe für node.kubernetes.io/not-ready
```

Steht die Toleration schon im Manifest, ergänzt der API-Server keine eigene.

MAC — Schritt 15

```
kubectl --context rke2 apply -f szenarien/13-nodes/fix-toleranz.yaml
```

Rollt `web` mit den kürzeren Tolerations neu aus.

**DAS SOLLTEST DU SEHEN**

Wiederholst du den Ausfall, steht nach etwa 50 Sekunden `NotReady` und weitere 30 Sekunden später der Ersatz-Pod auf `rke2-cp`. Starte den Agenten danach wieder.

**RKE2**

`systemctl stop rke2-agent` simuliert den harten Ausfall gut: Das Kubelet ist weg, die Container laufen weiter. Erst `rke2-killall.sh` würde auch sie beenden.

**TALOS**

`talosctl shutdown` und `talosctl reboot` cordonen und drainen den Knoten vorher. Die Pods ziehen dann sauber um, das Szenario tritt nicht auf. Erst `--force` überspringt Cordon und Drain. Einen echten Ausfall simulierst du, indem du die VM im Hypervisor hart ausschaltest.

## 13.5 Pods werden evicted

Das Kubelet schützt seinen Knoten. Wird der Speicher knapp, die Platte voll (auch bei den Inodes) oder gehen die Prozess-IDs aus, setzt es die Conditions `MemoryPressure`, `DiskPressure` oder `PIDPressure` und wirft Pods hinaus. Dieses Hinauswerfen heißt Eviction. Es greift aber auch ohne Knotennot, wenn ein Pod sein eigenes Limit für lokalen Speicher (`ephemeral-storage`) überschreitet. Das ist das Szenario hier, denn eine volle Platte möchtest du im Labor nicht wirklich provozieren.

DATEI szenarien/13-nodes/kaputt-evicted.yaml (Ausschnitt)

```
        command:
          - sh
          - -c
          - dd if=/dev/zero of=/daten/block bs=1M count=200 && sleep 3600
        resources:
          limits:
            ephemeral-storage: 100Mi
        volumeMounts:
          - name: daten
            mountPath: /daten
    volumes:
      - name: daten
        emptyDir: {}
```

200 MiB in ein `emptyDir`, erlaubt sind 100 MiB. Ein `emptyDir` ohne eigenes Medium liegt auf der Platte des Knotens und zählt zum lokalen Speicher des Pods.

MAC Schritt 16

```
kubectl --context rke2 apply -f szenarien/13-nodes/kaputt-evicted.yaml
```

Legt das Deployment `fueller` an. Auf Talos verhält es sich gleich.

**SYMPTOM**

```
$ kubectl --context rke2 get pods -n dbg-13 -l app=fueller
NAME                       READY   STATUS              RESTARTS   AGE
fueller-6f8b7d9c4b-2xjzq   0/1     Error               0          2m4s
fueller-6f8b7d9c4b-9tqwe   0/1     Error               0          71s
fueller-6f8b7d9c4b-mz5rk   1/1     Running             0          9s
```

Alle paar Sekunden ein neuer Pod, die alten bleiben mit `Error` liegen. Je nach Zeitpunkt siehst du auch `ContainerStatusUnknown`.

### Untersuchen

Die Spalte `STATUS` zeigt den Zustand des Containers. Der Grund, warum der Pod beendet wurde, steht eine Ebene höher im Pod-Status.

MAC Schritt 17

```
kubectl --context rke2 get pods -n dbg-13 -l app=fueller \
  -o custom-columns=NAME:.metadata.name,PHASE:.status.phase,REASON:.status.reason
kubectl --context rke2 get pod -n dbg-13 <pod> -o jsonpath='{.status.message}{"\n"}'
```

Phase und Reason jedes Pods, dann die Message eines der Pods mit `Error`.

```
NAME                       PHASE     REASON
fueller-6f8b7d9c4b-2xjzq   Failed    Evicted
fueller-6f8b7d9c4b-9tqwe   Failed    Evicted
fueller-6f8b7d9c4b-mz5rk   Running   <none>

Pod ephemeral local storage usage exceeds the total limit of containers 100Mi.
```

Dieselbe Meldung steht als Event mit dem Reason `Evicted` am Pod. In k9s öffnest du einen der Pods mit `d`. Oben in der Describe-Ansicht stehen `Status: Failed`, `Reason: Evicted` und die Message. Bei einer echten Knotennot sieht die Message anders aus, etwa `The node was low on resource: ephemeral-storage.`, und am Knoten steht `DiskPressure=True` mit dem Taint `node.kubernetes.io/disk-pressure`. Die Schwellen, ab denen das Kubelet eingreift, liest du über den API-Server direkt aus seiner laufenden Konfiguration:

MAC — Schritt 18

```
kubectl --context rke2 get --raw /api/v1/nodes/rke2-w1/proxy/configz \
  | jq .kubeletconfig.evictionHard
```

Der Endpunkt `configz` liefert die aktive Kubelet-Konfiguration. Das funktioniert auf beiden Distributionen gleich.

```
{
  "imagefs.available": "15%",
  "imagefs.inodesFree": "5%",
  "memory.available": "100Mi",
  "nodefs.available": "10%",
  "nodefs.inodesFree": "5%"
}
```

Das sind die Kubernetes-Standardwerte. Fällt der freie Platz unter 10 Prozent, beginnt das Kubelet zu räumen: erst ungenutzte Images und tote Container, dann Pods. Wie voll die Platte wirklich ist, siehst du auf RKE2 mit `df -h /var/lib` auf dem Knoten. Ohne SSH geht es auch mit einem Debug-Pod:

MAC — Schritt 19

```
kubectl --context rke2 debug node/rke2-w1 -n dbg-13 -it \
  --image=busybox:1.37 --profile=sysadmin -- df -h /host/var/lib
```

Startet einen privilegierten Pod im Host-Netzwerk von `rke2-w1`. Das Dateisystem des Knotens liegt unter `/host`. Der Pod heißt `node-debugger-rke2-w1-` plus Zufallsendung und bleibt danach als `Completed` stehen. Lösche ihn mit `kubectl delete pod` oder in k9s mit `Ctrl-d`.

**URSACHE**

Der Container belegt mit seinem `emptyDir` mehr lokalen Speicher, als `limits.ephemeral-storage` erlaubt. Das Kubelet prüft das bei jedem Durchlauf seines Eviction-Managers und beendet den Pod sofort, ohne Schonfrist. Ein evicted Pod startet nicht neu: Er bleibt mit Phase `Failed` stehen, das Deployment ersetzt ihn durch einen neuen, der dasselbe tut.

### Beheben

Der Fix setzt das Limit auf den echten Bedarf und begrenzt das Volume zusätzlich direkt. Mit `sizeLimit` am `emptyDir` lautet die Message bei Überschreitung `Usage of EmptyDir volume "daten" exceeds the limit "250Mi".`. Damit weißt du sofort, welches Volume schuld ist.

MAC — Schritt 20

```
kubectl --context rke2 apply -f szenarien/13-nodes/fix-evicted.yaml
kubectl --context rke2 delete pods -n dbg-13 --field-selector=status.phase=Failed
```

Neues Limit von 300 MiB, `sizeLimit: 250Mi` am Volume. Die zweite Zeile räumt die liegengebliebenen evicted Pods weg. Kubernetes löscht sie sonst erst, wenn der Pod-Garbage-Collector eine sehr hohe Schwelle erreicht.

**DAS SOLLTEST DU SEHEN**

Ein einziger `fueller`-Pod steht dauerhaft auf `Running`, `RESTARTS` bleibt bei 0, es kommen keine neuen Pods nach.

**RKE2**

`kubectl debug node/...` funktioniert ohne Weiteres, mit `chroot /host` hast du sogar die Werkzeuge des Knotens. Auf dem Knoten selbst zeigen `df -h` und `crictl stats` den Verbrauch.

**TALOS**

`kubectl debug node/...` braucht einen Namespace mit `enforce: privileged`, sonst lehnt die clusterweite PSA `baseline` den Pod ab. In neu erzeugten 1.14-Clustern laufen Pods zudem in einer eigenen Sandbox (`workloadIsolation`) und sehen nicht alles vom Host. Besser direkt über die Talos-API: `talosctl mounts` zeigt die Belegung, `talosctl debug <image>` öffnet eine Debug-Shell, ganz ohne Kubelet.

## 13.6 Drain hängt an einem PodDisruptionBudget

Vor Wartungsarbeiten leerst du einen Knoten mit `kubectl drain`. Der Befehl cordont den Knoten und evictet dann jeden Pod über die Eviction-API. Diese API fragt vorher alle PodDisruptionBudgets, ob der Pod gehen darf. Ein zu strenges PDB sagt immer nein.

DATEI `szenarien/13-nodes/kaputt-pdb.yaml (Ausschnitt)`

```
kind: Deployment
metadata:
  name: api
```

```
spec:
  replicas: 2
# ...
---
kind: PodDisruptionBudget
metadata:
  name: api
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: api
```

Zwei Replikas, und das PDB verlangt, dass immer beide verfügbar sind. Das sieht nach maximaler Sicherheit aus und verbietet jede freiwillige Unterbrechung.

MAC

Schritt 21

```
kubectl --context rke2 apply -f szenarien/13-nodes/kaputt-pdb.yaml
kubectl --context rke2 drain rke2-w1 --ignore-daemonsets \
  --delete-emptydir-data --timeout=90s
```

`--ignore-daemonsets` ist nötig, weil auf jedem Knoten DaemonSet-Pods wie `rke2-canal` laufen. `--delete-emptydir-data` erlaubt das Evicten von Pods mit `emptyDir`, etwa dem `fueller`. Der Timeout sorgt dafür, dass der Befehl irgendwann aufgibt.

**SYMPTOM**

```
node/rke2-w1 cordoned
evicting pod dbg-13/api-5b8f9c7d6-hx2lw
error when evicting pods/"api-5b8f9c7d6-hx2lw" -n "dbg-13" (will retry after 5s):
  Cannot evict pod as it would violate the pod's disruption budget.
evicting pod dbg-13/api-5b8f9c7d6-hx2lw
error when evicting pods/"api-5b8f9c7d6-hx2lw" -n "dbg-13" (will retry after 5s):
  Cannot evict pod as it would violate the pod's disruption budget.
...
```

Alle fünf Sekunden derselbe Versuch, bis nach 90 Sekunden `global timeout reached` kommt. Liegen beide `api`-Pods zufällig auf `rke2-cp`, läuft der Drain durch. Dann prüfe mit `kubectl get pods -o wide` und drain den Knoten, auf dem ein `api`-Pod liegt.

## Untersuchen

MAC

Schritt 22

```
kubectl --context rke2 get pdb -n dbg-13
```

Die Spalte `ALLOWED DISRUPTIONS` ist die einzige, die zählt.

```
NAME   MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
api    2               N/A               0                     3m
```

Null erlaubte Unterbrechungen. Das ändert sich auch nicht, solange beide Pods laufen. In k9s erreichst du die Liste mit `:pdb`, den Knoten mit `:nodes`. Dort steht `rke2-w1` jetzt auf `Ready,SchedulingDisabled`.

**URSACHE**

`minAvailable: 2` bei zwei Replikas erlaubt null Evictions. Die Eviction-API antwortet deshalb mit HTTP 429 (`Too Many Requests`), und `kubectl drain` versucht es endlos weiter. Ein PDB schützt nur vor freiwilligen Unterbrechungen. Gegen einen Knotenausfall hilft es nicht, gegen Wartung blockiert es, wenn es zu streng ist.

## Beheben

**MAC** Schritt 23

```
kubectl --context rke2 apply -f szenarien/13-nodes/fix-pdb.yaml
kubectl --context rke2 drain rke2-w1 --ignore-daemonsets --delete-emptydir-data
kubectl --context rke2 uncordon rke2-w1
```

Das neue PDB erlaubt mit `maxUnavailable: 1` eine Unterbrechung. Der Drain läuft durch, danach gibst du den Knoten mit `uncordon` wieder frei. In k9s gehen Cordon, Uncordon und Drain in der Knotenansicht mit `c`, `u` und `r`.

**DAS SOLLTEST DU SEHEN**

`kubectl --context rke2 get pdb -n dbg-13` zeigt `ALLOWED DISRUPTIONS 1`, der Drain endete mit `node/rke2-w1 drained`. Nach `uncordon` steht der Knoten wieder auf `Ready` ohne `SchedulingDisabled`.

**STOLPERFALLE**

Ein abgebrochener Drain lässt den Knoten gecordont zurück. Neue Pods landen dann nie auf ihm, ohne dass irgendwo ein Fehler steht. Nach jedem Drain, auch einem gescheiterten, gehört ein Blick auf `kubectl get nodes` dazu.

**RKE2**

Drain machst du vor jeder Wartung selbst, etwa vor einem Upgrade von RKE2. Das PDB greift genau dabei.

**TALOS**

`talosctl reboot`, `shutdown` und `upgrade` cordonen und drainen den Knoten selbst, ebenfalls über die Eviction-API. Ein zu strenges PDB verzögert also auch diese Befehle. Prüfe die PDBs vor einem Talos-Upgrade.

## 13.7 Aufräumen

MAC Schritt 24

```
kubectl --context rke2 delete ns dbg-13
kubectl --context talos delete ns dbg-13
```

Löscht alle Objekte dieses Kapitels. Prüfe vorher, dass beide Worker `Ready` sind und keiner mehr gecordont ist.

**PROBIER ES SELBST**

Ein Kollege meldet: „Auf `talos-w1` startet seit gestern kein neuer Pod mehr, der Knoten ist aber `Ready`.“ Spiele das nach mit `kubectl --context talos cordon talos-w1` und lege danach das Deployment `web` mit vier Replikas an. Finde die Ursache nur über `kubectl get nodes`, `describe node` und die Events der Pods. Welche Zeile in `describe node` verrät es, und welcher Taint steht dabei?

Hinweis: Achte auf `Unschedulable: true` und den Taint `node.kubernetes.io/unschedulable:NoSchedule`. Mit `uncordon` ist der Spuk vorbei.

**WAS DU GELERNT HAST**

- `Unknown` in den Conditions heißt: Das Kubelet schweigt. `False` heißt: Das Kubelet läuft und meldet selbst ein Problem.
- Auf RKE2 untersuchst du das Kubelet mit `systemctl`, `journalctl` und `kubelet.log`, auf Talos mit `talosctl services`, `service kubelet` und `logs kubelet`.
- Nach 50 Sekunden ohne Meldung wird ein Knoten `NotReady`, nach weiteren 300 Sekunden werden seine Pods zum Löschen markiert und hängen in `Terminating`, bis das Kubelet zurück ist.
- Evicted Pods haben Phase `Failed` und Reason `Evicted`. Die Message nennt die Ressource und das Limit.
- Ein PDB mit `minAvailable` gleich der Replikazahl blockiert jeden Drain. `ALLOWED DISRUPTIONS` zeigt es auf einen Blick.

KAPITEL 14

# Control Plane, etcd und Webhooks

*Du prüfst API-Server und etcd auf beiden Distributionen, sicherst etcd vor jedem Eingriff, holst einen abgestürzten API-Server zurück und erkennst abgelaufene Zertifikate, hängende Webhooks und Namespaces, die nicht sterben wollen.*

**Dauer:** 90 Minuten **Voraussetzung:** Kapitel 2, 3 und 13

## 14.1 Was in diesem Kapitel kaputtgeht

Alles, was du bisher untersucht hast, lief über den API-Server. In diesem Kapitel ist er selbst das Problem, oder etwas, das er bei jeder Anfrage fragt: etcd, ein Webhook, eine aggregierte API. Fehler auf dieser Ebene treffen nicht einen Pod, sondern alle Benutzer des Clusters gleichzeitig.

| Szenario | Symptom | Wo die Antwort steht |
|---|---|---|
| API-Server startet nicht | `connection refused` bei jedem `kubectl` | Log des Containers `kube-apiserver` auf dem Knoten |
| Client-Zertifikat abgelaufen | `You must be logged in to the server` | `openssl x509 -enddate`, Log des API-Servers |
| Webhook nicht erreichbar | `failed calling webhook` beim Anlegen | `ValidatingWebhookConfiguration`, Endpoints des Service |
| Namespace hängt | `Terminating` ohne Ende | Conditions des Namespace, `kubectl get apiservice` |

Die Szenarien laufen im Namespace `dbg-14`. Zwei davon greifen in die Control Plane selbst ein. Lies deshalb den Abschnitt über etcd-Snapshots, bevor du anfängst.

## 14.2 Ist die API gesund?

Der API-Server beschreibt seinen Zustand selbst, über zwei Endpunkte. `/livez` beantwortet die Frage, ob der Prozess lebt und neu gestartet werden sollte. `/readyz` sagt, ob er Anfragen bedienen kann, also auch, ob etcd antwortet. Mit `?verbose` listet er jede Einzelprüfung auf.

MAC Schritt 1

```
kubectl --context rke2 get --raw='/readyz?verbose'
kubectl --context talos get --raw='/livez?verbose' | tail -n 3
```

Beide Aufrufe gehen über dieselbe Verbindung wie jedes `kubectl get`, mit deinen Admin-Rechten.

```
[+]ping ok
[+]log ok
[+]etcd ok
[+]etcd-readiness ok
[+]informer-sync ok
...
[+]shutdown ok
readyz check passed
```

Eine gescheiterte Prüfung steht mit `[-]` da, etwa `[-]etcd failed: reason withheld`. Den Grund hält der API-Server bewusst zurück, er steht in seinem eigenen Log. Einzelne Prüfungen fragst du direkt ab, zum Beispiel `/readyz/etcd`.

**WARUM DAS WICHTIG IST**

Kommt gar keine Antwort, sondern `The connection to the server 172.16.0.160:6443 was refused – did you specify the right host or port?`, läuft auf diesem Port kein API-Server. Dann hilft kein `kubectl` mehr. Du brauchst einen Weg, der nicht über die Kubernetes-API führt: SSH auf RKE2, die Talos-API auf Talos.

## 14.3 Vor jedem Eingriff: etcd prüfen und sichern

etcd ist das Gedächtnis des Clusters. Jedes Objekt, jedes Secret liegt dort. Bevor du an der Control Plane schraubst, prüfst du, ob etcd gesund ist, und ziehst einen Snapshot. Beide Labor-Cluster haben genau ein etcd-Mitglied, auf `rke2-cp` und `talos-cp`.

### RKE2

RKE2 betreibt etcd als Static Pod. `etcdctl` liegt im Image, die Client-Zertifikate liegen auf dem Knoten unter `/var/lib/rancher/rke2/server/tls/etcd/` und sind in den Pod eingehängt.

RKE2-CP — Schritt 2

```
etcdcontainer=$(crictl ps --name etcd --quiet)
TLS="--cacert /var/lib/rancher/rke2/server/tls/etcd/server-ca.crt
  --cert /var/lib/rancher/rke2/server/tls/etcd/server-client.crt
  --key /var/lib/rancher/rke2/server/tls/etcd/server-client.key"
crictl exec "$etcdcontainer" etcdctl $TLS endpoint health
crictl exec "$etcdcontainer" etcdctl $TLS endpoint status -w table
crictl exec "$etcdcontainer" etcdctl $TLS alarm list
```

In der RKE2-Umgebung aus Kapitel 2 als root. `$TLS` steht absichtlich ohne Anführungszeichen, die Shell soll es in einzelne Argumente zerlegen.

```
127.0.0.1:2379 is healthy: successfully committed proposal: took = 2.4ms
```

`endpoint status` zeigt Version, Datenbankgröße, Leader und Raft-Index als Tabelle. `alarm list` gibt nichts aus, solange kein Alarm aktiv ist. Ein `NOSPACE`-Alarm heißt: Die Datenbank hat ihre Quota erreicht, etcd nimmt keine Schreibzugriffe mehr an.

RKE2-CP — Schritt 3

```
rke2 etcd-snapshot save --name vor-kapitel-14
rke2 etcd-snapshot list
```

Ein Snapshot von Hand. `list` (Alias `ls`) zeigt Name, Ort, Größe und Zeitpunkt aller Snapshots.

**DAS SOLLTEST DU SEHEN**

Eine Zeile `vor-kapitel-14-rke2-cp-...` mit dem Ort `file:///var/lib/rancher/rke2/server/db/snapshots/...`. Dazu kommen die automatischen Snapshots, die RKE2 standardmäßig um 0 und 12 Uhr zieht und von denen es fünf behält. Manuelle Snapshots räumt RKE2 nicht automatisch weg.

### Talos

Auf Talos ist etcd kein Pod, sondern ein Talos-Dienst. Alles geht über die Talos-API, auch wenn Kubernetes gar nicht mehr antwortet.

MAC — Schritt 4

```
talosctl -n 172.16.0.170 service etcd
talosctl -n 172.16.0.170 etcd members
talosctl -n 172.16.0.170 etcd status
talosctl -n 172.16.0.170 etcd alarm list
```

Dienststatus, Mitgliederliste, Größe und Leader, zuletzt die Alarme.

```
NODE           MEMBER             DB SIZE   IN USE            LEADER
172.16.0.170   3f1c2b9a7d4e6a10   28 MB     9.1 MB (32.50%)   3f1c2b9a7d4e6a10
```

MAC — Schritt 5

```
mkdir -p ~/talos-backup
talosctl -n 172.16.0.170 etcd snapshot ~/talos-backup/etcd-$(date +%F).snapshot
```

Streamt einen konsistenten Snapshot auf den Mac. Die Datei enthält alle Secrets des Clusters. Sie gehört deshalb nicht ins Repo.

| RKE2 | TALOS |
| --- | --- |
| Wiederherstellen: `systemctl stop rke2-server`, dann `rke2 server --cluster-reset --cluster-reset-restore-path=<snapshot>`; danach `systemctl start rke2-server`. RKE2 legt Snapshots selbst nach Zeitplan an. | Wiederherstellen: etcd-Daten des Knotens löschen und neu bootstrappen mit `talosctl bootstrap --recover-from=<snapshot>` (Ablauf im Guide „Disaster Recovery“). Einen Zeitplan für Snapshots bringt Talos nicht mit, den baust du selbst, etwa als Cron-Job mit `talosctl`. |

## 14.4 Der API-Server startet nicht

Ein häufiger Grund für einen toten API-Server nach einem Upgrade: ein Argument, das die neue Version nicht mehr kennt. Feature-Gates sind dafür besonders anfällig. Ist ein Feature GA geworden, verschwindet sein Gate nach einigen Versionen, und wer es noch setzt, bekommt einen API-Server, der beim Start abbricht. Du baust das mit einem erfundenen Gate nach.

DATEI `szenarien/14-controlplane/rke2-config-kaputt.yaml (Ausschnitt)`

```
kube-apiserver-arg:
  - "feature-gates=DebugBuchGate=true"
```

Eine Drop-in-Datei für RKE2. Sie kommt nach `/etc/rancher/rke2/config.yaml.d/`, RKE2 liest sie nach der `config.yaml`. Zum Zurücknehmen löschst du sie einfach.

DATEI `labor/talos/patch-apiserver-featuregate.yaml (Ausschnitt)`

```
apiVersion: v1alpha1
kind: KubeAPIServerConfig
extraArgs:
  feature-gates: DebugBuchGate=true
```

Dasselbe für Talos. Seit 1.14 konfiguriert das Dokument `KubeAPIServerConfig` den API-Server, `cluster.apiServer` ist veraltet.

MAC — Schritt 6

```
scp szenarien/14-controlplane/rke2-config-kaputt.yaml <user>@172.16.0.160:/tmp/
ssh <user>@172.16.0.160
```

Kopiert die Datei auf `rke2-cp` und meldet dich dort an.

RKE2-CP — Schritt 7

```
install -D -m 600 /tmp/rke2-config-kaputt.yaml \
  /etc/rancher/rke2/config.yaml.d/90-kaputt.yaml
systemctl restart rke2-server --no-block
```

Als root. `--no-block` kehrt sofort zurück, statt auf einen Start zu warten, der nie fertig wird. Die Workloads laufen weiter, nur die Control Plane ist weg.

MAC — Schritt 8

```
talosctl -n 172.16.0.170 patch mc \
  --patch @labor/talos/patch-apiserver-featuregate.yaml
```

Talos rendert den Static Pod des API-Servers neu, ohne den Knoten neu zu starten.

**SYMPTOM**

```
$ kubectl --context rke2 get nodes
The connection to the server 172.16.0.160:6443 was refused - did you specify
the right host or port?
```

Auf dem Talos-Cluster dasselbe mit `172.16.0.170:6443`. Je nach Zeitpunkt siehst du auch einen Timeout oder `EOF`.

### Untersuchen auf RKE2

Der API-Server ist ein Container wie jeder andere. `crictl` fragt containerd direkt und braucht keinen API-Server.

RKE2-CP — Schritt 9

```
crictl ps -a --name kube-apiserver
crictl logs $(crictl ps -a --name kube-apiserver --quiet | head -n 1) 2>&1 \
  | tail -n 5
journalctl -u rke2-server --since "5 min ago" --no-pager | tail -n 10
```

Die Container-Liste zeigt `Exited`-Einträge mit steigender Zahl in der Spalte `ATTEMPT`. Das Log des jüngsten Versuchs nennt den Grund. Das Journal zeigt, dass RKE2 auf den API-Server wartet.

```
Error: invalid argument "DebugBuchGate=true" for "--feature-gates" flag:
unrecognized feature gate: DebugBuchGate
```

Dieselben Logs liegen als Dateien unter `/var/log/pods/kube-system_kube-apiserver-rke2-cp_*/kube-apiserver/`. Das Manifest, das RKE2 aus der Konfiguration erzeugt hat, liegt in `/var/lib/rancher/rke2/agent/pod-manifests/kube-apiserver.yaml`. Ein `grep feature-gates` darin bestätigt, woher das Argument kommt.

### Untersuchen auf Talos

MAC — Schritt 10

```
talosctl -n 172.16.0.170 containers --namespace cri | grep kube-apiserver
talosctl -n 172.16.0.170 logs --namespace cri \
  kube-system/kube-apiserver-talos-cp:kube-apiserver:<id>
```

Die erste Zeile zeigt die Pod-Sandbox und den Container des API-Servers mit seiner ID. Setze sie in die zweite Zeile ein. Weil der Container ständig neu startet, ändert sich die ID mit jedem Versuch.

Das Log enthält dieselbe Meldung wie auf RKE2. Welche Konfiguration Talos gerendert hat, zeigt `talosctl -n 172.16.0.170 get staticpods kube-apiserver -o yaml`. Einen schnellen Gesamtüberblick liefert `talosctl -n 172.16.0.170 health`: Es bleibt bei der Prüfung der Kubernetes-Komponenten hängen.

**URSACHE**

kube-apiserver bricht beim Parsen seiner Flags ab, weil es das Feature-Gate `DebugBuchGate` nicht kennt. Das Kubelet startet den Static Pod immer wieder neu, jeder Versuch endet sofort. Ohne API-Server antwortet keine Kubernetes-Schnittstelle mehr. Controller, Scheduler und `kubectl` stehen still, die laufenden Pods arbeiten aber weiter.

### Beheben

RKE2-CP — Schritt 11

```
rm /etc/rancher/rke2/config.yaml.d/90-kaputt.yaml
systemctl restart rke2-server
```

RKE2 schreibt das Manifest des API-Servers bei jedem Start neu, ohne das Gate.

MAC — Schritt 12

```
talosctl -n 172.16.0.170 edit mc
```

Lösche im Dokument `KubeAPIServerConfig` die Zeile `feature-gates: DebugBuchGate=true`. Talos rendert den Static Pod sofort neu.

**DAS SOLLTEST DU SEHEN**

`kubectl --context rke2 get --raw='/readyz'` und dasselbe für `talos` antworten mit `ok`. `talosctl -n 172.16.0.170 health` läuft ohne Fehler durch.

**RKE2**

Ohne API-Server gehst du per SSH auf den Knoten. `crictl`, `journalctl` und die Dateien unter `/var/log/pods/` funktionieren unabhängig von Kubernetes. Die Konfiguration korrigierst du in `config.yaml` oder `config.yaml.d/`.

**TALOS**

Die Talos-API (Port 50000) läuft weiter, wenn Kubernetes tot ist. `containers`, `logs`, `edit mc` und `etcd status` gehen alle über sie. Ist auch die Talos-API weg, bleibt nur die Konsole der VM.

## 14.5 Abgelaufenes Client-Zertifikat

Zertifikate laufen ab, und zwar genau dann, wenn niemand daran denkt. Du erzeugst dir deshalb ein Zertifikat, das nach zehn Minuten abläuft. Das Skript schickt einen CertificateSigningRequest

mit `expirationSeconds: 600` an den Cluster, genehmigt ihn und baut eine eigene kubeconfig für den Benutzer `dbg14-user`.

DATEI `szenarien/14-controlplane/kurzlebiges-zertifikat.sh (Ausschnitt)`

```
kubectl --context "$CTX" apply -f - <<YAML
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: ${NAME}
spec:
  request: $(base64 < "${DIR}/${NAME}.csr" | tr -d '\n')
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 600
  usages:
    - client auth
YAML
kubectl --context "$CTX" certificate approve "$NAME"
```

600 Sekunden sind das kleinste erlaubte `expirationSeconds`. Schlüssel und kubeconfig landen in `szenarien/14-controlplane/tmp-<kontext>/`, das Verzeichnis ist von Git ausgenommen.

MAC Schritt 13

```
kubectl --context rke2 apply -f szenarien/14-controlplane/kaputt-zertifikat.yaml
szenarien/14-controlplane/kurzlebiges-zertifikat.sh rke2
export KC=szenarien/14-controlplane/tmp-rke2/dbg14-user.kubeconfig
kubectl --kubeconfig $KC get configmaps
```

Legt Namespace, ConfigMap und RoleBinding an, erzeugt das Zertifikat und liest mit ihm die ConfigMaps in `dbg-14`. Das klappt. Jetzt zehn Minuten Pause.

**SYMPTOM**

```
$ kubectl --kubeconfig $KC get configmaps
error: You must be logged in to the server (Unauthorized)
```

Je nach kubectl-Version steht in der Klammer auch `the server has asked for the client to provide credentials`. An der kubeconfig hat sich nichts geändert.

### Untersuchen

`Unauthorized` heißt: Der API-Server weiß nicht, wer du bist. Bei Client-Zertifikaten ist der erste Verdacht das Ablaufdatum.

MAC Schritt 14

```
kubectl config view --kubeconfig $KC --raw \
  -o jsonpath='{.users[0].user.client-certificate-data}' \
  | base64 -d | openssl x509 -noout -subject -enddate
```

Holt das Zertifikat aus der kubeconfig und zeigt Inhaber und Ablaufzeit.

```
subject=CN=dbg14-user
notAfter=Nov  9 21:50:12 2026 GMT
```

Der Zeitpunkt liegt in der Vergangenheit. Die Gegenprobe steht im Log des API-Servers, das du mit deinem Admin-Zugang lesen kannst:

MAC Schritt 15

```
kubectl --context rke2 logs -n kube-system kube-apiserver-rke2-cp --since=15m \
  | grep -m 1 "certificate has expired"
```

Auf Talos heißt der Pod `kube-apiserver-talos-cp`.

Dort steht sinngemäß `"Unable to authenticate the request" err="[x509: certificate has expired or is not yet valid: current time ... is after ...]"`.

**URSACHE**

Das Client-Zertifikat ist abgelaufen. Der API-Server prüft bei jeder Anfrage die Gültigkeit und weist abgelaufene Zertifikate ab. Dem Client sagt er aus Sicherheitsgründen nur `Unauthorized`, den genauen Grund schreibt er ins eigene Log.

| Meldung beim Client | Was abgelaufen oder falsch ist |
|---|---|
| `You must be logged in to the server (Unauthorized)` | dein Client-Zertifikat (oder Token) aus der kubeconfig |
| `x509: certificate has expired or is not yet valid` | das Serving-Zertifikat des API-Servers, oder die Uhr deines Rechners geht falsch |
| `x509: certificate signed by unknown authority` | die CA in deiner kubeconfig passt nicht zum Cluster, etwa nach einer Neuinstallation |

### Beheben

Für den Testbenutzer erzeugst du einfach ein neues Zertifikat: das Skript noch einmal aufrufen. Wichtiger ist die Frage, wann die echten Zertifikate deiner Cluster ablaufen.

RKE2-CP Schritt 16

```
rke2 certificate check --output table
```

Listet alle Zertifikate von RKE2 mit Ablaufdatum, Restlaufzeit und Status.

MAC Schritt 17

```
talosctl -n 172.16.0.170 get kubernetesdynamiccerts -o yaml | head -n 20
talosctl config info
kubectl config view --context talos --minify --raw \
  -o jsonpath='{.users[0].user.client-certificate-data}' \
  | base64 -d | openssl x509 -noout -enddate
```

Die dynamischen Kubernetes-Zertifikate von Talos im PEM-Format, dann die Laufzeit deiner `talosconfig` in der Zeile `Certificate expires`, zuletzt das Ablaufdatum deines Admin-Zertifikats für den Kontext `talos`.

**DAS SOLLTEST DU SEHEN**

Nach erneutem Aufruf des Skripts liefert `kubectl --kubeconfig $KC get configmaps` wieder die ConfigMap `hallo`. Für den Rest des Kapitels brauchst du `$KC` nicht mehr: `unset KC`.

**RKE2**

Zertifikate gelten 365 Tage. RKE2 erneuert beim Start jedes Zertifikat, das abgelaufen ist oder in weniger als 120 Tagen abläuft. Von Hand: `systemctl stop rke2-server`, `rke2 certificate rotate`, `systemctl start rke2-server`. Danach musst du `/etc/rancher/rke2/rke2.yaml` erneut auf den Mac kopieren, denn auch das Admin-Zertifikat ist neu.

**TALOS**

Talos erneuert die Server-Zertifikate selbst, das Kubelet braucht dafür mindestens einen Neustart im Jahr. Client-Zertifikate sind deine Sache: Jedes `talosctl kubeconfig` erzeugt eine kubeconfig mit einem Jahr Laufzeit. Die `talosconfig` erneuerst du mit `talosctl config new`, solange die alte noch gilt.

## 14.6 Ein Webhook blockiert den Namespace

Admission-Webhooks fragt der API-Server bei jedem passenden Schreibzugriff. Ist der Webhook nicht erreichbar, entscheidet `failurePolicy`: `Ignore` lässt die Anfrage durch, `Fail` lehnt sie ab. Kyverno ist ein Beispiel (Kapitel 11), hier geht es um den allgemeinen Fall.

DATEI `szenarien/14-controlplane/kaputt-webhook.yaml (Ausschnitt)`

```yaml
kind: ValidatingWebhookConfiguration
metadata:
  name: dbg14-pruefung
webhooks:
  - name: pruefung.dbg14.example.com
    failurePolicy: Fail
    timeoutSeconds: 5
    clientConfig:
      service:
        name: pruefer
        namespace: dbg-14
        path: /validate
    namespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: dbg-14
    rules:
      - operations: ["CREATE", "UPDATE"]
        resources: ["configmaps"]
# ...
```

Der Service `pruefer` hat keinen einzigen Pod. Der `namespaceSelector` begrenzt den Schaden auf `dbg-14`.

MAC — Schritt 18

```
kubectl --context rke2 apply -f szenarien/14-controlplane/kaputt-webhook.yaml
kubectl --context rke2 create configmap test -n dbg-14 --from-literal=a=b
```

Legt Webhook und Service an und versucht danach, eine ConfigMap zu erstellen.

**SYMPTOM**

```
error: failed to create configmap: Internal error occurred: failed calling webhook
"pruefung.dbg14.example.com": failed to call webhook: Post
"https://pruefer.dbg-14.svc:443/validate?timeout=5s": <Grund>
```

Am Ende steht der eigentliche Grund. Er hängt davon ab, wie der API-Server den Service auflöst und was das Netz antwortet: `connect: connection refused`, `no endpoints available for service "pruefer"` oder, bei einem Backend, das nicht antwortet, `context deadline exceeded` nach `timeoutSeconds`.

### Untersuchen

Die Meldung nennt den Namen des Webhooks, nicht den der Konfiguration. Beide findest du in der Liste:

MAC — Schritt 19

```
kubectl --context rke2 get \
  validatingwebhookconfigurations,mutatingwebhookconfigurations
kubectl --context rke2 get validatingwebhookconfiguration dbg14-pruefung \
  -o jsonpath='{.webhooks[*].clientConfig.service}{"\n"}'
kubectl --context rke2 get endpointslices -n dbg-14
```

Alle Webhook-Konfigurationen, dann das Ziel des fraglichen Webhooks, zuletzt die Endpunkte des Ziel-Service.

```
NAME            ADDRESSTYPE   PORTS     ENDPOINTS   AGE
pruefer-x8k2d   IPv4          <unset>   <unset>     2m
```

In k9s öffnest du die Liste mit `:validatingwebhookconfigurations` und mit `y` das YAML des Eintrags. Keine Endpunkte, also kein Pod hinter dem Service.

**URSACHE**

Der API-Server muss für jede ConfigMap in `dbg-14` den Webhook fragen. Der Service dahinter hat keine Pods, der Aufruf scheitert. Wegen `failurePolicy: Fail` lehnt der API-Server die Anfrage ab. Ohne `namespaceSelector` träfe das jeden Namespace, und bei Pods als Ziel könnte der Cluster nicht einmal seine eigenen Komponenten neu starten.

### Beheben

Richtig ist, das Backend wieder zum Laufen zu bringen. Ist es dauerhaft weg, entfernst du die Konfiguration. Im Notfall schaltest du vorübergehend auf `Ignore`:

MAC Schritt 20

```
kubectl --context rke2 patch validatingwebhookconfiguration dbg14-pruefung \
  --type=json \
  -p='[{"op":"replace","path":"/webhooks/0/failurePolicy","value":"Ignore"}]'
kubectl --context rke2 delete validatingwebhookconfiguration dbg14-pruefung
```

Die erste Zeile ist der Notausgang, die zweite die eigentliche Lösung. Webhook-Konfigurationen selbst laufen nie durch Webhooks, du kannst sie also immer ändern oder löschen.

**DAS SOLLTEST DU SEHEN**

`kubectl --context rke2 create configmap test -n dbg-14 --from-literal=a=b` meldet `configmap/test created`.

## 14.7 Der Namespace bleibt in Terminating

Ein Namespace verschwindet erst, wenn der Namespace-Controller alles darin gelöscht hat. Dafür muss er wissen, welche Ressourcentypen es überhaupt gibt, und fragt die Discovery des API-Servers. Antwortet eine aggregierte API nicht, ist die Liste unvollständig, und der Controller löscht den Namespace sicherheitshalber nicht. Der Klassiker: Jemand entfernt den metrics-server, aber die APIService `v1beta1.metrics.k8s.io` bleibt stehen.

DATEI `szenarien/14-controlplane/kaputt-apiservice.yaml (Ausschnitt)`

```yaml
kind: APIService
metadata:
  name: v1alpha1.debug.dbg14.example.com
spec:
  group: debug.dbg14.example.com
  version: v1alpha1
  service:
    name: debug-api
    namespace: dbg-14
# ...
---
kind: ConfigMap
metadata:
  name: festgehalten
  namespace: dbg-14
  finalizers:
    - dbg14.example.com/aufraeumen
```

Eine API-Gruppe, deren Service keine Pods hat, und eine ConfigMap mit einem Finalizer, den kein Controller je entfernt. Zwei typische Gründe für hängende Namespaces in einer Datei.

MAC Schritt 21

```
kubectl --context rke2 apply -f szenarien/14-controlplane/kaputt-apiservice.yaml
kubectl --context rke2 delete ns dbg-14 --wait=false
```

`--wait=false`, weil `kubectl` sonst ewig auf das Ende wartet.

**SYMPTOM**

```
$ kubectl --context rke2 get ns dbg-14
NAME     STATUS        AGE
dbg-14   Terminating   14m
```

Auch nach Minuten keine Änderung. Manche `kubectl`-Befehle melden nebenbei Fehler zur Gruppe `debug.dbg14.example.com`.

### Untersuchen

Der Namespace-Controller schreibt in die Conditions des Namespace, woran er hängt.

**MAC** Schritt 22

```
kubectl --context rke2 get ns dbg-14 -o json | jq -r \
  '.status.conditions[] | select(.status=="True") | "\(.reason): \(.message)"'
```

Nur die Conditions mit Status `True`, also die tatsächlichen Probleme.

```
DiscoveryFailed: Discovery failed for some groups, 1 failing: unable to retrieve
  the complete list of server APIs: debug.dbg14.example.com/v1alpha1: ...
SomeResourcesRemain: Some resources are remaining: configmaps. has 1 resource
  instances
SomeFinalizersRemain: Some content in the namespace has finalizers remaining:
  dbg14.example.com/aufraeumen in 1 resource instances
```

**MAC** Schritt 23

```
kubectl --context rke2 get apiservice v1alpha1.debug.dbg14.example.com
kubectl --context rke2 get configmaps -n dbg-14
```

Die APIService der Gruppe und was im Namespace noch übrig ist. Ohne Namen listet `get apiservice` alle, die vom API-Server selbst tragen in `SERVICE` den Wert `Local`.

```
NAME                                SERVICE            AVAILABLE                  AGE
v1alpha1.debug.dbg14.example.com    dbg-14/debug-api   False (ServiceNotFound)    15m
```

Vor dem Löschen stand dort `False (MissingEndpoints)`. Jetzt hat der Namespace-Controller den Service schon entfernt, die APIService zeigt ins Leere. In k9s findest du beide Listen unter `:apiservices` und `:namespaces`.

**URSACHE**

Zwei Dinge halten den Namespace fest. Erstens ist die APIService nicht verfügbar, die Discovery bleibt unvollständig, und der Controller verweigert das endgültige Löschen. Weil die APIService cluster-weit gilt, betrifft das jeden Namespace, den du in dieser Zeit löschst. Zweitens trägt die ConfigMap einen Finalizer. Der API-Server löscht sie erst, wenn jemand ihn entfernt, und dieser Jemand existiert nicht.

### Beheben

MAC

Schritt 24

```
kubectl --context rke2 delete apiservice v1alpha1.debug.dbg14.example.com
kubectl --context rke2 patch configmap festgehalten -n dbg-14 \
  --type=merge -p '{"metadata":{"finalizers":null}}'
```

Erst die verwaiste APIService weg, dann den Finalizer entfernen. Bei echten Finalizern prüfst du vorher, welcher Controller ihn setzt und warum er ihn nicht entfernt.

**DAS SOLLTEST DU SEHEN**

Wenige Sekunden später meldet `kubectl --context rke2 get ns dbg-14` `NotFound`. `kubectl --context rke2 get apiservice` zeigt nur noch `True` in der Spalte `AVAILABLE`.

**STOLPERFALLE**

Im Netz kursiert der Tipp, den Finalizer `kubernetes` direkt am Namespace über `/finalize` zu entfernen. Der Namespace ist dann weg, seine Objekte aber nicht unbedingt: Sie bleiben verwaist in etcd und tauchen wieder auf, sobald jemand einen Namespace gleichen Namens anlegt. Behebe immer die Ursache.

**PROBIER ES SELBST**

Spiele das Webhook-Szenario auf `talos` nach, aber ändere vorher in einer Kopie der Datei `failurePolicy` auf `Ignore` und `timeoutSeconds` auf `10`. Starte dann im Service `pruefer` statt keines Pods einen Pod, der Verbindungen annimmt, aber kein TLS spricht (etwa `nicolaka/netshoot:v0.15` mit `nc -lk 8443`, Selektor und Label anpassen). Wie lange dauert jetzt `kubectl create configmap`, und was steht im Log des API-Servers?

Hinweis: Mit `Ignore` geht die ConfigMap durch, aber erst nach dem Timeout. Ein langsamer Webhook mit `Ignore` macht den Cluster träge statt kaputt. Räume danach mit `kubectl --context talos delete validatingwebhookconfiguration dbg14-pruefung` und `kubectl --context talos delete ns dbg-14` auf.

**WAS DU GELERNT HAST**

- `/readyz?verbose` und `/livez?verbose` zeigen die Einzelprüfungen des API-Servers. Kommt gar keine Antwort, brauchst du SSH oder die Talos-API.
- Vor jedem Eingriff an der Control Plane: etcd prüfen (`etcdctl` im etcd-Container bzw. `talosctl etcd status`) und einen Snapshot ziehen (`rke2 etcd-snapshot save` bzw. `talosctl etcd snapshot`).
- Ein API-Server, der nicht startet, schreibt den Grund in sein Container-Log: auf RKE2 über `crictl logs`, auf Talos über `talosctl logs --namespace cri`.
- `Unauthorized` heißt: dein Client-Zertifikat. `x509` beim Client heißt: das Zertifikat oder die CA des Servers.
- Ein Webhook mit `failurePolicy: Fail` und ohne Backend blockiert jede passende Anfrage. Webhook-Konfigurationen lassen sich trotzdem immer löschen.
- Hängende Namespaces erklären sich in ihren Conditions. Häufigste Ursachen: nicht verfügbare APIService und Finalizer ohne Controller.

KAPITEL 15

# Rollouts, Autoscaling und Helm

*Du erkennst hängende Rollouts und rollst sie zurück, bringst einen HPA mit `<unknown>` zum Messen, installierst dafür auf Talos den metrics-server, befreist ein festgefahrenes Helm-Release und liest, warum Jobs aufgeben und ConfigMap-Änderungen nicht ankommen.*

**Dauer:** 75 Minuten **Voraussetzung:** Kapitel 2, 3 und 6

## 15.1 Was in diesem Kapitel kaputtgeht

Die Fehler dieses Kapitels entstehen nicht beim ersten Deployment, sondern bei der zweiten Version, beim Skalieren und beim Automatisieren. Typisch für sie: Nichts stürzt sichtbar ab. Die alte Version läuft weiter, der HPA tut nichts, Helm verweigert sich. Wer nicht nachsieht, merkt es erst Tage später.

| Szenario | Symptom | Wo die Antwort steht |
|---|---|---|
| Rollout hängt | `rollout status` endet mit `exceeded its progress deadline` | Condition `Progressing`, Events des neuen Pods |
| HPA misst nicht | `TARGETS` zeigt `cpu: <unknown>/50%` | Events des HPA, `kubectl top` |
| Helm-Release hängt | `another operation ... is in progress` | `helm history`, Status `pending-upgrade` |
| Job gibt auf | Job `Failed`, mehrere Pods `Error` | Condition und Event `BackoffLimitExceeded` |
| ConfigMap ohne Wirkung | Pod zeigt den alten Wert | Wie die ConfigMap eingebunden ist |

Alle Szenarien laufen im Namespace `dbg-15`. Die Manifeste erfüllen `restricted`, auf Talos gibt es also keine Warnungen der Pod Security Admission.

## 15.2 Der Rollout hängt

Ein Deployment tauscht Pods schrittweise aus. Mit `maxSurge: 1` und `maxUnavailable: 0` startet es zuerst einen neuen Pod und entfernt erst dann einen alten, wenn der neue ready ist. Wird er nie ready, bleibt der Rollout stehen. Nach `progressDeadlineSeconds` erklärt der Deployment-Controller ihn für gescheitert, rollt aber nichts zurück.

MAC Schritt 1

```
kubectl --context rke2 apply -f szenarien/15-rollouts/web-v1.yaml
kubectl --context rke2 rollout status deploy/web -n dbg-15
```

Version 1 mit drei Replikas. `rollout status` endet mit `deployment "web" successfully rolled out`.

DATEI `szenarien/15-rollouts/kaputt-rollout.yaml (Ausschnitt)`

```
  annotations:
    kubernetes.io/change-cause: "v2: httpd auf 8081"
spec:
  progressDeadlineSeconds: 60
# ...
          command: ["httpd", "-f", "-p", "8081", "-h", "/tmp"]
          readinessProbe:
            tcpSocket:
              port: 8080
```

Version 2 lauscht auf einem neuen Port, die Readiness-Probe prüft weiter den alten. Die Frist ist mit 60 Sekunden kurz, der Standard sind 600.

MAC Schritt 2

```
kubectl --context rke2 apply -f szenarien/15-rollouts/kaputt-rollout.yaml
kubectl --context rke2 rollout status deploy/web -n dbg-15
```

Startet den Rollout auf Version 2 und beobachtet ihn.

**SYMPTOM**

```
Waiting for deployment "web" rollout to finish: 1 out of 3 new replicas have been
updated...
error: deployment "web" exceeded its progress deadline
```

Die Anwendung antwortet trotzdem: `kubectl get deploy -n dbg-15` zeigt `READY 3/3`, `UP-TO-DATE 1`. Die drei alten Pods laufen weiter.

### Untersuchen

MAC Schritt 3

```
kubectl --context rke2 describe deploy web -n dbg-15 \
  | sed -n '/^Conditions/,/^Events/p'
kubectl --context rke2 get rs,pods -n dbg-15 -l app=web
```

Die Conditions des Deployments, dann die ReplicaSets und Pods. Das neue ReplicaSet hat einen Pod, der `0/1` ready ist.

```
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    False   ProgressDeadlineExceeded
```

`Available True` und `Progressing False` zusammen heißen: Die Anwendung läuft, aber die neue Version kommt nicht an. Den Grund liefert der neue Pod:

MAC — Schritt 4

```
kubectl --context rke2 events -n dbg-15 --types=Warning
```

Nur Warnungen im Namespace.

```
LAST SEEN   TYPE      REASON      OBJECT                     MESSAGE
4s          Warning   Unhealthy   pod/web-6d5f8c9b7-tq4lz    Readiness probe ...
```

Die volle Message lautet `Readiness probe failed: dial tcp 10.42.1.23:8080: connect: connection refused`. In k9s öffnest du mit `:deploy dbg-15` die Deployments und mit `z` die ReplicaSets des gewählten Eintrags. Mit `Enter` gehst du weiter zu den Pods, mit `d` zu den Events.

**URSACHE**

Der neue Pod lauscht auf 8081, die Readiness-Probe klopft auf 8080 und bekommt `connection refused`. Der Pod wird nie ready, also entfernt das Deployment keinen alten Pod und startet keinen weiteren neuen. Nach 60 Sekunden ohne Fortschritt setzt der Controller `Progressing=False` mit `ProgressDeadlineExceeded`. Das ist nur eine Meldung, der Rollout bleibt in diesem Zustand stehen.

### Beheben

MAC — Schritt 5

```
kubectl --context rke2 rollout history deploy/web -n dbg-15
kubectl --context rke2 rollout undo deploy/web -n dbg-15
```

`history` listet die Revisionen mit der `change-cause`-Annotation. `undo` geht auf die vorherige Revision zurück, mit `--to-revision=<n>` auf eine bestimmte.

```
REVISION  CHANGE-CAUSE
1         v1: httpd auf 8080
2         v2: httpd auf 8081
```

Danach rollst du die korrigierte Version aus, in der Probe und Port zusammenpassen:

MAC — Schritt 6

```
kubectl --context rke2 apply -f szenarien/15-rollouts/fix-rollout.yaml
kubectl --context rke2 rollout status deploy/web -n dbg-15
```

Version 2 mit `readinessProbe` auf Port 8081.

**DAS SOLLTEST DU SEHEN**

`deployment "web" successfully rolled out`, `kubectl --context rke2 get deploy web -n dbg-15` zeigt `3/3` bei `UP-TO-DATE 3`. In `rollout history` steht jetzt die neue `change-cause`.

**STOLPERFALLE**

`kubectl apply` meldet bei einem kaputten Rollout `configured` und endet erfolgreich. Pipelines, die nur `apply` ausführen, merken nichts. Ein `kubectl rollout status --timeout=...` danach liefert bei überschrittener Frist einen Exit-Code ungleich 0 und lässt die Pipeline scheitern.

## 15.3 Der HPA misst nicht

Ein HorizontalPodAutoscaler mit dem Ziel `averageUtilization: 50` rechnet so: aktueller CPU-Verbrauch geteilt durch den CPU-Request, gemittelt über alle Pods. Dafür braucht er zwei Dinge: Messwerte aus der Resource-Metrics-API (`metrics.k8s.io`, geliefert vom metrics-server) und einen Request in jedem Container.

DATEI `szenarien/15-rollouts/kaputt-hpa.yaml (Ausschnitt)`

```
      containers:
        - name: rechner
          image: busybox:1.37
          resources:
            limits:
              memory: 32Mi
---
kind: HorizontalPodAutoscaler
spec:
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

Kein CPU-Request im Container. 50 Prozent von nichts lassen sich nicht berechnen.

**MAC** Schritt 7

```
kubectl --context rke2 apply -f szenarien/15-rollouts/kaputt-hpa.yaml
kubectl --context talos apply -f szenarien/15-rollouts/kaputt-hpa.yaml
```

Dasselbe Manifest auf beiden Clustern.

**SYMPTOM**

```
$ kubectl --context rke2 get hpa -n dbg-15
NAME      REFERENCE            TARGETS              MINPODS   MAXPODS   REPLICAS   AGE
rechner   Deployment/rechner   cpu: <unknown>/50%   1         4         1          2m
```

Auf beiden Clustern dasselbe Bild. Die Ursachen sind aber verschieden.

### Untersuchen

MAC Schritt 8

```
kubectl --context rke2 describe hpa rechner -n dbg-15 | sed -n '/^Conditions/,$p'
kubectl --context talos describe hpa rechner -n dbg-15 | sed -n '/^Events/,$p'
```

Conditions und Events des HPA auf beiden Clustern.

Auf RKE2 steht in der Condition `ScalingActive False` mit dem Reason `FailedGetResourceMetric`, das Event lautet:

```
failed to get cpu utilization: missing request for cpu in container rechner
of Pod rechner-7b9c6d8f5-l2kxn
```

Auf Talos scheitert es schon einen Schritt früher:

```
failed to get cpu utilization: unable to get metrics for resource cpu: unable
to fetch metrics from resource metrics API: the server could not find the
requested resource (get pods.metrics.k8s.io)
```

MAC Schritt 9

```
kubectl --context talos top pods -n dbg-15
kubectl --context talos get apiservice v1beta1.metrics.k8s.io
```

Auf Talos meldet `top` `error: Metrics API not available`, die APIService gibt es nicht (`NotFound`). Auf RKE2 liefert beides Werte, dort läuft `rke2-metrics-server`.

In k9s zeigt `:hpa dbg-15` die HPAs, `d` die Describe-Ansicht mit den Events. Fehlt die Metrics-API, bleiben in k9s auch die Spalten für CPU und Speicher in der Pod-Ansicht leer.

**URSACHE**

Auf RKE2 fehlt der CPU-Request: Der HPA bekommt Messwerte, kann sie aber nicht in Prozent umrechnen. Auf Talos fehlt zusätzlich die Messquelle, weil Talos keinen metrics-server mitbringt. In beiden Fällen setzt der HPA `ScalingActive=False` und ändert die Replikazahl nicht, weder nach oben noch nach unten.

### Beheben auf Talos: metrics-server installieren

metrics-server holt seine Werte über HTTPS vom Kubelet jedes Knotens. Das Kubelet von Talos signiert sein Serving-Zertifikat standardmäßig selbst, metrics-server würde es ablehnen. Die Talos-Doku beschreibt deshalb drei Schritte: Das Kubelet beantragt sein Zertifikat bei der Cluster-CA, ein Approver genehmigt diese Anträge, dann kommt der metrics-server.

DATEI `labor/talos/patch-kubelet-serving-certs.yaml (Ausschnitt)`

```
apiVersion: v1alpha1
kind: KubeletConfig
```

```
extraArgs:
  rotate-server-certificates: "true"
```

Das Kubelet stellt dann einen CertificateSigningRequest mit dem Signer `kubernetes.io/kubelet-serving`.

MAC — Schritt 10

```
talosctl -n 172.16.0.170,172.16.0.171 patch mc \
  --patch @labor/talos/patch-kubelet-serving-certs.yaml
kubectl --context talos apply -f https://raw.githubusercontent.com/\
alex1989hu/kubelet-serving-cert-approver/v0.12.1/deploy/standalone-install.yaml
kubectl --context talos get csr
```

Patch auf beiden Knoten, dann der Approver in der festen Version 0.12.1. Die CSRs mit Signer `kubernetes.io/kubelet-serving` stehen nach kurzer Zeit auf `Approved,Issued`.

**STOLPERFALLE**

Solange die Serving-CSRs auf `Pending` stehen, hat das Kubelet kein Zertifikat. Dann scheitern auch `kubectl logs` und `kubectl exec` auf diesem Knoten mit einem TLS-Fehler. Installiere den Approver deshalb direkt nach dem Patch. Ohne den Patch meldet metrics-server dagegen sinngemäß `x509: cannot validate certificate for 172.16.0.171 because it doesn't contain any IP SANs`. Die Abkürzung `--kubelet-insecure-tls` schaltet die Prüfung ab, die Talos-Doku geht bewusst den sicheren Weg.

DATEI `werkzeuge/metrics-server-values-talos.yaml (Ausschnitt)`

```
replicas: 1
args: []
resources:
  requests:
    cpu: 50m
    memory: 64Mi
```

Kein `--kubelet-insecure-tls`. Die Standard-Argumente des Charts fragen das Kubelet über die `InternalIP` und den Port aus dem Node-Status.

MAC — Schritt 11

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm --kube-context talos upgrade --install metrics-server \
  metrics-server/metrics-server --version 3.14.0 \
  -n kube-system -f werkzeuge/metrics-server-values-talos.yaml --wait
kubectl --context talos top nodes
```

Installiert metrics-server 0.9.0 nach `kube-system`. Nach etwa einer Minute liefert `top nodes` CPU und Speicher beider Knoten.

Jetzt zeigt auch der Talos-HPA dieselbe Meldung wie RKE2: `missing request for cpu`. Den Request setzt der Fix für beide:

MAC Schritt 12

```
kubectl --context rke2 apply -f szenarien/15-rollouts/fix-hpa.yaml
kubectl --context talos apply -f szenarien/15-rollouts/fix-hpa.yaml
```

Der Container fordert jetzt `cpu: 50m` an.

**DAS SOLLTEST DU SEHEN**

Nach ein bis zwei Minuten zeigt `kubectl get hpa -n dbg-15` auf beiden Clustern `cpu: 0%/50%` oder einen kleinen Prozentwert. `describe hpa` meldet `ScalingActive True`.

**RKE2**

metrics-server ist als `rke2-metrics-server` dabei und wird über den Helm-Controller von RKE2 verwaltet. Anpassungen gehen über eine `HelmChartConfig` in `/var/lib/rancher/rke2/server/manifests/`.

**TALOS**

Kein metrics-server, keine Metrics-API. Du installierst ihn selbst, nach dem Muster der Talos-Doku mit Serving-Zertifikaten aus der Cluster-CA. Damit hast du ihn auch selbst zu aktualisieren.

## 15.4 Das Helm-Release hängt

Helm speichert jede Revision eines Release als Secret im Namespace. Vor einem Upgrade legt es die neue Revision mit dem Status `pending-upgrade` an, am Ende setzt es `deployed` oder `failed`. Wird Helm dazwischen hart beendet, bleibt `pending-upgrade` stehen. Das ist zugleich die Sperre gegen parallele Upgrades. Typischer Auslöser in der Praxis: Ein CI-Runner bricht einen Job wegen Zeitüberschreitung ab, während `helm upgrade --wait` noch wartet.

DATEI `szenarien/15-rollouts/haenger.sh (Ausschnitt)`

```
helm --kube-context "$CTX" upgrade demo "$CHART" \
  -n dbg-15 --set readinessPort=9999 --wait --timeout 5m &
HELM_PID=$!
sleep 15
kill -9 "$HELM_PID"
```

Revision 2 wird nie ready, `helm` wartet. Nach 15 Sekunden kommt `SIGKILL`. Auf `Ctrl-C` oder `SIGTERM` würde Helm noch reagieren und das Release als `failed` markieren, auf `SIGKILL` nicht.

MAC Schritt 13

```
szenarien/15-rollouts/haenger.sh rke2
helm --kube-context rke2 upgrade demo \
  szenarien/15-rollouts/demo-chart -n dbg-15
```

Das Skript installiert Revision 1 aus dem Mini-Chart `demo-chart` und bricht das Upgrade auf Revision 2 ab. Danach der Versuch, normal weiterzuarbeiten.

**SYMPTOM**

```
Error: UPGRADE FAILED: another operation (install/upgrade/rollback) is in progress
```

Dabei läuft gar keine andere Operation. Jeder weitere Versuch endet genauso.

## Untersuchen

MAC — Schritt 14

```
helm --kube-context rke2 history demo -n dbg-15
kubectl --context rke2 get secrets -n dbg-15 -l owner=helm --show-labels
```

Die Revisionen des Release, dann die Secrets, in denen Helm sie speichert. Das Label `status` jedes Secrets entspricht der Spalte `STATUS`.

```
REVISION  UPDATED                   STATUS           CHART       ... DESCRIPTION
1         Mon Nov  9 22:41:03 2026  deployed         demo-0.1.0  ... Install complete
2         Mon Nov  9 22:41:20 2026  pending-upgrade  demo-0.1.0  ... Preparing upgrade
```

In k9s öffnest du mit `:helm` die Releases, `r` zeigt die Historie des gewählten Release. Auch `helm list -n dbg-15` zeigt das Release, denn Helm 4 listet standardmäßig alle Zustände. Mit `--pending` filterst du gezielt.

**URSACHE**

Die letzte Revision steht auf `pending-upgrade`. Helm prüft vor jedem Upgrade, ob die jüngste Revision in einem `pending`-Zustand ist, und bricht dann mit `another operation ... is in progress` ab. Der Prozess, der die Revision abschließen sollte, wurde mit `SIGKILL` beendet und konnte nichts mehr aufräumen.

## Beheben

MAC — Schritt 15

```
helm --kube-context rke2 rollback demo 1 -n dbg-15
helm --kube-context rke2 upgrade demo szenarien/15-rollouts/demo-chart \
  -n dbg-15 --wait
```

Der Rollback prüft die Sperre nicht und legt Revision 3 als Kopie von Revision 1 an. Danach ist ein normales Upgrade wieder möglich. In der k9s-Historie geht der Rollback mit `r` auf der gewünschten Revision.

**DAS SOLLTEST DU SEHEN**

`helm history demo -n dbg-15` endet mit einer Revision im Status `deployed`. Revision 3 trägt die Beschreibung `Rollback to 1`.

Hilft der Rollback nicht, etwa weil schon die erste Installation hängt (`pending-install`), bleibt das Löschen des Secrets der hängenden Revision: `kubectl delete secret sh.helm.release.v1.demo.v2 -n dbg-15`. Danach gilt wieder die vorherige Revision. Mach das nur, wenn du sicher bist, dass wirklich kein Helm-Prozess mehr läuft.

**RKE2**

RKE2 installiert seine mitgelieferten Charts (`rke2-traefik`, `rke2-coredns` …) über den eingebauten Helm-Controller, sichtbar an den Jobs `helm-install-rke2-...` in `kube-system`. Diese Releases änderst du nicht mit `helm upgrade`, sondern über `HelmChartConfig`.

**TALOS**

Talos hat keinen Helm-Controller. Alles, was du mit Helm installierst, auch metrics-server und Traefik, verwaltest du selbst mit der Helm-CLI.

## 15.5 Der Job gibt auf

Ein Job startet bei Fehlern neue Pods, bis `backoffLimit` erreicht ist. Die Wartezeit zwischen den Versuchen wächst dabei: 10 Sekunden, 20, 40 und so weiter. `activeDeadlineSeconds` begrenzt zusätzlich die Gesamtlaufzeit.

MAC — Schritt 16

```
kubectl --context rke2 apply -f szenarien/15-rollouts/kaputt-job.yaml
```

Der Job `import` erwartet `/daten/import.csv`, die Datei gibt es nicht. `backoffLimit: 2` erlaubt zwei Wiederholungen.

**SYMPTOM**

```
$ kubectl --context rke2 get jobs,pods -n dbg-15 -l app=import
NAME               STATUS   COMPLETIONS   DURATION   AGE
job.batch/import   Failed   0/1           52s        52s

NAME               READY   STATUS   RESTARTS   AGE
pod/import-7xq9d   0/1     Error    0          52s
pod/import-bk2mw   0/1     Error    0          41s
pod/import-zr4pt   0/1     Error    0          20s
```

### Untersuchen

MAC — Schritt 17

```
kubectl --context rke2 describe job import -n dbg-15 | sed -n '/^Events/,$p'
kubectl --context rke2 logs -n dbg-15 job/import
```

Die Events des Jobs, dann das Log eines seiner Pods. `logs job/...` sucht sich selbst einen Pod aus.

```
  Warning  BackoffLimitExceeded  2s  job-controller  Job has reached the specified
                                                     backoff limit
FEHLER: /daten/import.csv fehlt
```

In k9s zeigt `:jobs dbg-15` die Jobs. `Enter` führt zu den Pods, `l` zu den Logs und `p` zu den Logs des vorherigen Containers, falls einer neu gestartet wurde.

**URSACHE**

Jeder Versuch endet mit Exit-Code 1. Nach drei Pods (ein Versuch plus zwei Wiederholungen) setzt der Job-Controller die Condition `Failed` mit dem Reason `BackoffLimitExceeded` und startet nichts mehr. Wäre stattdessen `activeDeadlineSeconds` abgelaufen, stünde dort `DeadlineExceeded` mit `Job was active longer than specified deadline`.

### Beheben

Das Pod-Template eines Jobs ist unveränderlich. Ein `apply` mit geändertem Befehl scheitert mit `field is immutable`. Du löschst den Job und legst ihn neu an.

MAC — Schritt 18

```
kubectl --context rke2 delete job import -n dbg-15
kubectl --context rke2 apply -f szenarien/15-rollouts/fix-job.yaml
kubectl --context rke2 wait --for=condition=Complete job/import \
  -n dbg-15 --timeout=60s
```

Der korrigierte Job legt seine Eingabedatei selbst an. `wait` kehrt zurück, sobald der Job fertig ist.

**DAS SOLLTEST DU SEHEN**

`job.batch/import condition met`, `kubectl get jobs -n dbg-15` zeigt `Complete` und `1/1`.

**STOLPERFALLE**

`ttlSecondsAfterFinished: 3600` löscht den Job eine Stunde nach dem Ende, mitsamt Pods und Logs. Bei CronJobs räumen zusätzlich `successfulJobsHistoryLimit` und `failedJobsHistoryLimit` auf. Untersuche gescheiterte Jobs zeitnah. Einen CronJob testest du gezielt mit `kubectl create job --from=cronjob/<name> test-1`.

## 15.6 Die ConfigMap-Änderung kommt nicht an

MAC — Schritt 19

```
kubectl --context rke2 apply -f szenarien/15-rollouts/kaputt-configmap.yaml
kubectl --context rke2 patch configmap begruessung -n dbg-15 \
  --type=merge -p '{"data":{"GRUSS":"Hallo v2"}}'
```

Das Deployment `gruss` liest `GRUSS` per `envFrom` aus der ConfigMap und gibt den Wert alle zehn Sekunden aus. Danach änderst du die ConfigMap.

**SYMPTOM**

`kubectl --context rke2 logs -n dbg-15 deploy/gruss --tail 3` zeigt auch Minuten später nur `Hallo v1`.

**URSACHE**

Umgebungsvariablen setzt das Kubelet einmal beim Start des Containers. Eine spätere Änderung der ConfigMap erreicht den laufenden Prozess nie. Als Volume eingebundene ConfigMaps aktualisiert das Kubelet mit Verzögerung (Sync-Periode plus Cache, zusammen oft ein bis zwei Minuten), außer bei `subPath`-Mounts. Ob die Anwendung die geänderte Datei dann neu liest, ist ihre Sache.

MAC — Schritt 20

```
kubectl --context rke2 rollout restart deploy/gruss -n dbg-15
```

Startet die Pods neu, sie lesen die ConfigMap beim Start. In k9s geht das in der Deployment-Ansicht mit `r`.

**DAS SOLLTEST DU SEHEN**

Nach dem Neustart zeigt `kubectl --context rke2 logs -n dbg-15 deploy/gruss --tail 1` den Text `Hallo v2`.

In Helm-Charts löst du das dauerhaft mit einer Annotation am Pod-Template, die eine Prüfsumme der ConfigMap enthält, etwa `checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}`. Ändert sich die ConfigMap, ändert sich das Template, und Kubernetes rollt neu aus.

## 15.7 Aufräumen

MAC — Schritt 21

```
helm --kube-context rke2 uninstall demo -n dbg-15
kubectl --context rke2 delete ns dbg-15
kubectl --context talos delete ns dbg-15
```

Entfernt das Helm-Release und beide Namespaces. Der metrics-server auf Talos bleibt, du brauchst ihn in den weiteren Übungen und für k9s.

**PROBIER ES SELBST**

Setze in einer Kopie von `fix-rollout.yaml` das Image auf `busybox:1.37-gibtesnicht`, `progressDeadlineSeconds` auf `30` und die `change-cause` auf `v3: neues Image`. Wende sie an. Welche Condition hat das Deployment nach einer Minute, welchen Status der neue Pod, und welche Revision stellst du mit `rollout undo --to-revision` wieder her, damit `v2` mit korrigierter Probe läuft?

Hinweis: Wieder `Progressing False` mit `ProgressDeadlineExceeded`, diesmal steht der neue Pod auf `ImagePullBackOff` (Kapitel 5). `rollout history` zeigt dir, welche Revision die `change-cause` „Probe korrigiert“ trägt.

---

**WAS DU GELERNT HAST**

- Ein hängender Rollout zeigt `Progressing=False` mit `ProgressDeadlineExceeded`. Kubernetes rollt nicht von selbst zurück, das machst du mit `rollout undo`.
- `<unknown>` beim HPA hat zwei häufige Gründe: keine Metrics-API (Talos ohne metrics-server) oder kein Request im Container. Die Events des HPA unterscheiden beides.
- metrics-server auf Talos braucht Kubelet-Serving-Zertifikate aus der Cluster-CA: `rotate-server-certificates`, ein Approver für die CSRs, dann das Chart.
- `another operation (install/upgrade/rollback) is in progress` heißt: Die letzte Revision steht auf `pending-*`. `helm history` zeigt sie, `helm rollback` löst die Sperre.
- Jobs geben nach `backoffLimit` mit `BackoffLimitExceeded` auf. Ihr Template ist unveränderlich, also löschen und neu anlegen.
- ConfigMaps als Umgebungsvariablen wirken erst nach einem Neustart: `rollout restart` oder eine Prüfsummen-Annotation.

---

KAPITEL 16

# Anhang

*Fehlermeldungen, Befehle und Tasten zum schnellen Nachschlagen, wenn es brennt.*

## 16.1 Spickzettel: Von der Meldung zum ersten Befehl

Die linke Spalte ist das, was du siehst. Die mittlere nennt die häufigste Ursache, nicht die einzige. Die rechte sagt dir, wo du zuerst hinschaust.

| Meldung oder Status | Häufigste Ursache | Erster Blick | Kap. |
|---|---|---|---|
| `Pending`, `Insufficient cpu` | Requests größer als der freie Platz auf jedem Knoten | `describe pod`, Events | 4 |
| `didn't match Pod's node affinity/ selector` | Label fehlt auf dem Knoten oder Tippfehler im Selektor | `get nodes --show-labels` | 4 |
| `had untolerated taint(s)` | Taint ohne passende Toleration | `describe node`, `Taints:` | 4 |
| `unbound immediate PersistentVolumeClaims` | PVC ohne StorageClass oder mit falschem Namen | `describe pvc` | 4, 9 |
| Deployment ohne Pods, `FailedCreate` | ResourceQuota, LimitRange oder Pod Security lehnt ab | Events am ReplicaSet | 4, 10 |
| `ImagePullBackOff`, `not found` | Tippfehler im Tag oder Repository | `describe pod`, Events | 5 |
| `ImagePullBackOff`, `pull access denied` | Privates Repository ohne `imagePullSecrets` | Secret im selben Namespace | 5 |
| `toomanyrequests` | Rate-Limit von Docker Hub | Registry-Mirror auf dem Knoten | 5 |
| `CreateContainerConfigError` | Secret, ConfigMap oder Key fehlt | `describe pod` | 5 |
| `exec format error` | Skript ohne Shebang oder falsche Architektur | Image und Entrypoint prüfen | 5 |
| `CrashLoopBackOff`, Exit 127 | Befehl im Container nicht gefunden | `logs --previous` | 6 |
| `CrashLoopBackOff`, Exit 1 | Anwendung beendet sich mit Fehler | `logs --previous` | 6 |

| Meldung oder Status | Häufigste Ursache | Erster Blick | Kap. |
|---|---|---|---|
| `OOMKilled`, Exit 137 | Speicherlimit zu klein oder Speicherleck | `lastState`, Limits | 6 |
| Neustarts, `failed liveness probe` | Probe zu streng, Start dauert länger | Events, `startupProbe` | 6 |
| `Running`, aber `0/1` ready | Readiness-Probe schlägt fehl | Events, EndpointSlice | 6, 7 |
| `Read-only file system` | `readOnlyRootFilesystem` ohne beschreibbares Volume | `securityContext` | 6 |
| Service ohne Endpoints | Selektor passt nicht zu den Pod-Labels | `get endpointslices` | 7 |
| `connection refused` | Falscher `targetPort` oder Prozess lauscht woanders | `kubectl debug`, `ss -ltn` | 7 |
| `NXDOMAIN`, DNS-Timeout | Falscher Name, Namespace oder Egress auf Port 53 blockiert | `nslookup` aus einem Pod | 7 |
| NetworkPolicy wirkt nicht | CNI setzt keine Policies durch (Flannel auf Talos) | CNI prüfen | 7 |
| HTTP 404 vom Ingress | Falsche `ingressClassName`, falscher Host oder Service-Port | Controller-Logs | 8 |
| HTTP 503 vom Ingress | Kein bereiter Pod hinter dem Service | Endpoints, Readiness | 8 |
| `TRAEFIK DEFAULT CERT` | TLS-Secret falsch aufgebaut oder nicht gefunden | `describe secret`, Traefik-Log | 8 |
| `Permission denied` auf Volume | Prozess-UID darf nicht schreiben | `ls -ln` im Pod | 9 |
| `Multi-Attach`, Pod auf anderem Knoten hängt | RWO-Volume, Replikas auf mehreren Knoten | `get pods -o wide` | 9 |
| `Forbidden`, `cannot list resource` | RBAC: Binding fehlt, falscher Namespace oder Role statt ClusterRole | `auth can-i --as` | 10 |
| `violates PodSecurity` | Pod Security Admission des Namespace | Namespace-Labels | 10 |

| Meldung oder Status | Häufigste Ursache | Erster Blick | Kap. |
|---|---|---|---|
| `admission webhook ... denied the request` | Kyverno-Policy greift | Policy-Name aus der Meldung | 11 |
| `failed calling webhook` | Webhook-Backend nicht erreichbar, `failurePolicy: Fail` | `get validatingwebhookconfigurations` | 11, 14 |
| Falco-Alarm | Ein Prozess tut etwas, das eine Regel als verdächtig einstuft | Alarm-Felder, Pod, Container-ID | 12 |
| Knoten `NotReady` | Kubelet läuft nicht oder erreicht den API-Server nicht | Kubelet-Log auf dem Knoten | 13 |
| Pod `Terminating`, `Unknown` | Knoten ausgefallen, Eviction nach 300 s | `get nodes`, Tolerations | 13 |
| `Evicted` | Knoten oder Pod hat ein Limit für Speicher oder Platz überschritten | `status.message` | 13 |
| Drain hängt | PodDisruptionBudget erlaubt keine Unterbrechung | `get pdb` | 13 |
| `x509: certificate has expired` | Client-Zertifikat in der kubeconfig abgelaufen | Zertifikat dekodieren | 14 |
| Namespace bleibt `Terminating` | Nicht erreichbarer APIService oder Finalizer | `get apiservice`, Conditions | 14 |
| `ProgressDeadlineExceeded` | Neue Pods werden nie ready | `rollout status`, neue Pods | 15 |
| HPA zeigt `<unknown>` | metrics-server fehlt oder Pods ohne CPU-Requests | `top pods`, `describe hpa` | 15 |
| `another operation ... is in progress` | Abgebrochenes Helm-Release im Zustand `pending-*` | `helm history` | 15 |
| `BackoffLimitExceeded` | Job-Pods scheitern öfter als erlaubt | Logs der Job-Pods | 15 |

## 16.2 RKE2 und Talos: dieselbe Frage, zwei Befehle

Auf RKE2 laufen die Befehle auf dem Knoten, nach `sudo -i` und den Exporten aus Kapitel 2. Auf Talos laufen sie auf dem Mac, mit `-n` und der IP des Knotens.

| Frage | RKE2 (auf dem Knoten) | Talos (auf dem Mac) |
|---|---|---|
| Läuft der Knoten-Dienst? | `systemctl status rke2-server` bzw. `rke2-agent` | `talosctl -n <IP> services` |
| Log des Knoten-Dienstes | `journalctl -u rke2-server` | `talosctl -n <IP> logs kubelet` |
| Kubelet-Log | `/var/lib/rancher/rke2/agent/logs/kubelet.log` | `talosctl -n <IP> logs kubelet` |
| Kernel-Meldungen, OOM-Killer | `journalctl -k` | `talosctl -n <IP> dmesg` |
| Container des Knotens | `crictl ps -a` | `talosctl -n <IP> containers --namespace cri` |
| Log eines Containers | `crictl logs <id>` | `talosctl -n <IP> logs --namespace cri <id>` |
| Image prüfen oder holen | `crictl images`, `crictl pull` | `talosctl -n <IP> image list`, `image pull` |
| Überblick in Echtzeit | `top`, `df -h` | `talosctl -n <IP> dashboard` |
| Konfiguration | `/etc/rancher/rke2/config.yaml` | `talosctl -n <IP> get machineconfig -o yaml` |
| Konfiguration ändern | Datei bearbeiten, Dienst neu starten | `talosctl -n <IP> patch mc --patch @datei` |
| Registry-Mirror | `/etc/rancher/rke2/registries.yaml` | Dokument `RegistryMirrorConfig` |
| etcd-Zustand | `etcdctl` im etcd-Pod | `talosctl -n <IP> etcd status`, `etcd members` |
| etcd-Snapshot | `rke2 etcd-snapshot save` | `talosctl -n <IP> etcd snapshot <datei>` |
| Zertifikate prüfen | `rke2 certificate check` | `talosctl config info`, `get kubernetesdynamiccerts` |
| Alles einsammeln | Logs und `crictl`-Ausgaben von Hand | `talosctl -n <IP> support --no-encryption` |

## 16.3 k9s: die wichtigsten Tasten

| Taste | Wirkung |
|---|---|
| `:pods`, `:deploy`, `:events` | Ressourcenansicht wechseln, mit Namespace dahinter: `:pods dbg-04` |
| `:ctx` | Kontext wechseln |
| `0` | Alle Namespaces anzeigen |
| `/text`, `/!text`, `/-l app=web` | Filtern, invertiert filtern, nach Label filtern |
| `d`, `y`, `e` | Describe, YAML, Bearbeiten |
| `l`, `p` | Logs, Logs des vorherigen Laufs |

| Taste | Wirkung |
|---|---|
| `s` | Shell im Container |
| `Enter` | Container-Ansicht eines Pods öffnen |
| `o`, `Shift-j` | Zum Knoten springen, zum Besitzer springen |
| `Shift-f` | Port-Forward einrichten |
| `Ctrl-w` | Weitere Spalten einblenden |
| `Ctrl-d`, `Ctrl-k` | Löschen mit Bestätigung, sofort löschen |
| `:xray deploy`, `:pulses` | Abhängigkeitsbaum, Überblick über den Cluster |
| `Shift-D` | Plugin aus Kapitel 3: netshoot-Debug-Container am Pod |
| `Shift-E` | Plugin aus Kapitel 3: Events der markierten Ressource |
| `Esc`, `?` | Zurück, Hilfe mit allen Tasten der aktuellen Ansicht |

Starte k9s mit `--readonly`, wenn du nur suchst und nichts ändern willst.

## 16.4 Versionen

Alle Angaben gelten für den Stand dieses Buches, 9. November 2026.

| Komponente | Version |
|---|---|
| RKE2 | `v1.36.6+rke2r1` |
| Talos Linux und `talosctl` | `v1.14.5` |
| Kubernetes auf beiden Clustern | `1.36.6` |
| k9s | `v0.51.0` |
| Kyverno (Chart `kyverno/kyverno`) | `v1.19.2` (Chart `3.9.2`) |
| Falco (Chart `falcosecurity/falco`) | `0.45.2` (Chart `9.2.2`) |
| Traefik für Talos (Chart `traefik/traefik`) | `v3.7.17` (Chart `42.3.1`) |
| metrics-server für Talos (Chart `metrics-server/metrics-server`) | Chart `3.14.0` |
| local-path-provisioner | `v0.0.37` |
| netshoot | `nicolaka/netshoot:v0.15` |

## 16.5 Das Repo im Überblick

```
kubernetes-debuggen/
├── buch/            Quelle dieses Buches (Typst), book-en/ die englische Ausgabe
├── labor/
```

```
│   ├── rke2/        config-cp.yaml, config-w1.yaml, registries-mirror.yaml
│   └── talos/       patch-cp.yaml, patch-w1.yaml, patch-local-path.yaml,
│                    patch-registry.yaml und die Patches aus Kapitel 13 und 14
├── k9s/             plugins.yaml                                  (Kap. 2, 3)
├── werkzeuge/       Helm-Werte für Kyverno, Falco, Traefik, metrics-server,
│                    local-path für Talos
├── policies/        Kyverno-Policies mit Tests unter tests/       (Kap. 11)
└── szenarien/       ein Ordner pro Kapitel, darin kaputt-*.yaml und fix-*.yaml
```

Jede Datei `kaputt-*.yaml` legt ihren Namespace `dbg-<kapitel>` selbst an. Das Aufräumen ist deshalb immer dasselbe: `kubectl --context <cluster> delete namespace dbg-<kapitel>`.

## 16.6 Offizielle Dokumentation

- Kubernetes, Debugging: kubernetes.io/docs/tasks/debug
- RKE2: docs.rke2.io
- Talos Linux 1.14: docs.siderolabs.com/talos/v1.14
- k9s: k9scli.io
- Kyverno: kyverno.io/docs
- Falco: falco.org/docs
- Traefik: doc.traefik.io/traefik

---

**WAS DU GELERNT HAST**

- Fehlersuche ist eine Methode, kein Talent: Symptom, Schicht, Ursache, Fix, Prüfung.
- Die meisten Antworten stehen in Events, im `describe` und in den Logs des vorherigen Laufs.
- Auf RKE2 gehst du per SSH auf den Knoten, auf Talos fragst du die API mit `talosctl`.
- Policies, Admission und Laufzeit-Sicherheit sind eigene Fehlerquellen. Lies die Meldung bis zum Ende, sie nennt fast immer den Schuldigen.

---

KUBERNETES DEBUGGEN

Ein Pod hängt in `Pending`, ein anderer startet seit einer Stunde neu, und der Webhook lehnt jede Änderung ab. Wo fängst du an?

Dieses Buch macht Fehlersuche zur Routine. Du baust zwei Cluster, einen mit RKE2 und einen mit Talos Linux, und machst sie gezielt kaputt. Zu jedem Fehler siehst du das Symptom, die Untersuchung mit `kubectl` und k9s, die Ursache und den Fix, jeweils auf beiden Distributionen.

AUS DEM INHALT

| | |
|---|---|
| **Methode** | Vom Symptom über die Schicht zur Ursache |
| **Werkzeuge** | kubectl, k9s, crictl, talosctl und kubectl debug |
| **Workloads** | Pending, ImagePullBackOff, CrashLoopBackOff, OOMKilled |
| **Netzwerk** | Services, DNS, NetworkPolicies, Ingress und TLS |
| **Sicherheit** | RBAC, Pod Security, Kyverno und Falco als Fehlerquelle |
| **Plattform** | Nodes, etcd, Zertifikate, Webhooks und Rollouts |
| **Distributionen** | RKE2 und Talos im direkten Vergleich |

**Thomas Zachmann**
Autor auf Leanpub



Hands-on-Handbuch · Stand: 9. November 2026